

Algorísmia
Col·lecció d'Exàmens Resolts

Albert Atserias Amalia Duch Jordi Petit Salvador Roura

Facultat de Matemàtiques i Estadística
Universitat Politècnica de Catalunya

Índex

I	Enunciats	19
1	Parcial, 8 de novembre de 2024	21
1.1	Com sempre, un misteri	21
1.2	Internal Path Length	22
1.3	Comptant i ordenant	22
1.4	Arbres binaris de cerca òptims	22
2	Extraordinari, 4 de juliol de 2024	25
2.1	Nombres pràctics	25
2.2	Elements freqüents	25
2.3	Heaps	25
2.4	Element únic	26
2.5	Teoria de jocs	26
2.6	Daus amb lletres	26
2.7	Complexitat	26
3	Final, 16 de gener de 2024	27
3.1	NP-completesa	27
3.2	Ordenant amb repeticions	27
3.3	Càlcul eficient de recurrències	27
3.4	Quad-trees	28
3.5	Xarxa de carrers	28
3.6	Diverses consultes	28
3.7	THREE-DIMENSIONAL MATCHING	28
4	Parcial, 3 de novembre de 2023	31
4.1	Quick Select	31
4.2	Paraules amb repeticions	31
4.3	Procediment recursiu (1)	32
4.4	Parell de punts més propers en el pla	32
4.5	Procediment recursiu (2)	33

5	Extraordinari, 5 de juliol de 2023	35
5.1	Subparaules diferents	35
5.2	Complexitat	35
5.3	Codis de Huffman	35
5.4	Teoria de jocs	36
5.5	Heaps	36
5.6	Arbres AVL	36
5.7	Ordenació curiosa	36
5.8	Punts d'articulació i arestes pont	36
6	Final, 10 de gener de 2023	37
6.1	Recorreguts d'un arbre binari	37
6.2	Cerques binàries ineficients	38
6.3	Servidor web	38
6.4	Vèrtexs verds i blancs	39
6.5	Rotacions de paraules	39
6.6	Teoria de jocs	39
6.7	Punts d'articulació i arestes pont	40
6.8	Flux màxim	40
7	Parcial, 28 d'octubre de 2022	41
7.1	Sumar un	41
7.2	Cost d'alguns bucles	41
7.3	Codis de Huffman	41
7.4	Tallant taulons	42
7.5	Bandera belga	42
7.6	Com sempre, un codi misteriós	42
8	Final, 11 de gener de 2022	45
8.1	Quants a l'interval?	45
8.2	Ordenació sàvia	45
8.3	Punts en un cercle	45
8.4	Arbres de cerca	46
8.5	Teoria de jocs	46
8.6	Isomorfisme d'un subgraf	46
8.7	Flux màxim	47
8.8	Influencer	48
9	Parcial, 29 d'octubre de 2021	49
9.1	Ordenació boja	49
9.2	Multiplicant matrius	49
9.3	Codi misteriós	50
9.4	Xips defectuosos	50
9.5	I encara més codis misteriosos	50
10	Extraordinari, 1 de juliol de 2021	53
10.1	Quicksort, de nou	53
10.2	Dijkstra modificat	54
10.3	Arbres aleatoris dolents	54
10.4	Teoria de jocs	54
10.5	Un altre codi misteriós	55
10.6	Pintant un graf	55

11 Final, 18 de gener de 2021	57
11.1 Parells de primers bessons	57
11.2 Dividint un vector	57
11.3 Arbres de cerca aleatoris	57
11.4 Arbres de cerca aleatoritzats	58
11.5 Kruskal	58
11.6 Teoria de jocs	58
11.7 Taules de dispersió	58
11.8 Cicles hamiltonians	59
12 Parcial, 6 de novembre de 2020	61
12.1 Potències de 2	61
12.2 Rang d'un element	61
12.3 És un min_heap?	61
12.4 Quicksort millorat?	62
12.5 Maps i sets	62
12.6 Ordre d'una permutació	63
13 Extraordinari, 7 de juliol de 2020	65
13.1 Heaps	65
13.2 L'agència matrimonial	65
13.3 Cub de Rubik	66
13.4 Algunes demostracions	67
13.5 Taules dinàmiques	67
14 Final, 9 de gener de 2020	69
14.1 Element comú	69
14.2 Heaps	69
14.3 RSA	69
14.4 Arbres de cerca AVL	70
14.5 Quick-sort	70
14.6 2-SAT	71
14.7 Torneig de go	71
14.8 NP-completesa	72
15 Parcial, 5 de novembre de 2019	73
15.1 Exponenciació de matrius	73
15.2 Misteri	73
15.3 Els k més propers a la mediana	74
15.4 Euclides i Fibonacci	74
15.5 Programacions dinàmiques	74
16 Final, 15 de gener de 2019	77
16.1 Arbres diversos	77
16.2 Heap, heap,	77
16.3 Rangs en arbres 2d	78
16.4 Bonnie i Clide	78
16.5 Triangle de Sierpinski	78
16.6 Màxim nombre de camins independents	79
16.7 Knuth-Morris-Pratt	79
16.8 Teoria de jocs	80
16.9 Quatre nombres	80

17 Parcial, 5 de novembre de 2018	81
17.1 Recurrències	81
17.2 Misteri	81
17.3 Ordenació i més	82
17.4 Eficiència	82
17.5 Multiplicació súper-eficient	83
17.6 El nombre perdut	83
17.7 El rètol lumínic	83
18 Extraordinari, 29 de juny de 2018	85
18.1 Misteri	85
18.2 Suma de dos elements	85
18.3 Heaps	85
18.4 NP-completesa	86
18.5 Problema de l'aturada	86
18.6 Tall mínim	86
18.7 Teoria de jocs	86
18.8 Què calcula?	87
19 Final, 18 de gener de 2018	89
19.1 Recurrències	89
19.2 Els k més petits, en ordre	89
19.3 Arbres AVL	89
19.4 NP-completesa	90
19.5 2-SAT	90
19.6 Teoria de jocs	90
19.7 Què fan?	90
19.8 Codi estrany	91
20 Parcial, 6 de novembre de 2017	93
20.1 Ordenació	93
20.2 Cues de prioritats	93
20.3 Programació dinàmica	94
20.4 Multiplicació russo-francesa	94
20.5 Notació asimptòtica	94
20.6 Zeros i uns	94
21 Extraordinari, 27 de juny de 2017	97
21.1 Cerques i insercions	97
21.2 Encara un altre codi misteriós	98
21.3 I encara un altre	98
21.4 Arbres AVL	98
21.5 La motxilla	99
21.6 Els del mig	99
21.7 Collatz i Turing	99
21.8 El valor d'un graf	100
22 Final, 17 de gener de 2017	101
22.1 Hashing universal	101
22.2 Recurrències	101
22.3 MAX3SAT	102
22.4 Flux màxim	102

22.5	Teoria de jocs	102
22.6	Codi espatllat	102
22.7	Més codis misteriosos	103
23	Parcial, 15 de novembre de 2016	105
23.1	Recurrència simpàtica	105
23.2	Els més petits	105
23.3	Mínim local	105
23.4	Ordenació	106
23.5	Codi misteriós	106
23.6	El problema d'en Nikochan	106
24	Extraordinari, 7 de juliol de 2016	107
24.1	Recurrències	107
24.2	Escacs	107
24.3	Consultes en un llibre electrònic	107
24.4	Codis misteriosos	108
24.5	Algorisme de Kruskal	109
24.6	Teoria de jocs	110
25	Final, 19 de gener de 2016	111
25.1	Costos	111
25.2	Heaps	112
25.3	Cicles i camins Hamiltonians	112
25.4	Inaproximabilitat	112
25.5	Arbres aleatoritzats	112
25.6	Components fortament connexos	113
25.7	Algorisme de Prim	113
25.8	Flors i colors	113
26	Parcial, 3 de novembre de 2015	115
26.1	Pica-pica	115
26.2	Maleïdes recurrències	115
26.3	Element solitari	116
26.4	En posició!	116
26.5	Comptatge	116
26.6	Selecció	116
27	Extraordinari, 26 de juny 2015	117
27.1	Punts propers a l'origen	117
27.2	Propietats dels algorismes d'ordenació	117
27.3	Grafs	117
27.4	Teoria de jocs	118
27.5	Arbres AVL	118
27.6	Un altre codi espifiat	118
27.7	NP-completesa	118
28	Final, 16 de gener de 2015	119
28.1	Recurrència	119
28.2	Arbres AVL	119
28.3	Heaps ternaris	119
28.4	NP-completesa	120

28.5	Ordenació topològica	120
28.6	Codi espifat	120
28.7	Max-flow	121
28.8	Teoria de jocs	121
29	Parcial, 14 de novembre de 2014	123
29.1	Ordenació per selecció	123
29.2	Recurrència	123
29.3	Primer element repetit	123
29.4	Subvector de suma zero	124
29.5	Nombres de Fibonacci	124
29.6	Heaps	124
29.7	Ordenació topològica	124
30	Extraordinari, 7 de juliol de 2014	125
30.1	Recurrència	125
30.2	Interseccions	125
30.3	Anagrames	125
30.4	Creació de heaps	125
30.5	Quadrat d'una matriu	126
30.6	Arbres de cerca	126
30.7	Teoria de jocs	126
31	Final, 16 de gener de 2014	127
31.1	Sumeu zero	127
31.2	Ordenació de vectors ordenadets	127
31.3	Arbres de cerca	127
31.4	Teoria de jocs	128
31.5	Fluxos	128
31.6	Codi mort	129
32	Parcial, 8 de novembre de 2013	131
32.1	Heu preparat la llicó?	131
32.2	Polinomis	131
32.3	Heaps i heapsort	131
32.4	Comparacions del Quicksort	132
32.5	Múltiples de 66 en un conjunt	132
33	Final, 15 de gener de 2013	133
33.1	El mateix que el del parcial	133
33.2	Algorisme curiós	133
33.3	Problemes petits i bonics sobre vectors	134
33.4	Agència matrimonial	135
33.5	Tasques i màquines	135
34	Parcial, 13 de novembre de 2012	137
34.1	Ompliu els blancs	137
34.2	Comparacions del QuickSort	137
34.3	L'Euclides no hi és	138
34.4	La funció misteri ataca de nou	138
34.5	Nombres aborrits	139
34.6	Encara més misteris?	139

35 Extraordinari, 29 de juny de 2012	141
35.1 Miscel·lània	141
35.2 Algorismes d'aproximació	142
35.3 Complexitat i indecidibilitat	142
35.4 Programació dinàmica	142
35.5 Teoria de jocs	143
35.6 Codi misteriós	143
36 Final, 11 de gener de 2012	145
36.1 Codis de Huffman	145
36.2 Heaps	145
36.3 Teoria de jocs	145
36.4 Estructures de dades	146
36.5 NP-completesa i programació dinàmica	146
36.6 Algorismes d'aproximació	146
37 Parcial, 27 d'octubre de 2011	149
37.1 Ompliu els blancs	149
37.2 Costs	150
37.3 Sumar un	150
37.4 Heaps	151
37.5 Triangles en grafs	151
38 Extraordinari, 5 de juliol de 2011	153
38.1 Cues de prioritats	153
38.2 Anàlisi	153
38.3 NP-completesa	154
38.4 Indecidibilitat	154
38.5 Aproximació	154
38.6 Grafs	155
38.7 Arbres	155
38.8 Més arbres	155
39 Final, 11 de gener de 2011	157
39.1 Heaps i AVLs	157
39.2 Anàlisi	157
39.3 NP-completesa	158
39.4 Indecidibilitat	158
39.5 Aproximació	159
39.6 Grafs	159
39.7 Llistes	159
39.8 Arbres	160
40 Parcial, 28 d'octubre de 2010	161
40.1 Ompliu els blancs	161
40.2 Ordenació	162
40.3 Dividir i vèncer	162
40.4 Primalitat	162
40.5 Producte de matrius	162

II	Solucions	165
1	Parcial, 8 de novembre de 2024	167
1.1	Com sempre, un misteri	167
1.2	Internal Path Length	168
1.3	Comptant i ordenant	168
1.4	Arbres binaris de cerca òptims	169
2	Extraordinari, 4 de juliol de 2024	171
2.1	Nombres pràctics	171
2.2	Elements freqüents	171
2.3	Heaps	172
2.4	Element únic	173
2.5	Teoria de jocs	173
2.6	Daus amb lletres	173
2.7	Complexitat	173
3	Final, 16 de gener de 2024	175
3.1	NP-completesa	175
3.2	Ordenant amb repeticions	175
3.3	Càlcul eficient de recurrències	176
3.4	Quad-trees	176
3.5	Xarxa de carrers	176
3.6	Diverses consultes	176
3.7	THREE-DIMENSIONAL MATCHING	177
4	Parcial, 3 de novembre de 2023	179
4.1	Quick Select	179
4.2	Paraules amb repeticions	179
4.3	Procediment recursiu (1)	180
4.4	Parell de punts més propers en el pla	180
4.5	Procediment recursiu (2)	180
5	Extraordinari, 5 de juliol de 2023	183
5.1	Subparaules diferents	183
5.2	Complexitat	183
5.3	Codis de Huffman	184
5.4	Teoria de jocs	184
5.5	Heaps	184
5.6	Arbres AVL	185
5.7	Ordenació curiosa	185
5.8	Punts d'articulació i arestes pont	186
6	Final, 10 de gener de 2023	187
6.1	Recorreguts d'un arbre binari	187
6.2	Cerques binàries ineficients	187
6.3	Servidor web	188
6.4	Vèrtexs verds i blancs	188
6.5	Rotacions de paraules	189
6.6	Teoria de jocs	189
6.7	Punts d'articulació i arestes pont	189
6.8	Flux màxim	190

7	Parcial, 28 d'octubre de 2022	193
7.1	Sumar 1	193
7.2	Cost d'alguns bucles	193
7.3	Codis de Huffman	194
7.4	Tallant taulons	194
7.5	Bandera belga	195
7.6	Com sempre, un codi misteriós	195
8	Final, 11 de gener de 2022	197
8.1	Quants a l'interval?	197
8.2	Ordenació sàvia	197
8.3	Punts en un cercle	197
8.4	Arbres de cerca	198
8.5	Teoria de jocs	198
8.6	Isomorfisme d'un subgraf	198
8.7	Flux màxim	199
8.8	Influencer	200
9	Parcial, 29 d'octubre de 2021	201
9.1	Ordenació boja	201
9.2	Multiplicant matrius	201
9.3	Codi misteriós	202
9.4	Xips defectuosos	202
9.5	I encara més codis misteriosos	203
10	Extraordinari, 1 de juliol de 2021	205
10.1	Quicksort, de nou	205
10.2	Dijkstra modificat	206
10.3	Arbres aleatoris dolents	206
10.4	Teoria de jocs	206
10.5	Un altre codi misteriós	206
10.6	Pintant un graf	207
11	Final, 18 de gener de 2021	209
11.1	Parells de primers bessons	209
11.2	Dividint un vector	210
11.3	Arbres de cerca aleatoris	210
11.4	Arbres de cerca aleatoritzats	210
11.5	Kruskal	211
11.6	Teoria de jocs	212
11.7	Taules de dispersió	212
11.8	Cicles hamiltonians	213
12	Parcial, 6 de novembre de 2020	215
12.1	Potències de 2	215
12.2	Rang d'un element	215
12.3	És un min-heap?	216
12.4	Quicksort millorat?	217
12.5	Maps i sets	218
12.6	Ordre d'una permutació	218

13 Extraordinari, 7 de juliol de 2020	221
13.1 Heaps	221
13.2 L'agència matrimonial	222
13.3 Cub de Rubik	223
13.4 Algunes demostracions	223
13.5 Taules dinàmiques	224
14 Final, 9 de gener de 2020	225
14.1 Element comú	225
14.2 Heaps	226
14.3 RSA	226
14.4 Arbres de cerca AVL	227
14.5 Quick-sort	227
14.6 2-SAT	228
14.7 Torneig de go	229
14.8 NP-completesa	229
15 Parcial, 5 de novembre de 2019	231
15.1 Exponenciació de matrius	231
15.2 Misteri	231
15.3 Els k més propers a la mediana	232
15.4 Euclides i Fibonacci	232
15.5 Programacions dinàmiques	232
16 Final, 5 de gener de 2019	235
16.1 Arbres diversos	235
16.2 Heap, heap,	235
16.3 Rang en arbres 2d	236
16.4 Bonnie i Clyde	236
16.5 Triangle de Sierpinski	237
16.6 Màxim nombre de camins independents	237
16.7 Knuth-Morris-Pratt	238
16.8 Teoria de jocs	238
16.9 Quatre nombres	238
17 Parcial, 5 de novembre de 2018	239
17.1 Recurrències	239
17.2 Misteri	239
17.3 Ordenació i més	239
17.4 Eficiència	240
17.5 Multiplicació súper-eficient	240
17.6 El nombre perdut	240
17.7 El rètol lumínic	240
18 Extraordinari, 29 de juny de 2018	241
18.1 Misteri	241
18.2 Suma de dos elements	241
18.3 Heaps	241
18.4 NP-completesa	242
18.5 Problema de l'aturada	242
18.6 Components fortament connexos	242
18.7 Tall mínim	243

18.8	Teoria de jocs	244
18.9	Què calcula?	244
19	Final, 18 de gener de 2018	245
19.1	Recurrències	245
19.2	Els k més petits, en ordre	245
19.3	Arbres AVL	245
19.4	NP-completesa	246
19.5	2-SAT	246
19.6	Teoria de jocs	246
19.7	Què fan?	246
19.8	Codi estrany	247
20	Parcial, 6 de novembre de 2017	249
20.1	Ordenació	249
20.2	Cues de prioritats	249
20.3	Programació dinàmica	250
20.4	Multiplicació russo-francesa	250
20.5	Notació asimptòtica	250
20.6	Zeros i uns	250
21	Extraordinari, 27 de juny de 2017	251
21.1	Cerques i insercions	251
21.2	Encara un altre codi misteriós	251
21.3	I un altre	252
21.4	Arbres AVL	252
21.5	La motxilla	252
21.6	Els del mig	253
21.7	Collatz i Turing	253
21.8	El valor d'un graf	253
22	Final, 17 de gener de 2017	255
22.1	Hashing universal	255
22.2	Recurrències	255
22.3	MAX3SAT	255
22.4	Flux màxim	256
22.5	Teoria de jocs	256
22.6	Codi espatllat	256
22.7	Més codis misteriosos	257
23	Parcial, 15 de novembre de 2016	259
23.1	Recurrència simpàtica	259
23.2	Els més petits	259
23.3	Mínim local	260
23.4	Ordenació	260
23.5	Codi misteriós	260
23.6	El problema d'en Nikochan	260

24 Extraordinari, 7 de juliol de 2016	261
24.1 Recurrències	261
24.2 Escacs	261
24.3 Consultes en un llibre electrònic	262
24.4 Codis misteriosos	262
24.5 Algorisme de Kruskal	263
24.6 Teoria de jocs	263
25 Final, 19 de gener de 2016	265
25.1 Costos	265
25.2 Heaps	266
25.3 Cicles i camins Hamiltonians	267
25.4 Inaproximabilitat	267
25.5 Arbres aleatoritzats	268
25.6 Components fortament connexts	268
25.7 Algorisme de Prim	268
25.8 Flors i colors	269
26 Parcial, 3 de novembre de 2015	271
26.1 Pica-pica	271
26.2 Maleïdes recurrències	271
26.3 Element solitari	272
26.4 En posició!	272
26.5 Comptatge	272
26.6 Selecció	272
27 Extraordinari, 26 de juny de 2015	273
27.1 Punts propers a l'origen	273
27.2 Propietats dels algorismes d'ordenació	273
27.3 Grafs	273
27.4 Teoria de jocs	274
27.5 Arbres AVL	274
27.6 Un altre codi espifat	274
27.7 NP-completesa	274
28 Final, 16 de gener de 2015	275
28.1 Recurrència	275
28.2 Arbres AVL	276
28.3 Heaps ternaris	276
28.4 NP-completesa	277
28.5 Ordenació topològica	277
28.6 Codi espifat	278
28.7 Max-flow	278
28.8 Teoria de jocs	278
29 Parcial, 14 de novembre de 2014	279
29.1 Ordenació per selecció	279
29.2 Recurrència	280
29.3 Primer element repetit	280
29.4 Subvector de suma zero	281
29.5 Nombres de Fibonacci	281
29.6 Heaps	281

29.7 Ordenació topològica	281
30 Extraordinari, 7 de juliol de 2014	283
30.1 Recurrència	283
30.2 Interseccions	283
30.3 Anagrames	283
30.4 Creació de heaps	284
30.5 Quadrat d'una matriu	284
30.6 Arbres de cerca	284
30.7 Teoria de jocs	284
31 Final, 16 de gener de 2014	285
31.1 Sumeu zero	285
31.2 Ordenació de vectors ordenadets	285
31.3 Arbres de cerca	286
31.4 Teoria de jocs	286
31.5 Fluxos	286
31.6 Codi mort	286
32 Parcial, 8 de novembre de 2013	287
32.1 Heu preparat la lliçó?	287
32.2 Polinomis	287
32.3 Heaps i heapsort	288
32.4 Comparacions del Quicksort	289
32.5 Múltiples de 66 en un conjunt	289
33 Final, 15 de gener de 2013	291
33.1 El mateix que el del parcial	291
33.2 Algorisme curiós	291
33.3 Problemes petits i bonics sobre vectors	292
33.4 Agència matrimonial	292
33.5 Tasques i màquines	293
34 Parcial, 13 de novembre de 2012	295
34.1 Ompliu els blancs	295
34.2 Comparacions del QuickSort	295
34.3 L'Euclides no hi és	296
34.4 La funció misteri ataca de nou	297
34.5 Nombres aborrits	297
34.6 Encara més misteris?	297
35 Extraordinari, 29 de juny de 2012	299
35.1 Miscel·lània	299
35.2 Algorismes d'aproximació	299
35.3 Complexitat i indecidibilitat	300
35.4 Programació dinàmica	300
35.5 Teoria de jocs	301
35.6 Codi misteriós	301

36 Final, 11 de gener de 2012	303
36.1 Codis de Huffman	303
36.2 Heaps	303
36.3 Teoria de jocs	304
36.4 Estructures de dades	304
36.5 NP-completesa i programació dinàmica	305
36.6 Algorismes d'aproximació	306
37 Parcial, 27 d'octubre de 2011	307
37.1 Ompliu els blancs	307
37.2 Costos	308
37.3 Sumar un	308
37.4 Heaps	308
37.5 Triangles en grafs	309
38 Extraordinari, 5 de juliol de 2011	311
38.1 Cues de prioritats	311
38.2 Anàlisi	311
38.3 NP-completesa	311
38.4 Indecidibilitat	312
38.5 Aproximació	312
38.6 Grafs	312
38.7 Arbres	312
38.8 Més arbres	312
39 Final, 11 de gener de 2011	313
39.1 Heaps i AVLs	313
39.2 Anàlisi	314
39.3 NP-completesa	314
39.4 Indecidibilitat	315
39.5 Aproximació	315
39.6 Grafs	315
39.7 Llistes	316
39.8 Arbres	316
40 Parcial, 28 d'octubre de 2010	319
40.1 Ompliu els blancs	319
40.2 Ordenació	320
40.3 Dividir i vèncer	320
40.4 Primalitat	320
40.5 Producte de matrius	321

Prefaci

Aquesta és una col·lecció de problemes i d'exàmens resolts per a l'assignatura **Algorísmia** del Grau de Matemàtiques de la Facultat de Matemàtiques i Estadística.

Els exàmens recollits en aquesta col·lecció són els que s'han fet en aquesta assignatura des de la seva creació. Els enunciats, amb les corresponents solucions, s'han ordenat expressament de més a menys recents.

La nostra voluntat és anar ampliant i millorant aquesta col·lecció d'exàmens resolts amb el pas del temps.

Salvador i Jordi
Barcelona, 6 de juliol de 2011.

Part I

Enunciats

Parcial, 8 de novembre de 2024

1.1 Com sempre, un misteri

Considereu aquest codi. Sigui $n = v.size()$. Podeu suposar $0 \leq e \leq d < n$.

```
void misteri (vector<double>& v, int e, int d) {  
    if (e + 1 == d) {  
        if (v[e] > v[d]) swap(v[e], v[d]);  
    }  
    else if (e + 2 == d) {  
        if (v[e+1] < v[e]) swap(v[e+1], v[e]);  
        if (v[d] < v[e+1]) swap(v[d], v[e+1]);  
        if (v[e+1] < v[e]) swap(v[e+1], v[e]);  
    }  
    else if (e + 3 ≤ d) {  
        int n = d - e + 1;  
        int i = e + n/4;  
        int j = d - n/4;  
        int m = (i + j)/2;  
        misteri(v, m, d);  
        misteri(v, i, j);  
        misteri(v, e, m);  
        misteri(v, i, d);  
    }  
}
```

- (a) Digueu raonadament què fa una crida a `misteri(v, e, d)`, i justifiqueu-ne la correctesa.
- (b) Calculeu una fita inferior i una fita superior al cost d'una crida a `misteri(v, 0, n - 1)`.
- (c) Donades les fites calculades a l'apartat anterior, digueu si `misteri()` fa la seva feina de manera eficient.

1.2 Internal Path Length

L'*Internal Path Length* (IPL) d'un arbre binari de cerca es defineix com la suma, sobre tots els nodes x de l'arbre, de la longitud (mesurada com el nombre d'arestes) del camí que va des d' x fins a l'arrel de l'arbre. Per exemple, l'IPL d'un arbre amb un sol node és 0, i l'IPL d'un arbre amb dos nodes és 1.

- Doneu un exemple d'un arbre binari de cerca amb $n = 6$ nodes, i el valor del seu IPL.
- Donat un arbre binari de cerca (quasi) complet d' n nodes, doneu una recurrència $C(n)$ que descriu el seu IPL. Resoleu la recurrència.
- Considereu ara un arbre binari de cerca construït a partir de la inserció successiva d'una permutació aleatòria de les claus $\{1, 2, \dots, n\}$ en un arbre inicialment buit. Doneu una recurrència $P(n)$ per a l'IPL esperat de l'arbre obtingut després d'aquest procés. Digueu quina n'és la solució.
- Quina implicació té el resultat anterior en el cost de les operacions de cerca, inserció i esborrat en arbres binaris de cerca construïts a partir de permutacions aleatòries?

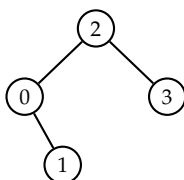
1.3 Comptant i ordenant

Teniu un vector $V[0..n-1]$ amb n enters, on n és gros (p.ex., $n = 10^6$). Sigui d el nombre d'elements diferents que hi ha a V .

- Proposeu dos algorismes eficients per calcular d . Un que ho sigui en el cas pitjor i l'altre que ho sigui en el cas mitjà. Calculeu el cost dels dos algorismes.
- Suposeu que coneixeu d , i que és molt més petit que n (p.ex., $d = 10^3$). Proposeu un algorisme per obtenir V ordenat. El cost esperat, asimptòticament quan n i d creixen (amb d seguint sent molt més petit que n) del vostre algorisme ha de ser millor que el dels algorismes d'ordenació estàndard.

1.4 Arbres binaris de cerca òptims

Considereu una situació on tenim n objectes identificats amb claus, que per senzillesa suposarem que són $0, \dots, n-1$. Cada clau i té una probabilitat d'accés $p_i > 0$ coneguda, fixa i independent. Volem construir un arbre binari de cerca (ABC) que minimitzi el cost esperat (mesurat com a nombre de nodes visitats) d'accés als objectes. Per exemple, amb $n = 4$, $p_0 = 0.3$, $p_1 = 0.1$, $p_2 = 0.5$ i $p_3 = 0.1$, l'ABC



té un cost esperat de $0.3 \cdot 2 + 0.1 \cdot 3 + 0.5 \cdot 1 + 0.1 \cdot 2 = 1.6$. Es pot comprovar que amb aquestes probabilitats tots els altres ABCs tenen un cost més gran.

Per a tot parell (e, d) amb $0 \leq e \leq d < n$, sigui $f(e, d)$ el cost esperat d'un accés a una clau de l'ABC òptim corresponent a les claus de $[e, d]$. Per exemple, $f(0, 3) = 1.6$ i $f(0, 1) = 0.75 \cdot 1 + 0.25 \cdot 2 = 1.25$. En el segon cas, les probabilitats de les claus de $[0, 1]$ s'han ajustat de forma proporcional perquè sumin 1 (p_0 segueix sent tres vegades més gran que p_1).

- (a) Mostreu una n i una combinació de probabilitats amb la qual posar recursivament la clau amb la màxima probabilitat a l'arrel de cada subarbre $[e, d]$ no produeixi un arbre total òptim.
- (b) Sense escriure codi, mostreu una recurrència per a $f(e, d)$. Per al cas general, useu notació matemàtica o paraules si us cal per explicar el càlcul que faríeu. Quin cost en temps i quin cost en espai tindria una programació dinàmica basada en la vostra solució?
- (c) Sigui $a_{e,d}$ l'arrel òptima que cal triar per construir el subarbre $[e, d]$. (Per senzillesa, suposeu que és única.) Per exemple, $a_{0,3} = 2$ i $a_{0,1} = 0$. Òbviament, tenim $a_{x,x} = x$. Per al cas general, es pot demostrar que es compleix

$$a_{e,d-1} \leq a_{e,d} \leq a_{e+1,d} .$$

Doneu alguna explicació intuïtiva per justificar les dues desigualtats anteriors. No us en demanem una demostració rigurosa, la qual aquí seria massa difícil.

- (d) Expliqueu com podríeu fer servir les desigualtats anteriors per reduir el cost en temps de la programació dinàmica. Quin cost podeu aconseguir?

Pista: En l'anàlisi del cost surt quelcom semblant a una "suma telescòpica".

Extraordinari, 4 de juliol de 2024

2.1 Nombres pràctics

En aquest problema, anomenarem un nombre pràctic (un nombre real computable, entre 0 i 1, en base 10) a un programa que escriu una seqüència infinita de dígit entre 0 i 9, que interpretarem com els infinits decimals del nombre. Per exemple, un programa que escrigui $50000\dots$ es correspon a $1/2$, un que escrigui $3333\dots$ es correspon a $1/3$, un que escrigui $14159265359\dots$ es correspon a $\pi - 3$, etc.

En Pepet afirma que ha fet un programa X tal que, donat qualsevol programa Y , X és capaç de determinar en un nombre finit de passos si Y és un nombre pràctic, és a dir, si Y escriu un nombre infinit de dígit entre 0 i 9 (i res més). Pot ser que en Pepet tingui raó?

2.2 Elements freqüents

Donat un vector amb n paraules d' m caràcters cadascuna, cal trobar totes les paraules que apareguin estrictament més de $n/4$ vegades.

Proposeu un algorisme que usi taules hash per resoldre aquest problema. Analitzeu el cost en el cas pitjor i en el cas mitjà de la vostra solució.

Proposeu un algorisme diferent que tingui cost $\Theta(mn)$ en el cas pitjor.

2.3 Heaps

Siguin A i B dos max-heaps amb n i m enters, respectivament. Se sap que $n = 2^x - 1$ per a alguna $x \geq 1$, i que $2^{(x-1)} \leq m \leq n + 1$. Dissenyeu un algorisme amb cost $\Theta(x)$ en el cas pitjor que produeixi un max-heap amb tots els elements d' A i B . Expliqueu el vostre algorisme amb un exemple amb $x = 3$.

2.4 Element únic

Sigui V un vector amb n enters. Sabem que exactament un element apareix una vegada, i tots els altres apareixen dues vegades, i consecutivament. Per exemple, podríem tenir $V = [4, 4, 2, 2, 3, 3, 7, 1, 1, 8, 8]$. Sense donar codi, expliqueu amb prou detall un mètode eficient per trobar l'element únic (7, a l'exemple). Quin cost té el vostre algorisme en el cas pitjor? Ha de ser inferior a $\Theta(n)$.

2.5 Teoria de jocs

Considereu aquesta variant del joc del Nim, amb "piles" de dues dimensions: Donats diversos rectangles, cadascun de mides $n_i \times m_i$, per torns cal triar un dels rectangles i esborrar-li les x files de baix (amb $1 \leq x \leq n_i$), o les y columnes de la dreta (amb $1 \leq y \leq m_i$). Per exemple, si el rectangle triat té mides 2×4 , el rectangle resultant pot quedar buit, o tenir mides 1×4 , 2×3 , 2×2 , o 2×1 . Com sempre, perd qui no pot jugar.

Quin és el número d'un rectangle 2×4 ?

Una posició té dos rectangles, un de mides 2×4 , i l'altre de mides 1×3 . És la posició guanyadora o perdedora? Si fos guanyadora, mostreu totes les jugades que guanyin.

2.6 Daus amb lletres

Suposeu un alfabet amb un nombre arbitràriament gran de caràcters. Sigui s una paraula amb m caràcters. Teniu $n \geq m$ daus, cadascun amb c cares. Cada dau té un caràcter diferent a les seves cares. És possible que es pugui construir s tirant tots els daus, descartant els que sobrin, i posant en un ordre adequat els altres?

Per exemple, sigui $s = \text{"pas"}$. Si tenim quatre daus, cadascun amb quatre cares, amb aquests caràcters: "sati", "quin", "esco", i "pare", la resposta és positiva: descartem el segon dau, i agafem la 'p' de l'últim dau, la 'a' del primer, i la 's' del tercer.

Dissenyau un algorisme polinòmic en la mida de l'entrada (és a dir, en $m + n \cdot c$) per resoldre aquest problema.

[Pista: *Inspireu-vos en un algorisme dels grafs.*]

2.7 Complexitat

Independentment de si heu sabut resoldre el problema anterior, responeu les preguntes següents:

- Podem afirmar que el problema pertany a NP, o que no hi pertany?
- Podem afirmar que el problema és NP-difícil, o que no ho és?
- Que implicaria que la resposta a les dues preguntes anteriors fos positiva?

Final, 16 de gener de 2024

3.1 NP-completesa

Considereu l'algorisme següent per determinar si un graf conte una clique (un subgraf complet) de mida k : Primer, generem tots els possibles subconjunts de vèrtexs de mida k . N'hi ha $O(n^k)$. Després, per a cada subconjunt, comprovem si és un subgraf complet. Digueu si aquest algorisme és o no un algorisme polinòmic per resoldre el problema CLIQUE. Si la resposta és afirmativa, què implicaria respecte a com són les classes P i NP?

3.2 Ordenant amb repeticions

Donada una seqüència d' n enters amb moltes repeticions, de tal manera que el nombre d'enters diferents és $O(\log(n))$, dissenyeu un algorisme que ordeni els n nombres amb $O(n \log(\log(n)))$ comparacions en el cas pitjor. Expliqueu per què en aquest cas no es compleix la fita inferior $\Omega(n \log(n))$ en el cas pitjor dels algorismes d'ordenació basats en comparacions.

3.3 Càlcul eficient de recurrències

Siguin f_0, f_1, f_2, a, b i c sis nombres enters qualssevol. Podem definir aquesta recurrència: $F(0) = f_0, F(1) = f_1, F(2) = f_2$, i $F(n) = a \cdot F(n-1) + b \cdot F(n-2) + c \cdot F(n-3)$ per a tota $n \geq 3$. Per exemple, la combinació $f_0 = c = 0, f_1 = f_2 = a = b = 1$ defineix els nombres de Fibonacci.

Doneu un algorisme que calculi, per a qualsevol combinació donada dels sis paràmetres i d'una n , $F(n)$ amb el millor cost asimptòtic (en n) possible. Supposeu que disposeu d'un ordinador que no pot treballar amb nombres reals, però que pot fer operacions aritmètiques amb enters arbitràriament grans amb cost $\Theta(1)$.

3.4 Quad-trees

Recordeu que un quad-tree és una estructura de dades similar a un arbre binari de cerca, però per a punts en dues dimensions. En tots els apartats d'aquest problema, considereu el cost $C(T)$ d'una cerca sense èxit en el cas pitjor en un quad-tree T amb n punts.

(a) Supposeu $n = 5$, que les coordenades x dels punts són 0, 1, 2, 3 i 4, i que les coordenades y dels punts són 5, 6, 7, 8 i 9. Associeu les x amb les y per construir cinc punts, i amb aquests punts construïu un quad-tree. El resultat final ha de tenir el cost més baix possible d'una cerca sense èxit en el cas pitjor d'entre totes les eleccions que podeu fer.

Sigui n arbitràriament gran. Teniu llibertat per construir el quad-tree que vulgueu.

(b) Quin és el creixement asimptòtic de la funció

$$C(n) = \min\{C(T), T \text{ és un quad-tree amb } n \text{ punts}\}?$$

(c) Responen la mateixa pregunta canviant min per max.

3.5 Xarxa de carrers

Considereu la xarxa de carrers d'una ciutat, modelada com un graf dirigit, on els vèrtexs es corresponen a encreuaments, i els arcs a carrers on només es pot circular en un dels dos sentits. Proposeu un algorisme eficient per determinar si és possible anar des de qualsevol encreuament fins a qualsevol altre. Quin cost té el vostre algorisme?

3.6 Diverses consultes

(a) Sigui t un string amb n lletres minúscules, i w un string amb m lletres minúscules, amb $m \leq n$. Expliqueu breument l'algorisme Rabin-Karp per determinar amb cost $\Theta(n)$ i amb alta probabilitat d'encert si w es troba dins de t .

(b) Sigui V un vector amb n nombres naturals. Donades dues posicions e i d , amb $0 \leq e \leq d < n$, sigui $S(e, d) = \sum_{e \leq i \leq d} V[i]$. Supposeu que es faran n consultes per conèixer $S(e, d)$ per a diversos parells e i d . Expliqueu quin precàlcul es podria fer perquè el cost total del precàlcul i de les n consultes sigui només $\Theta(n)$. Supposeu que $S(0, n-1)$ no pateix sobreiximent. Pista: Considereu inclusió-exclusió.

(c) Aquí, direm que una paraula és un quadrat si és la concatenació d'una paraula amb ella mateixa. Per exemple, "holahola" és un quadrat.

Sigui s un string amb n lletres minúscules. Supposeu que es faran n consultes, cadascuna definida amb dues posicions e i d , amb $0 \leq e \leq d < n$, i tals que $d - e$ és senar, per saber si $s[e..d]$ és un quadrat. Inspirant-vos en els dos apartats anteriors, expliqueu quins precàlculs es podrien fer perquè el cost total dels precàlculs i de les n consultes sigui només $\Theta(n)$, i que les respostes siguin correctes amb alta probabilitat.

3.7 THREE-DIMENSIONAL MATCHING

El problema THREE-DIMENSIONAL MATCHING es pot explicar així: Tenim n parells de sabates s_i , n vestits v_j , i n barrets b_k , amb n^3 combinacions possibles (s_i, v_j, b_k) . Una matriu $M[n \times n \times n]$ de booleans conté a cada posició $M[i][j][k]$ si la combinació (s_i, v_j, b_k) quedaria estèticament bé (segons l'opinió d'algú). El problema consisteix a determinar si hi ha n combinacions que quedarien bé i que no tinguin elements en comú.

Sabent que THREE-DIMENSIONAL MATCHING és NP-difícil, expliqueu si és probable que es pugui resoldre en temps polinòmic. Aquest problema pertany a NP?

Suposeu ara que tenim dues matrius $A[n \times n]$ i $B[n \times n]$ de booleans. $A[i][j]$ és certa si el parell (s_i, v_j) quedaria bé, i $B[j][k]$ és certa si el parell (v_j, b_k) quedaria bé. Suposeu que $M[i][j][k]$ és certa si i només si $A[i][j]$ és certa i $B[j][k]$ és certa. Sota aquestes condicions, digueu si el problema es pot resoldre en temps polinòmic. Si la resposta fos afirmativa, expliqueu com es podria fer.

Parcial, 3 de novembre de 2023

4.1 Quick Select

El problema de la selecció consisteix a, donats un vector $[e_1, e_2, \dots, e_n]$ amb n elements diferents i un enter k , $1 \leq k \leq n$, trobar el k -èsim element més petit del vector. L'algorisme d'ordenació *Quick_Sort*, vist a classe, es pot modificar fàcilment per resoldre el problema de la selecció. L'algorisme resultant es coneix com a *Quick_Select*.

Expliqueu com seria el vostre mètode *Quick_Select*, i doneu el cost en els casos millor i pitjor del vostre algorisme. El vostre *Quick_Select* és òptim en el cas pitjor?

4.2 Paraules amb repeticions

Considereu dues paraules $A = a_1a_2 \dots a_n$ i $B = b_1b_2 \dots b_m$ formades a partir d'un mateix alfabet finit, i tals que $m \leq n$. Diem que B és una subseqüència d' A si existeixen m índexs $i_1 < i_2 < \dots < i_m$ tals que, per a tot $1 \leq j \leq m$, tenim que $b_j = a_{i_j}$. És a dir, B és una subseqüència d' A si es pot construir B seleccionant alguns elements d' A preservant-ne l'ordre.

(a) Proposeu un algorisme per determinar si B és subseqüència d' A en temps $O(n)$.

Donada una paraula B , definim B^i com la paraula on cada caràcter de B apareix i vegades de manera consecutiva. Per exemple, si $B = xyzzx$, llavors $B^3 = xxxxyyzzzzzzxxx$. Com a cas particular, tenim que B^0 és la paraula buida.

(b) Useu l'algorisme proposat a (a) per dissenyar un algorisme que trobi la màxima i per a la que B^i és una subseqüència d' A en temps $O(n \log(n/m))$.

4.3 Procediment recursiu (1)

Sigui V un *vector* `<double>` ordenat creixentment, amb possibles repetits. Considereu aquest procediment recursiu:

```
int f(int e, int d, double x, const vector<double>& V) {
    if (e > d) return e;
    int m = (e + d)/2;
    if (V[m] <= x) return f(m + 1, d, x, V);
    if (m > 0 and V[m-1] <= x) return m;
    return f(e, m - 1, x, V);
}
```

En general, què retorna $f(0, V.size() - 1, x, V)$?

Sigui $n = V.size()$. Analitzeu el cost de $f(0, V.size() - 1, x, V)$.

4.4 Parell de punts més propers en el pla

Donats $n \geq 2$ punts (x_i, y_i) en el pla amb coordenades reals, volem trobar la distància euclídea (habitual) mínima entre tots els parells de punts. Per senzillesa, suposem que no hi ha x 's ni y 's repetides.

Un mètode possible consisteix en, abans que res, ordenar eficientment tots els punts respecte d' x . Després, apliquem aquest algorisme recursiu: Si n no és més gran que una certa constant petita, la resposta es troba trivialment. Suposem ara que n és prou gran. Considereu una línia vertical, definida amb $x = X$, que separa els n punts en dos subconjunts amb aproximadament $n/2$ punts cadascun: $P = \{\text{els punts amb } x < X\}$, i $G = \{\text{els punts amb } x > X\}$. Resolem recursivament cadascun dels dos subproblemes sobre P i sobre G , cosa que ens dona d_p i d_g (la distància mínima dels punts amb x 's petites i grans, respectivament). Sigui $d = \min(d_p, d_g)$.

Ara, trobem $A = \{\text{tots els punts } (x_i, y_i) \text{ tals que } |x_i - X| \leq d\}$. És fàcil veure que la resposta al problema és el mínim entre d i el mínim de les distàncies entre tots els parells de punts d' A . Fixeu-vos que A pot tenir un nombre $\Theta(n)$ de punts.

(a) Trobeu una fita superior al cost d'aquest algorisme en el cas pitjor.

(b) Suposeu que l'algorisme retorna, com efecte lateral, tots els punts donats ordenats segons les y 's. Argumenteu com obtindríeu aquesta llista a partir de les llistes obtingudes recursivament sobre P i sobre G . Quin cost té el vostre mètode?

(c) A partir d'aquesta llista ordenada per y , podem obtenir trivialment amb cost $\Theta(n)$ una llista L també ordenada per y només amb els punts d' A . Suposeu que es pogués demostrar que per obtenir la resposta només cal calcular la distància de cada punt d' L amb els $O(\sqrt{n})$ anteriors i posteriors dins d' L . Calculeu el nou cost de l'algorisme aprofitant aquesta hipòtesi.

(d) De fet, amb arguments geomètrics es pot demostrar que només cal calcular la distància de cada punt d' L amb els 7 anteriors i posteriors dins d' L . Calculeu de nou el cost de l'algorisme.

4.5 Procediment recursiu (2)

Sigui V un *vector* `<int>` de mida n . Considereu aquest procediment recursiu:

```
void f(int i) {  
    if (i == n) {  
        for (int j = 0; j < n - 2; ++j)  
            if (V[j] + V[j+1] + V[j+2] == 0) return;  
        for (int x : V) cout << x;  
        cout << endl;  
        return;  
    }  
    V[i] = 0;  
    f(i + 1);  
    V[i] = 1;  
    f(i + 1);  
}
```

Què escriu $f(0)$? Com faríeu que aquest codi fos més eficient? No cal que calculeu el cost del nou codi.

Analitzeu el cost d'una crida a $f(0)$. Doneu el resultat fent servir la notació $\Theta()$.

Extraordinari, 5 de juliol de 2023

5.1 Subparaules diferents

Donada una paraula s amb n lletres, es vol comptar quantes subparaules diferents conté. Per exemple, $s = \text{"abab"}$ conté set subparaules (no buides) diferents: "a", "b", "ab", "ba", "aba", "bab" i "abab".

L'algorisme obvi per resoldre aquest problema guardaria cada subparaula en un conjunt (implementat amb un arbre de cerca o amb una taula de dispersió), i escriuria la mida final del conjunt. Expliqueu com resoldríeu aquest problema (potser amb una petita probabilitat d'error) de forma més eficient en general. Quin cost té el vostre algorisme?

5.2 Complexitat

Aquí, considereu que tots els problemes són decisionals (la sortida és SÍ o NO). Recordeu que EXP és la classe dels problemes que es poden resoldre amb algun algorisme de cost exponencial. Un problema és EXP-complet si pertany a EXP i qualsevol problema d'EXP s'hi pot reduir polinòmicament. Recordeu que $P \subseteq NP \subseteq EXP$, i que se sap que $P \neq EXP$.

- a) Sigui X un problema EXP-complet. Pot ser que X es resolgui en temps polinòmic?
- b) Sigui Y un problema NP-complet. Es pot reduir polinòmicament Y a X ?
- c) Què implicaria que X es pogués reduir polinòmicament a Y ?

5.3 Codis de Huffman

a) Calculeu un codi de Huffman per als 5 símbols a, b, c, d, e amb freqüències respectives 0.10, 0.05, 0.60, 0.20, 0.05. Quina és la longitud esperada de la codificació d'un missatge d' n símbols generat segons aquesta distribució?

b) Dels dos lemes que configuren la demostració d'optimalitat dels codis de Huffman vista a classe, demostreu-ne el primer:

Lema: *D'entre els codis instantanis òptims, n'hi ha algun que té els dos símbols amb freqüències més petites com a germans a l'arbre associat. A més, s'hi troben a nivell 0 (és a dir, a profunditat màxima).*

5.4 Teoria de jocs

Considereu aquesta variant del joc del nim: *A cada torn, no es poden treure més pedres que les que l'adversari ha tret en el torn anterior. Inicialment, se'n pot treure qualsevol nombre.*

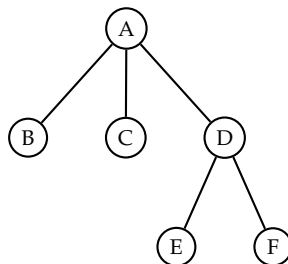
Argumenteu si aquesta variant es pot estudiar amb nímbers. Si la resposta és afirmativa, expliqueu com es podria analitzar. Altrament, expliqueu com calcularíeu si una posició inicial és guanyadora o perdedora.

5.5 Heaps

Donat un vector V amb els elements 1, 7, 2, 0, 4, 5, 8, 6, 3, 9 en aquest ordre, mostreu pas a pas com es transformaria en un max-heap amb l'algorisme de cost linial explicat a classe (*heapify*). Dibuixeu la seqüència d'arbres binaris quasi-complets (tots de mida 10) que il·lustra aquest procés. En tot moment, ignoreu la posició que típicament es deixa buida a l'inici de V .

5.6 Arbres AVL

Donat un arbre arrelat, definim la profunditat de cada node com la distància a l'arrel, suposant que la profunditat de l'arrel és 1. Per exemple, a l'arbre



A té profunditat 1, B, C i D tenen profunditat 2, i E i F tenen profunditat 3.

Considereu un arbre de cerca AVL amb les claus $\{1, 2, \dots, 12\}$. A quina profunditat màxima es pot trobar la clau 6? Mostreu un exemple.

5.7 Ordenació curiosa

Suposeu que modifiqueu l'algorisme d'ordenació per fusió (*mergesort*), de manera que una de les dues crides recursives es substitueix per una ordenació trivial quadràtica. Quin cost tindria l'algorisme resultant?

5.8 Punts d'articulació i arestes pont

a) Considereu un graf no dirigit G amb n vèrtexs. Quants punts d'articulació pot tenir G com a màxim?

b) Pot G tenir exactament un punt d'articulació però cap aresta pont? Si la resposta és afirmativa, mostreu-ne un exemple amb una petita explicació. Altrament, argumenteu per què no pot ser.

Final, 10 de gener de 2023

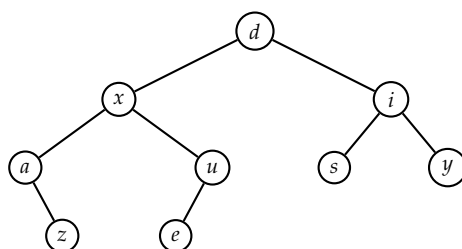
6.1 Recorreguts d'un arbre binari

Sigui A un arbre binari amb elements diferents. Si x és l'element a l'arrel d' A , i E i D són els subarbres esquerre i dret d' A , llavors el preordre $p(A)$, l'inordre $i(A)$, i el postordre $t(A)$ d' A són les seqüències definides recursivament així:

$$\begin{aligned} p(A) &= x + p(E) + p(D), \\ i(A) &= i(E) + x + i(D), \\ t(A) &= t(E) + t(D) + x. \end{aligned}$$

En tots tres casos, el símbol '+' denota la concatenació de seqüències. Si A és buit, llavors tots tres $p(A)$, $i(A)$, i $t(A)$ es defineixen com la seqüència buida.

Per exemple, amb l'arbre binari



el preordre és *dxazueisy*, el postordre és *zaxeusyid*, i l'inordre és *azxeudsiy*.

És possible reconstruir un A desconegut si es coneixen $p(A)$ i $i(A)$? I si es coneixen $p(A)$ i $t(A)$? Si la resposta és afirmativa, argumenteu-ho. Altrament, doneu un contraexemple.

6.2 Cerques binàries ineficients

Sigui V un vector amb n reals en ordre estrictament creixent. Aquest codi recursiu, ben conegut, diu si x es troba dins de $V[e..d]$:

```
bool busca(double x, int e, int d, vector<double> V) {
    if (e > d) return false;
    int m = (e + d)/2;
    if (x < V[m]) return busca(x, e, m - 1, V);
    if (x > V[m]) return busca(x, m + 1, d, V);
    return true;
}
```

Lamentablement, el programador d'aquest codi ha oblidat passar V per referència. Quin és el cost en el cas pitjor de $\text{busca}(x, 0, n - 1, V)$?

Suposeu que la funció $\text{extreu}(V, e, d)$ retorna amb cost $\Theta(d - e + 1)$ un vector amb els elements de $V[e..d]$, mantenint-ne l'ordre. Aquest codi fa els mateixos passos que l'anterior, però passant com a paràmetre només el tros actual T del vector original V :

```
bool busca(double x, vector<double> T) {
    if (T.empty()) return false;
    int m = (T.size() - 1)/2;
    if (x < T[m]) return busca(x, extreu(T, 0, m - 1));
    if (x > T[m]) return busca(x, extreu(T, m + 1, T.size() - 1));
    return true;
}
```

Quin és el cost en el cas pitjor de $\text{busca}(x, V)$?

6.3 Servidor web

Volem detectar les adreces IP (32 bits) que es connecten a un servidor web més d'un cop durant un dia. Ho implementem amb una taula de hash $T[1, \dots, m]$ d' m bits que es reinicialitzen a 0 cada dia a les 00:00. Quan una adreça x es connecta al servidor, es consulta el bit $T[h(x)]$, on $h: \{0, 1\}^{32} \rightarrow [m]$ és la funció de hash. Si el bit és 0, es posa a 1. Si és 1, s'afegeix x a una llista de possibles repetits (que es guarda fora de la taula).

Per simplificar les anàlisis, suposeu que cada aplicació d'una funció de hash $h(x)$ és una variable aleatòria que pren valors a $[m]$ amb probabilitat uniforme, i que és independent de totes les altres. A tots els apartats, suposeu que en el dia en qüestió s'han tractat n adreces diferents. Escriviu les fórmules de probabilitat en funció d' m i n .

a) Calculeu la probabilitat que la següent adreça diferent que es connecti doni un fals positiu, és a dir, que s'afegeixi a la llista tot i que sigui el primer cop que es connecta.

Per reduir els falsos positius, decidiu usar dues funcions de hash h_1, h_2 : Quan arriba una adreça x , es consulten els bits $T[h_1(x)]$ i $T[h_2(x)]$. Si els dos bits són 1, s'afegeix x a la llista. Altrament, es posen els dos bits a 1 (potser un ja ho era).

b) Donada una posició i de la taula T , calculeu la probabilitat $P(i)$ que $T[i] = 1$.

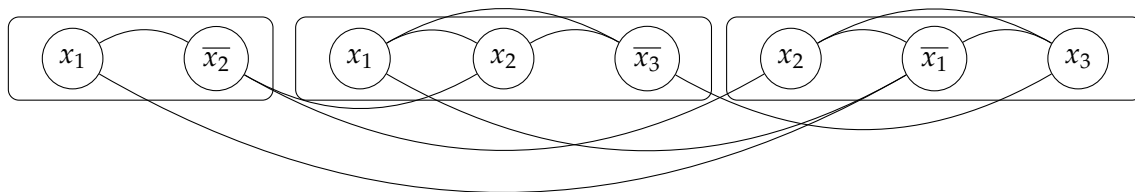
c) Donades dues posicions i i j de T , sigui $P(i, j)$ la probabilitat que $T[i] = T[j] = 1$. Encara amb l'esquema de les dues funcions de hash h_1, h_2 , digueu quina de les dues quantitats $P(i, j)$ o $P(i)P(j)$ és més gran quan $i \neq j$. Feu servir aquest fet i la resposta de la caixa anterior per donar una (bona) fita superior a la probabilitat que la següent adreça diferent que es connecti doni un fals positiu.

6.4 Vèrtexs verds i blancs

Recordeu que un literal és una variable x_i o la seva negació $\bar{x}_i$, i que una clàusula és un conjunt de literals diferents. Recordeu també el problema SAT: Donades una fórmula F formada per m clàusules $C_1, \dots, C_m$ en n variables booleanes $x_1, \dots, x_n$, cal determinar si F és satisfactible; és a dir, si es poden assignar valors CERT o FALS a cada variable x_i de manera que cada clàusula tingui com a mínim un literal cert.

A partir d'una fórmula F , construïm un graf no dirigit $G(F)$: Per a cada clàusula C d' F i cada literal ℓ de C , afegim un vèrtex amb etiqueta (C, ℓ) . Dos vèrtexs diferents etiquetats amb (C, ℓ) i (C', ℓ') es connecten amb una aresta si, o bé $C = C'$, o bé $C \neq C'$ però els literals ℓ i ℓ' són incompatibles: l'un és x_i i l'altre és $\bar{x}_i$ per a algun i .

Per exemple, si F és la fórmula amb les tres clàusules $C_1 = \{x_1, \bar{x}_2\}$, $C_2 = \{x_1, x_2, \bar{x}_3\}$, i $C_3 = \{x_2, \bar{x}_1, x_3\}$, llavors $G(F)$ és el graf següent (les caixes hi són només per indicar quins vèrtexs estan etiquetats amb la mateixa primera component C que, per claredat, s'ha suprimit):



Volem pintar els vèrtexs de $G(F)$ amb dos colors, verd i blanc, de manera que no hi hagi dos vèrtexs veïns de color verd.

Discutiu quin és el nombre màxim de vèrtexs de $G(F)$ que es poden pintar de color verd segons si F és satisfactible o no.

Amb el que sabeu de teoria, discutiu si existeix un algorisme polinòmic que, donat un graf, determini el màxim nombre de vèrtexs que es poden pintar de color verd.

6.5 Rotacions de paraules

Donada una paraula s de mida n només amb lletres minúscules, en definim la i -èsima rotació (per a $0 \leq i < n$) com $s_i s_{i+1} \dots s_{n-1} s_0 \dots s_{i-2} s_{i-1}$.

Donades dues paraules s i t , amb el mateix nombre de lletres de cada, cal calcular quantes i -èsimes rotacions d' s són iguals a t . El vostre algorisme ha de tenir cost $\Theta(n)$.

Per exemple, per a $s = \text{"abbabb"}$ i $t = \text{"babbab"}$ la resposta és 2, corresponent a $i = 2$ i $i = 5$. Mostreu com funcionaria el vostre algorisme amb aquest exemple.

Pista: Considereu la concatenació d' s amb ella mateixa.

6.6 Teoria de jocs

Considereu una variant del joc del Nim, on a cada torn només es poden treure una o dues pedres d'una pila.

Digueu quin és el númer d'una pila en funció del seu nombre de pedres p .

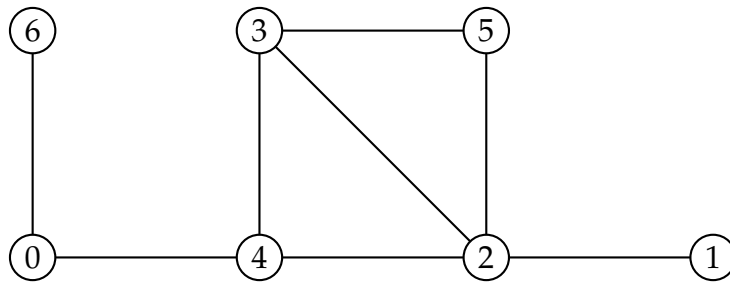
En els dos apartats següents, no us demanem demostracions rigoroses, només explicacions prou convincents.

Suposeu que teniu n piles. El nombre de pedres p_i de cada pila i s'escull uniformement i independentment a l'atzar entre 0 i 2000. Quan n tendeix a infinit, la probabilitat que la posició del joc sigui perdedora tendeix a algun límit? En cas afirmatiu, a quin?

Suposeu que ara la situació és similar, però si amb la p_i escollida el número de la pila i seria 0, es descarta aquesta p_i i es torna a triar, tantes vegades com calgui. Quan n tendeix a infinit, la probabilitat que la posició del joc sigui perdedora tendeix a algun límit? En cas afirmatiu, a quin?

6.7 Punts d'articulació i arestes pont

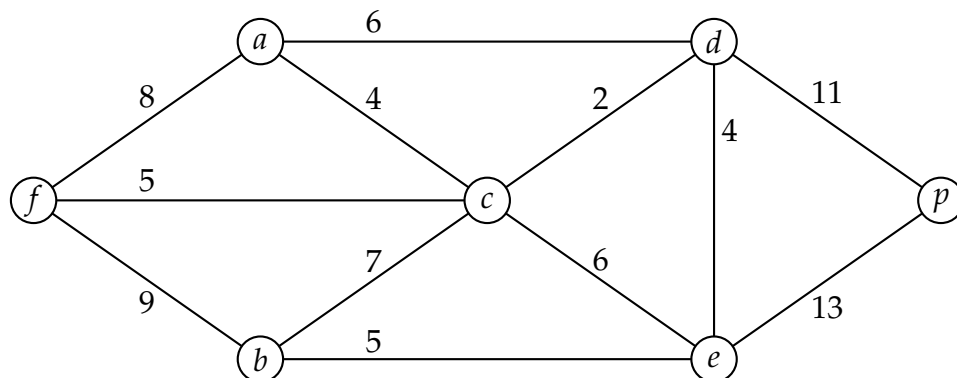
Expliqueu què són els punts d'articulació i les arestes pont d'un graf, i mostreu-los en aquest graf d'exemple:



Expliqueu com funciona l'algorisme linial vist a classe per trobar els punts d'articulació i les arestes pont. Useu el graf anterior d'exemple, mostrant el contingut final dels comptadors de cada vèrtex. Suposeu que l'algorisme comença al vèrtex 0, i que els veïns de cada vèrtex estan emmagatzemats de petit a gran.

6.8 Flux màxim

Calculeu un flux màxim d'aquest graf. Marqueu el flux de cada aresta en el mateix graf. Si hi ha més d'una solució, mostreu-ne una qualsevol. Expliqueu breument com heu calculat aquest flux màxim.



En un graf de fluxos qualsevol, suposeu que es pot augmentar tant com es vulgui la capacitat d'exactament una aresta. Caracteritzeu per a quines arestes és potencialment interessant augmentar-ne la capacitat, si es vol aconseguir incrementar el flux màxim del graf. En el graf anterior, quin és aquest conjunt d'arestes? Per a cadascuna, quin és l'increment del flux màxim que es pot aconseguir?

Parcial, 28 d'octubre de 2022

7.1 Sumar un

Considereu un nombre x amb n bits escollits uniformement a l'atzar, independentment. Així doncs, x pot tenir zeros a l'esquerra. Calculeu el cost en el cas mitjà de l'algorisme escolar per sumar 1 a x .

L'algorisme rep x en un vector amb $n + 1$ bits representats amb enters que són 0 o 1, excepte per un 0 extra a l'extrem esquerre per si la suma incrementés la mida del nombre. (Per exemple, de $x = 7 = 111_2$ a $x' = 8 = 1000_2$ quan $n = 3$.) El resultat final es deixa en el mateix vector, modificant només els bits que canvien.

7.2 Cost d'alguns bucles

Sigui n un enter positiu. Considereu aquest codi (que probablement no fa res útil):

```
for (int i = 0; i < n; ++i)
    for (int j = 0; j < i*i ; ++j)
        if (j%i == 0) f(n);
```

Digueu quin cost té el codi si $f(n)$ té cost $\Theta(1)$.

I si $f(n)$ té cost $\Theta(n)$?

7.3 Codis de Huffman

Digueu quines d'aquestes afirmacions sobre l'algorisme de Huffman són certes. Per a les que siguin certes, digueu per què. Per a les que siguin falses, doneu un contraexemple. En tots els casos, les freqüències sumen 1.

- a) "Si tots els símbols tenen freqüències més petites que 0.5, llavors tots els símbols es codificaran amb almenys dos bits."
- b) "Un símbol amb freqüència més gran o igual que 0.5 mai es codificarà amb dos o més bits."
- c) "Si tots els símbols tenen freqüències més petites que 0.33, llavors tots els símbols es codificaran amb dos o més bits."

7.4 Tallant taulons

Disposeu d'un tauló de longitud ℓ , que heu de tallar en n trossos més petits. Sabeu la longitud final ℓ_i de cada tros, amb $\sum_{1 \leq i \leq n} \ell_i = \ell$. No importa de quina part del tauló surt cada tros. Heu de calcular el cost mínim M de fer tots els talls, sabent que tallar un tros de mida x costa x .

Per exemple, suposeu $\ell = 30$, i que es volen obtenir cinc trossos de longituds 3, 9, 1, 7 i 10. En aquest cas podem tallar el tauló en el punt 11, cosa que resulta en un tros de mida 11 i un de mida 19. Ara tallem el tros de mida 11 pel punt 4, obtenint 4 i 7. Ara tallem el tros de mida 4 pel punt 3, obtenint 3 i 1. Finalment, tallem el tros de mida 19 pel punt 10, obtenint 10 i 9. El cost total es $30 + 11 + 4 + 19 = 64$. Es pot demostrar que no és possible tallar el tauló amb cost total més petit, així que $M = 64$.

Expliqueu com resoldríeu aquest problema amb el mínim cost temporal possible (en tot cas, polinòmic). Quin és aquest cost? Si feu servir algun algorisme explicat a classe, demostreu que la conversió d'un problema a l'altre és correcta.

7.5 Bandera belga

Ens donen un vector amb n elements. Cada element és un *struct* amb diversos camps, un dels quals és un caràcter que pot ser 'B', 'G' o 'R' (representant els colors *black*, *gold* i *red*). El problema de la bandera belga consisteix a reordenar el vector de manera que a les primeres posicions quedin tots els elements de color 'B', a continuació tots els elements de color 'G', i finalment tots els elements de color 'R'. L'ordre relatiu final entre els elements del mateix color és irrellevant.

a) Proposeu un algorisme de cost $\Theta(n)$ que resolgui aquest problema fent servir memòria addicional $O(1)$. Si us cal, podeu usar com a subrutina algun algorisme explicat durant el curs.

b) Suposeu que ara tenim un vector d' n caràcters indistingibles escollits entre 'B', 'G' i 'R'. Proposeu un altre algorisme lineal, més fàcil de programar que l'anterior, que obtingui el mateix resultat fent servir memòria addicional $O(1)$.

7.6 Com sempre, un codi misteriós

Disposem d' n objectes numerats entre 0 i $n - 1$. Cada objecte té pes $P[i]$ i valor $V[i]$, on P i V són vectors d'enters estrictament positius. Suposeu que s és un enter donat estrictament positiu, i que INF és una constant entera més gran que qualsevol altre enter. Considereu aquest codi:

```
vector<int> R(s + 1, -INF);
R[0] = 0;
for (int j = 0; j < n; ++j)
    for (int i = s; i ≥ P[j]; --i) R[i] = max(R[i], R[i - P[j]] + V[j]);

int res = 0;
for (int x : R) res = max(res, x);
cout << res << endl;
```

Què escriu el codi anterior? Ajudeu la vostra explicació amb aquest petit exemple: $n = 3$, $P = [2, 4, 2]$, $V = [3, 5, 4]$, $s = 7$. Aquest codi implementa implícitament un algorisme de programació dinàmica; considereu la funció recursiva associada. Quins paràmetres

tindria i què calcularia? Quin rang de valors vàlids tindria cada paràmetre? Quina seria la recurrència per al cas general? I els valors per als casos base?

Final, 11 de gener de 2022

8.1 Quants a l'interval?

Considereu un multiconjunt S amb n naturals en el rang $[1 \dots k]$, on k és una constant. Expliqueu com respondríeu eficientment a consultes de *quants a l'interval?*, en què donats dos naturals a i b , amb $a \leq b$, es determini quants $x \in S$ compleixen $a \leq x \leq b$. Per exemple, si $S = \{2, 2, 2, 10, 23, 23, 30, 42, 50\}$, $a = 11$ i $b = 42$, la resposta hauria de ser 4. L'entrada comença amb n i els n nombres en qualsevol ordre, seguits de moltes consultes amb a i b . El vostre algorisme pot processar els n nombres amb cost $O(n + k)$, però després ha de respondre cada consulta en temps constant.

8.2 Ordenació sàvia

Donat un vector V amb n elements diferents, recordeu que una inversió és qualsevol parell de posicions (i, j) tals que $0 \leq i < j < n$ i $V[i] > V[j]$.

Quin és el mínim nombre d'inversions i el màxim nombre d'inversions que pot tenir V en funció d' n ? Caracteritzeu com ha de ser V per aconseguir aquests valors.

Suposeu que sabeu que V té $\Theta(\sqrt{n})$ inversions. En aquest cas, quin seria el cost en el cas pitjor i el cost en el cas mitjà d'ordenar V amb la variant de *quicksort* que particiona amb un pivot escollit a l'atzar? Expliqueu com es podria ordenar V amb cost estrictament millor tant en el cas pitjor com en el cas mitjà. Quins serien aquests nous costs?

8.3 Punts en un cercle

Considereu aquest mètode senzill per generar n punts amb coordenades reals dins del cercle unitari: Repetidament, s'escullen una x i una y de l'interval $[-1, 1]$ uniformement i independentment. (Suposeu que x i y tenen "molts decimals".) Si $0 \leq x^2 + y^2 \leq 1$, guardem el punt (x, y) ; altrament el descartem. Quan tenim n punts, parem.

Proposeu un algorisme tal que, donats n punts obtinguts amb el mètode anterior, els ordeni segons la distància a $(0, 0)$. L'algorisme ha de tenir cost linial en el cas mitjà.

8.4 Arbres de cerca

Considereu aquest codi sobre arbres binaris de cerca:

```
using abc = struct node*;

struct node {
    int clau;
    abc f_esq, f_dret;
};

void misteri(abc a) {
    if (a) {
        misteri(a->f_esq);
        cout << a->clau << endl;
        misteri(a->f_dret);
    }
}
```

Mostreu un exemple d'arbre binari de cerca a amb uns pocs elements, i el resultat de cridar a `misteri(a)`. Digueu què fa en general una crida a `misteri(a)`. Siguin n el nombre de vèrtexs d' a , i h l'alçada d' a . Quin cost té `misteri(a)`?

8.5 Teoria de jocs

Considereu una variant del joc del Nim: Hi ha diverses piles de pedres, i també diverses caixes amb pedres. Dos jugadors, per torns, han d'escollir o bé una pila o bé una caixa que no estigui buida. Si és una pila, se'n pot treure qualsevol nombre de pedres (almenys una). Si és una caixa, se n'ha de treure exactament una pedra. Perd qui no pot jugar.

Suposeu una situació amb dues piles amb 7 i 10 pedres, i tres caixes amb 10, 23 i 42 pedres. Digueu si la situació és guanyadora o perdedora. Si és guanyadora, mostreu totes les jugades que guanyen. No es valorarà cap resposta que no usi nùmbers.

8.6 Isomorfisme d'un subgraf

El problema CLIQUE consisteix a, donat un graf no dirigit $G = (V, E)$ i una k , trobar una clique de mida k dins de G , o dir que no en té cap. És a dir, cal trobar un subconjunt $S \subseteq V$ amb k vèrtexs tal que, $\forall x, y \in S, \{x, y\} \in E$. Se sap que CLIQUE és NP-complet.

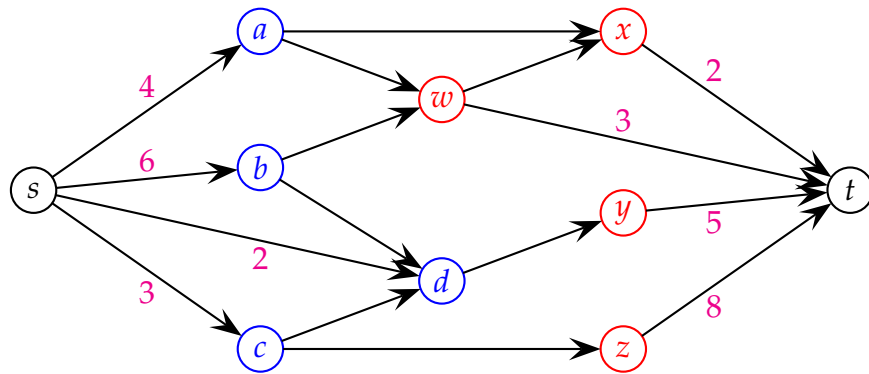
El problema SUBGRAF-ISOMORFISME consisteix a, donats dos grafs no dirigits $G_1 = (V_1, E_1)$ i $G_2 = (V_2, E_2)$, decidir si G_1 és isomorf a algun subgraf de G_2 . Sigui n_1 el nombre de vèrtexs de V_1 . Formalment, cal trobar algun subconjunt $T \subseteq V_2$ de mida n_1 i alguna bijecció $f : V_1 \rightarrow T$ tal que $\{x, y\} \in E_1$ sii $\{f(x), f(y)\} \in E_2$, o dir que no existeixen.

Demostreu que SUBGRAF-ISOMORFISME és NP-complet.

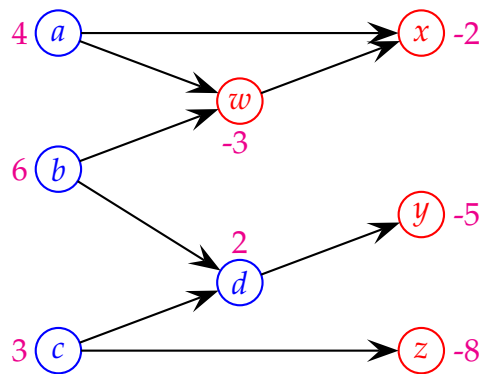
Recordeu el problema GRAF-ISOMORFISME: Donats dos grafs no dirigits G_3 i G_4 , cal decidir si G_3 i G_4 són isomorfs, i en cas afirmatiu proporcionar una bijecció g entre els vèrtexs dels grafs. Expliqueu si es pot reduir polinòmicament GRAF-ISOMORFISME a SUBGRAF-ISOMORFISME. I a l'inrevès? Si alguna de les dues respostes és incerta, expliqueu per què.

8.7 Flux màxim

Calculeu el flux màxim d'aquest graf (els arcs sense etiqueta tenen capacitat ∞). Marqueu el flux en el mateix graf, i expliqueu breument els passos de l'algorisme.



El graf $G = (V, E)$ següent descriu un conjunt de tasques V , les dependències entre tasques, i el premi $p(x)$ de cada tasca $x \in V$, indicat al costat. Els vèrtexs blaus tenen premi positiu, i els vermells, negatiu (és a dir, tenen cost). Per exemple, l'arc $a \rightarrow w$ indica que si no es fa w (amb cost 3) no es pot fer a (amb premi 4). Volem escollir un subconjunt de tasques per maximitzar el benefici tot respectant les dependències. És a dir, volem triar un $S \subseteq V$ (potser buit), de manera que si $x \in S$ i $(x, y) \in E$, llavors $y \in S$, i aconseguir que $\sum_{x \in S} p(x)$ sigui màxima.



Donat un graf G com el de l'exemple, sempre es pot construir un altre graf G' segons el patró del graf anterior (s connectada amb tots els vèrtexs blaus, tots els vèrtexs vermells connectats amb t , etc), i calcular-ne el flux màxim F . Expliqueu per què el màxim benefici possible de G coincideix amb

$$\left(\sum_{x \in V, p(x) > 0} p(x) \right) - F .$$

No us en demanem una demostració rigorosa, només una explicació prou convincent. També, expliqueu com trobar S a partir del càlcul del flux màxim.

8.8 Influencer

Diem que un element x en un vector V de mida n és un *influencer* si x apareix més de $\lfloor \frac{n}{2} \rfloor$ vegades a V . Per exemple, 7 és un influencer de $[7, 9, 7, 7, 2]$, però no de $[8, 7, 7, 3]$.

(a) Proposeu un algorisme, el més eficient possible en el cas pitjor, per determinar si V conté un influencer, i si és així quin és. Quin cost té l'algorisme?

(b) Si V està ordenat, es pot determinar més eficientment si conté un influencer i quin és. Com? Amb quin cost?

(c) Supposeu que ens garanteixen que un vector d'enters V té un influencer. Digueu si aquest codi el trobaria o no i per què:

```
int num = 0;
int y = 42;
for (int z : V)
    if (num == 0) { y = z; num = 1; }
    else if (z == y) ++num;
    else --num;
cout << y << endl;
```

(No podeu usar de cap manera aquesta (pretesa) solució a l'apartat (a).)

Parcial, 29 d'octubre de 2021

9.1 Ordenació boja

Sigui $V[i..j]$ un (sub)vector amb $n = j - i + 1$ elements. Considereu aquest algorisme d'ordenació:

- Si $n \leq 2$, ordenem el vector trivialment.
- Altrament, si $n \geq 3$, suposem per senzillesa que n és múltiple de 3. (Si no fos així, podríem adaptar l'algorisme.) Sigui $t = n/3$, $e = i + t$, i $d = j - t$. Primer ordenem recursivament $V[i..d]$, després $V[e..j]$, i finalment de nou $V[i..d]$.

- a) Argumenteu per què aquest algorisme funciona. Mostreu un petit exemple.
- b) Doneu una recurrència per al temps que triga aquest algorisme, i resoleu-la.
- c) Suposeu que podeu modificar l'algorisme, i canviar una de les tres crides recursives per una crida a una funció que ordeni amb l'algorisme de selecció. Quin cost tindria aquest nou algorisme modificat?
- d) Finalment, suposeu que podeu modificar l'algorisme original, i canviar una de les crides recursives per una crida a una funció que particioni. (Particionar un vector de mida N consisteix a deixar els E elements més petits a la banda esquerra, i els $N - E$ més grans a la banda dreta. L'ordre relatiu amb què queden els elements a les dues bandes no és important.) Expliqueu quina de les tres crides substituiríeu i com, i per què aquest nou algorisme seguiria ordenant correctament. Si volem minimitzar el cost en el cas pitjor, quin mètode escolliríeu per particionar, i quin cost en el cas pitjor aconseguiríeu?

9.2 Multiplicant matrius

- a) Expliqueu breument l'algorisme de Strassen per multiplicar matrius de mida $n \times n$. No cal que doneu les crides recursives concretes. Quin cost té i com es calcula?
- b) Actualment es coneixen algorismes per multiplicar matrius amb cost aproximat $\Theta(n^{2.37})$. Amb el mateix esquema de l'algorisme de Strassen, digueu raonadament quantes crides recursives s'haurien de fer per aconseguir un cost asimptòtic que sigui millor.

9.3 Codi misteriós

Suposeu que V és un vector amb n enters diferents, amb $n \geq 2$ parell, i que $m = n/2$. Considereu aquest codi:

```
priority_queue <int> Q;
for (int i = 0; i < m; ++i) Q.push(V[i]);
for (int i = m; i < n; ++i) {
    Q.push(V[i]);
    cout << ' ' << Q.top();
    Q.pop();
}
cout << endl;
```

- Què escriu el codi si $n = 10$ i $V = [4, 8, 10, 9, 12, 2, 23, 1, 42, 7]$?
- Doneu una explicació d'alt nivell (és a dir, en general, sense comentar cada línia en particular) sobre quins valors s'escriuen amb una V qualsevol.
- Quin cost en el cas pitjor té el codi, en funció d' n ?
- Expliqueu com es podrien escriure els mateixos nombres (potser en un ordre diferent), però més eficientment. Quin seria el cost del nou algorisme?

9.4 Xips defectuosos

Teniu n xips per reciclar, alguns dels quals poden ser defectuosos. Per distingir quins són bons, teniu un dispositiu que permet connectar-ne dos entre si momentàniament. Durant la connexió, cada xip emet un diagnòstic sobre l'altre xip (diu si és bo o dolent). A cada test, un xip bo sempre diu la veritat, mentre que un xip dolent pot mentir. Així doncs, si els dos xips diuen que l'altre xip és bo, podem deduir que o bé els dos xips són bons, o bé els dos xips són dolents. Qualsevol altra combinació de respostes implica per exemple que almenys un dels dos xips és dolent. En el que segueix, sigui b el nombre de xips bons, i sigui $d = n - b$ el nombre de xips dolents.

- Suposeu $n \geq 3$, i que sabem que $d = 1$. Expliqueu com podríeu identificar el xip dolent minimitzant el nombre de tests en el cas pitjor. Quin seria aquest nombre?
- Suposeu que cada xip té un identificador intern únic i que, a cada test, els xips detecten amb quin altre xip s'han connectat. Suposeu $n = 30$, i que sabem que $b = 10$. Expliqueu per què, si els xips dolents "es confabulen" en les respostes que donaran, és impossible destriar els xips bons dels dolents per molts tests que es facin.
- Suposeu que no coneixem b , però sí que sabem que $b > n/2$. Expliqueu com podríeu identificar un xip bo amb $\Theta(n)$ tests en el cas pitjor.

Pistes: Considereu un algorisme dividir-i-vèncer que divideixi n aproximadament per la meitat. Aneu en compte amb el vostre mètode quan n sigui senar.

9.5 I encara més codis misteriosos

Sigui n un enter positiu. Considereu aquest tros de codi:

```
vector <int> F(n);
for (int p = 2; p < n; ++p)
    if (F[p] == 0)
        for (int i = p; i < n; i += p) F[i] = p;
```

a) Expliqueu què calcula. Demostreu que el cost és $O(n \log n)$.

A continuació tenim aquest altre tros de codi:

```
int q = 0;
for (int i = 2; i < n; ++i) {
    int x = i;
    while (x > 1) {
        ++q;
        int f = F[x];
        while (x%f == 0) x /= f;
    }
}
cout << double(q)/n << endl;
```

b) Expliqueu què compta q , i en conseqüència què és el que s'escriu. Demostreu que la quantitat escrita és $O(\log n)$.

10

Extraordinari, 1 de juliol de 2021

10.1 Quicksort, de nou

Volem ordenar un vector global V amb n elements diferents. Considereu el quicksort amb aquesta estructura més o menys habitual:

```
void quicksort (int e, int d) {  
    if (e + 1 ≤ d) {  
        if (e + 1 == d and  $V[e] > V[d]$ ) swap( $V[e]$ ,  $V[d]$ );  
        return;  
    }  
    int p = particio (e, d);  
    quicksort (e, p - 1);  
    quicksort (p + 1, d);  
}
```

Sigui $m = d - e + 1$ el nombre d'elements del subvector a particionar. Supposeu que el procediment *particio* (e , d) és la partició de Lomuto, amb algunes diferències: Primer, la mida m ha de ser almenys 3 (per això ordenem si cal amb un *swap*() quan $e + 1 == d$, és a dir, quan $m = 2$). Segon, i més important, en comptes d'escollir el pivot uniformement a l'atzar, s'escullen 3 elements uniformement a l'atzar, i es fa servir com a pivot la mediana d'aquests 3 elements. Per exemple, si els elements escollits fossin 89, 23 i 42, es faria servir el 42 com a pivot.

Per a cada $i \in [0, m - 1]$, sigui p_i la probabilitat que el pivot escollit acabi a la posició i . Tenint en compte que hi ha $\binom{m}{3}$ maneres equiprobables d'escollir els tres elements, p_i ara no és $1/m$. Per exemple, $p_0 = p_{m-1} = 0$. I si $c = \lfloor m/2 \rfloor$, llavors p_c és la màxima de les p_i . (Si m és parell, p_{c-1} també és màxima per simetria.)

Calculeu p_i raonadament.

Calculeu el nombre de comparacions en el cas mitjà d'aquesta variant de quicksort. No us conformeu amb l'ordre de creixement, calculeu la constant que multiplica el terme dominant. Aquesta constant és millor que la del quicksort bàsic?

Pista:

$$\int x^2 \ln x \, dx = \frac{x^3 \ln x}{3} - \frac{x^3}{9} \quad , \quad \int x^3 \ln x \, dx = \frac{x^4 \ln x}{4} - \frac{x^4}{16}$$

Calculeu el cost en el cas pitjor d'aquesta variant de quicksort. N'hi ha prou amb l'ordre de creixement asimptòtic.

10.2 Dijkstra modificat

Considereu aquesta variant de l'algorisme de Dijkstra vist a classe:

```

int dijkstra (int u, int v) {
    priority_queue <P> Q;
    VE dist(n, INF);
    dist[u] = 0;
    Q.push(P(0, u));
    while (not Q.empty()) {
        P a = Q.top(); Q.pop();
        int d = -a.first;
        int x = a.second;
        if (d == dist[x]) {
            if (x == v) return d;
            for (P arc : G[x]) {
                int y = arc.second;
                int d2 = max(d, arc.first); // (*)
                if (d2 < dist[y]) {
                    dist[y] = d2;
                    Q.push(P(-d2, y));
                }
            }
        }
    }
    return INF;
}

```

La diferència fonamental es troba a la línia marcada amb un asterisc.

Expliqueu breument què calcula aquesta variant. Dibuixeu un petit graf per ajudar-vos en l'explicació. Quin cost té aquest codi en el cas pitjor?

10.3 Arbres aleatoris dolents

Sigui $P(n)$ la probabilitat que un arbre binari de cerca aleatori amb $n \geq 1$ claus tingui alçada n . Per exemple, $P(2) = 1$. Trobeu raonadament una fórmula tancada per a $P(n)$.

10.4 Teoria de jocs

Considereu aquesta frase: "Per a tota variant del joc del nim que es pugui analitzar amb nùmers, i per a tota posició guanyadora, totes les jugades guanyadores (les que deixen a l'altre jugador en una posició perdedora) decrementen el nùmer de la pila on es fa la jugada."

Si la frase és certa, raoneu per què. Si és falsa, proposeu un joc, calculeu-ne els nímbers, i mostreu una posició i una jugada que contradigui la frase.

10.5 Un altre codi misteriós

Sigui s un string de mida n , compost només amb lletres minúscules (recordeu que n'hi ha 26), i sigui m un enter tal que $1 \leq m \leq n$. Considereu aquest codi:

```

long long h = 0;
long long pw = 1;
for (int i = m - 1; i ≥ 0; --i) {
    h += pw*(s[i] - 'a');
    if (i > 0) pw *= 26;
}

set<long long> S;
S.insert(h);
for (int i = m; i < n; ++i) {
    h -= pw*(s[i-m] - 'a');
    h *= 26;
    h += s[i] - 'a';
    S.insert(h);
}
cout << S.size() << endl;

```

Expliqueu què calcula el codi anterior.

Calculeu el cost en el cas pitjor del codi anterior. Quin és el cas pitjor?

Si m és prou gran, aquest codi fallaria. Per què? Feu alguna proposta de modificació del codi de manera que tingui el mateix cost, no tingui errors d'execució, i faci la mateixa funció amb alta probabilitat. (No cal que quantifiquem aquesta probabilitat; n'hi ha prou amb una bona intuïció.)

10.6 Pintant un graf

En tot aquest problema, suposeu que G és un graf no dirigit i connex amb n vèrtexs i m arestes. Recordeu que una k -coloració de G és una assignació, a cada vèrtex, d'un color escollit entre $\{1, \dots, k\}$, de manera que cap aresta tingui els dos vèrtexs del mateix color.

Expliqueu l'algorisme vist a classe que, donat un G que és 2-colorable, en troba una 2-coloració en temps linial en la mida de G .

Suposeu que tot vèrtex de G té com a molt d veïns. Proposeu i analitzeu un algorisme polinòmic (idealment, linial) que trobi una $(d + 1)$ -coloració de G .

Sigui $a = \lceil \sqrt{n} \rceil$. Useu els dos apartats anteriors per proposar un algorisme polinòmic que, donat un G que és 3-colorable, en trobi una coloració amb com a molt $c \cdot a$ colors, per a alguna constant c .

- Pista 1: Considereu dos casos: Quan tots els vèrtexs tenen grau més petit que a , i quan almenys un vèrtex té grau a o més gran.
- Pista 2: Quants colors calen, com a molt, per pintar el veïnatge de qualsevol vèrtex?

Sigui k -COL el problema consistent a k -colorar un graf. Se sap que 3-COL és NP-complet. Marqueu cada afirmació amb un SI, amb un NO, o amb un NO SE SAP:

- 4-COL es pot reduir polinòmicament a 3-COL:
- 2-COL es pot reduir polinòmicament a 3-COL:
- 3-COL es pot reduir polinòmicament a 2-COL:

Expliqueu breument cadascuna de les tres respostes.

Final, 18 de gener de 2021

11.1 Parells de primers bessons

Els parells de primers bessons són aquells parells de nombres naturals $(x, x + 2)$ tals que tant x com $x + 2$ són nombres primers. Els primers parells són $(3, 5)$, $(5, 7)$, $(11, 13)$, $(17, 19)$, $(29, 31)$, ...

Conjectura: Hi ha infinits parells de primers bessons.

Aquesta famosa conjectura encara no ha estat demostrada. Supposeu que el problema de l'aturada (*halting problem*) fos computable. Expliqueu com construiríeu una funció que acabi en un nombre finit de passos i que al final retorni **true** o **false** en funció de si la conjectura és certa o falsa.

11.2 Dividint un vector

Donat un vector V amb n nombres enters diferents, i un natural k que és un divisor d' n , heu de repartir els elements de V en k vectors $T_1, \dots, T_k$ d'aquesta manera:

- Cada T_i ha de contenir n/k elements de V .
- T_1 ha de contenir els n/k elements més petits de V , T_2 ha de contenir els següents n/k elements de V , ..., i T_k ha de contenir els n/k elements més grans de V .

Els elements poden aparèixer en qualsevol ordre dintre de cada T_i . Per exemple, si $V = [7, 5, 9, 1, 2, 4]$ i $k = 3$, el resultat podria ser $T_1 = [1, 2]$, $T_2 = [5, 4]$, $T_3 = [9, 7]$.

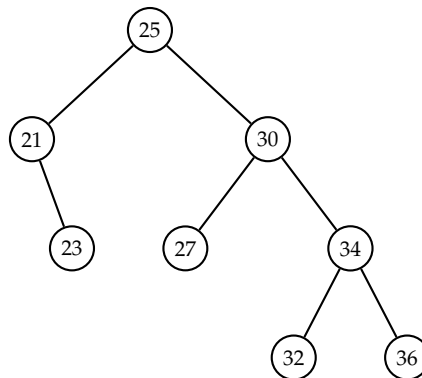
Proposeu un algorisme eficient per resoldre aquest problema. Quin cost té en el cas pitjor en funció d' n i k ? Per a $k = \Theta(\log n)$ el vostre algorisme ha de tenir cost estrictament més petit que $\Theta(n \log n)$.

11.3 Arbres de cerca aleatoris

Analitzeu el cost mitjà, mesurat com el nombre esperat de nodes visitats, de buscar la clau més petita en un arbre de cerca aleatori amb n claus. No us conformeu amb l'ordre de creixement, calculeu la constant que multiplica al terme dominant.

11.4 Arbres de cerca aleatoritzats

Considereu l'arbre de cerca següent:



Mostreu els possibles arbres, amb les probabilitats corresponents, resultat d'inserir un 28 amb l'algorisme dels arbres de cerca aleatoritzats.

11.5 Kruskal

Sigui G un graf connex amb pesos estrictament positius a les arestes. Suposant que tots els pesos són diferents, demostreu que l'algorisme de Kruskal realment obté un arbre generador de cost mínim. Pot ser que G tingui més d'un arbre generador de cost mínim?

11.6 Teoria de jocs

Considereu una variant del joc del nim, en la qual només es poden treure 2, 3 o 4 pedres a cada jugada, com sempre d'una sola pila. En particular, això implica que una pila que només té una pedra és una posició perdedora.

- Donada una pila amb n pedres, trobeu una fórmula per al seu númer en funció d' n .
- Suposeu una situació del joc amb tres piles: una amb 10 pedres, una altra amb 20, i una tercera amb un nombre indeterminat de pedres x . En funció d' x , digueu si la posició és guanyadora o perdedora. També en funció d' x , digueu quines jugades guanyadores es poden fer a la tercera pila.

11.7 Taules de dispersió

Suposeu un `unordered_map<string, int>` implementat amb una taula de hash $T[1\dots m]$. Per a una certa funció de hash h , cada $T[i]$ conté un vector amb els parells $\langle c, v \rangle$ amb les claus c tals que $h(c) = i$, en cap ordre particular. T conté n parells $\langle \text{clau}, \text{valor} \rangle$ en total.

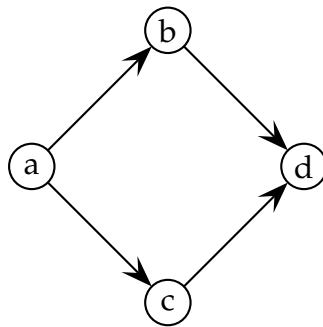
Quan s'analitza el cost de les operacions en una taula de hash, típicament s'ignora el cost de calcular $h(c)$, cosa que és una simplificació. Aquí, suposeu que totes les claus c són strings de longitud s , i que calcular $h(c)$ té cost $\Theta(s)$.

Considereu l'algorisme que troba la v corresponent a una c que pertany a T . Quin cost té, en funció d' s i del factor de càrrega $\alpha = n/m$? Calculeu-lo en el cas mitjà, assumint que el valor $h(c)$ està uniformement distribuït a $\{1, \dots, m\}$. Sobre com estan formades les claus c internament, suposeu que cada dues coincideixen en quasi tots els caràcters, però no sabeu en quines posicions.

Suposant que els $\langle c, v \rangle$ que conté la taula es coneixen en temps de compilació i que no es modifiquen durant l'execució, proposeu una manera d'implementar la taula de hash per aconseguir que el cost de la consulta de l'apartat anterior sigui, sota les mateixes suposicions, una funció lineal d' s i d' α .

11.8 Cicles hamiltonians

Sigui DCH el problema consistent a decidir si un graf *dirigit* conté un cicle hamiltonià, i sigui NCH el problema consistent a decidir si un graf *no dirigit* conté un cicle hamiltonià. Reduïu polinòmicament DCH a NCH. Dibuixeu el resultat de la reducció aplicat al graf



12

Parcial, 6 de novembre de 2020

12.1 Potències de 2

En aquest problema, suposem que les posicions dels vectors comencen en 1. Sigui V un vector amb n nombres enters diferents, i sigui S el vector que s'obtingria si s'ordenés V . Descriviu amb prou detall un algorisme eficient que retorni un vector R que contingui, en ordre, $S[1], S[2], S[4], \dots, S[2^i], \dots, S[2^{\lfloor \log_2 n \rfloor}]$.

Per exemple, amb $V = [7, 3, 17, 14, 5, 10, 20, 16, 23, 12]$ el vostre algorisme hauria de retornar $R = [3, 5, 10, 17]$. La vostra solució ha de tenir cost inferior a $\Theta(n \log n)$ en el cas pitjor. Quin és el seu cost?

12.2 Rang d'un element

a) Expliqueu quina informació cal emmagatzemar als nodes d'un arbre binari de cerca T amb n claus diferents per poder implementar aquesta operació: Donada una clau x , cal retornar en quina posició (començant en 0) es troba x entre les n claus de T (cal retornar un -1 si x no està dins de T). Per exemple, si x és la clau més petita, cal retornar 0, i si és la més gran, cal retornar $n - 1$. Feu un dibuix d'un arbre amb 6 claus per ajudar a la vostra explicació.

b) Descriviu amb paraules prou detallades o amb codi C++ com implementaríeu l'operació demanada. El vostre algorisme ha de tenir cost $O(h)$, on h és l'alçada de T .

12.3 És un min_heap?

Expliqueu breument i doneu el codi C++ d'un procediment

```
bool es_min_heap(const vector<int>& V);
```

per determinar si un `vector<int> V` de mida $n \geq 1$ conté un min-heap o no. Suposeu, com és habitual amb els heaps, que la primera posició de V no es fa servir. Quin és el cost en el cas millor i en el cas pitjor del vostre procediment?

12.4 Quicksort millorat?

Volem ordenar un vector amb n elements diferents. Considereu el quicksort amb aquesta estructura habitual:

```
void quicksort (int e, int d) {
    if (e ≥ d) return;
    int p = particio (e, d);
    quicksort (e, p - 1);
    quicksort (p + 1, d);
}
```

Sigui $m = d - e + 1$ el nombre d'elements del subvector a particionar, i sigui $q = \lfloor m/4 \rfloor$. Supposeu que el procediment *particio* (e, d) no és la partició de Lomuto, sinó una nova partició amb un efecte similar, però amb dues diferències:

- La posició p després de la crida és qualsevol de l'interval $[e + q, d - q]$ escollida de forma equiprobable. Les posicions $[e, e + q - 1]$ i $[d - q + 1, d]$ tenen probabilitat 0.
- Aquesta partició fa $3m$ comparacions.

a) Calculeu el nombre de comparacions en el cas pitjor d'aquesta variant de quicksort. No us conformeu amb l'ordre de creixement, calculeu la constant que multiplica al terme dominant. Respecte dels casos pitjors i considerant només les comparacions, aquesta variant és més eficient que el quicksort bàsic?

b) Torneu a respondre totes les preguntes anteriors, però ara respecte al nombre esperat de comparacions (és a dir, en el cas mitjà).

12.5 Maps i sets

Sigui M un *map*<string, int>, i sigui S un *set*<string>. Aquest codi pretén escriure els enters corresponents a les paraules de M que no estan a S :

```
auto it1 = M.begin();
auto it2 = S.begin();
while (it1 ≠ M.end()) {
    if (it1 → first > *it2) ++it2;
    else {
        if (it1 → first < *it2) cout << it1 → second << endl;
        ++it1;
    }
}
```

a) Expliqueu breument com pretén funcionar el codi donat. Proporcioneu un petit exemple si us ajuda en l'explicació.

b) El codi donat conté un o més errors. Arregleu-lo fent-hi els mínims canvis necessaris. Supposeu $n = S.size() = M.size()$ per a una certa n . Un cop corregit el codi, quin cost té en el cas millor i en el cas pitjor?

12.6 Ordre d'una permutació

(a) Sigui S_n el conjunt de les permutacions de $[n] := \{1, \dots, n\}$. Donades dues permutacions $\sigma \in S_n$ i $\tau \in S_n$, el seu producte $\sigma\tau$ es defineix com la permutació de S_n tal que $(\sigma\tau)(i) = \sigma(\tau(i))$ per a tot $i \in [n]$.

A la resta del problema, suposeu que σ és qualsevol permutació de S_n donada. Assumint que les posicions dels vectors comencen en 1, representarem σ amb un vector d'enters $p[1..n]$ tal que $p[i] = \sigma(i)$ per a $i = 1, \dots, n$.

L'ordre de σ es defineix com el mínim enter $m \geq 1$ tal que σ^m és la permutació identitat (és a dir, $\sigma^m(i) = i$ per a tota i). Descriviu un algorisme que, donada σ i un enter $m \geq 1$, calculi l' m -èsima potència σ^m . L'algorisme ha de tenir cost $O(n \log m)$.

(b) Qualsevol σ es pot descompondre de forma única en un producte de k permutacions cicle: Existeix una partició $A_1, \dots, A_k$ de $[n]$ tal que $\sigma = \sigma_1 \cdots \sigma_k$, on cada σ_i és una permutació que a $[n] \setminus A_i$ és com la identitat, i $A_i = \{a_i, \sigma_i(a_i), \sigma_i^2(a_i), \dots, \sigma_i^{\ell_i-1}(a_i)\}$, on a_i és un element qualsevol d' A_i i ℓ_i és l'ordre de σ_i .

Descriviu un algorisme de cost $\Theta(n)$ que, donada σ , determini k i retorni un vector $\ell[1..k]$ amb els ordres de $\sigma_1, \dots, \sigma_k$.

(c) Per a aquest apartat feu servir el Teorema dels Nombres Primers: Si $\pi(x)$ és la quantitat de nombre primers dins de l'interval $[1, x]$, llavors $\pi(x) = x / \ln(x) + o(x / \ln(x))$.

Es pot veure que si σ es descompon en k permutacions cicle d'ordres $\ell_1, \dots, \ell_k$, llavors l'ordre de σ és $\text{mcm}(\ell_1, \dots, \ell_k)$. Demostreu que l'ordre de σ pot ser $2^{\Omega(\sqrt{n \log n})}$.

Pistes: La quantitat de nombres primers a l'interval $(x/2, x]$ és $\pi(x) - \pi(x/2)$. Tots aquests primers són més grans que $x/2$.

(d) Se sap que l'ordre màxim de σ és asimptòticament $e^{\sqrt{n \ln n}}$, i per tant $2^{\Theta(\sqrt{n \log n})}$. Feu servir aquest fet, així com la resposta a (b), per respondre a aquest últim apartat.

Descriviu un algorisme, tan eficient com pugueu, per computar un vector amb els dígit de l'ordre de σ escrit en binari. Calculeu el cost del vostre algorisme assumint que només les operacions aritmètiques amb nombres més petits que n tenen cost constant. Hi ha un algorisme de cost $\Theta(n)$, i un altre, lleugerament pitjor, de cost $O(n^{1.0001})$.

Pistes: Descomponeu els ordres $\ell_1, \dots, \ell_k$ de les permutacions cicle en factors primers, i recordeu que, òbviament, $\ell_i \leq n$ per a $i = 1, \dots, k$.

Extraordinari, 7 de juliol de 2020

13.1 Heaps

Expliqueu com seria una implementació d'un max-heap on els nodes interns tinguessin (per defecte) tres fills en lloc de dos. En particular:

- (a) Mostreu un exemple d'un max-heap amb 9 elements (l'arbre ternari) i com s'emmagatzemaria internament en un vector.
- (b) Expliqueu amb paraules però amb prou detall com s'haurien d'adaptar totes les operacions fonamentals dels heaps, *heapify* inclosa. Quin cost asimptòtic tindrien?
- (c) Supposeu que mesurem el cost de les operacions com el nombre de nodes de l'arbre visitats. Compareu el cost de fer un *push(x)* i de fer un *pop()* amb aquesta variant respecte als costos dels heaps clàssics. No us conformeu amb els costos asimptòtics: compareu les constants multiplicatives.

13.2 L'agència matrimonial

En certes cultures s'admet que els homes tinguin més d'una dona. Supposeu que teniu una agència matrimonial, i que la informació de què disposeu és:

- h : el nombre d'homes;
- d : el nombre de dones;
- k : el màxim nombre de dones que pot tenir cada home ($k = 1$ es correspon a una societat tradicional);
- Per a cada parell (home, dona), si s'agraden prou entre si perquè aquest matrimoni sigui possible.

Sigui p el nombre de parells factibles. Supposeu que cada home i cada dona pertanyen almenys a un d'aquests parells. Així doncs, tenim $p \geq \max(h, d)$. La vostra tasca és maximitzar el nombre de dones casades.

(a) Expliqueu com reduir aquest problema a un algorisme explicat a l'assignatura. La vostra solució ha de tenir cost polinòmic. Mostreu com funcionaria el vostre algorisme amb aquest exemple: $h = 3$, $d = 6$, $k = 2$, i aquesta matriu que indica els parells potencialment possibles (els homes s'identifiquen amb lletres, les dones amb nombres, les p creus indiquen els parells factibles):

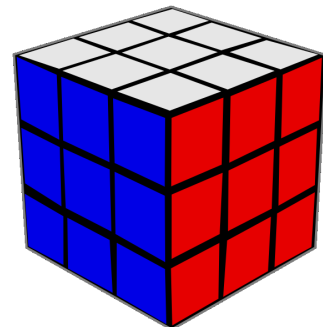
	1	2	3	4	5	6
a	x	x	x			
b		x				
c			x	x	x	x

(b) Sigui x el resultat del vostre algorisme (el nombre de dones que s'han pogut casar). Doneu la fita superior més ajustada possible per al cas pitjor del vostre algorisme. Expliqueu per què aquest cost és polinòmic.

(c) Supposeu que existís un algorisme correcte i de cost constant per resoldre el problema anterior. Expliqueu com el podríeu usar per resoldre aquest problema similar: Donades les mateixes dades *excepte* la k , cal trobar la mínima k que permeti que totes les dones es puguin casar. La vostra proposta ha de tenir el millor cost possible en el cas pitjor. Quin és aquest cost?

13.3 Cub de Rubik

Heu jugat mai al cub de Rubik? Probablement, perquè és el joc més venut del món. És un trencaclosques mecànic amb forma de cub, amb cada cara d'un color diferent, i subdividit en n^3 petites peces cúbiques. En la versió més popular, que podeu veure a la dreta, tenim $n = 3$. Les peces es poden reordenar fent rotacions sobre els eixos. A partir de qualsevol configuració inicial obtinguda fent tantes rotacions com es vulgui, l'objectiu del joc és aconseguir que cada cara del cub torni a ser d'un únic color.



Sigui RUBIK el problema consistent a, donades n , la configuració actual del cub (amb sis matrius $n \times n$), i un natural k , decidir si és possible resoldre el cub amb k o menys rotacions. Està demostrat que RUBIK és un problema NP-complet.

(a) Sigui X un problema NP-difícil. Existeix forçosament una reducció en temps polinòmic de X a RUBIK? I de RUBIK a X ?

(b) Sigui Y un problema a la classe P. Existeix forçosament una reducció en temps polinòmic de Y a RUBIK? I de RUBIK a Y ?

(c) Fixeu-vos que la k de l'entrada pot ser, per exemple, 2^n . Tenint en compte això, doneu un motiu plausible que expliqui com pot ser que RUBIK sigui un problema que pertanyi a la classe NP.

13.4 Algunes demostracions

Sigui p un nombre primer. Per a tot $(a, b) \in \mathbb{Z}_p \times \mathbb{Z}_p$, sigui $f_{a,b} : \mathbb{Z}_p \rightarrow \mathbb{Z}_p$ la funció definida per $f_{a,b}(x) = ax + b \pmod{p}$. Sigui $H_p = \{f_{a,b} : (a, b) \in \mathbb{Z}_p \times \mathbb{Z}_p\}$ el conjunt de totes aquestes funcions. Fixeu $(x_1, x_2, y_1, y_2) \in \mathbb{Z}_p \times \mathbb{Z}_p \times \mathbb{Z}_p \times \mathbb{Z}_p$ i suposeu que $x_1 \neq x_2$.

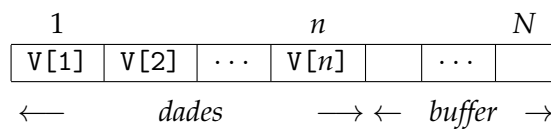
(a) Demostreu que si $f_{a,b}$ s'escull uniformement a l'atzar a H_p , llavors la probabilitat que $f_{a,b}(x_1) = y_1$ és $1/p$. Cal que p sigui primer?

(b) Demostreu que si $f_{a,b}$ s'escull uniformement a l'atzar a H_p , llavors la probabilitat que $f_{a,b}(x_1) = y_1$ i $f_{a,b}(x_2) = y_2$ és $1/p^2$. Cal que p sigui primer?

(c) Sigui F_p el conjunt de totes les funcions $f : \mathbb{Z}_p \rightarrow \mathbb{Z}_p$. Calculeu la probabilitat que una funció f escollida uniformement a l'atzar a F_p provoqui la col·lisió $f(x_1) = f(x_2)$. Feu el mateix amb la probabilitat que una funció $f_{a,b}$ escollida uniformement a l'atzar a H_p provoqui la col·lisió $f_{a,b}(x_1) = f_{a,b}(x_2)$. Podeu fer servir els apartats anteriors.

13.5 Taules dinàmiques

Considereu un vector V amb $V.size() = n$. Per claredat, suposeu que les posicions de V es numeren de la 1 a la n . Recordeu que `push_back(x)` insereix l'element x al final de V , i que `pop_back()` esborra l'element del final de V (assumint $n \geq 1$). La implementació interna consta d'una taula d' N posicions de memòria consecutives, amb $N \geq n$, dividida en dues parts contigües: la part de les *dades* i la part del *buffer*:



La part de les dades té mida n i conté els elements en l'ordre que s'han inserit. La part del buffer té mida $N - n$ i serveix per encabir-hi els nous elements que puguin arribar.

Siguin a i b dos paràmetres enters fixats. Es garanteix que N , a , i b són potències de 2, i que $a \geq b \geq 2$. El disseny és tal que en qualsevol moment l'estat (n, N) de la taula satisfà $N \geq a$. A més, si $N \geq ab$, llavors també es compleix $n \geq N/a$.

Per mantenir aquest invariant les implementacions de `push_back(x)` i `pop_back()` fan el següent:

- `push_back(x)`: Si a l'estat actual tenim $n < N - 1$, llavors s'insereix x a la primera posició lliure del buffer, i l'estat passa a ser $(n + 1, N)$. En canvi, si l'estat actual compleix $n \geq N - 1$, llavors es crea una taula nova en l'estat $(n + 1, bN)$, copiant-hi els n elements de la part de les dades de la taula anterior, a més d' x . El cost d'aquesta operació és $\Theta(1)$ en el primer cas, i $\Theta(N)$ en el segon cas.
- `pop_back()`: Assumim que $n \geq 1$. Si a l'estat actual tenim $N < ab$ o $n > N/a + 1$, llavors simplement s'esborra l'últim element de la part de les dades, i la taula passa a l'estat $(n - 1, N)$. En canvi, si l'estat actual compleix $N \geq ab$ i $n \leq N/a + 1$, llavors es crea una taula nova en l'estat $(n - 1, N/b)$, copiant-hi els $n - 1$ primers elements de la part de les dades de la taula anterior. El cost d'aquesta operació també és $\Theta(1)$ en el primer cas, i $\Theta(N)$ en el segon cas.

- (a) Demostreu que les operacions `push_back(x)` i `pop_back()` efectivament mantenen tots els invariants.
- (b) Supposeu $a = b = 2$, i que l'estat inicial és $(n_0, 2n_0)$, amb $n_0 = 32$. Demostreu que qualsevol seqüència d' m operacions `push_back(x)` (sense cap `pop_back()`) té cost total $\Theta(m)$. És a dir, cost $\Theta(1)$ per operació, en mitjana.
- (c) Seguiu suposant $a = b = 2$, i un estat inicial $(n_0, 2n_0)$, amb $n_0 = 32$. Proposeu una seqüència d' m operacions, amb m arbitràriament gran, que combini tant `push_back(x)` com `pop_back()` i que tingui cost total $\Theta(m^2)$. És a dir, cost $\Theta(m)$ per operació, en mitjana.
- (d) Ara supposeu $a = 4$, $b = 2$ i que l'estat inicial és $(n_0, 2n_0)$, amb $n_0 = 32$. Existeix alguna seqüència d' m operacions, amb m arbitràriament gran, que combini `push_back(x)` i `pop_back()` i que tingui cost total $\Theta(m^2)$? En cas afirmatiu, proposeu-ne una. En cas negatiu, raoneu per què.

Final, 9 de gener de 2020

14.1 Element comú

Diem que un element x d'un vector d'enters V de mida $4n$ és un element comú si x apareix a V almenys n vegades. Per exemple, 5 és un element comú de $[7, 9, 7, 5, 2, 5, 3, 7]$.

Suposeu que V té almenys un element comú. Proposeu un algorisme eficient per trobar un element comú qualsevol de V . El vostre algorisme només pot usar espai addicional $O(1)$. Quin cost esperat té el vostre algorisme? Hi ha una solució amb cost esperat $\Theta(n)$.

14.2 Heaps

A cadascun dels dos apartats següents, mostreu cada pas de l'algorisme explicat a classe.

(a) Inseriu un 33 en el max-heap següent:

1	2	3	4	5	6	7	8	9	10
42	28	23	27	25	15	23	17	22	22

(b) Esborreu l'arrel del max-heap següent:

1	2	3	4	5	6	7	8	9	10
42	28	23	27	25	15	23	17	22	22

14.3 RSA

Cada treballador d'una empresa té associat un parell de claus pública i privada de RSA. La clau pública del treballador i és (E_i, N_i) i la privada és (D_i, N_i) . Cada parell de claus s'ha creat amb una execució de l'algorisme generador de claus que vam veure a classe, amb $n = 32$ bits, i amb una llavor aleatòria independent per a cada treballador. Com que cada N_i té 32 bits, satisfà $2^{31} \leq N_i \leq 2^{32} - 1$.

L'empresa ha decidit publicar, per a cada treballador i , el seu incentiu salarial s_i (un enter entre 1000 i 4999). La taula publicada, però, registra s_i xifrat amb la clau pública (E_i, N_i) segons l'algorisme de xifrat de RSA (tractant s_i com el missatge a xifrar):

Id	Incentiu
001	3563266347
002	3424133328
003	2828571910
...	...

D'aquesta manera, cada treballador i pot conèixer s_i fent servir la clau privada (D_i, N_i) . Tanmateix, en principi, ningú podria saber l'incentiu de cap altre treballador a menys que conegués la seva clau privada.

(a) Només coneixent les claus públiques de tots els treballadors, com modificaríeu la taula per doblar l'incentiu d'un o més treballadors qualssevol sense haver de factoritzar els N_i 's? Ignoreu el fet que el doble de l'incentiu pugui superar els 4999 EUR.

(b) De fet, només coneixent les claus públiques dels treballadors, com podríeu desxifrar els incentius d'un o més treballadors qualssevol sense haver de factoritzar els N_i 's?

(c) Fent servir les mateixes claus públiques, proposeu una manera alternativa de xifrar els incentius que sigui més segura. Hi ha diverses maneres: escolliu-ne una i sigueu molt clars i concisos.

14.4 Arbres de cerca AVL

(a) Dibuixeu un arbre de cerca AVL amb 12 claus per al qual el cost de trobar la clau més petita sigui mínim. No us oblideu de posar claus dins de tots els nodes.

(b) Determineu quin és el cost asimptòtic en el *cas millor* de trobar la clau més petita en un arbre de cerca AVL amb $n \geq 1$ claus, i demostreu-ho.

14.5 Quick-sort

Considereu el codi següent per al quick-sort sobre un *vector* **<double>** V global:

```
// Particiona V[e..d]. Retorna la posició del pivot.
int particio (int e, int d) {
    double pivot = V[d];
    int i = e;
    for (int j = e; j < d; ++j)
        if (V[j] < pivot) swap(V[i++], V[j]);
    swap(V[i], V[d]);
    return i;
}

// Ordena amb quick-sort V[e..d].
void quick_sort (int e, int d) {
    if (e < d) {
        int p = particio (e, d);
        quick_sort (e, p - 1);
        quick_sort (p + 1, d);
    }
}
```

Aquest codi realment ordena el vector. Considerem-ne doncs el cost. En el que segueix, podeu usar el resultat de l'anàlisi del cost mitjà del quick-sort vist a classe, però heu de raonar igualment les respostes.

- (a) Supposeu que V conté inicialment una permutació aleatòria d' n nombres diferents. Quin cost esperat té *quick_sort* (0, $n - 1$)?
- (b) Supposeu ara que V conté inicialment n nombres diferents en ordre creixent. Quin cost té *quick_sort* (0, $n - 1$)?
- (c) Ara V conté n nombres iguals. Quin cost té *quick_sort* (0, $n - 1$)?
- (d) Finalment, suposeu que afegim

swap($V[\text{random}(e, d)], V[d]$);

com a primera línia de la funció *particio* (e, d), on la crida *random*(e, d) retorna un nombre escollit a l'atzar uniformement entre e i d . Amb aquesta línia, digueu quin és ara el cost esperat dels casos (a), (b) i (c).

14.6 2-SAT

Considereu aquesta fórmula booleana:

$$(a \vee b) \wedge (\bar{a} \vee c) \wedge (\bar{c} \vee d) \wedge (\bar{b} \vee c) \wedge (\bar{c} \vee \bar{d})$$

Usant l'algorisme polinòmic explicat a classe, trobeu una assignació de valors booleans a les variables que satisfaci la fórmula, o demostreu que no n'hi ha cap. Feu servir, sense explicar-ne el funcionament, l'algorisme que troba els components fortament connexos d'un graf. A la vostra explicació, deixeu molt clara la propietat que voleu comprovar.

14.7 Torneig de go

S'està celebrant un torneig de go amb n participants que juguen entre si moltes vegades. Els jugadors reben un punt per cada partida guanyada (no hi ha empats). Coneixeu el nombre de punts que duu cada jugador fins al moment, i les partides que falten per jugar.

Per exemple, suposeu $n = 4$, i que els punts actuals de cada jugador i les partides que falten per jugar són:

A	B	C	D
8	11	21	20

	A	B	C	D
A	-	6	3	6
B	-	-	5	4
C	-	-	-	6
D	-	-	-	-

A les taules podem veure que A duu 8 punts, que B i C encara s'han d'enfrontar entre si 5 vegades, etc.

Volem saber si el jugador A encara pot guanyar el torneig, encara que sigui empatant a punts amb altres jugadors. A l'exemple la resposta és negativa, perquè encara que A guanyi les 15 partides que li falten per jugar, obtenint un total de 23 punts, és fàcil veure que C o D obtindrà necessàriament almenys 24 punts.

Doneu un algorisme polinòmic per decidir, a partir de dues taules donades, si un jugador determinat pot guanyar un torneig amb n jugadors. Expliqueu el vostre algorisme usant l'exemple anterior.

Pista: Reduïu aquest problema a un max-flow en un graf que, a part de la font i el pou, sigui bipartit amb $(n - 1)(n - 2)/2$ vèrtexs a una banda i $n - 1$ vèrtexs a l'altra.

14.8 NP-completesa

(a) Sigui V el conjunt $\{1, \dots, n\}$. El problema GRAPH ISOMORPHISM consisteix a, donats dos grafs no dirigits $G_1 = (V, E_1)$ i $G_2 = (V, E_2)$, decidir si G_1 i G_2 són en el mateix graf. És a dir, cal decidir si existeix una permutació $f : V \rightarrow V$ tal que x i y són adjacents en G_1 si i només si $f(x)$ i $f(y)$ són adjacents en G_2 . Ningú sap si aquest problema es pot resoldre en temps polinòmic.

Suposeu que algú descobrí un algorisme de cost polinòmic per al problema 3-SAT. Digueu raonadament si això implicaria necessàriament que es podria resoldre GRAPH ISOMORPHISM en temps polinòmic.

Per respondre aquesta pregunta no és necessari fer cap reducció explícita, sigui possible fer-la o no.

(b) Considereu el problema PARTITION: Donats n nombres naturals $A = \{a_1, \dots, a_n\}$ amb $\sum_{a \in A} a = 2x$ per a un cert $x > 0$, cal trobar un subconjunt $S \subseteq A$ tal que $\sum_{a \in S} a = x$, o determinar que no hi ha cap S que compleixi la condició. Sabent que SUBSET SUM és un problema NP-complet, demostreu que PARTITION també ho és.

Pista: Suposeu que els naturals de SUBSET SUM sumen y , i que la suma objectiu és t , amb $0 < t < y$. A la reducció, considereu els nombres $y + t$ i $2y - t$.

Parcial, 5 de novembre de 2019

15.1 Exponenciació de matrius

En tot aquest problema, suposeu que el cost de multiplicar dues matrius $n \times n$ és $\Theta(n^3)$. Donada una matriu d'enters M de mida $n \times n$, suposeu que heu de fer un programa que llegeixi diversos enters x , i que per a cadascun calculi M^x .

a) Raoneu quin cost tindria, en funció d' n i x , un algorisme que implementés directament la recurrència $M^x = M \cdot M^{x-1}$, amb $M^0 = I$.

b) Raoneu ara quin cost tindria, en funció d' n i x , un algorisme que implementés la recurrència $M^x = (M^{x/2})^2$ per a x parell, i $M^x = M \cdot (M^{\lfloor x/2 \rfloor})^2$ per a x senar.

c) Assumiu que totes les x són més petites que un cert nombre 2^B . Suposeu que disposem de $M^1, M^2, \dots, M^{2^i}, \dots, M^{2^{B-1}}$ en una taula T . Expliqueu com es podria calcular cada M^x usant T . Quin cost tindria el càlcul de M^x en el cas pitjor? I en el cas millor? I en el cas mitjà, suposant que x s'escull a l'atzar entre 0 i $2^B - 1$? En els tres casos, ignoreu el cost de precalcular T .

15.2 Misteri

En el codi següent, suposeu que el vector T només conté nombres estrictament positius, i que sum conté un nombre estrictament positiu.

```
using P = pair<int, int>;

void misteri (const vector<int>& T, int sum) {
    int n = T.size ();
    vector<P> V(n);
    for (int i = 0; i < n; ++i) V[i] = P(T[i], i);
    sort (V.begin (), V.end ());
```

```

int s = 0, i = n;
while (s < sum and i > 0) {
    --i;
    s += V[i].first ;
}

if (s < sum) cout << "sense solucio" << endl;
else {
    cout << "nombre minim = " << n - i << endl;
    for (int j = i; j < n; ++j) cout << V[j].second << endl;
}

```

- Expliqueu què calcula la funció *misteri* () i com ho fa. Doneu un exemple.
- Expliqueu quin cost té la funció *misteri* () .
- Funcionaria bé la funció *misteri* () si *T* contingues zeros o nombres negatius?

15.3 Els k més propers a la mediana

Considereu un vector V amb n enters diferents, amb n senar. Donat un nombre k entre 1 i n , heu de trobar els k elements més propers a la mediana de V . Definim la distància entre un element x i la mediana m com $|x - m|$.

Per exemple, amb $k = 3$ i $V = [8, 4, 1, 9, 2, 6, 7]$, el qual té $m = 6$, un possible resultat és 8, 6, 7. L'altre possible resultat és 4, 6, 7. L'ordre en què es presenten els k elements no és important.

Doneu amb prou grau de detall un algorisme el més eficient possible per resoldre aquest problema. Mostreu un exemple. Quin cost té en el cas pitjor?

Pista: Es pot resoldre amb cost linial en el cas pitjor.

15.4 Euclides i Fibonacci

Considereu l'algorisme d'Euclides:

```

int mcd(int a, int b) {
    if (b == 0) return a;
    return mcd(b, a%b);
}

```

Sigui F_k el k -èsim nombre de Fibonacci: $F_0 = 0, F_1 = 1, F_k = F_{k-1} + F_{k-2}$ per a $k \geq 2$. Doneu una fórmula exacta per al nombre de crides recursives durant l'execució de $\text{mcd}(F_{k+2}, F_k)$ per a $k \geq 0$. Quant val el resultat de la funció?

15.5 Programacions dinàmiques

Donats dos enters $n \geq 0$ i $t \geq 0$ i un conjunt $D = \{d_1, \dots, d_k\}$ amb tots els d_i diferents entre 0 i 9, volem computar el nombre $a_{n,t}$ de seqüències d' n dígitos escollits de D tals que la suma dels n dígitos és t .

Per exemple, amb $D = \{0, 4, 5\}$ tenim $a_{4,9} = 12$, perquè hi ha 12 seqüències de longitud 4 amb suma 9: 0045, 0054, ..., 5400.

En aquest problema podeu assumir que les operacions aritmètiques amb enters, reals o complexos tenen cost constant (i precisió infinita en el cas dels reals o complexos).

- a) Formuleu un sistema de recurrències lineals que expressin els $a_{n,t}$ en termes dels $a_{n-1,s}$, per als s adequats. No oblideu els casos base.
- b) Calculeu el cost en funció d' n d'un algorisme $A(n)$ de programació dinàmica basat en aquesta recurrència que computi els $a_{n,t}$ per a tots els $0 \leq t \leq 9n$.
- c) Formuleu un sistema de recurrències quadràtiques que expressin els $a_{n,t}$ en termes dels $a_{\lfloor n/2 \rfloor, s}$ i els $a_{\lfloor n/2 \rfloor, r}$, per als s i r adequats.
- d) Assumint ara que n és una potència de dos, expliqueu amb prou detall com usàrieu la recurrència anterior per fer un algorisme $B(n)$ de programació dinàmica que computi el mateix que $A(n)$, però amb cost asimptòticament millor. Quin cost té $B(n)$?

16

Final, 15 de gener de 2019

16.1 Arbres diversos

a) Inseriu, dibuixant l'arbre a cada pas, les claus 4, 5, 7, 2, 1, 3, 6 en aquest ordre en un arbre binari de cerca inicialment buit.

b) Inseriu les mateixes claus en el mateix ordre en un AVL inicialment buit. Digueu per a quines claus s'ha de rebalancejar l'arbre, i amb quin tipus de rotació: senzilla o doble.

16.2 Heap, heap, ...

Suposeu que la funció *min_heapify*(*vector* <int>& *v*, int *i*, int *j*) converteix en min-heap el subvector *v*[*i* .. *j*]. La funció *max_heapify*(*vector* <int>& *v*, int *i*, int *j*) és similar. I la funció *enfonsa*(*vector* <int>& *v*, int *i*, int *j*) recoloca l'element de la posició *i* de tal manera que el subvector *v*[*i* .. *j*] es converteixi en un (min o max, segons el cas) heap, tenint en compte que prèviament ja ho seria si no fos per *v*[*i*].

Considereu aquests codis per a $1 \leq m \leq n$:

```
void misteri_1 (vector<int>& v, int m) {
    int n = v.size ()-1;
    min_heapify(v, 0, n);
    for (int j = 0; j < m; ++j) { swap(v[0], v[n-j]); enfonsa(v, 0, n-j-1); }
    for (int j = 0; j < m; ++j)
        if (j ≤ n/2) swap(v[j], v[n-j]);
}

void misteri_2 (vector<int>& v, int m) {
    int n = v.size () - 1;
    max_heapify(v, 0, m-1);
    for (int j = m; j ≤ n; ++j) {
        if (v[0] > v[j]) { swap(v[0], v[j]); enfonsa(v, 0, m-1); }
    }
    for (int j = 0; j < m; ++j) { swap(v[0], v[m-j-1]); enfonsa(v, 0, m-j-2); }
}
```

Què fa *misteri_1()*? I *misteri_2()*? Quin cost té cadascuna? En funció d' m i n , quina escolliríeu i per què?

16.3 Rangs en arbres 2d

Considereu una col·lecció d' n claus 2-dimensionals (x, y) amb coordenades reals, sense x repetides ni y repetides. Recordeu que un arbre 2d es defineix així: L'arbre buit és un arbre 2d. Per a $n > 0$, un arbre 2d és un arbre binari on a cada node es discrimina per la coordenada x o la y de forma cíclica, començant amb la x . Per exemple, si l'arrel conté una clau (a, b) , el subarbre esquerre conté les claus amb $x < a$, i el subarbre dret conté les claus amb $x > a$.

Una consulta per rang consisteix a, donats $x_1 < x_2$ i $y_1 < y_2$, trobar totes les claus (x, y) tals que $x_1 < x < x_2$ i $y_1 < y < y_2$.

- Dibuixeu un arbre 2d amb 6 claus, i mostreu una consulta que retorni 3 de les claus.
- Sense donar codi ni pseudo-codi, expliqueu un algorisme per fer consultes per rang en un arbre 2d. En general, el vostre algorisme ha de ser millor que recórrer tot l'arbre comprovant cada clau.
- Considereu un arbre 2d perfectament balancejat, amb alçada a i $n = 2^a - 1$ claus. Argumenteu quin és el cost (respecte al nombre de nodes visitats) del vostre algorisme en els casos millor i pitjor.

16.4 Bonnie i Clide

Bonnie i Clide acaben de robar un banc. Tenen una bossa amb n xecs en valors enters de diners (curiosament tots al seu nom: "A pagar a Bonnie o a Clide") i se la volen dividir de tal manera que cadascú rebi exactament la mateixa suma de diners.

Aquest problema pertany a la classe NP ? I a la classe P ? És NP -complet? Justifiqueu les vostres respostes.

16.5 Triangle de Sierpinski

Probablement coneixeu el triangle de Sierpinski. En aquest problema, suposarem que comença amb una fila amb una X , i que el triangle s'estén infinitament cap avall. Aquestes en són les primeres 16 files:

```

      X
     X X
    X  X
   X X X
  X  X X
 X X  X X
X X X X X
 X X  X X
X X  X X
X X X X X
 X X  X X
X X  X X
X X X X X
 X X  X X
X X  X X
X X X X X

```

Fixeu-vos que les files de la 9 a la 16 són com les 8 primeres files, però duplicades. Així doncs, per obtenir 16 files més hauríem de copiar les 16 primeres files dues vegades a sota, una al costat de l'altra, i així successivament.

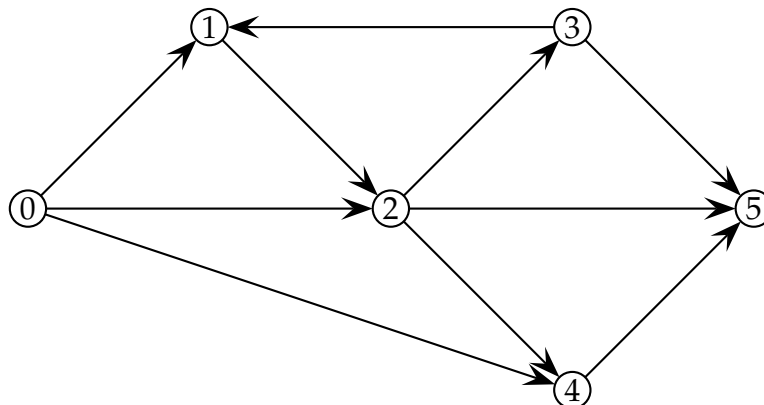
Suposeu que D i T són dos vectors prou grans inicialitzats amb les potències de 2 i de 3. Per exemple, $D[4]$ conté un 16 i $T[4]$ un 81. Ignorant possibles sobreeximents, aquesta funció calcula quantes X hi ha en les $n \geq 1$ primeres files del triangle:

```
int f(int n) {
    int i = 0;
    while (D[i] < n) ++i;
    if (n == D[i]) return T[i];
    return T[i-1] + 2*f(n - D[i-1]);
}
```

- Expliqueu per què aquesta funció és correcta. Exemplifiqueu-ho amb $n = 12$.
- Analitzeu el cost de $f(n)$ en el cas millor i en el cas pitjor. En l'anàlisi, ignoreu el cost d'inicialitzar els vectors D i T .
- Expliqueu com es podria millorar el cost en el cas pitjor de $f(n)$. Podeu incloure codi o pseudo-codi si aquest és breu i comprensible.

16.6 Màxim nombre de camins independents

Donat un graf dirigit amb n vèrtexs, volem calcular quin és el màxim nombre de camins independents des de 0 fins a $n - 1$. Els camins triats només poden tenir en comú els vèrtexs 0 i $n - 1$. Per exemple, si el graf és



llavors es poden aconseguir dos camins independents. Una de les possibles maneres és $0 \rightarrow 2 \rightarrow 3 \rightarrow 5$ i $0 \rightarrow 4 \rightarrow 5$.

Sense escriure codi ni pseudo-codi, però amb detall suficient, doneu un algorisme per resoldre aquest problema. Useu el graf de l'exemple en la vostra explicació.

Pista: Considereu una variant del max-flow.

16.7 Knuth-Morris-Pratt

Considereu el codi per a l'algorisme KMP explicat a classe. Per exemple, el codi calcula el vector $[-1, 0, 0, 0, 1, 1, 2]$ per a "abcaab".

- Doneu una paraula per a la qual KMP calculi el vector $[-1, 0, 1, 0, 0, 1, 2, 2, 3, 1, 2]$.
- Expliqueu raonadament per què no hi ha cap paraula per a la qual KMP calculi el vector $[-1, 0, 0, 1, 2, 0, 1, 2, 2]$.

16.8 Teoria de jocs

Considereu una variant del joc del nim, en la qual hi ha pedres de dos colors, blanc i negre. Com és habitual, s'ha de treure almenys una pedra d'una sola pila, i perd qui no pot jugar. Però aquí tenim una restricció: a cada torn, el nombre de pedres blanques que es pot treure ha de ser diferent del nombre de pedres negres.

Per exemple, si una pila té 3 pedres blanques i 2 pedres negres, algunes jugades vàlides sobre aquesta pila són: treure 2 pedres blanques, o 2 pedres negres, o 3 de blanques i 1 de negra. Però no seria vàlid treure 2 pedres de cada color, ni 1 pedra de cada color.

- a) Si considerem només piles individuals, digueu raonadament quines tenen númer 0.
- b) Supposeu una situació del joc amb tres piles: Una pila amb 1 pedra blanca i 4 negres, una altra amb 3 pedres blanques i 2 negres, i una altra amb 5 pedres blanques i 5 negres. Argumenteu si la posició és guanyadora o perdedora. Si fos guanyadora, mostreu una jugada que guanyi.

16.9 Quatre nombres

Considereu un vector amb n nombres enters arbitràriament grans, però sense repetits. Sense escriure codi ni pseudo-codi, però amb detall suficient, expliqueu com es poden trobar quatre nombres diferents a, b, c i d del vector tals que $a + b = c + d$, o indicar que no hi ha cap solució. Si n'hi hagués més d'una, l'algorisme en pot escollir la que vulgui.

El vostre algorisme hauria de ser tan eficient com sigui possible. Hi ha una solució amb cost esperat $\Theta(n^2)$.

Funcionaria el vostre algorisme si hi hagués nombres repetits? (No cal que ho faci.)

Parcial, 5 de novembre de 2018

17.1 Recurrències

Suposeu que el cost d'un algorisme satisfà la recurrència $T(n) = f(n) + 7T(n/4)$, amb $f(n) = \Theta(n^2)$.

- a) Doneu una fita asimptòticament ajustada per a $T(n)$.
- b) Si poguéssim trobar maneres més eficients de fer la part no recursiva, podríem reduir el cost $T(n)$ a mesura que reduïm $f(n)$? Hi ha algun punt a partir del qual no val la pena esforçar-s'hi per reduir $f(n)$ encara més? Doneu una resposta quantitativa.
- c) Si només poguéssim reduir el nombre de crides recursives sense canviar $f(n)$, podríem reduir el cost $T(n)$? Hi ha algun punt a partir del qual no val la pena esforçar-s'hi per reduir el nombre de crides recursives encara més? Doneu una resposta quantitativa.

17.2 Misteri

Considereu el codi següent:

```
int parteix (vector<int>& v, int e, int d) {
    int p = v[e], i = e, j = d + 1;
    while (true) {
        while (v[++i] < p);
        while (v[--j] > p);
        if (i >= j) {swap(v[e], v[j]); return j;}
        swap(v[i], v[j]);
    }
}

int misteri (vector<int>& V, int i, int j, int e, int d) {
    if (j - i >= d - e) {
        int r = agafa_un(e, d);
        return V[r];
    }
}
```

```

    int k = parteix(V, e, d);
    if (j < k) return misteri(V, i, j, e, k - 1);
    else if (k < i) return misteri(V, i, j, k + 1, d);
    else return V[k]; // k pertany a [i, j]
}

```

agafa_un(*e*, *d*); retorna, en temps constant, un enter qualsevol de [*e*, *d*]. La crida inicial del programa és *misteri*(*V*, *i*, *j*, 0, *V.size*() - 1); amb $0 \leq i \leq j < V.size()$.

- Calculeu el cost en el cas pitjor de la crida inicial a *misteri*() en funció d' $n = V.size()$.
- Digueu què calcula la funció *misteri*(). Doneu un exemple.
- Es pot fer el mateix més eficientment? Com? Amb quin cost?

17.3 Ordenació i més

Considereu l'algorisme següent camuflat misteriosament:

```

bool misteri (vector<int>& v, int x) {
    sort(v);
    int n = v.size();
    for (int i = 0; i < n; ++i)
        if (cerca_dicotomica(v, x - v[i])) return true;
    return false;
}

```

On:

- El procediment *sort*(*v*) ordena el vector *v*.
- El paràmetre *v* és un vector d'enters no negatius.
- El paràmetre *x* és un enter no negatiu.
- El procediment *cerca_dicotomica*(*v*, *x*) és una cerca binària de *x* dins del vector *v*.

a) Analitzeu els costos en temps en els casos pitjor i mitjà de la crida *misteri*(*v*,*x*) segons si el procediment *sort*() dels vectors en C++

- fa ordenació per selecció,
- és mergesort,
- o és quicksort.

b) Digueu en una frase què calcula aquest algorisme.

17.4 Eficiència

Tenim un algorisme *A* amb cost en el cas pitjor $\Theta(\sqrt{n})$ i un altre algorisme *B* amb cost en el cas pitjor $\Theta(2^{\sqrt{\log_2 n}})$. Quin dels dos és més eficient? O tenen la mateixa eficiència asimptòtica? Justifiqueu la vostra resposta.

17.5 Multiplicació súper-eficient

Donats dos nombres amb exactament 1000 dígit cadascun, guardats cadascun en un *vector* `<int>` de mida 1000, cal escriure'n el producte. Quina es la complexitat temporal de l'algorisme més eficient possible per resoldre aquest problema?

17.6 El nombre perdut

Suposeu que teniu un vector ordenat amb $n - 1$ enters diferents entre 1 i n . Expliqueu, sense escriure codi ni pseudocodi, com trobaríeu eficientment l'únic nombre que falta.

17.7 El rètol lumínic

Un comerç antic té un rètol consistent en una graella d' n files amb m bombetes cadascuna. Un sistema informàtic controla les bombetes, amb una matriu d'enters M de mida $n \times m$ on un 1 significa "encesa" i un 0 significa "apagada".

Un tècnic informàtic ha de fer el manteniment del rètol, i ha de fer moltes proves sobre sub-rectangles. En particular, necessita saber, donades i_1, i_2, j_1 i j_2 , amb $0 \leq i_1 \leq i_2 < n$, i $0 \leq j_1 \leq j_2 < m$, si alguna bombeta dins de $M[i_1..i_2][j_1..j_2]$ està encesa. El podeu ajudar?

Suposeu que l'entrada del vostre programa és M , seguida de p preguntes, cadascuna amb una combinació i_1, i_2, j_1 i j_2 . Expliqueu, sense escriure codi ni pseudocodi, com podríeu respondre cada pregunta de la forma més eficient possible. Si us cal, podeu fer algun càlcul sobre M abans de començar a respondre les preguntes. Quin seria el cost temporal total del vostre programa?

Extraordinari, 29 de juny de 2018

18.1 Misteri

Considereu el codi següent:

```
bool f(const string& a, const string& b) {  
    int n = a.size();  
    if (b.size() != n) return false;  
    int k = 0;  
    while (k < n and a[k] != b[0]) ++k;  
    if (k == n) return false;  
    for (int i = 0; i < n; ++i)  
        if (a[(k+i)%n] != b[i]) return false;  
    return true;  
}
```

- a) Digueu raonadament quin és el cost, en cas pitjor, de $f(a, b)$ en funció d' n .
- b) Supposeu que dins d' a no hi ha cap caràcter repetit, i el mateix per a b . Què fa $f(a, b)$?
- c) Expliqueu amb un exemple que passaria si tant a com b tinguessin caràcters repetits.

18.2 Suma de dos elements

Considereu un vector V amb n enters diferents i un altre enter x . Expliqueu, només amb paraules, com trobaríeu si a V hi ha almenys dos elements diferents que sumin exactament x , amb cost total $\Theta(n)$ en el cas mitjà.

18.3 Heaps

Donat el vector $[4, 1, 3, 2, 16, 9, 10, 14, 8, 7]$, feu servir l'algorisme *heapify* per convertir-lo en un max-heap, dibuixant com es transforma a cada pas.

18.4 NP-completesa

- Donats dos problemes NP-difícils X i Y , existeix forçosament una reducció en temps polinòmic d' X a Y ?
- Sigui X un problema NP-complet. Sigui Y un problema tal que X es redueix a Y . Podem deduir que Y és NP-complet?
- Sigui X un problema NP-complet. Sigui Y un problema tal que Y es redueix a X . Podem deduir que Y és NP-complet?

18.5 Problema de l'aturada

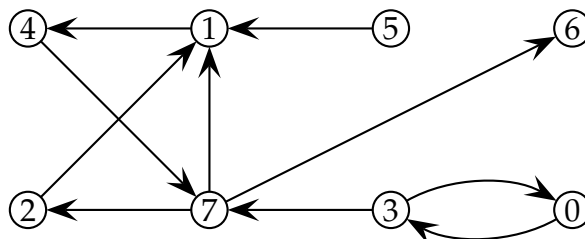
Considereu aquest problema: Donat un natural n , cal decidir si hi ha algun programa (en C++, per exemple) de mida n (és a dir, tal que el seu codi tingui n caràcters) que s'aturi per a totes les entrades.

És aquest problema decidable?

6. Components fortament connexos

(1'5 punts)

Calculeu els components fortament connexos del graf següent. Per fer-ho, heu d'usar l'algorisme explicat a classe. Mostreu els diversos passos de l'algorisme amb prou detall.



18.6 Tall mínim

Considereu un graf no dirigit G , amb pesos estrictament positius a les arestes. Siguin x , y i z tres vèrtexs diferents qualssevol de G . Si el tall mínim entre x i y val 10, i el tall mínim entre y i z val 10, pot ser que el tall mínim entre x i z valgui 100? Si és possible, mostreu-ne un exemple. Altrament, expliqueu per què és impossible.

18.7 Teoria de jocs

Demostreu que la composició de dues posicions d'un joc imparcial és perdedora si i només si les dues posicions tenen el mateix número.

A la demostració, podeu fer servir totes les propietats dels números, excepte la que diu que el número de la composició de dues posicions és l'“or exclusiu” dels seus números.

18.8 Què calcula?

Considereu aquest codi:

```
int main() {  
    int N;  
    cin >> N;  
    vector<int> V(N + 1, 0);  
    V[0] = 1;  
    for (int x = 1; x ≤ N; ++x)  
        for (int i = x; i ≤ N; ++i) V[i] += V[i-x];  
  
    for (int x = 1; x ≤ N; ++x) cout << V[x] << endl;  
}
```

Quins són els sis primers nombres que escriu? (Suposeu que N és almenys 6.) Quin cost té aquest codi? Ignorant els sobreiximents, què calcula?

19

Final, 18 de gener de 2018

19.1 Recurrències

Considereu aquest codi per a $n \geq 1$:

```
void escriu (int n) {  
    stack<int> pila;  
    pila .push(n);  
    while (not pila .empty()) {  
        int x = pila .top ();  
        pila .pop ();  
        cout << ' ' << x;  
        if (x > 1) {  
            pila .push(x - 1);  
            pila .push(x - 1);  
        } } }  
}
```

- a) Doneu un codi recursiu equivalent a l'anterior per a *escriu* (n).
- b) Doneu i resoleu una recurrència que expressi el cost del vostre procediment.

19.2 Els k més petits, en ordre

Considereu un vector desordenat amb n enters diferents. Expliqueu només amb paraules com en trobaríeu els $k = n / \log n$ enters més petits, ordenats de petit a gran, amb cost total $\Theta(n)$ en el cas pitjor.

19.3 Arbres AVL

Inseriu, dibuixant l'arbre a cada pas, les claus 94, 33, 50, 76, 96 i 67 en aquest ordre en un AVL inicialment buit. Digueu per a quines claus s'ha de rebalancejar l'arbre, i amb quin tipus de rotació: senzilla o doble.

19.4 NP-completesa

Suposeu que l'entrada de tots els algorismes d'aquest problema són b bits que codifiquen un numero x entre 2^{b-1} i $2^b - 1$. És un fet conegut que comprovar si x és un nombre primer es pot fer en temps polinòmic en b . Però ningú no sap com factoritzar x com a producte de primers en temps polinòmic en b .

- El problema de factoritzar un nombre donat x pertany a NP?
- Si es demostrés que no existeix cap algorisme polinòmic en b per factoritzar un nombre donat x , podríem deduir $P = NP$ o bé $P \neq NP$?
- Si es trobés algun algorisme polinòmic en b per factoritzar un nombre donat x , podríem deduir $P = NP$ o bé $P \neq NP$?

19.5 2-SAT

Considereu aquesta fórmula booleana: $(\bar{a} \vee c) \wedge (a \vee d) \wedge (\bar{b} \vee \bar{c}) \wedge (\bar{c} \vee \bar{d}) \wedge (\bar{b} \vee c)$
Fent servir l'algorisme explicat a classe, digueu si té alguna solució. Raoneu quantes.

19.6 Teoria de jocs

Recordem el joc del Nim: Hi ha diverses piles de pedres, i dos jugadors, per torns, han de treure almenys una pedra d'una pila no buida; perd qui no pot jugar. Considereu una variant on només es pot treure, o bé una pedra, o bé un nombre parell. Per exemple, d'una pila amb 6 pedres, no se'n poden treure ni 3 ni 5.

Suposeu una situació amb quatre piles amb 1, 3, 7 i 9 pedres. Digueu raonadament si treure 1 pedra de la pila amb 9 pedres és una jugada guanyadora.

19.7 Què fan?

Sigui V un un VE (vector d'enters) amb $n > 0$ enters positius en qualsevol ordre.

```
bool f(const VE& V) {
    set<int> F;
    for (int x : V) {
        if (F.find(x) != F.end()) return true;
        F.insert(x);
    }
    return false;
}
```

```
bool g(const VE& V) {
    VE G = V;
    sort(G.begin(), G.end());
    for (int i = 0; i + 1 < G.size(); ++i)
        if (G[i] == G[i+1]) return true;
    return false;
}
```

```

bool h(const VE& V) {
    int n = V.size ();
    vector<VE> H(n);
    for (int x : V) {
        int i = (37*x + 23)%n;
        for (int y : H[i])
            if (y == x) return true;
        H[i].push_back(x);
    }
    return false;
}

```

Què calcula cada funció? Breument, com ho fan? Quin cost té cadascuna en el seu cas millor i en el seu cas pitjor? Quins són aquests casos?

19.8 Codi estrany

Assumiu aquestes definicions de tipus:

```

using P = pair<int, int>;
using VP = vector<P>;
using Graf = vector<VP>;

```

Suposeu que G conté un *Graf* no dirigit i connex, amb n vèrtexs entre 0 i $n-1$, i amb pesos positius a les arestes, on per a cada aresta entre dos vèrtexs x i y amb cost c , $G[x]$ conté un parell $P(y, c)$ i $G[y]$ conté un parell $P(x, c)$. Considereu el codi següent:

```

int res = 0;
vector<bool> vist(n, false);
priority_queue<P> Q;
Q.push(P(0, 0));
while (not Q.empty()) {
    P p = Q.top(); Q.pop();
    int x = p.second;
    if (not vist[x]) {
        vist[x] = true;
        res += -p.first;
        for (P aresta : G[x]) Q.push(P(-aresta.second, aresta.first));
    }
}
cout << res << endl;

```

Què calcula aquest codi? Quin cost té? Expliqueu breument el seu funcionament, incloent un exemple d'execució amb un graf petit.

Parcial, 6 de novembre de 2017

20.1 Ordenació

Sigui s un string amb n lletres minúscules. Sense escriure codi ni pseudo-codi, expliqueu com ordenaríeu s de la forma més eficient possible. Per exemple, si s fos “abracadabra”, el resultat hauria de ser “aaaaabbcdrr”.

El cost del vostre algorisme hauria de ser inferior a $O(n \log n)$. Expliqueu per què això no contradiu la fita inferior sobre els algorismes d'ordenació explicada a classe.

20.2 Cues de prioritats

Sigui V un vector amb n enters, i sigui x un natural entre 1 i $n - 1$. Considereu aquest codi:

```
priority_queue<int> Q;
for (int i = 0; i < n; ++i) {
    Q.push(V[i]);
    if (i ≥ x) Q.pop();
}
while (not Q.empty()) {
    cout << Q.top() << endl;
    Q.pop();
}
```

- a) Què escriu i en quin ordre?
- b) Quin cost té, en funció d' n i x ?
- c) Ara, suposeu $x = \sqrt{n}$. Si només ens importa què s'escriu, i no en quin ordre, expliqueu amb paraules un algorisme alternatiu que sigui més eficient. Quin cost té?

20.3 Programació dinàmica

Sigui k un enter estrictament positiu, i sigui V un vector amb n enters diferents estrictament positius. Per a valors de i entre 0 i n , i valors de j entre 0 i k , sigui la recurrència

$$f(i, j) = f(i-1, j) + f(i-1, j - V[i-1]) + f(i-1, j - 2V[i-1]) ,$$

amb casos base

$$f(i, j) = 0 \text{ per a } j < 0, \quad f(0, j) = 0 \text{ per a } j > 0, \quad f(0, 0) = 1.$$

- Expliqueu amb paraules comunes què calcula aquesta recurrència.
- El codi corresponent, quin cost tindria en temps? Sense empitjorar el cost en temps, quin és el cost total en espai més petit que es pot aconseguir?

20.4 Multiplicació russo-francesa

Considereu el mètode següent per multiplicar dos naturals de n bits x i y :

```
int multiply(int x, int y) {
    if (y == 0) return 0;
    return 2*multiply(x, y/2) + (y%2 == 0 ? 0 : x);
}
```

- Calculeu el cost de *multiply* en funció de n , considerant que els nombres internament són com vectors de bits. Per exemple, dividir per 2 un nombre de n bits suposa moure tots els bits una posició a la dreta, amb cost $\Theta(n)$.
- Es pot multiplicar més eficientment? Com? Amb quin cost?

20.5 Notació asimptòtica

En els casos següents, indiqueu si $f = O(g)$, $f = \Omega(g)$, o $f = \Theta(g)$. En cada cas, cal triar-ne exactament una. No cal justificar res.

$f(n)$	$g(n)$	$f(n) = X(g(n))$
$n^{1/3}$	$n^{2/3}$	
$100 \cdot n + \log n$	$n + (\log n)^2$	
$n \cdot 2^n$	3^n	
$10 \cdot \log n$	$\log(n^2)$	
2^n	$2^{n/2}$	

20.6 Zeros i uns

Considereu el codi següent:

```
int parteix(vector<int>& v, int e, int d) {
    int p = v[e], i = e + 1;
    while (true) {
        while (v[i] ≤ p and i < d) ++i;
        while (v[d] ≥ p and d ≥ i) --d;
        if (i ≥ d) { swap(v[e], v[d]); return d; }
    }
}
```

```
        swap(v[i], v[d]);
    } }

void sort(vector<int>& v, int e, int d) {
    if (e < d) {
        int p = parteix(v, e, d);
        sort(v, e, p - 1); sort(v, p + 1, d);
    } }

void sort(vector<int>& v) { sort(v, 0, v.size() - 1); }
```

- a) Digueu quin és el cost de *sort* si tots els elements de *v* són 0.
- b) I quin és el cost mitjà si cada $v[i]$ és 0 o 1 amb probabilitat $1/2$ independent?

Extraordinari, 27 de juny de 2017

21.1 Cerques i insercions

Ompliu la taula següent per descriure el cost de l'operació de cerca d'una clau existent en un conjunt amb n claus implementat amb diverses estructures de dades. Per al cas mitjà, suposeu que les claus s'han inserit en ordre aleatori, i que es busca qualsevol de les claus existents amb la mateixa probabilitat. Aquí no heu de raonar res.

Cerca	Cas pitjor	Cas mitjà
Taula		
Taula ordenada		
Arbre binari de cerca		
Arbre AVL		
Taula de hash		
Perfect hashing		

Repetiu-ho per al cost d'inserir una nova clau. (L'algorisme d'inserció no pot suposar que la clau és nova, ho ha de comprovar.) Per al cas mitjà, suposeu que la nova clau té la mateixa probabilitat de ser més petita que totes les claus ja existents, o de ser més gran que la mínima actual però més petita que la segona, ..., o de ser més gran que totes les claus ja existents. Aquí tampoc no heu de raonar res.

Inserció	Cas pitjor	Cas mitjà
Taula		
Taula ordenada		
Arbre binari de cerca		
Arbre AVL		
Taula de hash		

Expliqueu per què no tindria sentit fer la pregunta sobre les insercions per al perfect hashing.

21.2 Encara un altre codi misteriós

Per resoldre aquest exercici, suposeu que els `int`'s de C++ són arbitràriament grans, és a dir, que no pateixen mai sobreiximents. També, suposeu que `int next(int n)`; és una funció que calcula en temps constant un enter que només és funció d' n . Considereu aquest codi:

```
int main() {
    int n;
    cin >> n;

    set<int> S;
    S.insert(n);
    n = next(n);
    while (S.find(n) == S.end()) {
        S.insert(n);
        n = next(n);
    }

    int q = 1;
    int seg = next(n);
    while (seg != n) {
        ++q;
        seg = next(seg);
    }
    cout << q << endl;
}
```

Expliqueu breument què calcula i com ho fa. Quin cost té?

21.3 I encara un altre

Sigui G una matriu de booleans que descriu un graf no dirigit, tal que $G[x][y]$ i $G[y][x]$ són certs si i només si els vèrtexs x i y estan connectats directament amb una aresta. Considereu aquest codi:

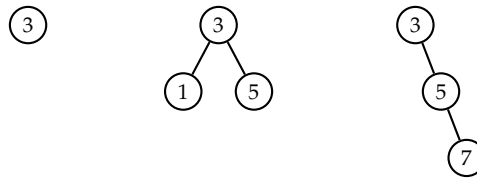
```
int n = G.size ();
for (int i = 0; i < n; ++i)
    for (int j = 0; j < n; ++j)
        for (int k = 0; k < n; ++k)
            G[i][j] = G[i][j] or (G[i][k] and G[k][j]);
```

Què pretén calcular i de quina manera? Aquest codi té un error. Quin és i com s'hauria de corregir?

21.4 Arbres AVL

Donat un arbre binari de cerca, definim la seva alçada com el màxim nombre de nodes en qualsevol camí de l'arrel a les fulles. Per exemple, aquests són arbres binaris de cerca

amb alçades respectives 1, 2 i 3:



Si no existeix cap arbre binari de cerca AVL amb 12 nodes i alçada 5, expliqueu per què. Si existeix, dibuixeu-ne un i expliqueu breument quines propietats té que el fan un arbre binari de cerca AVL.

21.5 La motxilla

Recordeu el problema de la motxilla: Donats n objectes amb pesos $p_1, \dots, p_n$ i beneficis $b_1, \dots, b_n$ i una motxilla de capacitat C , es vol trobar un subconjunt S dels objectes que maximitzi el benefici total $\sum_{i \in S} b_i$ tot garantint que el seu pes total no excedeix la capacitat de la motxilla: $\sum_{i \in S} p_i \leq C$. Els objectes con es poden partir.

1. Demostreu que l'algorisme que ordena els objectes segons la seva relació b_i/p_i en ordre decreixent i que tria els primers que, sumats, no excedeixen la capacitat de la motxilla, no és un algorisme d'aproximació de raó constant.
2. Modifiqueu lleugerament l'algorisme anterior per tal d'aconseguir un algorisme d'aproximació de raó 2. Pista: Penseu què fer amb el primer objecte que ja no cap a la motxilla.

21.6 Els del mig

Donat un vector v $[0..n-1]$ d' n elements, on n és múltiple de 4, es vol saber quins són els $n/2$ elements del mig, és a dir, aquells que estarien entre les posicions $n/4$ i $3n/4 - 1$, ambdues incloses, si s'ordenés el vector. Sense donar codi ni pseudo-codi, descriuiu amb prou detall un algorisme de cost $O(n)$ en el cas pitjor que resolgui aquest problema.

21.7 Collatz i Turing

Sigui n un natural estrictament positiu. Considereu el procés següent: Si n és parell, dividiu-lo per dos. Altrament, multipliqueu-lo per 3 i sumeu-li 1. Quan arribeu a 1, pareu. Per exemple, començant en 3, s'obté la seqüència 3, 10, 5, 16, 8, 4, 2, 1. Collatz va conjeturar l'any 1937 que aquest procés acaba per a qualsevol n inicial, però encara no ho ha sabut demostrar ningú.

Suposeu ara que el problema de l'aturada fós decidible: És a dir, disposeu d'una funció T que, donat qualsevol programa A i qualsevol entrada x , calcula i retorna si A s'aturaria eventualment quan s'apliqués sobre l'entrada x .

Com utilitzaríeu T per saber si la conjetura de Collatz és certa o no?

21.8 El valor d'un graf

El problema del recobriment de vèrtexs (VERTEX-COVER) consisteix a, donat un graf no dirigit $G = (V, E)$ i un natural K , determinar si existeix un subconjunt $S \subseteq V$ amb $|S| \leq K$ tal que $\forall \{u, v\} \in E, u \in S \text{ o } v \in S$. Recordeu que VERTEX-COVER és **NP**-complet.

El problema del valor d'un graf (GRAPH-VALUE) consisteix a, donat un graf no dirigit $G = (V, E)$ amb $n = |V|$ vèrtexs, una llista de n naturals $\langle a_1, \dots, a_n \rangle$, i un natural B , determinar si existeix una permutació $\pi : \{1, \dots, n\} \leftrightarrow V$ de forma que

$$\chi(\pi) = \sum_{\{u, v\} \in E} a_{\pi(u)} \cdot a_{\pi(v)}$$

sigui menor o igual que B .

Demostreu que GRAPH-VALUE és **NP**-complet.

Final, 17 de gener de 2017

22.1 Hashing universal

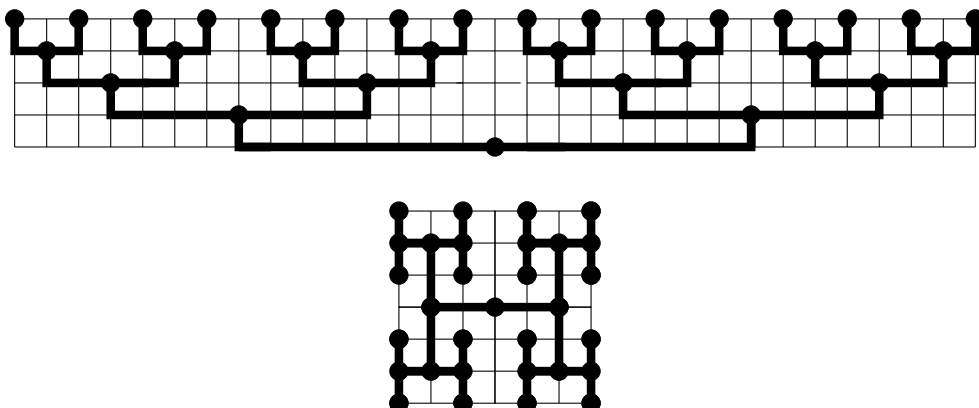
Sigui $\{0, 1, \dots, 2^n - 1\}$ un univers d'elements x que es vol "hashejar" en una taula de 2^m posicions. Els elements de l'univers i les entrades de la taula es representen a través de vectors de bits de mides n i m , respectivament. Considereu la família següent de funcions de hash:

$$\mathcal{H} = \{h_M \mid M \text{ és una matriu de } 0/1 \text{ de mida } m \times n, \text{ i } h_M(x) = (M \cdot x) \bmod 2\}$$

- Definiu el concepte de família universal de funcions de hash.
- Demostreu que $\mathcal{H}$ és una família universal de funcions de hash.

22.2 Recurrències

Tots els arbres es poden representar, sense cap intersecció entre arestes, en una graella rectangular que sigui prou gran. A continuació es mostren dues representacions d'un arbre binari complet de $2^4 = 16$ fulles (la primera té alçada 4 i amplada 30; la segona té longitud de costat 6):



- a) Seguint el patró recursiu de la *primera* representació, doneu i resoleu les recurrències que expressen l'alçada $H(n)$ i l'amplada $W(n)$, respectivament, de la graella necessària per representar un arbre binari complet de 2^n fulles.
- b) Seguint el patró recursiu de la *segona* representació, doneu i resoleu la recurrència que expressa la longitud $L(n)$ del costat de la graella quadrada necessària per representar un arbre binari complet de 4^n fulles.
- c) Si l'objectiu és minimitzar l'àrea de la graella usada per a un arbre binari complet de 4^n fulles, expliqueu usant raonaments asimptòtics quina de les dues construccions escolliríeu. Fixeu-vos que $4^n = 2^{2n}$.

22.3 MAX3SAT

El problema MAX3SAT es defineix així: Donada una fórmula booleana ϕ en forma normal conjuntiva, i amb exactament tres literals per clàusula (3-CNF), cal trobar una assignació de valors booleans a les variables de ϕ de forma que se satisfacin el màxim nombre de clàusules possible.

- a) Doneu una assignació que maximitzi el nombre de clàusules de

$$\phi = (x_1 \vee \neg x_2 \vee x_3) \wedge (x_2 \vee x_3 \vee \neg x_4) \wedge (\neg x_1 \vee x_2 \vee \neg x_4) \wedge (x_1 \vee \neg x_3 \vee x_4).$$

- b) Existeix algun algorisme polinòmic per resoldre MAX3SAT?
- c) Doneu un algorisme eficient i senzill que, donada una fórmula en 3-CNF, en retorni una assignació que satisfaci, almenys, la meitat de les clàusules.
- d) Demostreu que qualsevol fórmula en 3-CNF amb n clàusules té una assignació que en satisfà, almenys, $\frac{7}{8}n$ de les clàusules.

22.4 Flux màxim

Sigui G un graf no dirigit on les arestes tenen capacitats positives. Siguin x , y i z tres vèrtexs qualssevol de G . Supposeu que el flux màxim entre x i y és 100, i que el flux màxim entre y i z és 200. Doneu raonadament la fita inferior més ajustada i la fita superior més ajustada per al flux màxim entre x i z . Al vostre raonament, us pot servir la relació entre flux màxim i tall mínim.

22.5 Teoria de jocs

A partir de la definició dels nímbers, demostreu que qualsevol posició d'un joc imparcial és perdedora si i només si el seu númer és zero.

22.6 Codi espatllat

En aquest problema i el següent, supposeu aquestes definicions de tipus:

```
typedef vector<int> VE;
typedef vector<VE> VVE;
```

Considereu el codi següent. A tot arreu, supposeu que M és una matriu 3×3 que només conté valors -1 , 0 i 1 , i que *juga* val 1 o -1 .

```

map<VVE, int> R;

bool guanya(const VVE& M, int juga) {
    for (int i = 0; i < 3; ++i)
        if (M[i][0] == juga and M[i][1] == juga and M[i][2] == juga) return true;
    for (int j = 0; j < 3; ++j)
        if (M[0][j] == juga and M[1][j] == juga and M[2][j] == juga) return true;
    if (M[0][0] == juga and M[1][1] == juga and M[2][2] == juga) return true;
    if (M[0][2] == juga and M[1][1] == juga and M[2][0] == juga) return true;
    return false;
}

int f(const VVE& M, int juga) {
    auto it = R.find(M);
    if (it != R.end()) return it->second;

    int res = -juga;
    for (int i = 0; i < 3; ++i)
        for (int j = 0; j < 3; ++j)
            if (M[i][j] == 0) {
                VVE C = M;
                C[i][j] = juga;
                if (guanya(C, juga)) return R[M] = juga;
                int r2 = f(C, -juga);
                if (r2 == juga) return R[M] = juga;
                else if (r2 == 0) res = 0;
            }
    return R[M] = res;
}

```

- a) Què pretén calcular la funció f ? Expliqueu breument el seu funcionament.
- b) Aquest codi no és correcte. Per què falla? Com es podria arreglar fàcilment i sense incrementar el cost asimptòtic afegint o modificant unes poques línies?

22.7 Més codis misteriosos

Sigui G un VVE que conté un graf no dirigit amb $n > 0$ vèrtexs i m arestes de la forma habitual: per a tot vèrtex x , $G[x]$ conté els vèrtexs veïns de x . Considereu aquest codi:

```

bool ok(int x, int c, const VVE& G, VE& color) {
    if (color[x]) return color[x] == c;
    color[x] = c;
    for (int y : G[x])
        if (not ok(y, 3 - c, G, color)) return false;
    return true;
}

void misteri(const VVE& G) {
    VE color(G.size(), 0);
    cout << (ok(0, 1, G, color) ? "si" : "no") << endl;
}

```

Què calcula *misteri*(G)? Com funciona? Quin cost té en el cas pitjor? Quin és un cas pitjor?

Parcial, 15 de novembre de 2016

23.1 Recurrència simpàtica

Trobeu la solució de la recurrència $T(n) = 7T(n/7) + \Theta(n)$ i demostreu-la per inducció.

23.2 Els més petits

Donat un vector amb n elements diferents, se'n vol trobar els k elements més petits, per a $k \in \{1, \dots, n-1\}$.

- Expliqueu amb prou detall, i analitzeu, un algorisme el més eficient possible en el cas pitjor per resoldre aquest problema.
- Expliqueu amb prou detall, i analitzeu, un algorisme el més eficient possible en el cas mitjà per resoldre aquest problema.
- Expliqueu els avantatges i inconvenients dels dos algorismes.

23.3 Mínim local

Sigui X un vector amb $n \geq 2$ elements diferents. Un mínim local de X és una posició i tal que $X[i-1] > X[i] < X[i+1]$. La primera posició també pot ser un mínim local, si $X[0] < X[1]$. La darrera posició també pot ser un mínim local, si $X[n-2] > X[n-1]$.

- Demostreu que X té, almenys, un mínim local.
- Expliqueu amb prou detall, i analitzeu, un algorisme el més eficient possible per trobar un mínim local en X .

23.4 Ordenació

Demostreu que no pot existir cap algorisme d'ordenació de propòsit general que trigui temps lineal en mitjana, suposant que les $n!$ permutacions dels elements són equiprobables. Per fer-ho, demostreu que no existeix cap algorisme d'ordenació de propòsit general que funcioni en temps lineal per, almenys, la meitat de les possibles $n!$ entrades de mida n .

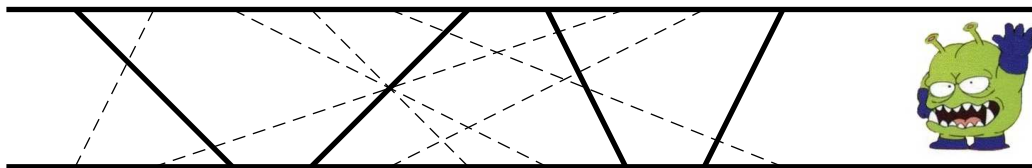
23.5 Codi misteriós

Considereu aquest codi, suposant $n \geq 1$:

```
int n;
cin >> n;
vector<double> V(n + 1, 0);
for (int i = 1; i ≤ n; ++i) {
    cin >> V[i];
    V[i] += V[i-1];
}
sort(V.begin(), V.end());
double res = ∞;
for (int i = 1; i ≤ n; ++i) res = min(res, V[i] - V[i-1]);
cout << res << endl;
```

- Quin cost té?
- Si l'entrada és 5 -6 10 -8 -10 24 quina és la sortida?
- En general, què calcula aquest codi?

23.6 El problema d'en Nikochan



El rei Nikochan vol travessar el túnel del dibuix des de la seva posició de la dreta fins a l'esquerra. Però un sistema automàtic tancarà tantes portes com pugui sense que es creuin entre si (al dibuix, quatre) per evitar que el monstre s'escapi. Cada porta i es defineix amb un parell d'enters (f_i, s_i) que indiquen la posició dels seus extrems a les parets del túnel. A partir de la informació de n portes, i assumint que no hi ha f_i 's repetides ni s_i 's repetides, cal trobar el màxim nombre de portes que es poden tancar al mateix temps.

Per exemple, amb les portes $(0,0)$, $(1,2)$ i $(3,3)$, totes tres es poden tancar alhora. Però amb les portes $(2,3)$, $(1,8)$ i $(5,1)$, només se'n pot tancar una (la que sigui).

Suposeu que teniu la informació de les n portes en un `vector<pair<int, int>> V` de mida n . Sense escriure codi ni pseudo-codi, però de forma prou clara i detallada, doneu un algorisme tan eficient com sigui possible per calcular el màxim nombre de portes que es poden tancar. Quin cost té la vostra solució? La solució esperada té cost $\Theta(n \log n)$.

Extraordinari, 7 de juliol de 2016

24.1 Recurrències

Doneu fites asimptòtiques per a les recurrències

$$A(n) = 9A(n/4) + O(n^2),$$

$$B(n) = 3B(n/3) + \Theta(\log^{3/2} n).$$

24.2 Escacs

Considereu el problema següent: Donada una posició vàlida del joc dels escacs, en la qual toca jugar a les blanques, és aquesta posició guanyadora? És a dir, poden guanyar les blanques sempre, independentment de les jugades que facin les negres?

Expliqueu raonadament si aquest problema és o no a la classe **P**, si és o no a la classe **NP**, si és o no **NP-complet**, i si és o no indecidible.

24.3 Consultes en un llibre electrònic

Heu d'implementar un sistema per cercar paraules a les pàgines d'un llibre electrònic. Concretament, cal trobar eficientment una llista ordenada amb els números de totes les pàgines que contenen totes les paraules que l'usuari consulta.

Com a exemple petitó, suposeu que tenim un llibre amb quatre pàgines, i que cada pàgina conté els mots següents:

- 1 : cada dia al mati canta el gall quiriquiqui
- 2 : la merda de la muntanya no fa pudor
- 3 : el nen canta a la gallina
- 4 : la gallina canta i el gall dorm

Llavors, el resultat de la consulta *[el, gall, canta]* hauria de ser *[1, 4]*, el resultat de la consulta *[canta]* hauria de ser *[1, 3, 4]*, i el resultat de la consulta *[guinardo]* hauria de ser la llista buida *[]*.

El sistema funciona en dues fases. En la primera fase (feta per l'editorial en un ordinador potent), es pre-processen d'alguna forma les pàgines del llibre. A la segona (feta en un dispositiu lector relativament lent), es llegeixen les consultes i s'escriuen les respostes. S'espera que hi hagi moltes pàgines, i totes amb moltes paraules. Es vol contestar molt ràpid cada consulta. Cadascuna de les consultes sol tenir molt poques paraules.

Fixem la nomenclatura següent:

M	=	nombre de pàgines
N	=	nombre total de paraules en totes les pàgines
n	=	nombre de paraules diferents en totes les pàgines
P	=	nombre màxim de pàgines on apareix cada paraula
k	=	nombre màxim de paraules en una consulta

Part 1: Considereu el cas on cada consulta conté una sola paraula ($k = 1$).

- Expliqueu quines estructures de dades i quin pre-procés usaríeu a la primera fase. Expliqueu com ho utilitzaríeu per tractar cada consulta.
- Quantifiqueu l'espai necessari per a les vostres estructures de dades.
- Quantifiqueu el temps necessari per a la primera fase de la vostra solució.
- Quantifiqueu el temps necessari per tractar cada consulta amb la vostra solució.

Part 2: Considereu ara el cas general ($k \geq 1$).

- Expliqueu com utilitzaríeu la mateixa estructura de dades de la Part 1 per processar cada consulta.
- Quantifiqueu el temps necessari per tractar cada consulta amb aquesta solució.

Observacions:

- Us pot ser útil descriure la vostra solució usant estructures de dades de la llibreria estàndard de C++. Però no doneu codi complet.
- Les solucions per a aquest problema que tinguin cost $\Omega(N)$ per a l'apartat (d) es consideraran insuficients i no es valoraran.

24.4 Codis misteriosos

En tots els apartats d'aquest problema, suposeu que V és un vector d'enters de mida n , amb $n \geq 2$ parell, i que $m = n/2$.

Considereu aquest codi:

```
priority_queue<int> Q;
for (int i = 0; i < m; ++i) Q.push(V[i]);
cout << Q.top() << endl;
for (int i = m; i < n; ++i) {
    Q.push(V[i]);
    cout << Q.top() << endl;
}
```

- (a) Què escriu el codi si $n = 10$ i $V = [4, 8, 10, 9, 12, 2, 23, 1, 42, 7]$?
- (b) Doneu una explicació d'alt nivell sobre quins valors s'escriuen amb un V qualsevol si els enters són tots diferents.
- (c) Quin cost asimptòtic té el codi, en funció de n ?

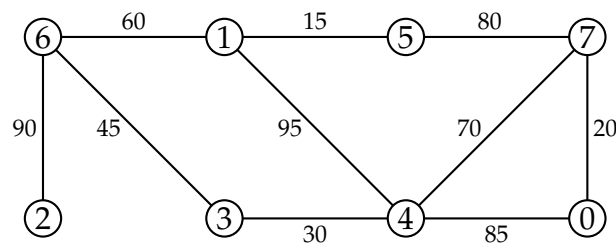
Considereu ara aquest codi:

```
set<int> S;
for (int i = 0; i < m; ++i) S.insert(V[i]);
cout << *S.begin() << endl;
for (int i = m; i < n; ++i) {
    S.insert(V[i]);
    S.erase(V[i-m]);
    cout << *S.begin() << endl;
}
```

- (d) Què escriu aquest codi amb l'exemple de l'apartat (a)?
- (e) Doneu una explicació d'alt nivell sobre quins valors s'escriuen amb un V qualsevol si els enters són tots diferents.
- (f) Quin cost asimptòtic té el codi, en funció de n ?
- (g) Si els elements de V estiguessin repetits, alguns dels dos codis podria fer alguna cosa diferent a les mencionades en els apartats (b) i (e)? En els dos casos, si la resposta és afirmativa, mostreu un exemple; si la resposta és negativa, expliqueu per què.
- (h) Escriviu en C++ un codi alternatiu a un dels dos codis donats, que calculi el mateix (suposant que tots els enters són diferents), però amb cost asimptòtic estrictament menor al calculat a l'apartat (c) o a l'apartat (f). Quin cost té aquest nou codi?

24.5 Algorisme de Kruskal

- (a) Apliqueu l'algorisme de Kruskal al graf següent, amb prou detall perquè quedi clar què calcula i quin és el seu funcionament.



- (b) Demostreu que l'algorisme de Kruskal és correcte per a qualsevol graf on els pesos de totes les arestes són diferents.

24.6 Teoria de jocs

Recordem el joc del Nim: Hi ha diverses piles de pedres, i dos jugadors, per torns, han de treure almenys una pedra d'una pila no buida; perd qui no pot jugar.

Considereu aquesta variant: per a cada pila, si té n pedres, llavors només se'n poden treure entre 1 i $\lceil n/2 \rceil$. Per exemple, si una pila té 20 pedres, se'n poden treure com a màxim 10, i si una pila en té 21, se'n poden treure com a màxim 11.

Suposeu una posició amb 9 piles, amb 1, 2, 3, 999, 4, 5, 6, 7 i 999 pedres respectivament. La posició és guanyadora o perdedora? Si és guanyadora, mostreu raonadament una jugada que guanyi.

Final, 19 de gener de 2016

25.1 Costos

Doneu raonadament el cost de les cinc funcions següents, suposant $n \geq 0$:

```
int f1 (int n) { // 0.25 punts
    int x = 0;
    for (int i = 0; i < n; ++i) ++x;
    return x;
}
```

```
int f2 (int n) { // 0.25 punts
    int x = 0;
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < i; ++j) ++x;
    return x;
}
```

```
int f3 (int n) { // 0.25 punts
    if (n == 0) return 0;
    return f3(n/2) + f1(n) + f3(n/2);
}
```

```
int f4 (int n) { // 0.25 punts
    if (n ≤ 1) return 1;
    return 2*f4(n-1) + 3*f4(n-2) + 8;
}
```

```
int f5 (int n) { // 0.5 punts
    int s = 0;
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < i*i; ++j)
            if (j%i == 0)
```

```

        for (int k = 0; k < n; ++k) ++s;
    return fl(s);
}

```

25.2 Heaps

Mostreu un max-heap amb 16 elements, tots diferents, on el cost d'esborrar el màxim del heap amb l'algorisme explicat a classe sigui el més petit possible.

25.3 Cicles i camins Hamiltonians

Considereu els dos problemes següents:

- HAM: Donat un graf G , conté un cicle Hamiltonià?
- CAM: Donat un graf G , conté un camí Hamiltonià?

Sabent que $\text{HAM} \in \mathbf{NP-C}$, demostreu que $\text{CAM} \in \mathbf{NP-C}$.

25.4 Inaproximabilitat

Considereu els dos problemes $\mathbf{NP}$ -complets següents:

- HAM: Donat un graf G , conté un cicle Hamiltonià?
- TSP: Donats un natural K i una matriu quadrada M amb n^2 naturals, existeix una permutació π de $\{1, \dots, n\}$ tal que

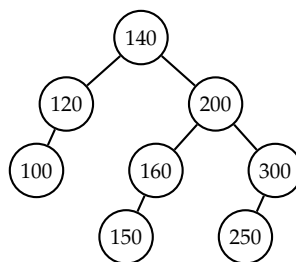
$$M[\pi_n, \pi_1] + \sum_{i=1}^{n-1} M[\pi_i, \pi_{i+1}] \leq K?$$

Demostreu que, si $\mathbf{P} \neq \mathbf{NP}$, llavors $\text{TSP} \notin \mathbf{APX}$.

Pista: Demostreu que, per a qualsevol $\rho \in \mathbf{Q}$, no existeix cap algorisme d'aproximació de factor ρ per a TSP (a no ser que $\mathbf{P} = \mathbf{NP}$). Utilitzeu una reducció a l'absurd utilitzant el fet que HAM és $\mathbf{NP}$ -complet.

25.5 Arbres aleatoritzats

Doneu tots els possibles arbres binaris de cerca resultat d'inserir, amb l'algorisme dels arbres aleatoritzats, un 170 en l'arbre següent:



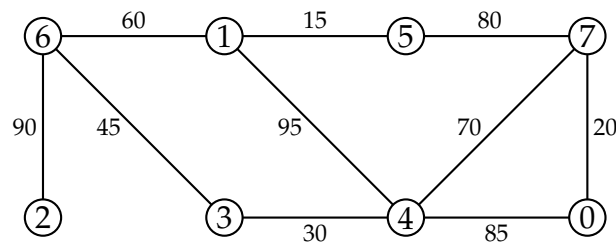
Amb quina probabilitat s'obté cadascun dels arbres?

25.6 Components fortament connexos

Considereu un graf dirigit G amb n vèrtexs, que no té arcs repetits ni arcs d'un vèrtex a ell mateix. Supposeu que G té exactament n components fortament connexos. Amb aquesta informació, digueu raonadament quin és el nombre màxim d'arcs m que pot tenir G .

25.7 Algorisme de Prim

a) Apliqueu l'algorisme de Prim al graf següent, amb prou detall perquè quedi clar què calcula i quin és el seu funcionament. Comenceu en el vèrtex 6.



b) Demostreu que l'algorisme de Prim és correcte en general. A la demostració, suposeu que tots els pesos de les arestes són diferents.

25.8 Flors i colors

Suposeu que teniu n classes de flors i n colors, i que sabeu de quins colors pot ser cada flor. Per exemple, podeu tenir $n = 3$, les flors campaneta, clavell i rosella, els colors blau, blanc i vermell, i aquestes $c = 5$ possibles combinacions: campanetes blaves, campanetes blanques, clavells blancs, clavells vermells, i roselles vermelles.

Voleu fer un ram amb el màxim nombre de flors, però sense repetir ni classes de flors ni colors. En l'exemple, podríeu crear un ram amb tres flors: una campaneta blava, un clavell blanc, i una rosella vermella. Però si els clavells no poguessin ser blancs, llavors només podríeu fer un ram amb dues flors.

Sigui f és el màxim nombre de flors que pot tenir el vostre ram. Expliqueu amb prou detall un algorisme per trobar f . Doneu una fita superior al cost temporal del vostre algorisme en funció de n , c i f .

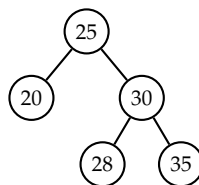
Parcial, 3 de novembre de 2015

26.1 Pica-pica

1. Demostreu que per a qualssevol funcions $f, g : \mathbb{N} \rightarrow \mathbb{N}$,

$$f(n) + g(n) = O(\max\{f(n), g(n)\}).$$

2. Supposeu que s'implementa un conjunt amb taules de dispersió, però no guardant els elements que col·lisionen en llistes, com es fa usualment, sinó en arbres AVL. Quins avantatges i desavantatges té en comparació?
3. Digueu quins són tots els possibles valors enters que provocarien una rotació doble si s'inserissin a l'arbre AVL següent. Inserir-hi qualsevol d'aquests valors (digueu quin) i mostreu l'arbre AVL resultat.



26.2 Maleïdes recurrències

Teniu tres solucions dividir-i-vèncer per resoldre un problema de mida n :

- Dividint en 3 subproblemes de mida $n/2$, i combinant les subsolucions en cost $\Theta(n^2\sqrt{n})$.
- Dividint en 4 subproblemes de mida $n/2$, i combinant les subsolucions en cost $\Theta(n^2)$.
- Dividint en 5 subproblemes de mida $n/2$, i combinant les subsolucions en cost $\Theta(n \log^2 n \log \log n)$.

Quina alternativa és la més eficient?

26.3 Element solitari

1. Considereu un vector amb n enters, amb n senar. Supposeu que cada element hi apareix exactament dos cops tret d'un element, que hi apareix exactament una vegada. Doneu un algorisme de cost linial per trobar l'element solitari.

Per exemple, per a $[4, 7, 4, 1, 3, 3, 6, 1, 7]$, cal retornar 6.

2. Supposeu ara, a més, que el vector està ordenat. Doneu un algorisme el més eficient possible per trobar l'element solitari.

Per exemple, per a $[1, 1, 3, 3, 4, 4, 6, 7, 7]$, cal retornar 6.

Analitzeu el cost en temps i en espai dels vostres dos algorismes.

26.4 En posició!

Donats un vector d'enters diferents v i un enter x , la funció següent examina en ordre (com a molt) les $n = v.size()$ posicions de v i retorna quina conté x , o bé retorna -1 si x no hi és.

```
int posicio (const vector<int>& v, int x) {
    for (int i = 0; i < v.size (); ++i) {
        if (v[i] == x) return i;
    }
    return -1;
}
```

Suposeu que la probabilitat que x estigui a $v[i]$ és $1/2^{i+1}$ per a $0 \leq i < n - 1$, i que estigui a $v[n - 1]$ és $1/2^{n-1}$. Com que aquestes probabilitats sumen 1, x sempre està present. Calculeu el cost asimptòtic de *posicio()* en el cas mitjà sota aquesta distribució de probabilitats.

26.5 Comptatge

Descriviu amb prou detall (però sense escriure codi ni pseudocodi) un algorisme eficient que, donat un vector amb n enters $[x_1, \dots, x_n]$, compti quantes posicions i , amb $1 \leq i \leq n$, compleixen la propietat

$$|\{j : 1 \leq j \leq n \text{ i } x_j \leq x_i\}| = i .$$

Analitzeu el cost en temps i en espai del vostre algorisme.

Per exemple, per a $[12, 14, 14, 17]$ cal retornar 3, i per a $[23, 33, 23]$ cal retornar 0.

26.6 Selecció

Ja sabem que, amb n elements diferents, l'algorisme de selecció per mediana de medianes triga temps $\Theta(n)$ en el cas pitjor quan s'utilitzen grups de 5 elements. Quin cost tindria en el cas pitjor si s'usen grups de 3 elements?

Extraordinari, 26 de juny 2015

27.1 Punts propers a l'origen

Donats n punts a l'espai tridimensional, es vol saber quins són els k punts més propers a l'origen, amb $k = \Theta(n)$. Expliqueu (sense escriure gens de codi) un algorisme amb cost estrictament inferior a $\Theta(n \log n)$ per resoldre aquest problema.

27.2 Propietats dels algorismes d'ordenació

Ompliu (en un altre full) una taula com la següent que inclogui raonadament quines són les propietats principals d'aquests cinc algorismes per ordenar un vector de n elements:

	selection-sort	insertion-sort	merge-sort	quick-sort	heap-sort
Temps en cas pitjor					
Temps en cas millor					
Temps en mitjana					
Mida espai addicional					
És estable?					

[Una ordenació és estable si manté l'ordre relatiu que tenien originalment els elements amb claus iguals. És a dir, si dos elements x i y tenen la mateixa clau, i x està a l'esquerra de y abans d'ordenar, llavors x estarà a l'esquerra de y després d'ordenar.]

27.3 Grafs

Donat un graf no dirigit amb n vèrtexs i m arestes, totes amb pesos positius, i un vèrtex qualsevol x , cal trobar la distància més curta de x a la resta de vèrtexs. Supposeu $m = O(n)$. Quin cost en el cas pitjor tindria l'algorisme de Dijkstra sota aquestes condicions?

Suposeu ara que, a més, se sap que els pesos de les arestes són naturals entre 1 i 10. Proposeu un algorisme alternatiu al de Dijkstra que sigui asimptòticament més eficient.

27.4 Teoria de jocs

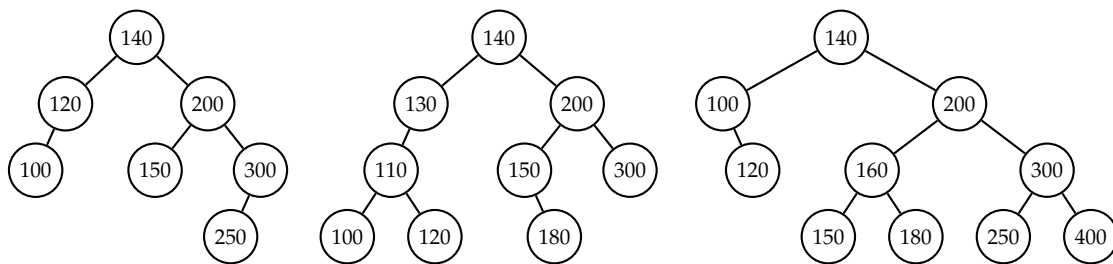
Recordem el joc del Nim: Hi ha diverses piles de pedres, i dos jugadors, per torns, han de treure almenys una pedra d'una pila no buida; perd qui no pot jugar.

Hi ha cinc piles amb el següent nombre de pedres: 12, 9, 3, 20, 16. Digueu raonadament totes les jugades guanyadores que hi ha, o si no n'hi ha cap.

27.5 Arbres AVL

a) Quin és el cost en el cas pitjor de trobar un element present en un AVL amb n nodes? I en el cas mitjà? Raoneu les dues respostes.

b) Dels arbres binaris següents, quins no són AVL? Per què?



27.6 Un altre codi espifat

Considereu el programa següent (amb el preàmbul de les incusions omès):

```
int main() {
    set<string> S;
    string s;
    while (cin >> s) {
        if (S.find(s) == S.end()) S.insert(s);
        else S.erase(s);
    }
    for (set<string>::iterator it = S.begin(); it <= S.end(); ++it) cout << *it << endl;
}
```

Què pretén fer aquest programa? El codi té un error. Quin? Com s'ha d'arreglar? Un cop arreglat, quin cost té aquest programa en el cas millor? I en el cas pitjor?

27.7 NP-completesa

Considereu el problema DIVISOR: Donat (en binari) un natural n , cal dir si existeix algun divisor x de n tal que $1 < x < n^{1/3}$. Ni es coneix cap algorisme polinòmic per resoldre DIVISOR, ni se sap si DIVISOR és NP-complet.

- Si es demostrés $P = NP$, això implicaria $DIVISOR \in P$, o bé $DIVISOR \notin P$?
- Si es demostrés $P \neq NP$, això implicaria $DIVISOR \in P$, o bé $DIVISOR \notin P$?
- Si es demostrés que $DIVISOR \notin P$, això implicaria $P = NP$, o bé $P \neq NP$?
- Si es trobés un algorisme polinòmic per resoldre DIVISOR, això implicaria $P = NP$, o bé $P \neq NP$?

Final, 16 de gener de 2015

28.1 Recurrència

El nombre esperat de comparacions $C(n)$ fetes per l'algorisme de quicksort per ordenar un vector de n elements compleix la recurrència

$$C(n) = \sum_{i=1}^n \frac{1}{n} ((n-1) + C(i-1) + C(n-i)) = (n-1) + \frac{2}{n} \sum_{i=1}^{n-1} C(i).$$

Demostreu per inducció que $C(n) = O(n \log n)$.

28.2 Arbres AVL

Assumiu que l'alçada d'un arbre amb un sol node és 1.

- Quin és el cost en el cas pitjor de cercar un element en un arbre AVL amb $n2^n$ nodes?
- Quin és l'arbre AVL resultat d'inserir les claus 1, 2, 3, 4, 5, 6 en aquest ordre en un arbre inicialment buit?
- Quina és l'alçada màxima d'un arbre AVL amb 7 nodes? Mostreu un exemple.

28.3 Heaps ternaris

Un heap ternari és com un min-heap, però on els nodes tenen tres fills enlloc de dos.

Expliqueu com representàrieu un heap ternari en memòria (doneu la declaració de tipus *Heap3*) i com us mouríeu als fills i al pare d'un node donat.

Implementeu una funció **void** *insert* (*Heap3* & *h*, *Elem* *x*); que insereixi un element *x* en un heap ternari *h* i analitzeu-ne el cost.

28.4 NP-completesa

Proveu que CLICA i GRAN-CLICA es poden reduir polinòmicament entre si, en ambdós sentits:

CLICA: Donat un graf $G = (V, E)$ i donat un nombre natural k , cal determinar si G té un subgraf complet de k vèrtexs. És a dir, cal dir si existeix un subconjunt $U \subseteq V$ de k vèrtexs tal que per a tot parell de vèrtexs diferents $u, v \in U$, $\{u, v\} \in E$.

GRAN-CLICA: Donat un graf $G = (V, E)$, cal determinar si G té un subgraf complet de $\lceil |V|/2 \rceil$ vèrtexs.

28.5 Ordenació topològica

Considereu un graf dirigit amb n vèrtexs que té exactament una ordenació topològica. Per exemple, per a $n = 1$ només tenim un possible graf, per a $n = 2$ en tenim dos, i per a $n = 3$ en tenim dotze. Exactament, quants grafs diferents hi pot haver, en funció de n ?

28.6 Codi espifiat

Considereu el programa següent (amb el preàmbul de les inclusions omès):

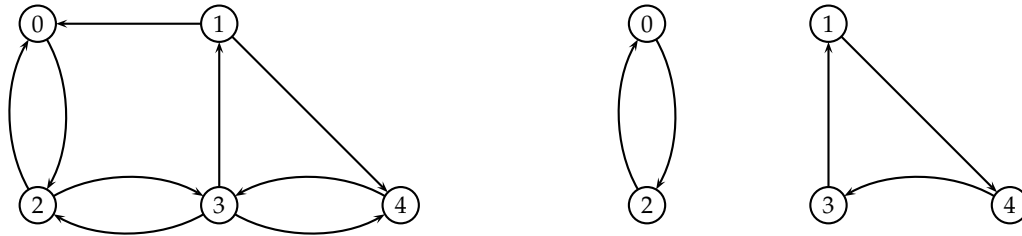
```
int repre(int x, vector<int>& pare) {
    return (pare[x] == -1 ? x : repre(pare[x], pare));
}

int main() {
    int n, m;
    cin >> n >> m;
    vector<int> pare(n, -1);
    while (n > 1 and m-->) {
        int x, y;
        cin >> x >> y;
        int rx = repre(x, pare);
        int ry = repre(y, pare);
        if (rx != ry) {
            pare[y] = x;
            --n;
        }
    }
    cout << (n == 1 ? "si" : "no") << endl;
}
```

Què pretén calcular aquest programa? El codi té un error. Quin? Com s'ha d'arreglar? Un cop arreglat, quin cost té aquest programa en el cas millor? I en el cas pitjor?

28.7 Max-flow

Donat un graf dirigit, es vol saber si es poden escollir n arcs de manera que cada vèrtex quedi inclòs en exactament un cycle de dos o més vèrtexs. Per exemple, a la dreta es pot veure una manera (única, en aquest cas) de descompondre en cicles el graf de l'esquerra:



Descriviu amb prou detall com resoldre aquest problema reduint-lo a un *max-flow*. Useu el graf de l'exemple a la vostra explicació. Pista: Construïu un nou graf amb els vèrtexs duplicats.

28.8 Teoria de jocs

Recordem el joc del Nim: Hi ha diverses piles de pedres, i dos jugadors, per torns, han de treure almenys una pedra d'una pila no buida; perd qui no pot jugar.

Algú diu que ha trobat una situació del joc on, dins de la mateixa pila, hi ha exactament dues jugades guanyadores. Si això és possible, mostreu un exemple i justifiqueu la vostra resposta. Si és impossible, expliqueu per què. La vostra resposta ha d'usar números.

Parcial, 14 de novembre de 2014

29.1 Ordenació per selecció

Aquesta és una implementació de l'algorisme d'ordenació per selecció:

```
void sel_sort (vector<elem>& v) {  
    int n = v.size ();  
    for (int i = 0; i < n; ++i) {  
        int m = i;  
        for (int j = i + 1; j < n; ++j) {  
            if (v[j] < v[m]) {  
                m = j; (★)  
            }  
        }  
        swap(v[i], v[m]);  
    }  
}
```

1. Calculeu exactament quantes assignacions d'elements es fan.
2. Calculeu exactament quantes comparacions entre elements es fan.
3. Assumint que l'entrada d'aquest codi és qualsevol de les permutacions de n elements de forma equiprobable, calculeu exactament quin és el nombre esperat de cops que s'executa la instrucció (★).

29.2 Recurrència

Demostreu que la recurrència $T(n) = T\left(\frac{1}{5}n\right) + T\left(\frac{3}{4}n\right) + O(n)$ té com a solució $T(n) = O(n)$.

29.3 Primer element repetit

Donat un vector de n naturals, s'hi vol trobar el primer element repetit. És a dir, cal trobar l'element que aparegui més d'una vegada tal que l'índex de la seva primera ocurrència sigui mínim. Si no hi ha cap element repetit cal retornar -1.

Per exemple, per a [10, 5, 3, 4, 3, 5, 6] cal retornar 5; per a [8, 6, 5, 4, 6, 120, 4, 8, 6] cal retornar 8; i per a [4, 7, 8, 2, 3] cal retornar -1.

1. Doneu un algorisme el més ràpid possible per resoldre aquest problema i analitzeu-ne el cost en temps i en espai, suposant que l'espai auxiliar està limitat a $\Theta(1)$.
2. Repetiu la pregunta anterior si la memòria addicional no està limitada.

29.4 Subvector de suma zero

Donat un vector de n enters, es vol determinar si conté alguna subseqüència no buida d'elements consecutius que sumi zero.

Per exemple, suposant que la primera posició té índex 0, per a [4, 2, -3, 1, 6] hi ha un subvector que suma zero des de l'índex 1 fins al 3; per a [4, 2, 0, 1, 6] hi ha un subvector que suma zero des de l'índex 2 fins al 2; i per a [4, 2, -4, 1, 6] no hi ha cap subvector que sumi zero.

Respongueu a les preguntes següents sense suposar res sobre els valors continguts en el vector. (Per exemple, si els nombres estan repetits, o si són petits en valor absolut, etc.)

1. Doneu un algorisme el més ràpid possible per resoldre aquest problema en el cas pitjor i analitzeu-ne el cost en temps i en espai.
2. Doneu un algorisme el més ràpid possible per resoldre aquest problema en el cas mitjà i analitzeu-ne el cost en temps i en espai.

29.5 Nombres de Fibonacci

Sigui F_n l'enèsim nombre de Fibonacci, definit com és habitual: $F_0 = 0$, $F_1 = 1$, i $F_n = F_{n-1} + F_{n-2}$ per a $n \geq 2$. Expliqueu amb prou detall però sense escriure codi com es pot calcular F_n en temps $O(\log n)$. El vostre algorisme no pot usar nombres reals.

29.6 Heaps

Considereu el heap següent:

47	41	31	37	29	13	19	17	11	23
----	----	----	----	----	----	----	----	----	----

Inseriu-hi un 43 i esborreu-ne el màxim. Feu les dues operacions independentment, és a dir, ambdues sobre el heap original. Mostreu els passos intermedis, amb una petita explicació.

29.7 Ordenació topològica

Considereu un graf dirigit amb n vèrtexs i sense cap cicle. Quin és el nombre màxim d'ordenacions topològiques que pot tenir? I el nombre mínim? Mostreu un exemple de cada.

Extraordinari, 7 de juliol de 2014

30.1 Recurrència

Sigui $F(n) = 2F(n/2) + 2F(n/3) + \Theta(n^2)$. Demostreu que $F(n)$ és $O(n^2 \log n)$.

30.2 Interseccions

Es vol calcular la intersecció de K vectors de n elements arbitraris cadascun.

- Doneu un algorisme el més eficient possible en el cas pitjor per resoldre aquest problema. Analitzeu el seu cost en temps i en espai.
- Doneu un algorisme el més eficient possible en el cas mitjà per resoldre aquest problema. Analitzeu el seu cost en temps i en espai.

30.3 Anagrames

Es disposa d'una llista L de n paraules i una llista de consultes. Cada consulta és una paraula x i la seva resposta ha de ser totes les paraules de L que són anagrames de x .

Descriviu una estructura de dades per tal de respondre eficientment cada consulta. Descriviu com usar aquesta estructura de dades per respondre a una consulta.

Analitzeu el cost en espai de la vostra estructura de dades. Analitzeu el cost en temps de la construcció de la vostra estructura de dades. Analitzeu el cost en temps del procés de cada consulta.

30.4 Creació de heaps

Doneu un algorisme el més eficient possible per convertir un vector de n elements arbitraris en un max-heap. Analitzeu el cost del vostre algorisme.

30.5 Quadrat d'una matriu

Per calcular el quadrat d'una matriu M de mida $n \times n$, l'algorisme obvi pren temps $O(n^3)$. Com segur que sabeu, existeixen algorismes més avançats de multiplicació de matrius que tenen un cost més baix, però no se'n coneix cap de cost $O(n^2)$. De totes formes, hom podria pensar que calcular el quadrat d'una matriu és més fàcil que fer el producte de dues matrius i, per tant, que existís algun algorisme $O(n^2)$ per calcular quadrats de matrius.

Demostreu que si existeix un algorisme $O(n^2)$ per calcular quadrats de matrius $n \times n$, llavors també existeix un algorisme $O(n^2)$ per calcular productes de parells de matrius $n \times n$.

30.6 Arbres de cerca

Suposeu un arbre de cerca que a cada node té, a més d'una clau real x i dels punters al fill esquerra *esq* i al fill dret *dre*, un comptador m amb la mida (el nombre de nodes) del subarbre del qual x és arrel. Usant les definicions

```
struct node {
    double x;
    node* esq;
    node* dre;
    int m;
};
```

implementeu una funció `int posicio (double y, node* p);` que retorni la posició de y dins de l'arbre apuntat per p . Per exemple, si l'arbre té 9 nodes, llavors si y n'és el mínim cal retornar 1, si n'és el màxim cal retornar 9, i si n'és la mediana cal retornar 5. Si y no es troba dins de p cal retornar -1 .

La vostra funció ha de prendre temps proporcional a l'alçada de l'arbre.

30.7 Teoria de jocs

Considereu qualsevol joc que es pugui resoldre amb nímbers. Suposeu una posició del joc composta per m subjocs $S_1, S_2, \dots, S_m$ tals que $n_1 > n_2 > \dots > n_m$, on n_i denota el númer del subjoc S_i . A més, suposeu que la posició total és guanyadora.

Sota aquestes hipòtesis, considereu l'afirmació "de totes les jugades guanyadores, n'hi ha almenys una dins del subjoc S_1 ". Si l'afirmació és certa, demostreu-la. Altrament, mostreu-ne un contraexemple.

Final, 16 de gener de 2014

31.1 Sumeu zero

Considereu que cadascuna de les estructures de dades següents emmagatzema n enters diferents. Per a cada problema donat, expliqueu un algorisme el més eficient possible que el resolgui i analitzeu el seu cost en temps en el cas pitjor:

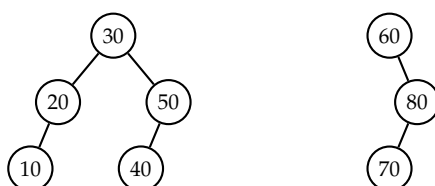
1. Donat un arbre AVL, hi ha algun element que sigui zero?
2. Donat un *max-heap*, hi ha algun element que sigui zero?
3. Donat un vector, hi ha dos elements que sumin zero?
4. Donat un vector ordenat, hi ha dos elements que sumin zero?
5. Donat un vector ordenat, hi ha dos elements que no sumin zero?
6. Donat un arbre binari de cerca, hi ha dos elements que sumin zero?
7. Donat un vector, hi ha tres elements que sumin zero?

31.2 Ordenació de vectors ordenadets

Diem que un vector de n elements es troba *ordenadet* si cada element no es troba a distància més gran o igual que $\lceil \log n \rceil$ de la seva posició un cop ordenat. Doneu un algorisme per ordenar vectors ordenadets. El vostre algorisme ha de ser el més eficient possible en el cas pitjor. Analitzeu el seu cost en temps i en espai.

31.3 Arbres de cerca

Mostreu raonadament quins arbres es poden obtenir i amb quines probabilitats si es fusionen els arbres següents amb l'algorisme join dels arbres aleatoritzats explicat a classe:



31.4 Teoria de jocs

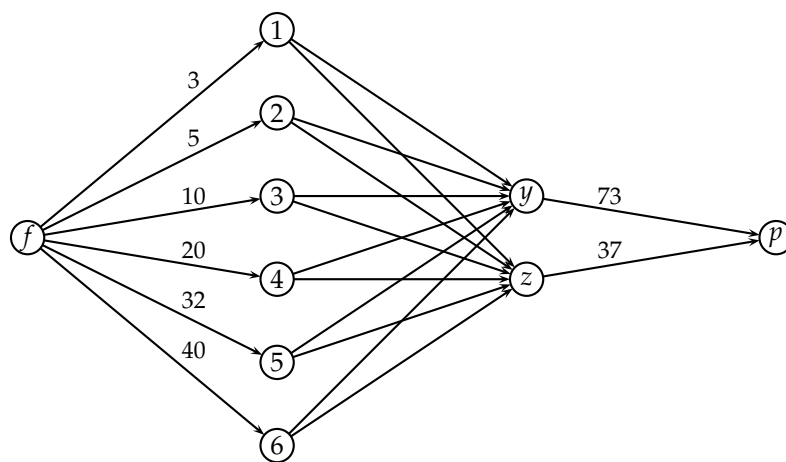
Recordem el joc del Nim: Hi ha diverses piles de pedres, i dos jugadors, per torns, han de treure almenys una pedra d'una pila no buida; perd qui no pot jugar. Considereu una variant amb una regla addicional: a cada torn, com a molt, es poden treure tres pedres.

Suposeu un joc que té nou piles amb aquestes pedres: 1, 7, 8, 21, 23, 25, 102, 884, 1000. La posició és guanyadora o perdedora? Per què? Si és guanyadora, mostreu de forma raonada una jugada que guanyi.

31.5 Fluxos

Recordem el problema *subset sum*: Donats n nombres naturals x_i i un natural y , cal decidir si hi ha algun subconjunt dels nombres que sumi exactament y . Sigui s la suma de tots els nombres, i sigui $z = s - y$. Considerem aquesta reducció a max-flow: Creem un graf amb $n + 4$ vèrtexs: un per a cada x_i , un per a y , un per a z , un per a la font i un altre per al pou. Connectem la font amb cada x_i amb un arc de capacitat x_i . Connectem y i z amb el pou amb arcs de capacitats y i z , respectivament. Finalment, connectem tots els x_i amb y i z , cadascun amb dos arcs de capacitat infinita.

Per exemple, si els x_i són 3, 5, 10, 20, 32 i 40, i $y = 73$, llavors $s = 110$ i $z = 37$, i aquest és el graf que s'obté (les capacitats ∞ no es mostren):



Ara n'hi ha prou de fer un *max-flow* sobre aquest graf per resoldre el problema original. En efecte, com que el problema té solució ($y = 3 + 10 + 20 + 40$), llavors tot el flux que arriba als vèrtexs 1, 3, 4 i 6 pot arribar fins a y , i la resta (els dels vèrtexs 2 i 5) pot arribar fins a z , de manera que el total $s = 110$ pot arribar fins al pou. Recíprocament, si el total $s = 110$ no pogués arribar fins al pou, seria perquè no existeix cap subconjunt de suma y que es pogués dirigir fins aquest vèrtex. Per tant, el problema original té solució si i només si el max-flow permet passar tot el flux s .

- Expliqueu per què aquesta reducció és incorrecta.
- Expliqueu quines conseqüències tindria que hi hagués una reducció correcta (i en temps polinòmic) de *subset sum* a *max-flow*.

31.6 Codi mort

En Genís està preparant un compilador optimitzador. Un dels objectius que té és eliminar codi mort, és a dir, codi que no pot mai ser executat independentment de l'entrada. Per exemple, en el programa següent,

```
void paritat (int n) {  
    if (n%2 == 0) cout << "parell";  
    else if (n%2 == 1 or n%2 == -1) cout << "senar";  
    else cout << "que estrany";  
}
```

la línia marcada amb un asterisc no es pot executar per a cap valor de n .

En general, doncs, en Genís vol fer un algorisme que, donat un programa p i una de les seves instruccions i , indiqui si i és codi mort o no.

Podeu donar un algorisme per resoldre el problema d'en Genís?

Parcial, 8 de novembre de 2013

32.1 Heu preparat la lliçó?

Definiu formalment $O(f)$, $\Omega(f)$ i $\Theta(f)$.

32.2 Polinomis

Considereu que un vector p amb n reals representa el polinomi $p(x) = \sum_{i=0}^{n-1} p[i]x^i$.

1. Descriviu un algorisme per avaluar un polinomi p en un punt real x donat. Analitzeu el cost del vostre algorisme.
2. Descriviu un algorisme de cost $\Theta(n^2)$ per multiplicar dos polinomis de grau $n - 1$.
3. Mostreu com obtenir el polinomi que resulta de multiplicar dos polinomis linears $ax + b$ i $cx + d$ fent només tres productes. Pista: Un dels productes és $(a + b)(c + d)$.
4. Utilitzant la idea anterior, i suposant que n és una potència de dos, doneu un algorisme de cost $\Theta(n^{\log_2 3})$ per multiplicar dos polinomis de grau $n - 1$. Justifiqueu el cost del vostre algorisme.

32.3 Heaps i heapsort

1. Implementeu una funció **bool** *is_max_heap*(**const** *vector* <**double**>& v); que digui si els elements de v formen un max-heap. Calculeu en detall el cost en temps de la vostra funció.
2. Implementeu una funció **void** *heapify*(*vector* <**double**>& v); que reorganitzi els elements de v en forma de max-heap, gastant només espai addicional $\Theta(1)$ i temps $\Theta(n)$. Calculeu formalment el cost en temps de la vostra funció.
3. Useu la funció anterior per implementar **void** *heapsort*(*vector* <**double**>& v); en temps $\Theta(n \log n)$ i espai $\Theta(1)$.

32.4 Comparacions del Quicksort

De les $n!$ permutacions de $\{1, \dots, n\}$, quantes (i quines) d'elles provoquen que quicksort realitzi el nombre màxim de comparacions possibles?

32.5 Múltiples de 66 en un conjunt

Implementeu una funció **void** *elimina_multiples_de_66* (*set* <int>& s); que elimini tots els elements múltiples de 66 de s utilitzant espai addicional constant. Doneu el cost de la vostra funció en el cas millor i en el cas pitjor.

Final, 15 de gener de 2013

33.1 El mateix que el del parcial

Demostreu que el nombre esperat de comparacions per ordenar un conjunt S de n elements diferents amb QuickSort és $\Theta(n \log n)$.

Per concreció, considereu el següent algorisme aleatoritzat:

- Si $n < 2$, S ja està ordenat.
- Altrament,
 1. Es tria a l'atzar un element x de S .
 2. Tot comparant cada element y de S amb x , es construeixen els conjunts $S_1 = \{y \in S \mid y < x\}$ i $S_2 = \{y \in S \mid y > x\}$.
 3. S'ordenen S_1 i S_2 recursivament.
 4. Es retorna l'ordenació de S_1 , seguida de x , seguida de l'ordenació de S_2 .

33.2 Algorisme curiós

Considereu aquest algorisme: Donat un nombre natural $x \geq 1$ que inicialment té n bits, si x és parell el dividim per dos; altrament, fem que x passi a valer el seu nombre de bits (inicialment, n). Parem quan x arriba a 1.

1. En funció de n , quin cost té l'algorisme en el cas pitjor? Doneu un exemple de x amb aquest cost.
2. En funció de n , quin cost té l'algorisme en el cas millor? Doneu un exemple de x amb aquest cost.

33.3 Problemes petits i bonics sobre vectors

1. Donat un vector v amb n posicions, es vol saber si v conté tots els elements de 1 a n exactament una vegada.

Per a cadascun dels algorismes següents, digueu si resol o no el problema i, si ho fa, doneu el seu cost en temps (CP: en el cas pitjor, CM: en el cas mitjà) i en espai (en el cas pitjor). Per respondre, ompliu una taula com la següent:

Algorisme	Funciona	Temps CP	Temps CM	Espai CP
a				
b				
c				
d				
e				
f				
g				
h		—		
i		—		
j				

- (a) Sumem tots els elements de v . Retornem cert ssi la suma és igual a $n(n+1)/2$.
 - (b) Ordenem els elements de v amb l'algorisme d'ordenació per inserció, i verifiquem que la primera posició conté un 1, que la darrera conté una n , i que no hi ha dos elements consecutius repetits.
 - (c) Com abans, però amb l'algorisme d'ordenació per fusió.
 - (d) Com abans, però amb l'algorisme d'ordenació ràpida.
 - (e) Creem un conjunt d'enters (*set* `<int>`) inicialment buit. Hi inserim cada element de v . Recorrem ordenadament el conjunt per comprovar que és igual a $\{1, \dots, n\}$.
 - (f) Creem un min-heap inicialment buit. Hi inserim cada element de v . Recorrem en pre-ordre el min-heap per comprovar que és igual a $\{1, \dots, n\}$.
 - (g) Creem una taula b de $n+1$ booleans, tots inicialitzats a fals. Per a cada element x de v , si $x < 1$ o $x > n$, retornem fals; sinó, si $b[x]$ retornem fals; sinó posem $b[x]$ a cert i continuem. Si no s'ha retornat mai fals, retornem cert.
 - (h) Per a cada índex i de v , comprovem que $1 \leq v[i] \leq n$, i que, per a cada índex j de v amb $j > i$, $v[i] \neq v[j]$.
 - (i) Com abans, però aquest cop utilitzem una cerca dicotòmica.
 - (j) Per a cada permutació p de $[1, \dots, n]$, si $p = v$, retornem cert. Si no s'ha retornat mai cert, retornem fals.
2. Donat un vector v amb $n-1$ posicions que només conté elements de 1 a n sense repeticions, es vol saber quin és l'element de 1 a n que falta. Proposeu un algorisme per resoldre aquest problema en temps lineal i espai constant.
 3. Donat un vector v amb $n-2$ posicions que només conté elements de 1 a n sense repeticions, es vol saber quins són els dos elements de 1 a n que falten. Proposeu un algorisme per resoldre aquest problema en temps lineal i espai constant.

33.4 Agència matrimonial

En certes cultures s'admet que els homes tinguin més d'una dona. Supposeu que munteu una agència matrimonial, i que la informació de què disposeu és: el nombre d'homes h ; el nombre de dones d ; per a cada home, quines dones li agraden; per a cada dona, quins homes li agraden; i per a cada home, quantes dones vol com a màxim. L'objectiu es fer el màxim nombre possible de matrimonis entre persones que s'agradin l'una a l'altra.

Expliqueu breument però amb prou detall com reduiríeu aquest problema a un explicat a l'assignatura. Mostreu el funcionament del vostre algorisme amb un exemple petit. Supposeu que els homes es numeren de 1 a h , i les dones de $h + 1$ a $h + d$.

33.5 Tasques i màquines

Tenim m màquines idèntiques que han de processar n tasques. La tasca i -èsima triga t_i unitats de temps en ser processada. Cada màquina només pot processar una tasca en cada moment, i cada tasca s'ha de processar en una sola màquina. Estem interessats a distribuir les n tasques a les m màquines, de forma que el temps final per completar totes les tasques sigui mínim.

La solució del problema és doncs una *assignació* de cada tasca a una màquina i a un temps d'inici, tot garantint les restriccions. El *temps de procés* d'una assignació és el temps necessari per completar totes les tasques segons aquella assignació.

Per exemple, per a tres màquines i set tasques que requereixen temps 6, 9, 3, 3, 7, 1, 3 respectivament, el diagrama següent representa una assignació amb temps de procés 12, que es pot comprovar que és el mínim necessari.

màquina 1:	-2.4mm 1	-2.4mm 7	-2.4mm 3
màquina 2:	-2.4mm 3	-2.4mm 6	-2.4mm 3
màquina 3:	-2.4mm 9		

1. Demostreu que la versió decisonal d'aquest problema d'assignació de tasques és **NP-completa**.
2. Demostreu que l'estratègia següent és un algorisme d'aproximació de raó 2 per al problema d'optimització:
 - (a) Ordenem de forma arbitrària les tasques.
 - (b) En ordre, assignem cada tasca a la màquina que acabarà abans.

Pista: Penseu en quin estat es troben les màquines quan comença l'última tasca.

Parcial, 13 de novembre de 2012

34.1 Ompliu els blancs

Ompliu els blancs de la forma més curta i precisa possible.

- Definiu formalment $O(f)$:

- Si $T(n) = 2T(n-3) + n^2 - 2n + 1$, llavors $T(n) = \Theta(\text{ })$.
- Per ordenar un vector amb n elements, l'algorisme d'ordenació per inserció triga $\Theta(\text{ })$ passos en el cas millor i $\Theta(\text{ })$ passos en el cas pitjor.
- Si féssim una cerca dicotòmica partint el vector en tres trossos enlloc de dos i buscant en el terç adequat, el cost resultant seria $\Theta(\text{ })$.
- Suposeu que hi ha una manera de multiplicar dues matrius 3×3 amb 25 productes de reals i 50 sumes de reals. Assumint que n és una potència de 3, el cost que tindria un algorisme recursiu basat en aquest mètode per multiplicar matrius $n \times n$ seria $\Theta(\text{ })$.

34.2 Comparacions del QuickSort

Demostreu que el nombre esperat de comparacions per ordenar un conjunt S de n elements diferents amb QuickSort és $\Theta(n \log n)$.

Per concreció, considereu el següent algorisme aleatoritzat:

- Si $n < 2$, S ja està ordenat.
- Altrament,

1. Es tria a l'atzar un element x de S .
2. Tot comparant cada element y de S amb x , es construeixen els conjunts $S_1 = \{y \in S \mid y < x\}$ i $S_2 = \{y \in S \mid y > x\}$.
3. S'ordenen S_1 i S_2 recursivament.
4. Es retorna l'ordenació de S_1 , seguida de x , seguida de l'ordenació de S_2 .

34.3 L'Euclides no hi és

Probablement recordeu l'algorisme d'Euclides per calcular el màxim comú divisor (mcd) de dos nombres. En aquest problema estudiem un algorisme alternatiu i l'apliquem a nombres grans.

1. Verifiqueu les identitats següents:

$$\text{mcd}(a, b) = \begin{cases} 2\text{mcd}(a/2, b/2) & \text{si } a \text{ i } b \text{ parells,} \\ \text{mcd}(a, b/2) & \text{si } a \text{ senar i } b \text{ parell,} \\ \text{mcd}((a-b)/2, b) & \text{si } a \text{ i } b \text{ senars.} \end{cases}$$

2. Basant-vos en aquestes identitats, escriviu una funció `int mcd (int a, int b)` per calcular el màxim comú divisor de a i b . Analitzeu el cost del vostre algorisme en funció de a i de b .
3. Considereu que un nombre gran es representa amb un vector de booleans, amb la declaració `typedef vector<bool> bigint;`. Per guardar un natural x , la posició i del `bigint` corresponent emmagatzema l' i -èsim bit de més pes de x . Per exemple, 12 es representaria com a $[1, 1, 0, 0]$.

Escriviu una funció `bigint mcd (bigint a, bigint b)` per calcular el màxim comú divisor de a i b . Analitzeu el cost del vostre algorisme. Si v és un `bigint` i b és un boolèa, recordeu que les operacions `v.push_back(b)`; i `v.pop_back()`; tenen cost constant.

34.4 La funció misteri ataca de nou

Considereu la funció següent:

```
int misteri (const vector<int>& v) {
    int n = v.size ();
    int m = v[0];
    for (int i=1; i<n; ++i) {
        if (v[i] < m) {
            m = v[i];          (*)
        }
    }
    return m;
}
```

1. Digueu què calcula aquesta funció.
2. Quants cops s'executa la instrucció (*) en el cas millor i quin és aquest cas?
3. Quants cops s'executa la instrucció (*) en el cas pitjor i quin és aquest cas?
4. Considerant que v conté una permutació aleatòria de $[1, n]$, quin és el nombre esperat de cops que s'executa la instrucció (*)?

Extraordinari, 29 de juny de 2012

35.1 Miscel·lània

Contesteu de forma breu però precisa i raonada les preguntes següents:

- a) Quin cost té aquest programa?

```
// pre:  $0 \leq e \leq d < V.size()$ 
void f (int e, int d, vector<int>& V) {
    if (e < d) {
        int m = (e + d)/2;
        intercanvia (V[e], V[m]);
        f(e, m, V);
        intercanvia (V[e], V[m]);
        f(e, m, V);
        intercanvia (V[m+1], V[d]);
        f(m + 1, d, V);
        intercanvia (V[m+1], V[d]);
    }
}
```

- b) Com escriuríeu el programa anterior perquè fes el mateix (molt) més ràpid?
- c) Considereu el següent problema decisonal per al joc dels escacs, en la posició inicial, encara sense cap moviment fet: “Blanques juguen. Guanyen?”. Digueu si el problema és decidible o no, si és **NP** o no, i si és **P** o no.
- d) Se sap que cert problema es pot resoldre amb un algorisme de cost $\Theta(n^{\log n})$, on n és la mida de l'entrada. Digueu si el problema és tractable (és a dir, si té solució de cost polinòmic), intractable, o alguna altra cosa.
- e) (0'1 extra) Fa una setmana es va celebrar el centenari del naixement d'un famós informàtic. Qui?

35.2 Algorismes d'aproximació

Donat un graf no dirigit $G = (V, E)$ (amb $n = |V|$ i $m = |E|$), i un subconjunt $S \subseteq V$, $t(S)$ és el nombre d'arestes amb un extrem en S i l'altre fora de S . El problema del tall màxim (MAXCUT) consisteix a trobar un subconjunt $S \subseteq V$ tal que $t(S)$ sigui màxim. Se sap que la versió decisional de MAXCUT és un problema **NP**-complet.

Considereu l'algorisme aleatoritzat A següent: Començant amb $S = \emptyset$, per a cada vèrtex v es tira una moneda, i si curt cara, v es posa en S .

Demostreu que, en mitjana (sobre les tirades de la moneda), A és un algorisme d'aproximació de raó $\frac{1}{2}$.

35.3 Complexitat i indecidibilitat

Considereu els dos problemes següents:

- SAT: Donada una fórmula booleana sobre diverses variables, hi ha alguna assignació de valors cert o fals a les variables que faci que la fórmula sigui certa?
- ATURADA: Donat un programa (algorisme) A i una entrada x , s'atura A quan s'executa sobre l'entrada x ?

Doneu una reducció en temps polinòmic de SAT a ATURADA. Sabent que SAT és **NP**-complet, podem deduir que ATURADA també ho és?

35.4 Programació dinàmica

La setmana passada va ser Sant Joan. La colla d'en Jonny, en Roy i la Steffy tenien previst passar la revetlla menjant coca, vevent cava i tirant coets. Els nostres amics van ser els responsables de comprar els coets. Entre ells va tenir lloc la conversa següent:

JONNY: He aconseguit recaptar P euros. La gent m'ha demanat que aquest any els coets facin molt de soroll, però que no en comprem cap de repetit.

ROY: Doncs en aquest catàleg hi ha N tipus de coets disponibles. Per a cada tipus hi figura el seu preu $p[i]$, el soroll que fa quan explota $s[i]$ i els grams de pólvora que conté $g[i]$.

JONNY: Perfecte! Mirem quins coets cal comprar per fer el màxim de soroll sense passar-nos del pressupost ni triar-ne dos del mateix tipus, agafem el cotxe de la Steffy i els comprem.

STEFFY: Ep, nois... Cal tenir a més en compte que en Felip Puig acaba de prohibir transportar més de G grams de pólvora en un vehicle privat!

- Escriuiu una recurrència (inclosos els casos base) per calcular el màxim soroll que es pot aconseguir triant entre els N coets de manera que en total costin P euros o menys i continguin G grams o menys de pólvora. Digueu el significat de cada paràmetre de la vostra recurrència i com es calcularia el resultat final del problema.
- Expresseu en funció de N , P i G quin cost temporal tindria l'algorisme de programació dinàmica resultant. Raoneu d'on surt aquest cost.

Suposeu que el soroll que fan diversos coets és la suma dels sorolls individuals. Suposeu també que tots els valors són naturals. No escrigueu gens de codi. Gens!

35.5 Teoria de jocs

Demostreu que si X i Y són dos jocs imparcials, el número del joc resultant d'ajuntar X i Y és el “xor” del número de X i del número de Y .

35.6 Codi misteriós

Considereu el codi següent:

```
const int P = 1000003;

int modul (int x) { int r = x%P; return r < 0 ? r + P : r; }

int num (char c) { return c - 'a'; }

bool f (string s, string t) {
    int n = s.size ();
    int m = t.size ();
    int x = 0;
    for (int i = 0; i < n; ++i) x = modul(26*x + num(s[i]));
    int y = 0;
    for (int i = 0; i < n; ++i) y = modul(26*y + num(t[i]));
    if (y == x) return true;
    int pot = 1;
    for (int i = 0; i < n - 1; ++i) pot = modul(26*pot);
    for (int i = n; i < m; ++i) {
        y = modul(26*(y - pot*num(t[i-n])) + num(t[i]));
        if (y == x) return true;
    }
    return false;
}
```

Suposeu que tant s com t només contenen lletres minúscules, i que $n \leq m$.

Què retorna la funció f ? Com funciona? Quin cost té en els casos millor i pitjor? Pot retornar **false** incorrectament? Suposant que n i m són “grans”, amb quina probabilitat (aproximada) retorna **true** incorrectament?

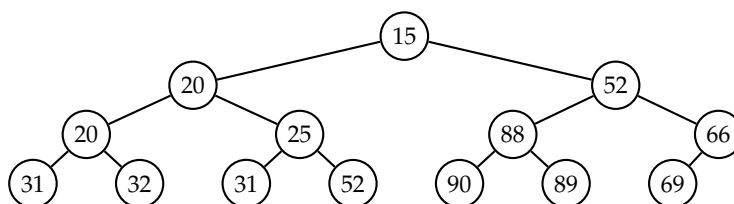
Final, 11 de gener de 2012

36.1 Codis de Huffman

Cal transmetre símbols entre 'a' i 'e', cadascun amb les probabilitats independents següents: $p_a = 0.34$, $p_b = 0.28$, $p_c = 0.18$, $p_d = 0.11$, i $p_e = 0.09$. Mostreu una possible codificació de Huffman. Expliqueu breument com l'heu obtinguda. Quants bits calen en mitjana per transmetre un missatge amb n símbols?

36.2 Heaps

- a) Dibuixeu els dos heaps resultants d'afegir les claus 21 i 12 l'una després de l'altra en el heap següent:



- b) Dibuixeu els dos heaps resultants d'esborrar el mínim dos cops (l'un després de l'altre) del heap anterior (abans de fer les insercions).

36.3 Teoria de jocs

Demostreu que si X i Y són dos jocs imparcials, el número del joc resultant d'ajuntar X i Y és el "xor" del número de X i del número de Y .

36.4 Estructures de dades

Sobre el conjunt $S = \{1, \dots, n\}$, una relació $R \subseteq S \times S$ ve definida per un conjunt de m parells ordenats $C = \{(u_1, v_1), \dots, (u_m, v_m)\}$, amb $u_i, v_i \in S$. Direm que u i v estan relacionats segons R si i només si $(u, v) \in C$.

- Descriuiu un algorisme i les seves estructures de dades associades per tal de saber si R és una relació simètrica en temps $O(m)$ en mitjana.
- Descriuiu un algorisme i les seves estructures de dades associades per tal de saber si R és una relació d'equivalència (reflexiva, simètrica i transitiva) en temps $O(n + m \log m)$ en el cas pitjor.

36.5 NP-completesa i programació dinàmica

Considereu els dos problemes següents (totes les entrades són nombres naturals):

- PARTICIÓ:** Donats m nombres, es poden repartir en dos subconjunts disjunts que sumin el mateix?
 - MOTXILLA:** Tenim una motxilla que pot aguantar fins a P unitats de pes, i n objectes, cadascun amb un pes p_i i un valor v_i . Es pot escollir un subconjunt dels objectes de forma que la suma dels pesos no excedeixi P , i que la suma dels valors sigui almenys V ?
- Sabent que PARTICIÓ és un problema **NP-complet**, demostreu que MOTXILLA també ho és.
 - Utilitzant programació dinàmica, doneu una solució al problema MOTXILLA (escriuiu la recurrència i el cas o casos base). Quin és el cost de la vostra solució en temps i en espai en el cas pitjor?
 - Per a la pregunta anterior, el gran professor Petit ha trobat un algorisme de cost $\Theta(nP)$. Es mereix un milió de dòlars per haver demostrat que **P = NP**?

36.6 Algorismes d'aproximació

Donat un graf no dirigit $G = (V, E)$ (amb $n = |V|$ i $m = |E|$), i un subconjunt $S \subseteq V$, $t(S)$ és el nombre d'arestes amb un extrem en S i l'altre fora de S .

El problema del tall màxim (MAXCUT) consisteix a trobar un subconjunt $S \subseteq V$ tal que $t(S)$ sigui màxim. Se sap que la versió decisional de MAXCUT és un problema **NP-complet**.

Considereu l'algorisme *cerca* següent:

```

S = ∅;
while (true) {
    if (hi ha algun u tal que t(S ∪ {u}) > t(S)) S = S ∪ {u};
    else if (hi ha algun u tal que t(S − {u}) > t(S)) S = S − {u};
    else return S;
}

```

- Demostreu que l'algorisme *cerca* acaba.

- b)** Demostreu que l'algorisme *cerca* és un algorisme d'aproximació de raó 2 per al problema MAXCUT.

Parcial, 27 d'octubre de 2011

37.1 Ompliu els blancs

Ompliu els blancs de la forma més curta i precisa possible.

- Si $T(n) = 4T(n/2) + 3n^3 - 2n^2 + 1$, llavors $T(n) = \Theta(\text{ })$.
- Si $T(n) = 3T(n-2) + n^2 - 2n + 1$, llavors $T(n) = \Theta(\text{ })$.
- Qualsevol algorisme per trobar el k -èsim element més petit d'una taula de talla n necessita () passos.
- En un min-heap amb n elements, en el cas pitjor, consultar l'element mínim té cost () , consultar l'element màxim té cost () , inserir un element té cost () i esborrar el mínim té cost () .
- Per ordenar un vector amb n elements tots iguals, l'algorisme de merge sort triga $\Theta(\text{ })$ passos i l'algorisme de quick sort (amb la partició de Hoare) en triga $\Theta(\text{ })$.
- Quick sort triga $\Theta(\text{ })$ passos per ordenar un vector amb n elements si sempre tria com a pivot la mediana dels elements a particionar.

Dibuixeu una línia des de cadascuna de les cinc funcions en el centre al millor valor Ω de l'esquerra, i al millor valor O de la dreta.

$\Omega(1/n)$		$O(1/n)$
$\Omega(1)$		$O(1)$
$\Omega(\log \log n)$		$O(\log \log n)$
$\Omega(\log n)$		$O(\log n)$
$\Omega(\log^2 n)$		$O(\log^2 n)$
$\Omega(n/\log n)$	$1/(\log n)$	$O(n/\log n)$
$\Omega(n)$	$7n^5 - 3n + 2$	$O(n)$
$\Omega(n^{1'00001})$	$(n^2 + n)/(\log^2 n + \log n)$	$O(n^{1'00001})$
$\Omega(n^2/\log^2 n)$	$2^{\log^2 n}$	$O(n^2/\log^2 n)$
$\Omega(n^2/\log n)$	3^n	$O(n^2/\log n)$
$\Omega(n^2)$		$O(n^2)$
$\Omega(n^{3/2})$		$O(n^{3/2})$
$\Omega(2^n)$		$O(2^n)$
$\Omega(5^n)$		$O(5^n)$
$\Omega(n^n)$		$O(n^n)$
$\Omega(n^{n^2})$		$O(n^{n^2})$

37.2 Costs

Considereu les funcions següents:

```

int cosa (int n) {
    int r = 0;
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < i; ++j) {
            for (int k = j; k < i + j; ++k) {
                ++r;
            }
        }
    }
    return r;
}

int farandula (int n) {
    if (n == 0) return 0;
    else return cosa(n) + 3 * farandula(n/3);
}

```

- Quan triga $cosa(n)$?
- Quan triga $farandula(n)$?

37.3 Sumar un

Per representar nombres naturals molts grans, es pot utilitzar una taula de bits. Per exemple, la taula $\langle 0, 1, 0, 1, 1 \rangle$ representaria el natural 26.

Considereu l'algorisme obvi per afegir una unitat a un natural representat amb una taula de n bits.

- Quin és el cost d'aquesta operació en el cas millor?
- I en el cas pitjor?
- I en el cas mitjà? (Suposeu que cadascun dels n bits és 1 amb probabilitat $\frac{1}{2}$.)

37.4 Heaps

Recordeu que les cues de prioritat de la STL estan implementades amb max-heaps.

Considereu el programa següent:

```
priority_queue<int> pq;
int x;
while (cin >> x) {
    pq.push(x);
}
while (not pq.empty()) {
    cout << pq.front() << endl;
    pq.pop();
}
```

- a) Digueu (utilitzant un màxim de dos verbs) què fa aquest algorisme.
- b) Demostreu que aquest algorisme triga temps $\Theta(n \log n)$ en el cas pitjor.
- c) Quin temps triga aquest algorisme en el cas millor? Quin és el cas millor?

37.5 Triangles en grafs

Un *triangle* en un graf no dirigit $G = (V, E)$ és un conjunt de tres vèrtexs distincts $u, v, w \in V$ tals que $\{u, v\} \in E$, $\{v, w\} \in E$, i $\{u, w\} \in E$.

Doneu un algorisme de temps lineal que, donat un graf no dirigit sense cicles de llargada 5 o més, indiqui si el graf conté algun triangle.

Extraordinari, 5 de juliol de 2011

38.1 Cues de prioritats

Expliqueu (molt breument!) quines operacions principals té una cua de prioritats, amb quina estructura de dades se sol implementar i quin és el cost de les operacions amb aquesta estructura de dades.

38.2 Anàlisi

a) Analitzeu el cost de la funció següent:

```
int intrigant (int n) {
    int k = 1 + n%2;
    int x = 0;
    for (int i = 1; i ≤ n; i += k)
        ++x;
    for (int j = 1; j ≤ n; ++j)
        for (int i = 1; i*i ≤ n; ++i)
            ++x;
    for (int j = 1; j ≤ n; ++j)
        for (int i = 1; i ≤ n; i += i)
            ++x;
    return x;
}
```

b) Analitzeu el cost de la funció següent:

```
int confus (int n) {
    if (n == 0) return 2;
    else return n*n*intrigant(n) + confus(n/2);
}
```

38.3 NP-completesa

Considereu els dos problemes següents:

- a) Donats n objectes amb pesos $p_1, \dots, p_n$ i una alforja amb capacitat suficient, cal decidir si es poden repartir equitativament els objectes en les dues bosses de l'alforja.
 Recordeu que una alforja és un sac obert pel mig i tancat pels caps, els quals formen dues bosses grosses, ordinàriament quadrangulars. Aleshores, es demana si es poden repartir tots els objectes entre les dues bosses de manera que cada objecte s'ha de ficar en una bossa i les dues bosses porten exactament la mateixa càrrega.
- b) Donats n objectes amb pesos $p_1, \dots, p_n$ i una motxilla amb capacitat C , cal decidir si amb exactament tres objectes es pot carregar la motxilla completament (és a dir, si la suma dels pesos dels tres objectes és C).

Sota la hipòtesi $P \neq NP$, digueu raonadament, per a cadascun d'aquests problemes, si pertany o no a la classe P .

38.4 Indecidibilitat

El problema de l'aturada consisteix a determinar si, donat un programa i una entrada, aquest programa s'atura o no amb aquesta entrada.

El problema de l'aturada total consisteix en determinar si, donat un programa, aquest programa s'atura o no sobre totes les entrades.

Sabent que el problema de l'aturada és indecidible, què podeu dir del problema de l'aturada total?

38.5 Aproximació

El problema *Minimum makespan scheduling* (MMS) és un problema **NP**-difícil que es formula així:

Donats els temps de procés $p_1, \dots, p_n$ de n treballs, i m màquines idèntiques, cal trobar una assignació dels n treballs a les m màquines de forma que el temps de finalització de tots els treballs sigui mínim.

- a) Sigui OPT el temps de finalització mínim. Demostreu les fites inferiors següents:

- $\max_{i \in 1..n} p_i \leq OPT$.
- $\frac{1}{m} \sum_{i \in 1..n} p_i \leq OPT$.

Considereu l'algorisme següent:

1. Ordenem els treballs arbitràriament.
2. Assignem els treballs a les màquines en aquest ordre, tot assignant el següent treball a la màquina que ha rebut menys quantitat de treball fins al moment.

b) Considereu una assignació obtinguda a través d'aquest algorisme. Sigui t un instant de temps on totes les màquines estan ocupades. Demostreu que

$$t \leq \frac{1}{m} \sum_{i \in 1..n} p_i.$$

c) Demostreu que l'algorisme donat és un algorisme d'aproximació de factor 2 per al problema del MMS.

38.6 Grafs

Considereu l'algorisme de Kruskal per trobar un arbre generador mínim d'un graf no dirigit amb pesos positius a les arestes. Suposeu que els pesos són tots diferents, i que les arestes ja venen ordenades per pes. Sabent que el graf és complet (és a dir, té n vèrtexos i $n(n-1)/2$ arestes), quin cost tindria una execució en el cas pitjor? Quina operació domina el cost i quin cost té? I en el cas millor? Mostreu un exemple de cada amb $n = 5$.

38.7 Arbres

Usant la definició

```
struct Node {
    Node* esq;
    Node* dre;
};

typedef Node* Punter;
```

feu una funció **int** *distancia_mes_curta* (Punter p); que, donat un punter p a l'arrel d'un arbre, en retorni la distància més curta (mesurada en nombre de nodes) fins a una fulla. Per exemple, un arbre amb un sol node té distància 1. Escriviu la funció recursivament.

Quin cost té la vostra funció?

Sense escriure codi, expliqueu com es podria calcular millor la distància, visitant només els nivells imprescindibles de l'arbre. La vostra nova solució pot ser recursiva o iterativa. Quin cost tindria aquesta nova funció en el cas millor?

38.8 Més arbres

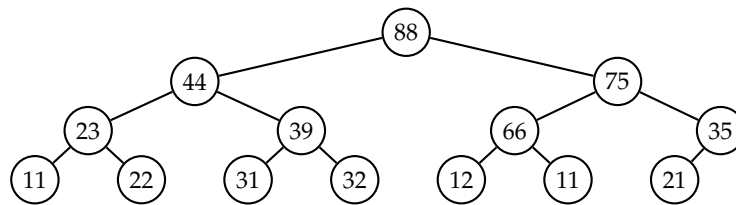
Considereu els arbres binaris de cerca aleatoritzats, amb una modificació: Ara, a cada pas de les insercions s'usa sempre probabilitat 0.5 per decidir si el nou element es guarda en aquella posició o no. (Si s'arriba a un subarbre buit, l'element es guarda allà.)

Mostreu la distribució de probabilitats dels arbres obtinguda després de la inserció de tres elements. És igual a la dels arbres aleatoris?

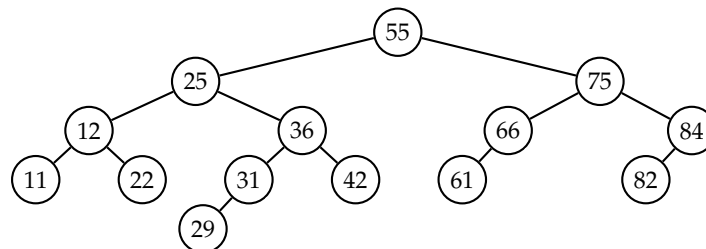
Final, 11 de gener de 2011

39.1 Heaps i AVLs

- a) Dibuixeu els dos heaps resultants d'afegir les claus 13 i 99 l'una després de l'altra en el heap següent:



- b) Dibuixeu els dos heaps resultants d'esborrar el màxim dos cops (l'un després de l'altre) del heap anterior (abans de fer les insercions).
- c) Dibuixeu els tres arbres AVL resultants d'afegir les claus 35, 34 i 40 l'una després de l'altra en l'arbre AVL següent:



39.2 Anàlisi

- a) Analitzeu el cost de la funció següent:

```

int misteri (int n) {
    int x = 0;
    for (int i = 1; i ≤ n; i += 2)
        ++x;
    for (int i = 1; i ≤ n; ++i)
        for (int j = 1; j ≤ i; ++j)
            for (int k = 1; k ≤ j; ++k)
                ++x;
    for (int i = n; i ≥ 1; i /= 2)
        for (int j = 1; j ≤ n; j++)
            ++x;
    return x;
}

```

b) Analitzeu el cost de la funció següent:

```

int rebuscat (int n) {
    if (n == 0) return 42;
    else return misteri(n) + 66*rebuscat(n - 1);
}

```

39.3 NP-completesa

Considereu els dos problemes següents:

- 3-COLORABILITAT: Donat un graf, es poden pintar els seus vèrtexs usant com a molt 3 colors diferents de manera que no hi hagi dos vèrtexs veïns del mateix color?
- 2-COLORABILITAT: Donat un graf, es poden pintar els seus vèrtexs usant com a molt 2 colors diferents de manera que no hi hagi dos vèrtexs veïns del mateix color?

Sabent que 3-COLORABILITAT és un problema **NP-complet**, responeu de forma breu però raonada si cadascuna de les afirmacions següents és certa, és falsa, o si no es pot assegurar ni una cosa ni l'altra:

- a) 2-COLORABILITAT es pot reduir polinòmicament a 3-COLORABILITAT.
- b) 3-COLORABILITAT es pot reduir polinòmicament a 2-COLORABILITAT.

39.4 Indecidibilitat

El darrer teorema de Fermat, afirma que l'equació diofàntica $x^n + y^n = z^n$ no té cap solució entera per a $n > 2$, essent x , y i z diferents de zero.

Aquest és un dels teoremes més famosos de la història de les matemàtiques, i fins l'any 1995 no se'n va trobar una demostració. El matemàtic francès Pierre de Fermat fou el primer a proposar el teorema, però malauradament la demostració que suposadament havia realitzat no s'ha trobat mai. Fermat només va deixar escrit en un marge de la seva còpia de l'Aritmètica de Diofant el plantejament del teorema i l'afirmació que havia trobat una demostració del teorema. En les seves pròpies paraules:

Cubum autem in duos cubos, aut quadrato-quadratum in duos quadrato-quadratos, et generaliter nullam in infinitum ultra quadratum potestatem in duos eiusdem nominis fas est dividere cuius rei demonstrationem mirabilem sane detexi. Hanc marginis exiguitas non caperet.

Mostreu (no cal que useu un estret marge de pàgina) com reduir el problema de Fermat al problema de l'aturada.

39.5 Aproximació

Recordeu el problema de VERTEX COVER (que en versió decisional és **NP**-complet):

Donat un graf connex $G = (V, E)$, cal determinar un subconjunt $S \subseteq V$ de talla mínima tal que tota aresta $e \in E$ contingui almenys un element de S .

Considereu l'algorisme següent:

1. Executem un recorregut en profunditat (DFS) en el graf G .
 2. Retornem els vèrtexs que no són les fulles de l'arbre de DFS (incloent l'arrel).
- a) Demostreu que la sortida d'aquest algorisme és realment un cobriment de vèrtexs.
- b) Demostreu que l'algorisme proposat és un algorisme d'aproximació de factor 2 per al problema del VERTEX COVER.

39.6 Grafs

Sigui G un graf dirigit amb costs als arcs. Considereu l'algorisme de Dijkstra per trobar el cost mínim d'anar des d'un vèrtex x a cadascun dels altres vèrtexs. Supposeu que G té arcs amb cost negatiu, però que no té cap cicle amb cost negatiu.

- a) Mostreu un exemple on l'algorisme falli.
- b) Sigui $-c$ el cost negatiu més gran en valor absolut dels arcs de G . Supposeu que sumem c a totes els arcs, per fer-los no-negatius. Sigui G' el graf resultant. Justifiqueu si aplicar l'algorisme de Dijkstra a G' ens permet trobar els costs mínims de G .

Les vostres respostes han de ser prou detallades perquè sigui evident que sabeu com funciona l'algorisme de Dijkstra.

39.7 Llistes

Considereu una llista d'enters ordenada de forma estrictament creixent. Usant la definició

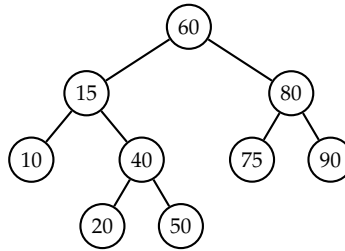
```
struct Node {
    int elem;
    Node* seguent;
};
```

```
typedef Node* Punter;
```

feu una funció `Punter esborra(int x, Punter p)`; que, donats un enter x i una llista identificada amb un punter p al seu primer node, retorni la llista un cop se li ha esborrat el node que conté x . Si x no hi és, cal retornar la llista tal qual. Podeu escriure la funció recursivament o iterativament.

39.8 Arbres

Considereu els arbres binaris de cerca aleatoritzats. Supposeu que en un determinat moment l'arbre emmagatzemat és



Mostreu quins arbres es poden obtenir, i amb quina probabilitat, quan a l'arbre anterior se li insereix un 30 amb l'algorisme aleatoritzat.

Parcial, 28 d'octubre de 2010

40.1 Ompliu els blancs

Ompliu els blancs de la forma més curta i precisa possible.

- Definiu $O(f)$:

- El teoreme mestre de resolució de recurrències divisores afirma que si tenim una recurrència de la forma $T(n) = aT(n/b) + \Theta(n^k)$ amb $b > 1$ i $k \geq 0$, llavors, fent $\alpha = \log_b a$,

$$T(n) = \begin{cases} \text{ } & \text{si } \alpha < k, \\ \text{ } & \text{si } \alpha = k, \\ \text{ } & \text{si } \alpha > k. \end{cases}$$

- Si $T(n) = 3T(n/2) + n^2 - 2n + 1$, llavors $T(n) = \Theta(\text{ })$.
- Si $T(n) = 2T(n-1) + n^2 - 2n + 1$, llavors $T(n) = \Theta(\text{ })$.
- Mergesort ordena n elements en temps en el cas pitjor, en temps en el cas mitjà, i en temps en el cas millor. L'espai auxiliar requerit és .
- Quicksort ordena n elements en temps en el cas pitjor, en temps en el cas mitjà, i en temps en el cas millor.
- Insertionsort ordena n elements en temps en el cas pitjor, en temps en el cas mitjà, i en temps en el cas millor.

- Per multiplicar dos nombres naturals grans eficientment podem fer servir l'algorisme de .
- Tot algorisme per trobar la mediana de n elements necessita passos.
- Un algorisme d'ordenació és estable si

40.2 Ordenació

Escriviu en C++ un procediment de cost $O(n)$ que ordeni una taula de n enters sabent que tots ells es troben entre 0 i nK (K és algun valor constant conegut). Quin és l'espai requerit pel vostre algorisme?

Expliqueu perquè aquest algorisme no contradiu el fet de que qualsevol algorisme de propòsit general necessita $\Omega(n \log n)$ passos per ordenar n valors.

40.3 Dividir i vèncer

En una habitació fosca tenim n cargols, tots d'un ample diferent, i les seves n femelles corresponents. Volem aparellar cada cargol amb la seva femella corresponent, però a causa de la foscor no es poden comparar els cargols amb els cargols ni les femelles amb les femelles. L'única comparació possible és intentar cargolar un cargol a una femella per comprovar si és massa gran, si és massa petit o si encaixa perfectament.



Descriviu un algorisme per resoldre aquest problema en $\Theta(n \log n)$ passos en el cas mitjà.

40.4 Primalitat

Expliqueu molt breument l'algorisme clàssic que, donat un natural x , determina si x és un nombre primer o no. Quin és el cost del vostre algorisme en funció de x ? Expliqueu perquè, malgrat les aparències, aquest algorisme és de cost exponencial. Podem doncs concloure que determinar si un nombre és primer és un problema intractable?

(Recordeu que es diu que un problema és intractable si no existeix cap algorisme de cost polinòmic per resoldre'l.)

40.5 Producte de matrius

L'algorisme de Strassen és un algorisme de dividir-vèncer per multiplicar dues matrius $n \times n$ en $O(n^{2.81})$ passos. La idea clau és que dues matrius 2×2 es poden multiplicar usant només set multiplicacions enlloc de les vuit usuals, i que aquesta idea es pot adaptar recursivament.

- a) L'algorisme de Strassen presentat a classe assumeix que n és una potència de 2. Com modificariem l'algorisme quan n no sigui necessàriament una potència de 2 tot mantenint el seu cost asimptòtic?

- b) Supposeu que trobéssim una variant de l'algorisme de Strassen basat en el fet que podem multiplicar matrius 3×3 utilitzant m multiplicacions enlloc de les 27 usuals. Com de petit hauria de ser m per tal que aquest algorisme sigui més eficient que l'algorisme de Strassen?
- c) L'algorisme de Coppersmith i Winograd multiplica matrius $n \times n$ en $O(n^{2'376})$ passos. Com de petit hauria de ser m per tal de millorar aquest cost?

Part II

Soluciones

Parcial, 8 de novembre de 2024

1.1 Com sempre, un misteri

(a) La funció `misteri` ordena el vector v en ordre creixent. La correctesa es pot justificar per inducció sobre la mida n del vector v . Quan $n \leq 3$ el vector s'ordena trivialment amb les primeres línies del codi. Suposant que `misteri` ordena correctament els vectors de mida inferior a n , després de la crida `misteri(v, m, d)` el subvector de $v[m..d]$ està ordenat correctament. En particular, tots els elements en les posicions de m a j són més petits o iguals que els elements en les posicions de $j + 1$ a d . Després de la crida `misteri(v, i, j)`, els elements de $v[i..m]$ són més petits o iguals que els de $v[m + 1..d]$. Per tant, després de la crida `misteri(v, e, m)` els elements més petits de v ja estaran col·locats correctament a les posicions de $v[e..i - 1]$, perquè els elements de les posicions i a m eren més petits o iguals als de les posicions posteriors abans de la crida recursiva, i després de la crida recursiva els elements de $v[e..i - 1]$ seran més petits o iguals que els de $v[i..m]$ i, conseqüentment, són els més petits de v . L'última crida recursiva ordena la part complementària del vector v i, per tant, al final tot el vector està ordenat correctament.

(b) Sigui $T(n)$ el cost d'ordenar un vector d' n elements amb `misteri`. Com que es fan tres crides recursives a subproblemes de mida $n/2$ i una crida recursiva a un subproblema de mida $3n/4$, tenim que

$$T(n) = 3T(n/2) + T(3n/4),$$

i llavors

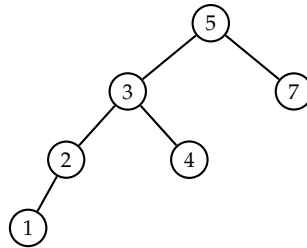
$$4T(n/2) \leq T(n) \leq 4T(3n/4).$$

Fent servir el teorema mestre dues vegades, obtenim $T(n) = \Omega(n^{\log_2 4}) = \Omega(n^2)$ i també $T(n) = O(n^{\log_{4/3} 4}) = O(n^{4.82})$.

(c) De tots els algorismes d'ordenació estudiats a classe, els que tenen el cost més alt en el cas pitjor són $\Theta(n^2)$. Per tant, podem dir que `misteri` és molt ineficient.

1.2 Internal Path Length

(a) L'IPL de l'arbre binari de cerca de la figura és $2 \cdot 1 + 2 \cdot 2 + 3 = 9$, perquè hi ha 2 nodes a distància 1 de l'arrel, 2 a distància 2 i 1 a distància 3.



(b) Observem primer que l'IPL de qualsevol arbre es pot calcular recursivament sumant $n - 1$ a la suma dels IPLs dels subarbres esquerra i dret de l'arbre original, perquè per a tot node x del subarbre esquerra de l'arrel es compleix que la distància d' x a l'arrel es pot calcular sumant un a la distància d' x a l'arrel del subarbre esquerra (i similarment amb el subarbre dret).

Llavors, per a un arbre complet d' n nodes, tenim $C(n) \simeq 2C(n/2) + n - 1$ i, per tant, $C(n) = \Theta(n \log n)$. L'IPL d'un arbre quasi complet d'alçada h està fitat per dalt per l'IPL de l'arbre complet d'alçada h i per baix per l'IPL de l'arbre complet d'alçada $h - 1$. Com que el nombre de nodes de l'arbre complet d'alçada $h - 1$ és la meitat del d'alçada h , l'IPL d'un arbre quasi complet de n nodes també és $\Theta(n \log n)$.

(c) Considerem ara un arbre binari de cerca de mida $n > 0$ aleatori, és a dir, construït a partir d'una permutació aleatòria d' n claus diferents. Com que la probabilitat que la clau $(i + 1)$ -èsima es trobi a l'arrel de l'arbre és $1/n$, tenim

$$P(n) = n - 1 + \frac{1}{n} \sum_{i=0}^{n-1} (P(i) + P(n - i - 1)) = n - 1 + \frac{2}{n} \sum_{i=0}^{n-1} P(i).$$

Aquesta recurrència només difereix en una constant del terme no recursiu de la recurrència del quick-sort en cas mitjà, així que asimptòtica té la mateixa solució, és a dir, $\Theta(n \log n)$.

(d) Això implica que l'alçada esperada d'un arbre binari aleatori és $\Theta(\log n)$, la qual coincideix amb el cost esperat de les operacions de cerca, inserció i esborrat en aquest tipus d'arbres.

1.3 Comptant i ordenant

(a) Ordenem el vector amb heapsort i després, en temps $\Theta(n)$ addicional, recorrem el vector sumant u a un comptador d'elements diferents cada vegada que veiem un element diferent de l'anterior. El cost en temps en el cas pitjor és $\Theta(n \log n)$. El cost en espai addicional en el cas pitjor és $\Theta(1)$ si es fa servir el propi vector com a heap.

[Tant mergesort com quicksort tindrien cost superior a constant en espai addicional.]

Per obtenir un algorisme eficient per al cas mitjà, muntem una taula de hash on anem introduint els elements del vector, sense més informació associada a cada element que el fet que ja l'hem vist. Mantenim un comptador d'elements diferents que incrementem cada vegada que inserim un element que encara no era a la taula de hash. El temps esperat és $\Theta(n)$ comptant que calcular el hash d'un enter té cost $O(1)$. L'espai addicional és $\Theta(d)$ si redimensionem la taula de hash duplicant-la en mida cada vegada que el factor de càrrega superi 1, o $\Theta(n)$ si ja d'entrada la fem de mida n .

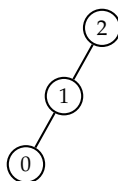
[En aquest cas, fer servir el mateix algorisme que el del cas pitjor podria ser, a efectes pràctics, competitiu amb aquest, especialment si es fa amb espai addicional $\Theta(1)$.]

(b) Muntem una taula de hash de mida d on anem introduint els elements del vector V . La informació associada a un element serà el nombre de vegades que l'hem vist. Al mateix temps, muntem un vector $U[1, \dots, m]$ on anem afegint-hi els elements nous que anem trobant. En acabar, ordenem U amb un algorisme d'ordenació estàndard, i construïm la versió ordenada de V , així: recorrem U d'esquerra a dreta, busquem a la taula de hash quantes vegades ha sortit l'element $U[i]$, i copiem $U[i]$ a V aquest número de vegades a partir de l'última posició de V que haguem visitat. Temps esperat: $\Theta(n) + \Theta(d \log d)$ comptant que calcular el hash d'un enter té cost $O(1)$. Espai addicional: $\Theta(d)$.

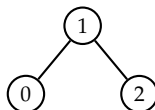
[Una alternativa subòptima seria mantenir un diccionari ordenat on les claus són els elements vistos i la informació associada a cada clau és quantes vegades l'hem vist. Així s'aconseguiria cost en temps $\Theta(n \log d)$ i en espai $\Theta(d)$. Quan $d \ll n$, això millora, en temps, un algorisme d'ordenació estàndard.]

1.4 Arbres binaris de cerca òptims

(a) Per exemple, amb $n = 3$, $p_0 = 0.25$, $p_1 = 0.35$, i $p_2 = 0.4$, l'ABC



té un cost esperat de $0.25 \cdot 3 + 0.35 \cdot 2 + 0.4 \cdot 1 = 1.85$, mentre que l'ABC



té un cost esperat de només $0.25 \cdot 2 + 0.35 \cdot 1 + 0.4 \cdot 2 = 1.65$. Així doncs, posar recursivament la clau amb la màxima probabilitat a l'arrel de cada subarbre no sempre produeix un arbre total òptim.

(b) Com a casos base, tenim $f(x, x - 1) = 0$ i $f(x, x) = 1$.

Sigui $p_{e,d} = \sum_{e \leq i \leq d} p_i$. Si fem servir "prefix sums", per exemple, cada $p_{e,d}$ es pot calcular amb cost constant. Per al cas general, cada possible arrel $a \in [e, d]$ generaria un arbre amb cost esperat $1 + p_{e,a-1}/p_{e,d} \cdot f(e, a - 1) + p_{a+1,d}/p_{e,d} \cdot f(a + 1, d)$. Cal comprovar totes les arrels i quedar-nos amb la que doni el cost mínim.

El cost en espai és clarament $\Theta(n^2)$, proporcional al nombre de parells (e, d) per als quals cal emmagatzemar $f(e, d)$.

El cost en temps és proporcional al nombre d'arrels que cal provar, és a dir, al sumatori $S = \sum_{0 \leq e < n} \sum_{e \leq d < n} d - e + 1$. La suma inclou els n casos amb $e = d$ per simplificar lleugerament els càlculs, aprofitant que el resultat final no canvia. Per començar, tenint en compte $d - e + 1 = O(n)$, podem afitar S trivialment per $O(n^3)$.

Per concloure que, de fet, $S = \Theta(n^3)$, amb algunes manipulacions una mica tedioses podem obtenir la fórmula tancada $S = n(n^2 + 3n + 2)/6$.

Alternativament, podem observar que en S l'expressió $d - e + 1$ val n una vegada, val $n - 1$ dues vegades, $\dots$, i val u n vegades. Així doncs,

$$S = \sum_{1 \leq m \leq n} (n + 1 - m) \cdot m \geq \sum_{n/3 \leq m \leq 2n/3} (n + 1 - m) \cdot m \geq \sum_{n/3 \leq m \leq 2n/3} n/3 \cdot n/3 \geq n^3/27.$$

(c) Per justificar intuïtivament la desigualtat $a_{e,d-1} \leq a_{e,d}$, suposem que afegim una clau amb una probabilitat insignificant a la dreta de $(e, d - 1)$, ajustant totes les probabilitats de forma proporcional perquè segueixin sumant u . En aquest cas, l'arbre òptim serà el mateix amb la nova clau a la dreta, al nivell més baix possible, i l'arrel òptima seguirà sent $a_{e,d-1}$. A la inversa, si la nova clau té una probabilitat propera a u , amb la resta de probabilitats properes a zero, l'arbre òptim tindrà la nova clau a l'arrel, amb l'anterior arbre òptim penjant a la seva esquerra. En aquest cas tindriem $a_{e,d} = d > a_{e,d-1}$. En general, com més gran sigui la probabilitat de la nova clau, més a la dreta estarà l'arrel òptima. En tot cas, no tindria sentit que afegir pes per la banda dreta suposés que l'arrel que d'alguna manera equilibra els pesos a esquerra i dreta es mogué en l'altre sentit.

El cas $a_{e,d} \leq a_{e+1,d}$ és simètric.

(d) En lloc de buscar l'arrel òptima per a cada $f(e, d)$ entre e i d , n'hi ha prou amb fer-ho entre $a_{e,d-1}$ i $a_{e+1,d}$. El cost ara és proporcional a

$$S' = \sum_{0 \leq e < n-1} \sum_{e < d < n} a_{e+1,d} - a_{e,d-1} + 1.$$

Anem a demostrar que la majoria de termes es cancel·len entre si, i que $S' = \Theta(n^2)$. Si definim

$$S_1 = \sum_{0 \leq e < n-1} \sum_{e < d < n} a_{e+1,d}, \quad S_2 = \sum_{0 \leq e < n-1} \sum_{e < d < n} a_{e,d-1}, \quad S_3 = \sum_{0 \leq e < n-1} \sum_{e < d < n} 1,$$

tenim que $S' = S_1 - S_2 + S_3$. Per començar, la suma S_3 compta el nombre de parells (e, d) , el qual sabem que és $\Theta(n^2)$. Respecte S_1 , fem el canvi de variable $e' = e + 1$, i separem els termes amb $d = n - 1$ de la resta:

$$S_1 = \sum_{1 \leq e' < n} \sum_{e' \leq d < n} a_{e',d} = \sum_{1 \leq e' < n} \sum_{e' \leq d < n-1} a_{e',d} + \sum_{1 \leq e' < n} a_{e',n-1}.$$

Amb S_2 , fem el canvi $d' = d - 1$, i ignorem els termes amb $e = 0$:

$$S_2 = \sum_{0 \leq e < n-1} \sum_{e \leq d' < n-1} a_{e,d'} \geq \sum_{1 \leq e < n-1} \sum_{e \leq d' < n-1} a_{e,d'}.$$

Tenint en compte que aquest últim sumatori és igual al primer de S_1 , posant-ho tot junt obtenim $S' \leq \sum_{1 \leq e < n} a_{e,n-1} + \Theta(n^2) = \Theta(n^2)$, perquè el sumatori té menys d' n termes, cadascun afegit per n .

Extraordinari, 4 de juliol de 2024

2.1 Nombres pràctics

Sabem que el problema de l'aturada és incomputable, és a dir, que no pot existir un programa A tal que, per a qualsevol programa B (sense entrada), A determini en un nombre finit de passos si B s'aturarà.

En Pepet no pot tenir raó. Demostrem-ho per contradicció. Suposem que X existís. Sigui B un programa per al qual volem saber si s'atura. Com a primer pas, li esborrem totes les instruccions d'escriptura per pantalla (els `cout`). Després, li afegim

```
while(true) cout << 1;
```

com a última línia. Anomenem Z al programa que, donat B , retorna el programa resultat de fer-li aquestes dues modificacions. El programa B s'atura si i només si el programa X ens diu que $Z(B)$ és un nombre pràctic. Per tant, si X existís podríem construir $A = X \circ Z$, el qual sabem que no existeix.

2.2 Elements freqüents

N'hi ha prou de fer servir un `unordered_map<string, int>` per comptar quantes vegades apareix cada paraula. Després, recorrem la taula de dispersió buscant les paraules amb el comptador prou gran.

Si la funció de dispersió està ben feta, el cost de calcular-la serà $\Theta(m)$. En el cas mitjà, cada inserció consisteix a calcular la funció de dispersió, més comparar la paraula amb un nombre constant de paraules. Cadascuna d'aquestes comparacions costa $O(m)$. Com que cada inserció costa $\Theta(m)$, en el cas mitjà el cost és $\Theta(nm)$.

En el cas pitjor, totes les paraules van a parar a la mateixa posició, i per veure que són diferents cal comparar $\Theta(m)$ caràcters cada vegada. Com que el nombre de comparacions entre paraules és $\Theta(n^2)$, el cost total en aquest cas és $\Theta(mn^2)$.

Per aconseguir cost $\Theta(mn)$ en el cas pitjor, fem servir l'algorisme de selecció que té cost linial (en n) en el cas pitjor. Busquem la mediana, el primer quartil, i el tercer quartil, que

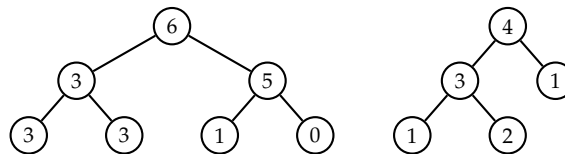
són els únics possibles candidats, i comptem quantes vegades apareixen. Això suposa $\Theta(n)$ comparacions de paraules tres vegades. Com que cada comparació costa $\Theta(m)$ en el cas pitjor, aconseguim el cost total desitjat.

Alternativament, podem ordenar totes les paraules fent servir Radix-Sort, el qual té cost $\Theta(mn)$ en el cas pitjor. Després, només cal mirar quines paraules tenen el nombre de repeticions demanades.

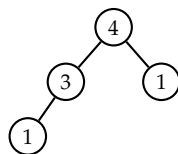
2.3 Heaps

Primer esborrem l'últim element de B amb cost $\Theta(1)$. Anomenem B' al heap resultant, i a a l'element esborrat. B' tindrà entre $2^{(x-1)} - 1$ i $2^x - 1$ elements. Després, posem a com arrel, A com a fill esquerre, i B' com a fill dret. Fixem-nos que les mides dels dos germans són correctes. Finalment, el que cal assegurar és que a quedi ben posada en el resultat, enfonsant-la, amb cost $\Theta(x)$ en el cas pitjor.

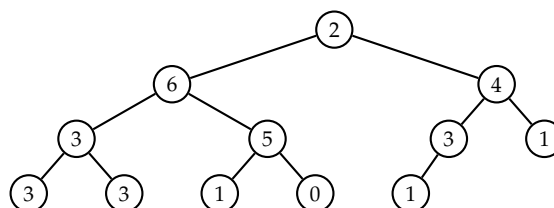
Per exemple, suposem que A i B són els max-heaps següents:



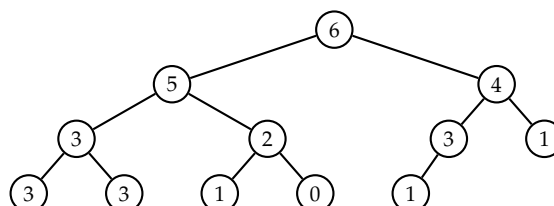
Esborrant l'últim element (2) de B , obtenim aquest B' :



Ara construïm un max-heap amb tots els elements, on només l'arrel pot estar malament:



Després d'enfonsar l'arrel, el resultat és un max-heap:



2.4 Element únic

Sigui $p(x)$ la posició de la primera aparició a V de cada element x . A l'exemple, $p(4) = 0$, $p(2) = 2$, $p(3) = 4$, $p(7) = 6$, $p(1) = 7$ i $p(8) = 9$. Hem de trobar l'últim element u tal que $p(u)$ sigui parell. Podem fer una mena de cerca binària: Sigui x l'element que estem veient. Si $p(x)$ és senar, ens movem cap a l'esquerra, i si $p(x)$ és parell, cap a la dreta. Per saber el valor de $p(x)$ és suficient amb comprovar els valors de V al voltant de la posició actual, vigilant de no sortir fora del vector. Com a cas base, podem aturar-nos quan el tros on busquem té un únic element, o quan veiem un element rodejat d'elements diferents. El cost en el cas pitjor és $\Theta(\log n)$, perquè fem com a màxim uns $\log_2 n$ passos, cadascun de cost constant.

2.5 Teoria de jocs

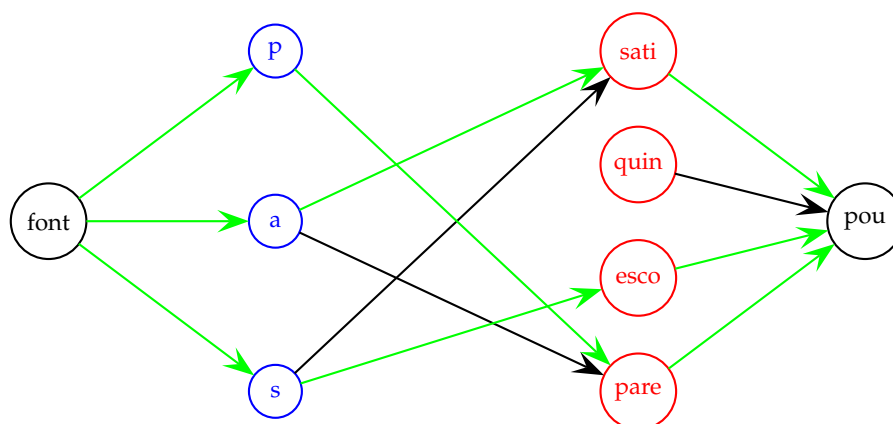
El número d'un rectangle buit és 0. Per als rectangles no buits, el seu número és el mínim número al qual no podem arribar. Calculant-los en ordre, veiem que $\text{nim}(1 \times 1) = 1$ (podem anar a 0), $\text{nim}(1 \times 2) = 2$ (podem anar a 0 i 1), $\text{nim}(1 \times 3) = 3$ (podem anar a 0, 1 i 2), $\text{nim}(1 \times 4) = 4$ (podem anar a 0, 1, 2 i 3), $\text{nim}(2 \times 2) = 1$ (podem anar a 0 i 2), $\text{nim}(2 \times 3) = 4$ (podem anar a 0, 1, 2 i 3), i $\text{nim}(2 \times 4) = 3$ (podem anar a 0, 1, 2 i 4).

El número de dos rectangles junts és el "or exclusiu" dels números dels rectangles. Com que ambdós rectangles tenen número 3, el número resultant és 0, i per tant la posició és perdedora.

2.6 Daus amb lletres

Es pot resoldre amb un max-flow, en particular, calculant un aparellament màxim: A una banda posem un vèrtex per a cada lletra d's. A l'altra, un vèrtex per a cada dau. Connectem cada lletra c amb cada dau d amb un arc si i només si d conté c . Ara calculem el flux màxim. El problema té solució si i només si el flux màxim és m .

Amb l'exemple de l'enunciat, tindríem (en verd es marquen els arcs per on passa flux):



2.7 Complexitat

Com que el problema té solució polinòmica, i $P \subseteq NP$, és immediat que el problema pertany a NP. Alternativament, és fàcil veure que hi ha testimonis de mida polinòmica que es poden comprovar en temps polinòmic quan el problema té solució positiva.

No podem afirmar que el problema sigui NP-difícil, perquè no es coneix cap reducció d'un problema NP-difícil a aquest. De fet, és "improbable" que existeixi, per l'apartat següent.

Si el problema fos NP-difícil, com que pertany a P , tindríem que $P = NP$, cosa que no està demostrada, i que la majoria d'experts de l'àrea consideren poc creïble.

Final, 16 de gener de 2024

3.1 NP-completesa

La funció $O(n^k)$ és polinòmica en n , però és exponencial en k . Com que k forma part de l'entrada, i en particular pot ser de l'ordre d' n , l'algorisme proposat no té cost d'execució polinòmic. Si el tinguèssim, com que el problema CLIQUE és NP-complet (SAT se l'hi redueix, per exemple), tots els problemes de la classe NP es podrien resoldre en temps polinòmic, reduint-los en temps polinòmic a CLIQUE, cosa que implicaria $P = NP$.

3.2 Ordenant amb repeticions

Es pot crear un diccionari (un map, implementat per exemple amb un AVL) de parells d'enters tal que el primer element sigui un element de la seqüència d'entrada i el segon en sigui el nombre d'aparicions. S'insereix cada element de la seqüència al diccionari: si ja hi és, n'incrementem en un el nombre d'aparicions, altrament inserim un nou element amb una aparició. Com que hi ha $O(\log n)$ elements diferents, el cost de cada inserció és $O(\log(\log n))$, i se'n fan n , amb cost total $O(n \log(\log n))$. Amb un recorregut final en inordre de l'AVL s'obtenen en ordre els $O(\log n)$ elements diferents (amb aquest cost), i se'n sap per a cadascun el nombre d'aparicions. Finalment, en temps $\Theta(n)$ s'escriuen, en ordre i tants cops com estiguin repetits, els elements de la seqüència. El cost total d'aquest algorisme és $O(n \log(\log n))$.

Alternativament, podem crear un vector de parells d'enters per mantenir ordenats els elements diferents de la seqüència (primer enter del parell) amb el seu nombre d'aparicions (segon enter del parell). Repetim n vegades el procediment següent: per a cada element de la seqüència, es busca al vector amb cerca dicotòmica. Si l'element és al vector, se n'incrementa el comptador d'aparicions. Altrament, l'inserim (amb insertion sort, per exemple) al vector ordenat amb una aparició. Com que la mida del vector és $O(\log n)$, cada cerca dicotòmica té cost $O(\log(\log n))$, i mantenir-lo ordenat amb insertion sort té cost total $O(\log^2 n)$. El cost de tot l'algorisme és $O(n \log(\log n))$.

La fita inferior $\Omega(n \log n)$ és certa quan tenim n elements diferents, però en aquest cas només n'hi ha $O(\log n)$.

3.3 Càlcul eficient de recurrències

Una manera eficient de resoldre aquest problema donades les condicions de la màquina de la qual disposem seria observar que es compleix la relació següent:

$$\begin{pmatrix} F(n) \\ F(n-1) \\ F(n-2) \end{pmatrix} = \begin{pmatrix} a & b & c \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}^{n-2} \begin{pmatrix} F(n-1) \\ F(n-2) \\ F(n-3) \end{pmatrix}$$

Per tant, podem calcular amb exponenciació ràpida la potència $(n-2)$ -èsima de la matriu 3×3 , que té mida constant, i multiplicar-la al final pel vector (f_2, f_1, f_0) . El cost només és $\Theta(\log n)$.

3.4 Quad-trees

(a) Qualsevol combinació de claus amb què el quad-tree resultant tingui, a part de l'arrel, un element a cadascun dels quatre subarbrs és correcta. Per exemple, l'arrel amb $(2, 7)$, i essent els altres quatre punts $(0, 5)$, $(1, 8)$, $(3, 6)$, i $(4, 9)$.

(b) L'alçada mínima que pot tenir un quad-tree amb n nodes és bàsicament $\log_4 n$. Es pot aconseguir posant un punt a l'arrel que divideixi la resta de punts en quatre subconjunts amb aproximadament $n/4$ punts cadascun. Així doncs, el mínim cost en el cas pitjor d'una cerca sense èxit segueix la recurrència $C(n) = \Theta(1) + 4C(n/4) = \Theta(\log n)$.

(c) L'alçada màxima que pot tenir un quad-tree amb n nodes és n . Es pot aconseguir posant un punt a l'arrel que faci que la resta de punts vagin tots a un dels fills. Així doncs, el màxim cost en el cas pitjor d'una cerca sense èxit segueix la recurrència $C(n) = \Theta(1) + C(n-1) = \Theta(n)$.

3.5 Xarxa de carrers

La pregunta demana com saber si un graf dirigit consisteix en un únic component fortament connex. La manera més directa per resoldre aquest problema fa servir l'algorisme explicat a classe, el qual realitza dues cerques en profunditat, la primera sobre el graf invers, i la segona sobre el graf original, i comprovar que la primera crida del segon recorregut visita tots els vèrtexs. El cost és doncs només linial en el nombre de vèrtexs i arcs.

Un mètode encara més senzill i amb el mateix cost fa primer un recorregut en el graf original començant en un vèrtex qualsevol x . Si no es visiten tots els vèrtexs, la resposta és negativa. Altrament, comprovem si des de tots els vèrtexs podem arribar a x fent un altre recorregut començant en el mateix vèrtex x , però en el graf invers. La resposta és afirmativa si i només si aquest recorregut també visita tots els vèrtexs, perquè en aquest cas es pot anar des de qualsevol vèrtex fins a qualsevol altre vèrtex passant per x .

3.6 Diverses consultes

(a) L'algorisme fa servir un "rolling hash" per calcular eficientment un valor per a w , i també per a totes les subparaules de t de mida m . Per exemple, podem imaginar w com un nombre escrit en base 26, associant-li el valor $h(w) = \sum_{i=0}^{m-1} 26^{m-1-i} \cdot (w[i] - 'a')$. Associem un valor similar als primers m caràcters de t . Després, calculem la resta valors de hash restant el producte de 26^{m-1} pel primer dígit, multiplicant per 26, i sumant

L'últim dígit, cosa que només té cost $\Theta(1)$. Si els valors són prou petits i no hi ha sobreiximents, cada subparaula de t és igual a w si i només si els valors de hash coincideixen. Altrament, fem els càlculs mòdul un enter p gros (típicament primer, per exemple per si hem de calcular inversos), cosa que pot provocar falsos positius amb una probabilitat aproximada de $1/p$. Es pot reduir aquesta probabilitat tant com es vulgui fent servir diverses funcions de "rolling hash".

(b) N'hi ha prou amb calcular les "sumes prefixades" $P[i] = \sum_{j=0}^i V[j]$ per a tota i , incrementalment des de $i = 0$ fins a $i = n - 1$, amb cost linial. És fàcil veure que es compleix $S(e, d) = P[d] - P[e - 1]$, excepte per a $e = 0$, on tenim $S(0, d) = P[d]$. El cost total del precàlcul i de les n consultes només és $\Theta(n)$.

(c) Primer, calculem el valor de hash de tots els prefixos d's, similarment a com s'ha explicat a l'apartat (a). Després, decidim que una subparaula és un quadrat (amb alta probabilitat) si les seves dues meitats tenen el mateix valor de hash. Per calcular el valor de hash d'una subparaula podem fer inclusió-exclusió com a l'apartat (b), però multipliant un dels termes per la potència de 26 adequada. Perquè aquesta operació tingui cost constant n'hi ha prou amb precalcular i guardar les n primeres potències de 26.

3.7 THREE-DIMENSIONAL MATCHING

Com que THREE-DIMENSIONAL MATCHING és NP-difícil, qualsevol problema dins d'NP s'hi pot reduir polinòmicament. Així doncs, si es pugués resoldre en temps polinòmic, qualsevol problema dins de NP també es podria resoldre en temps polinòmic, i com a conseqüència P seria igual a NP. La majoria d'investigadors de l'àrea consideren aquesta possibilitat poc probable.

Aquest problema clarament pertany a NP. Existeix un testimoni de mida polinòmica: n triplets (s_i, v_j, b_k) amb la suposada solució. Verificar que ho és requereix comprovar que les s_i , les v_j i les b_k són permutacions de $\{0, \dots, n - 1\}$ (es pot fer amb cost linial) i també consultar n booleans a la matriu M . El cost total seria només $\Theta(n)$.

Sota les noves hipòtesis el problema es pot resoldre amb cost polinòmic. Cal adonar-nos que només ens demanen fer un matching màxim en dos grafs bipartits: els que defineixen les matrius A i B . Ambdós grafs tenen $\Theta(n)$ vèrtexs i $O(n^2)$ arcs. Com que el max-flow en la versió Edmons-Karp en aquest cas particular fa com a màxim n recorreguts, cadascun de cost $O(n^2)$, el problema es pot resoldre amb cost total $O(n^3)$.

Parcial, 3 de novembre de 2023

4.1 Quick Select

La implementació de *Quick_Select* és molt semblant a la de *Quick_Sort*. Igual que per a *Quick_Sort*, es pren un element del conjunt (potser el primer) com a pivot, i es particiona el conjunt en tres parts: la dels elements més petits que pivot, el pivot, i la dels elements més grans que el pivot (observant que tots els elements són diferents). La partició retorna l'índex p que ocuparia l'element pivot al conjunt si aquest estigués ordenat. Llavors, si p és igual a k , el pivot és l'element que es busca; si $k < p$, es continua recursivament cercant l'element k -èssim a la part esquerra del conjunt; finalment, si $k > p$, s'ha de buscar recursivament l'element $k - p$ -èssim a la part dreta del conjunt.

A diferència de *Quick_Sort*, *Quick_Select* fa com a molt una única crida recursiva. Si la crida es fa a un subproblema de mida $n - c$ (amb $c > 0$ constant), llavors l'algorisme té cost $\Theta(n^2)$ (és el cost en el cas pitjor). D'altra banda, si fem per exemple crides recursives a subproblemes de mida n/b (amb $b > 1$ constant), llavors l'algorisme té cost $\Theta(n)$. També, si fem crides a subproblemes de mida c (amb $c > 0$ constant). Aquest algorisme no és òptim en el cas pitjor perquè l'algorisme de selecció en temps lineal té (com el seu nom indica) un cost en el cas pitjor que només és lineal.

4.2 Paraules amb repeticions

(a) Determinar si B és una subseqüència d' A en temps $\Theta(n)$ és fàcil. Es pot recórrer A fins a trobar la primera ocurrència de b_1 si n'hi ha, continuar a partir de la posició següent fins a trobar b_2 , i així successivament. La resposta és afirmativa si i només si s'aconsegueixen trobar totes les b_i . La correctesa d'aquest algorisme es pot provar fàcilment per inducció.

(b) Per a cada valor fixat d' i , es pot fer servir l'algorisme anterior per determinar en temps $O(n)$ si B^i és una subseqüència d' A . A més, es pot observar que per a tot parell $0 \leq i < j$, si B^j és una subseqüència d' A , B^i també ho és. El valor màxim d' i que hem de considerar no pot excedir $d = n/m$, perquè llavors la longitud de B^i seria més gran que la d' A . Per tant, fem una cerca binària sobre els possibles valors d' i , sabent que B^0 és una subseqüència, i que B^{d+1} no ho és. Per exemple, a la primera iteració comprovaríem

si B^i (amb $i = n/(2m)$) és una subseqüència d' A . Continuariem amb la cerca binària eliminant el rang inferior si B^i si és subseqüència d' A , i el superior en cas contrari. Fan falta $\Theta(\log(n/m))$ tests per determinar la i màxima. El cost total en el cas pitjor és per tant $O(n \log(n/m))$.

4.3 Procediment recursiu (1)

En general, una crida a $f(0, V.size() - 1, x, V)$ retorna l'última posició del vector on es podria inserir x mantenint el vector ordenat. Com a cas particular, retorna $V.size()$ si x és més gran que tots els elements de V . És el que es coneix com a *upper_bound* en C++.

Això es pot deduir observant els diversos casos del codi de la cerca dicotòmica. Per exemple, si $V = [2, 2, 2, 3, 5, 5, 7, 7, 7, 9, 9]$, els intervals d'una cerca del 5 són $[0, 11]$, $[6, 11]$, $[6, 7]$, i en aquest moment $m = 6$ compleix el cas base de la recursió ($V[m] > 5$ i $V[m-1] \leq 5$).

Com que és una versió d'una cerca binària, és evident que el cost de la funció en el cas pitjor és $\Theta(\log n)$. Alternativament, és la solució de la recurrència $F(n) = \Theta(1) + F(n/2)$. El cost en el cas millor és $\Theta(1)$, perquè podem trobar la posició buscada amb una sola crida.

[Si haguéssim de calcular el cost mitjà, suposant que tots els elements són diferents i que en busquem qualsevol amb probabilitat $1/n$, es podria veure que és $\Theta(\log n)$, per exemple observant que la meitat de les cerques es realitzen a les fulles d'un arbre binari virtual quasi-perfecte, i que aquestes $\Theta(n)$ fulles esta a profunditat $\Theta(\log n)$.]

4.4 Parell de punts més propers en el pla

Abans que res, cal dir que l'ordenació prèvia té cost $\Theta(n \log n)$, cosa que no afectarà la resta de càlculs.

(a) La recurrència del cas pitjor és $F(n) = O(n^2) + 2F(n/2)$. Aplicant el teorema mestre per a les recurrències divisores, la solució és $F(n) = O(n^2)$.

(b) N'hi ha prou de fer una fusió (*merge*) de les dues llistes, amb cost $\Theta(n)$.

(c) És fàcil veure que ara la recurrència per al cas pitjor és $F(n) = \Theta(n) + O(n \cdot \sqrt{n}) + 2F(n/2) = O(n^{3/2}) + 2F(n/2)$, la solució de la qual és $F(n) = O(n^{3/2})$.

(d) Finalment, la recurrència per al cas pitjor és $F(n) = \Theta(n) + O(n \cdot 1) + 2F(n/2) = \Theta(n) + 2F(n/2)$, la solució de la qual és $F(n) = \Theta(n \log n)$.

4.5 Procediment recursiu (2)

Una crida a $f(0)$ escriu totes les seqüències d' n bits sense tres o més zeros seguits. Per fer aquest codi (molt) més eficient, caldria assegurar que no es posen tres zeros seguits en la creació de la seqüència (comprovant que $V[i-2] + V[i-1] + V[i] > 0$ per a tota $i \geq 2$), i no al final dels n passos.

El cost és clarament $O(n \cdot 2^n)$, perquè cal comprovar cadascuna de les 2^n seqüències, i cada comprovació requereix mirar $O(n)$ bits. Però el cost no és $\Theta(n \cdot 2^n)$, sinó $\Theta(2^n)$. En efecte, una de cada vuit seqüències comença amb tres zeros, per la qual cosa el bucle de comprovació només fa un pas. De les seqüències que no comencen amb tres zeros, una fracció té tres zeros a les posicions 1, 2 i 3. De les que no, una fracció té tres zeros a les

posicions 2, 3 i 4, i així successivament. És clar que, fent la mitjana de les 2^n seqüències, després d'un nombre esperat constant c de passos trobarem tres zeros i ens aturarem.

[Es podria calcular aquesta constant c exactament quan n tendeix a infinit, però no calia per obtenir la puntuació màxima d'aquest problema.]

Extraordinari, 5 de juliol de 2023

5.1 Subparaules diferents

Fem servir *rolling hashing* com en l'algorisme Karp-Rabin. Sigui $p[0..n-1]$ la paraula. Un punter $i = 0, \dots, n-1$ recorrerà p d'esquerra a dreta. Després de cada increment de i , inicialitzem un rolling hash buit. Un segon punter $j = i, \dots, n-1$ recorrerà el sufix $p[i..n-1]$ afegint el caràcter $p[j]$ al rolling hash actual. El rolling hash es calcularà per exemple en base 52 mòdul P , on P és un nombre primer prou gran. Al principi del recorregut del sufix, i també després de cada increment de j , accedirem a una taula d'ocurrències $T[0..P-1]$ per determinar si la paraula $p[i..j]$ ja ha estat vista i, en cas que no, incrementarem un comptador de paraules diferents.

El cost d'aquest algorisme és $\sum_{i=1}^n \sum_{j=i}^n \Theta(1) = \Theta(n^2)$, perquè afegir un caràcter a un rolling hash té cost constant. La probabilitat d'error, és a dir, la probabilitat que alguna paraula nova compti com a ja vista, és negligible, i encara més si fem aquest procés amb diversos nombres primers alhora. Finalment, per evitar gastar espai proporcional a P , es pot fer servir una taula hash.

5.2 Complexitat

(a) No. Si X es pogués resoldre en temps polinòmic, aleshores $P = EXP$, perquè X és EXP -complet, però és sabut que $P \neq EXP$.

(b) Sí. Com que $NP \subseteq EXP$, el problema Y pertany a EXP , i per tant es redueix a X perquè X és EXP -complet.

(c) Implicaria $NP = EXP$ i, en particular, $P \neq NP$, perquè $P \neq EXP$.

5.3 Codis de Huffman

a) Una codificació possible és a : 000, b : 0010, c : 1, d : 01, e : 0011. L'arbre uneix b amb e , el resultat amb a , el resultat amb d , i el resultat amb c . La longitud esperada d'un símbol és $3 \times 0.10 + 4 \times 0.05 + 1 \times 0.60 + 2 \times 0.20 + 4 \times 0.05 = 1.7$. Per tant, la longitud esperada de la codificació d'un missatge d' n símbols és $1.7n$.

b) Sigui C un codi instantani. El transformem per aconseguir el que diu el lema sense empitjorar la longitud mitjana: Siguin a i b els dos símbols amb freqüències més petites. Si el símbol a no està associat a una fulla de nivell 0 (i.e., de profunditat màxima), llavors l'intercanviem amb el símbol x d'alguna de les fulles a nivell 0, i només podem guanyar, perquè la diferència en la longitud mitjana entre abans i després és

$$(L(x) - L(a))(P(x) - P(a)) \geq 0,$$

on $L(x)$ denota la longitud de la codificació d' x , i $P(x)$ denota la freqüència d' x . Si el símbol a està associat a una fulla de nivell 0 però aquesta no té cap germà (a nivell 0), llavors podem la fulla, associem el símbol a al seu pare, que ara és una fulla, i només podem guanyar. Suposem doncs que el símbol a està associat a una fulla de nivell 0 i que aquesta té un germà (a nivell 0). Si el símbol b no està associat al germà de la fulla d' a , llavors l'intercanviem amb el símbol associat al germà de la fulla d' a i només podem, o bé guanyar (si b no estava a nivell 0), o bé quedar-nos igual (si b estava a nivell 0), i en aquest últim cas ja tenim el que volíem.

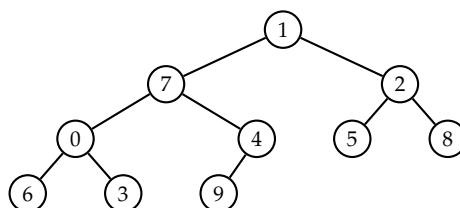
5.4 Teoria de jocs

Aquesta variant no es pot calcular amb nímbers. Una propietat imprescindible per poder aplicar la teoria associada és que, en ajuntar dues posicions, cap de les dues canviï d'estat. Però aquí, si tenim una posició on s'acaben de treure x pedres, i una altra posició on se n'acaben de treure y , amb $y \neq x$, i les ajuntem, almenys una de les dues posicions hauria de modificar-se (de x a y o a l'inrevés), perquè hi ha d'haver un únic nombre que limiti quantes pedres es poden treure de cada pila.

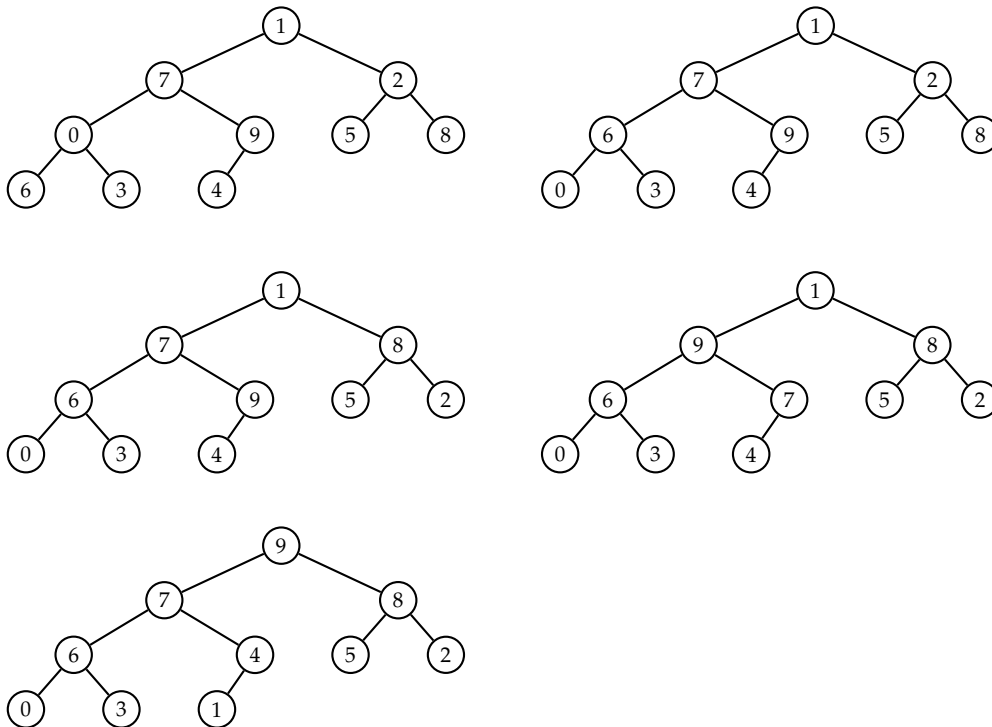
Es pot calcular si una posició és guanyadora implementant una programació dinàmica senzilla. L'estat està compost per un vector amb el nombre de pedres de cada pila, i un nombre que indica quantes pedres es poden treure com a màxim. Per evitar recalculat posicions que són bàsicament idèntiques, seria convenient mantenir el vector ordenat.

5.5 Heaps

Inicialment, tenim

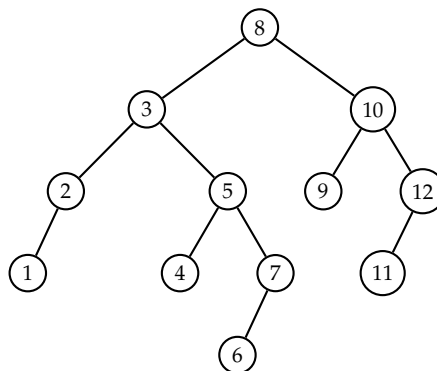


L'algorisme *heapify* enfonsa els elements del heap des de l'últim que no és una fulla fins al primer (l'arrel), és a dir, en aquest ordre: 4, 0, 2, 7 i 1. La seqüència d'arbres és:



5.6 Arbres AVL

Sabem que el mínim nombre d'elements per aconseguir un arbre AVL d'alçada h segueix aquesta recurrència: $M(0) = 0$, $M(1) = 1$, i $M(h) = 1 + M(h-1) + M(h-2)$ per a $h \geq 2$. Veiem que $M(5) = 12$, així que la clau 6 com a màxim podria estar a profunditat 5. Aquest exemple demostra que és possible:



5.7 Ordenació curiosa

El cost del nou algorisme segueix aquesta recurrència:

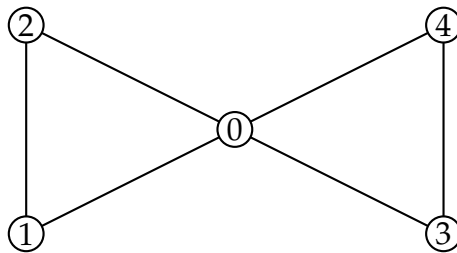
$$C(n) = C(n/2) + \Theta((n/2)^2) + \Theta(n) = \Theta(n^2) + C(n/2) .$$

Aplicant el teorema mestre, tenim $\log_2 1 = 0 < 2$, així que $C(n) = \Theta(n^2)$.

5.8 Punts d'articulació i arestes pont

a) Un punt d'articulació és un vèrtex que desconectaria G si l'esborréssim. Si $n \leq 2$, no hi pot haver cap punt d'articulació. Suposem ara $n \geq 3$, i que G és connex. Podem esborrar arestes dels cicles, sense desconectar G , perquè fent això cap punt d'articulació deixarà de ser-ho. Per tant, podem suposar que G és un arbre. En un arbre, els punts d'articulació són els nodes interns. Podem aconseguir fins a $n - 2$ nodes interns si G és una llista, la qual només té dues fulles. Finalment, és fàcil veure que tenint diversos components concrets amb n vèrtexs en total no aconseguiríem més de $n - 2$ punts d'articulació.

b) Sí que és possible. En aquest graf, el vèrtex 0 és un punt d'articulació, però no hi ha cap aresta tal que esborrant-la el graf es desconectaria:



Final, 10 de gener de 2023

6.1 Recorreguts d'un arbre binari

La primera resposta és afirmativa. Ho demostrem per inducció en la longitud de les seqüències. Si són buides, llavors A és l'arbre buit i no cal fer res més. Altrament, si x és el primer element de $p(A)$, llavors el que hi ha a l'esquerra d' x en el vector $i(A)$ és $i(E)$, i el que hi ha a la dreta és $i(D)$. Sabent això ja sabem els elements d' E i de D i, per tant, també podem extreure $p(E)$ i $p(D)$ de $p(A)$. Per hipòtesi d'inducció, $p(E)$ i $i(E)$ determinen E , i $p(D)$ i $i(D)$ determinen D . Per tant, $p(A)$ i $i(A)$ determinen A .

La segona resposta és negativa. Per exemple, si A és l'arbre de l'esquerra, i B és l'arbre de la dreta,



llavors $p(A) = p(B) = xy$ i $t(A) = t(B) = yx$, però $A \neq B$.

6.2 Cerques binàries ineficients

El primer cost és $\Theta(n \log n)$: En el cas pitjor es fan $\Theta(\log n)$ crides recursives, i cadascuna té cost $\Theta(n)$ perquè es copia el vector sencer cada vegada.

El segon cost és $\Theta(n)$. En aquest cas, si m denota la mida del vector de la crida recursiva en curs, llavors el cost de cada crida és $\Theta(m)$, i no $\Theta(n)$ com abans, i per tant en el cas pitjor el cost de la crida recursiva satisfà la recurrència $T(m) = T(m/2) + \Theta(m)$. La solució és $T(m) = \Theta(m)$ que, per a $m = n$, dóna $\Theta(n)$.

6.3 Servidor web

a) La resposta és $1 - (1 - 1/m)^n$. Per veure-ho, calculem la probabilitat que *no* sigui un fals positiu. Siguin $x_1, \dots, x_n$ les n adreces diferents ja tractades, i sigui x la següent adreça diferent. Perquè x no sigui un fals positiu cal i és suficient que $h(x_t) \neq h(x)$ per a tot $t = 1, \dots, n$. Per uniformitat de la variable $h(x_t)$, per a cada t fixat la probabilitat que $h(x_t) \neq h(x)$ és $1 - 1/m$. Per independència de totes les variables, la probabilitat que $h(x_t) \neq h(x)$ per a tot $t = 1, \dots, n$ és $(1 - 1/m)^n$. Per tant, la probabilitat que x sigui un fals positiu és $1 - (1 - 1/m)^n$.

[Una resposta mitjanament raonable però incorrecta és n/m , que seria una bona aproximació de $1 - (1 - 1/m)^n$ quan m és molt més gran que n , la qual cosa no podem assumir.]

b) La resposta és $1 - (1 - 2/m)^n$. Per veure-ho, calculem la probabilitat que $T[i] = 0$. Siguin $x_1, \dots, x_n$ les n adreces diferents ja tractades. Perquè $T[i] = 0$ cal i és suficient que $h_1(x_t) \neq i$ i $h_2(x_t) \neq i$ per a tot $t = 1, \dots, n$. Per uniformitat i independència de les variables $h_1(x_t)$ i $h_2(x_t)$, la probabilitat que $h_1(x_t) \neq i$ i $h_2(x_t) \neq i$ per a un $t \in [n]$ fixat és $(1 - 1/m)^2$. Per independència de totes les variables, la probabilitat que això passi per a tot $t \in [n]$ és $(1 - 1/m)^{2n}$. Per tant, la probabilitat que busquem és $1 - (1 - 1/m)^{2n}$.

[Una resposta mitjanament raonable però incorrecta és $2n/m$.]

c1) La resposta és $P(i, j) < P(i)P(j)$. Per veure-ho és suficient veure que $P(i | j) < P(i)$, on $P(i | j) = P(i, j)/P(j)$ és la probabilitat de $T[i] = 1$ condicionada a $T[j] = 1$. Per veure això notem que, sabent que existeix un índex $t \in [n]$ tal que $h_1(x_t) = j$ o $h_2(x_t) = j$, és menys probable que existeixi un índex $r \in [n]$ tal que $h_2(x_r) = i$ o $h_2(x_r) = i$ que si no ho sabéssim. En efecte, donat que $i \neq j$, en el primer cas sabem que per a algun índex $t \in [n]$ el valor j no estarà disponible per a $h_1(x_t)$, o no estarà disponible per a $h_2(x_t)$, mentre que en el segon cas no tenim la certesa que un tal t existeixi.

c2) Primera resposta vàlida: $(1/m)(1 - (1 - 1/m)^{2n}) + (1 - 1/m)(1 - (1 - 1/m)^{2n})^2$. Segona resposta vàlida: $(1 - (1 - 1/m)^{2n})^2$. Siguin x la següent adreça diferent. En cas que $h_1(x) = i$ i $h_2(x) = j$, la probabilitat de fals positiu és $P(i, j)$, que és més petit que $P(i)P(j)$ si $i \neq j$, i és $P(i)$ si $i = j$. Com que $i = j$ passa amb probabilitat $1/m$, això dona la primera resposta vàlida usant els apartats anteriors. La segona resposta vàlida s'obté ignorant el cas $i = j$. Això segueix sent una fita superior prou bona si m és gran.

[Una tercera resposta mitjanament raonable però incorrecta és $(2n/m)^2$.]

6.4 Vèrtexs verds i blancs

Si F és satisfactible, llavors el màxim és m o més (de fet, és m). Donada una assignació que satisfà totes les clàusules d' F , pintem un dels literals certs de cada clàusula de color verd i la resta de color blanc. Això dona una coloració vàlida amb m verds.

Si F és insatisfactible, llavors el màxim és $m - 1$ o menys (de fet, és el número màxim de clàusules que es poden satisfer amb una assignació). Si hi hagués una coloració amb m (o més) verds, llavors podríem assignar les variables de manera que els literals corresponents siguin cert sense que hi hagi conflictes en l'assignació, i llavors estaríem satisfent totes les clàusules d' F .

No existeix (de fet, no es coneix) cap algorisme polinòmic. L'apartat anterior demostra que la funció que transforma F en $G(F)$ és una reducció de SAT al problema de pintar el graf amb com a mínim m vèrtexs de color verd, i aquesta reducció es pot computar en temps polinòmic. Per tant, si hi hagués un algorisme com el suggerit a l'enunciat, llavors també hi hauria un algorisme per resoldre SAT en temps polinòmic, la qual cosa no se sap que sigui possible perquè implicaria $P = NP$.

6.5 Rotacions de paraules

La concatenació d' s amb s conté totes les i -èsimes rotacions d' s . Així doncs, construïm una paraula w que sigui la concatenació de t , un caràcter especial com ara '#', i s dues vegades. A l'exemple, tindriem $w = \text{"babbab#abbabbabbabb"}$. Ara n'hi ha prou amb aplicar l'algorisme KMP a w , i buscar n al vector d'enters resultat.

Amb l'exemple, els nombres obtinguts són

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
b	a	b	b	a	b	#	a	b	b	a	b	b	a	b	b	a	b	b
0	0	1	1	2	3	0	0	1	1	2	3	4	5	6	4	5	6	4

Observem que $n = 6$ es troba efectivament a dues posicions del vector resultat. El cost de l'algorisme KMP és lineal. Per tant, tenim cost $\Theta(n + 2n) = \Theta(n)$.

6.6 Teoria de jocs

És fàcil veure (per inducció, per exemple), que el nombre d'una pila amb p pedres és $p\%3$.

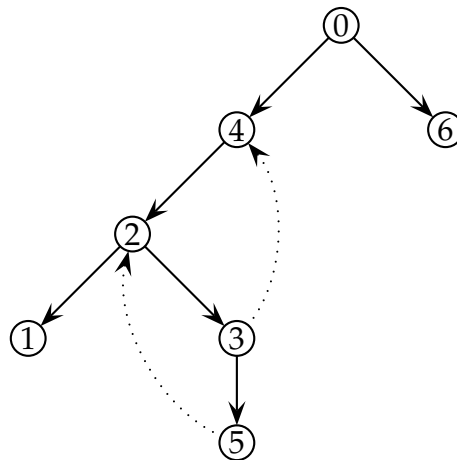
Entre 0 i 2000 hi ha $2001/3 = 667$ nombres amb residu 0, 1 i 2, així que la probabilitat que una pila tingui qualsevol d'aquests nombres és exactament $1/3$. Siguin n_0 , n_1 i n_2 el nombre de piles de cada nombre. Quan n tendeix a infinit, els tres comptadors tenen valors propers a $n/3$ amb alta probabilitat. La posició és perdedora si els dos bits del nombre total, calculat fent l'or exclusiu de tots els nombres individuals, són 0. Per tant, cal que n_1 i n_2 siguin parells. La probabilitat dels dos esdeveniments, que en el límit són independents, tendeix a $1/2$. Així doncs, el límit existeix i és $(1/2)^2 = 1/4$.

A la segona situació, la probabilitat de tenir nombre 1 o 2 és $1/2$. Quan n tendeix a infinit, $n_0 = 0$, i n_1 i n_2 tenen valors propers a $n/2$ amb alta probabilitat. Segueix sent necessari que n_1 i n_2 siguin parells, però ara no tenim nombres 0, així que els dos esdeveniments no són independents. En efecte, quan n és senar, exactament un de n_1 o n_2 és senar, i la posició és sempre guanyadora. Quan n és parell, n_1 i n_2 tenen la mateixa paritat, la qual és 0 amb probabilitat exactament $1/2$ (quan $n > 0$). La probabilitat que la posició sigui perdedora oscil·la: $1, 0, 1/2, 0, 1/2, 0, 1/2, \dots$ així que el límit no existeix.

6.7 Punts d'articulació i arestes pont

Els punts d'articulació són els vèrtexs que, si s'eliminen (ells i les arestes on apareixen), el graf resultat no és connex. A l'exemple, ho són els vèrtexs 0, 4 i 2. Similarment, les arestes pont són aquelles que, si s'eliminen, el graf resultat no és connex. A l'exemple, ho són les arestes 6-0, 0-4 i 2-1.

L'algorisme bàsicament consisteix en un recorregut en profunditat. Començant al vèrtex 0, i visitant els veïns de petit a gran, obtenim:



Per a cada vèrtex x , guardem el temps d'arribada t_x . A l'exemple, tenim $t_0 = 1$, $t_4 = 2$, $t_2 = 3$, $t_1 = 4$, $t_3 = 5$, $t_5 = 6$ i $t_6 = 7$.

També es guarda, per a cada vèrtex x , un valor m_x definit com el mínim comptador t_y de tots els vèrtexs y de totes les arestes $z \rightarrow y$ cap amunt (marcades amb punts a l'exemple) de tots els descendents z d' x (o t_x , si no es pot arribar a cap vèrtex y per sobre d' x). A l'exemple, tenim $m_0 = 1$, $m_1 = 4$, $m_2 = m_3 = m_4 = 2$, $m_5 = 3$ i $m_6 = 7$.

Un vèrtex x (que no sigui l'arrel) és un punt d'articulació si i només si existeix alguna aresta cap avall $x \rightarrow y$ tal que $m_y \geq t_x$. L'arrel és un punt d'articulació si i només si té almenys dues arestes cap avall.

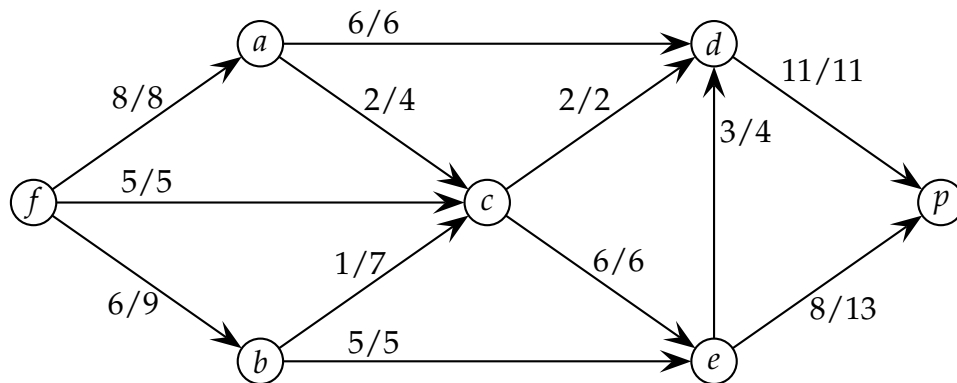
Les arestes cap amunt no són mai arestes pont. Una aresta cap avall $x \rightarrow y$ és una aresta pont si i només si $m_y > t_x$.

6.8 Flux màxim

L'algorisme troba repetidament camins augmentatius, fins que no en queden més. Per exemple, si suposem que es tria sempre el camí més amunt, es trobarien aquests camins:

- Flux 6 per $f \rightarrow a \rightarrow d \rightarrow p$.
- Flux 2 per $f \rightarrow a \rightarrow c \rightarrow d \rightarrow p$.
- Flux 3 per $f \rightarrow c \rightarrow e \rightarrow d \rightarrow p$.
- Flux 2 per $f \rightarrow c \rightarrow e \rightarrow p$.
- Flux 1 per $f \rightarrow b \rightarrow c \rightarrow e \rightarrow p$.
- Flux 5 per $f \rightarrow b \rightarrow e \rightarrow p$.

Un possible flux màxim (de valor 19) és aquest:



Per incrementar el flux màxim, només cal considerar augmentar les arestes d'un tall mínim, perquè la suma de capacitats d'aquestes arestes són una fita superior al valor del flux màxim. Al graf d'exemple, són les arestes que connecten vèrtexs que a l'últim recorregut des de f van ser accessibles, amb els que no: $a-d$, $c-d$, $c-e$ i $b-e$. Augmentar la capacitat de la primera permet incrementar el flux màxim en 2, i fent servir les altres arestes es pot incrementar en 3. En alguns casos cal tirar enrera el flux d'algunes arestes, aprofitant el graf residual.

[Una solució potencialment més eficient consistiria a trobar un tall fent un recorregut des de la font, buscar un altre tall fent un recorregut des del pou en el graf invers, i considerar només les arestes de la intersecció dels dos talls. A l'exemple, aquesta idea no reduiria el nombre d'arestes, perquè només hi ha un tall.]

Parcial, 28 d'octubre de 2022

7.1 Sumar 1

El cost de l'algorisme escolar per sumar 1 a un número x és proporcional al nombre de "carries" que es generen, és a dir, al nombre d'1's fins al primer 0 comptant des de la dreta. Escollint els n bits d' x a l'atzar, la probabilitat que x tingui el seu primer 0 a la posició t és 2^{-t-1} . Per tant, el cost en el cas mitjà és $\sum_{t=1}^n \Theta(t) \cdot 2^{-t-1}$. Atès que la sèrie infinita $\sum_{t=1}^{\infty} t/2^t$ convergeix a 2, la suma anterior està fitada per una constant i, per tant, el cost en el cas mitjà és $\Theta(1)$.

Una altra manera de veure-ho és que el nombre esperat de vegades que cal llençar una moneda a l'aire fins que surti cara és 2.

7.2 Cost d'alguns bucles

Suposant com és habitual que l'operació % (mòdul) té cost constant, el codi té cost

$$\sum_{i=1}^n i^2 \cdot \Theta(1) + \sum_{i=1}^n i \cdot (\Theta(1) + C_f(n)),$$

on C_f és el cost d' f . La primera suma compta el cost de les comparacions, els increments i els mòduls, i la segona suma compta el cost de les crides a $f(n)$, de les quals només se'n fan i perquè només hi ha i divisors d' i entre 0 i $i^2 - 1$.

Donat que la suma dels n primers quadrats és $\Theta(n^3)$, i que la suma dels n primers nombres és $\Theta(n^2)$, el resultat és

$$\Theta(n^3) + \Theta(n^2) + \Theta(n^2) \cdot C_f(n) = \Theta(n^3) + \Theta(n^2) \cdot C_f(n).$$

Quan $C_f(n) = \Theta(1)$ això és $\Theta(n^3)$, i quan $C_f(n) = \Theta(n)$ també és $\Theta(n^3)$.

7.3 Codis de Huffman

a) FALS: $(0.3, 0.3, 0.4)$ dona $(00, 01, 1)$.

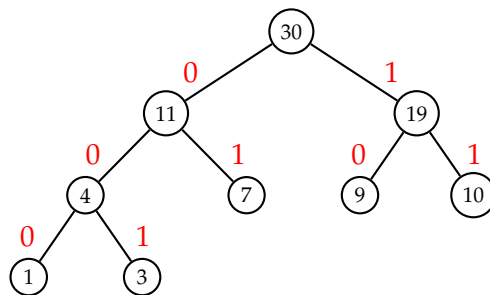
b) CERT: Com que la suma de les freqüències dels altres símbols és com a molt 0.5, el símbol en qüestió sortirà del heap a l'última iteració, i per tant quedarà penjat de l'arrel.

c) CERT: Suposem el contrari. Perquè un símbol es codifiqui amb un sol bit cal que acabi penjat de l'arrel de l'arbre final. Com que tots els símbols tenen freqüència més petita que 0.33, el subarbre germà T del símbol original acumularà una freqüència més gran que $1 - 0.33 = 0.67$. Aquest subarbre ha de tenir dos subarbres no buits, T_1 i T_2 , almenys un dels quals ha de tenir freqüència acumulada superior a $0.67/2 > 0.33$. Però llavors l'algorisme de Huffman hauria usat el símbol que penja de l'arrel abans que aquest subarbre en el moment de formar el subarbre T . Hem arribat a una contradicció.

7.4 Tallant taulons

N'hi ha prou d'aplicar l'algorisme que troba els codis de Huffman sobre les probabilitats $p_i = \ell_i/\ell$. L'arbre corresponent a la codificació obtinguda descriu on cal fer els $n - 1$ talls de forma òptima. El cost total és doncs només $\Theta(n \log n)$.

A l'exemple, una codificació de Huffman possible seria



amb aquests codis: 000 per a 1, 001 per a 3, 01 per a 7, 10 per a 9, i 11 per a 10.

Sigui b_i el nombre de bits del símbol i -èsim a la codificació de Huffman. El cost de la codificació és $H = \sum_{i \leq i \leq n} b_i \cdot p_i = \sum_{i \leq i \leq n} b_i \cdot \ell_i / \ell$. Sabem que Huffman minimitza H , i per tant que minimitza $H \cdot \ell = \sum_{i \leq i \leq n} b_i \cdot \ell_i$.

D'altra banda, qualsevol manera de tallar els taulons es pot representar amb un arbre binari, on els fills de cada tauló tallat de longitud x són les dues longituds obtingudes e i d , amb $e + d = x$. Per a cada tauló i , sigui t_i el nombre de talls en els quals el tauló està contingut. A l'exemple, $t_1 = 3$, perquè $\ell_1 = 3$ està contingut en el tall de 30, en el de 11, i en el de 4. No és difícil veure que el cost C de fer tots els talls coincideix amb $\sum_{i \leq i \leq n} t_i \cdot \ell_i$.

A l'exemple,

$$\begin{aligned}
 C &= 30 + 11 + 4 + 19 \\
 &= (1 + 3 + 7 + 9 + 10) + (1 + 3 + 7) + (1 + 3) + (9 + 10) \\
 &= 3 \cdot 1 + 3 \cdot 3 + 2 \cdot 7 + 2 \cdot 9 + 2 \cdot 10.
 \end{aligned}$$

Finalment, si interpretem l'arbre com una codificació d' n símbols, on cada tall comporta afegir un bit a la codificació, veurem que $t_i = b_i$, i podem concloure que $M = H \cdot \ell$.

7.5 Bandera belga

a) Primer particionem el vector respecte del camp color fent servir 'B' com a pivot. La partició ens dirà on acaben les 'B' i comencen les 'G' i les 'R'. Si la partició no ens retornés aquesta frontera, n'hi hauria prou amb fer un bucle linial per trobar-la. (Es podria trobar més eficientment amb cerca binària, però no cal per aconseguir la complexitat demanada a l'enunciat.) Ara n'hi ha prou amb particionar aquesta part dreta del vector fent servir 'G' com a pivot. Al final, el vector queda ordenat com cal. Com que particionar un vector de mida n té cost $\Theta(n)$, i aquí fem una partició sencera i una de parcial, el cost total també és $\Theta(n)$.

b) Si només tenim un vector de caràcters, el més senzill és comptar quantes lletres hi ha de cada amb tres variables, i després sobre escriure el vector d'esquerra a dreta amb tantes lletres de cada com calgui. El cost és clarament $\Theta(n)$, i només fem servir un nombre constant (i petit) de variables auxiliars.

7.6 Com sempre, un codi misteriós

El codi calcula la suma màxima possible de valors d'un subconjunt d'objectes que no pesin en total més d' s unitats. Aquesta és una variant de l'algorisme del "canvi de monedes" vist a classe (amb una moneda de cada).

La funció recursiva és

$f(j, i) =$ "màxim valor possible d'un subconjunt d'objectes que sumin exactament i , si podem usar els j primers objectes".

Les j anirien entre 0 i n , i les i entre 0 i s .

Amb l'exemple, el vector evoluciona així (les posicions del vector coincideixen amb la i , mentre que la j es correspon a l'evolució temporal del vector):

0	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$
0	$-\infty$	3	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$
0	$-\infty$	3	$-\infty$	5	$-\infty$	8	$-\infty$
0	$-\infty$	4	$-\infty$	7	$-\infty$	9	$-\infty$

En general, tenim $f(j, i) = \max(f(j-1, i), V[j-1] + f(j-1, i - P[j-1]))$.

Com a casos base podem agafar $f(j, i) = -\infty$ si $i < 0$ (per als valors invàlids d' i), $f(0, 0) = 0$, i $f(0, i) = -\infty$ si $i > 0$.

Final, 11 de gener de 2022

8.1 Quants a l'interval?

Donat que $1 \leq x \leq k$ per a tota $x \in S$, podem adaptar l'algorisme d'ordenació per comptatge. El primer pas és crear un vector $C[0..k]$ per comptar i guardar a cada posició i quants elements hi ha amb valor i . A continuació recorrem el vector C des d'1 fins a k , fent $C[i] += C[i-1]$. D'aquesta manera tenim a cada posició $C[i]$ el nombre d'elements d' S que hi ha a l'interval $[0, i]$. El temps de preprocés és doncs $\Theta(n + k)$. Ara podem contestar cada consulta de *quants a l'interval?* fent $C[b'] - C[a' - 1]$, amb $a' = \max(1, a)$ i $b' = \min(k, b)$.

8.2 Ordenació sàvia

El nombre d'inversions d'un vector n'és una mesura del "grau d'ordenació". Si el vector està ordenat creixentment no té cap inversió. D'altra banda, $\binom{n}{2}$ és una fita superior del nombre d'inversions, la qual s'aconsegueix amb el vector ordenat decreixentment.

Ordenant V fent servir la variant de *quicksort* que particiona amb un pivot escollit a l'atzar tenim un cost de $\Theta(n \log n)$ en el cas mitjà i de $\Theta(n^2)$ en el cas pitjor, perquè la manera d'ordenar d'aquesta variant és independent del nombre d'inversions del vector.

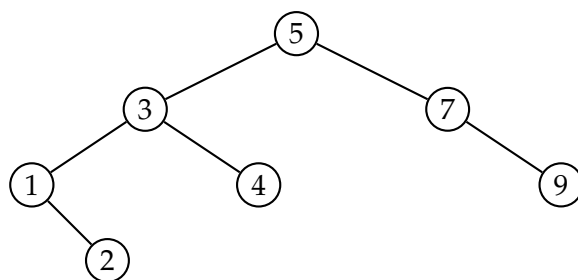
Fent servir *insertion sort* el cost és $\Theta(n)$ en tots els casos. Efectivament, l'algorisme ha de fer almenys $\Theta(n)$ comparacions per "verificar" si el vector ja està ordenat. La resta del cost d'aquest algorisme és proporcional al nombre d'intercanvis que fa, el qual coincideix amb el nombre d'inversions. Així doncs, amb $o(n)$ inversions, el cost és $\Theta(n)$, que és significativament millor que el de *quicksort*.

8.3 Punts en un cercle

Per la manera de generar els n punts, aquests queden distribuïts uniformement dins del cercle unitari $\mathcal{C}$ (amb centre a l'origen i radi 1). És a dir, la probabilitat de trobar un punt a qualsevol porció del cercle és proporcional a l'àrea de la porció. Així doncs, podem

ordenar els n punts amb *bucket sort* segons la seva distància a l'origen, dividint $\mathcal{C}$ en n regions disjunctes $\mathcal{C}_1, \mathcal{C}_2, \dots, \mathcal{C}_n$ (els *buckets*) totes amb la mateixa àrea π/n : $\mathcal{C}_1$ és el cercle amb centre a l'origen i radi $\sqrt{1/n}$, $\mathcal{C}_2$ és la corona formada pel cercle amb centre a l'origen i radi $\sqrt{2/n}$ menys $\mathcal{C}_1$, i així succesivament. Cada punt es guarda al bucket corresponent amb cost $\Theta(1)$. Com que el nombre esperat de punts a cada bucket és 1, el cost esperat d'ordenar cada bucket és $\Theta(1)$. El cost esperat total de l'algorisme és $\Theta(n)$.

8.4 Arbres de cerca



Una crida a *misteri* () amb l'arbre binari de cerca de la figura escriu 1, 2, 3, 4, 5, 7, 9.

Per a qualsevol arbre binari de cerca, una crida a *misteri* () visita tots els nodes en *inordre*. És a dir, recursivament visita el subarbre esquerre, l'arrel i el subarbre dret, en aquest ordre, i per tant escriu totes les claus en ordre creixent.

El cost de *misteri* () és proporcional al nombre de nodes que és visiten, el qual sempre és el nombre total de nodes de l'arbre.

8.5 Teoria de jocs

Sabem que el número d'una pila amb n pedres és n . És fàcil veure (per inducció, per exemple), que el número d'una caixa amb n pedres és $n\%2$. Per tant, la posició donada te número 12, resultat de fer l'or exclusiu de 7, 10, 0, 1 i 0. Com que el número no és 0, la posició és guanyadora. L'única jugada guanyadora consisteix a treure 4 pedres de la pila amb 10 pedres, fent que passi a tenir número 6 i el número total sigui 0. Si treïèssim una altra quantitat d'aquesta pila el resultat seria diferent de 0, i si juguèssim a l'altra pila o a qualsevol caixa, seria impossible posar a 0 el bit amb valor 2^3 del 10.

8.6 Isomorfisme d'un subgraf

Primer, veiem que SUBGRAF-ISOMORFISME és un problema NP. Per verificar-ne una solució n'hi ha prou a comprovar que T té mida n_1 , que els elements són diferents i pertanyen a V_2 , i que la funció f donada (per exemple, amb tots els parells $(x, f(x))$) és realment una bijecció entre V_1 i T que preserva les arestes entre els grafs. El certificat té clarament mida polinòmica en la mida de l'entrada, i la comprovació només requereix temps polinòmic.

Per demostrar que SUBGRAF-ISOMORFISME és NP-difícil, hi reduïm CLIQUE, que ja sabem que és NP-difícil. Donats una G i una k , si $k > |V|$ diem "no" (si no fem aquesta comprovació, la reducció no és polinòmica). Altrament, construïm G_1 com una clique amb k vèrtexs (tindrà mida $\Theta(k^2) = O(n^2)$), i fem que G_2 sigui G . Òbviament, SUBGRAF-ISOMORFISME dirà "sí" sii G conté una clique de mida k . És més, a partir de la f trobada es pot obtenir directament $S = T$.

Similarment a SUBGRAF-ISOMORFISME, GRAF-ISOMORFISME pertany a NP. Com que SUBGRAF-ISOMORFISME és NP-difícil, hi podem reduir qualsevol problema NP, en particular GRAF-ISOMORFISME.

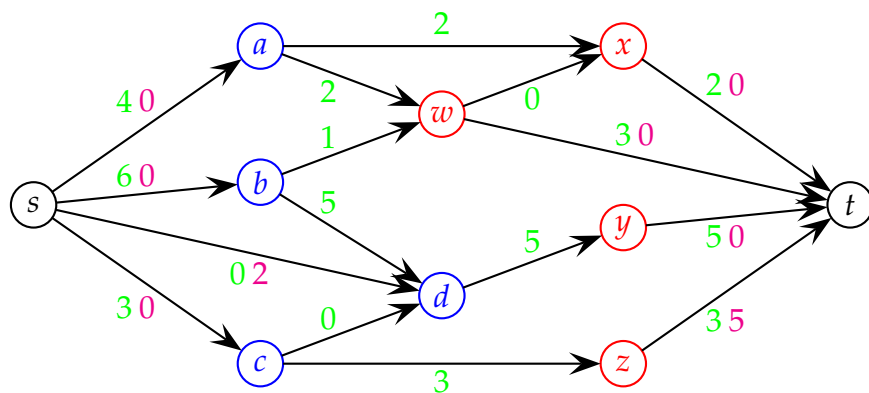
La reducció en sentit invers no és clar que existeixi. Actualment no s'ha pogut demostrar que GRAF-ISOMORFISME sigui NP-complet, ni que pertanyi a P. Si es demostrés que és NP-complet, per transitivitat en les reduccions segur que s'hi podria reduir SUBGRAF-ISOMORFISME. D'altra banda, si es demostrés que GRAF-ISOMORFISME pertany a P, llavors no existiria la reducció (a no ser que $P = NP$).

8.7 Flux màxim

Si les arestes se saturen de dalt a baix, els camins augmentatius són:

- $s \rightarrow a \rightarrow x \rightarrow t$, amb flux 2,
- $s \rightarrow a \rightarrow w \rightarrow t$, amb flux 2,
- $s \rightarrow b \rightarrow w \rightarrow t$, amb flux 1,
- $s \rightarrow b \rightarrow d \rightarrow y \rightarrow t$, amb flux 5,
- $s \rightarrow c \rightarrow z \rightarrow t$, amb flux 3.

El flux total és 13. En verd es mostra el flux que passa pels arcs, i en magenta les capacitats residuais dels arcs originals (les capacitats infinites i les dels arcs inversos no es mostren).



Suposant que poguéssim agafar tots els premis positius i cap dels negatius, el benefici seria $B = \sum_{x \in V, p(x) > 0} p(x)$ (a l'exemple, la suma és 15). D'altra banda, qualsevol tall (L, R) del graf G' que no talli arcs de capacitat infinita es correspon a un $S = L \setminus \{s\}$ vàlid. En efecte, si $x \in S$ i $(x, y) \in E$, llavors $y \in S$ (altrament tindríem un arc de capacitat infinita entre L i R). Podem calcular el benefici de cada S vàlid restant de B els $p(x) > 0$ que no hem agafat (els dels $x \in R$), i sumant els $p(y) < 0$ que hem agafat (els dels $y \in L$). Per la construcció de G' , la suma d'aquests $p(x)$ i $-p(y)$ que restem coincideix amb la capacitat del tall. Per maximitzar el benefici, volem el tall mínim de G' , que sabem que coincideix amb el flux màxim.

Per obtenir S , n'hi ha prou amb trobar L i eliminar-ne s . Això es pot fer comprovant quins vèrtexs s'han visitat a l'última iteració del max-flow, la que no ha trobat cap camí augmentatiu. A l'exemple, serien els vèrtexs $\{s, a, b, d, w, x, y\}$. (Per arribar a a, b, w i x cal usar algun arc invers del graf residual.) El benefici màxim és $4 + 6 + 2 - 3 - 2 - 5 = 2$, que efectivament coincideix amb $B - F = 15 - 13$.

8.8 Influencer

a) Si existeix un influencer, ha de coincidir amb la mediana m (o amb les dues medianes, si n és parell). Per tant, n'hi ha prou de seleccionar m amb temps lineal en el cas pitjor, i comptar quants elements hi ha que siguin iguals a m amb una passada sobre V .

b) Anem a la (una) posició del mig per trobar m . Després, fem una variant de cerca dicotòmica (igual a `lower_bound()`) per trobar la posició de la primera aparició d' m , amb cost logarítmic. Mirant si $\lfloor \frac{n}{2} \rfloor$ posicions més a la dreta també hi ha m , sabrem si és un influencer o no.

c) L'algorisme és correcte, i es pot veure amb aquest invariant: Després d' i iteracions, suposem que x ha aparegut j vegades, i la resta d'elements $k = i - j$ vegades. Llavors, si $j > k$, tenim $y = x$ i $num \geq j - k$. I si $j \leq k$, tenim $y = x$ o $num \leq k - j$. L'invariant es pot demostrar per inducció considerant uns quants casos. Un cop demostrat, trivialment tenim $y = x$ al final de l'execució.

Aquesta és una demostració més intuïtiva: Suposem que aparellem els elements, ajuntant-ne sempre dos de diferents, fins que no puguem seguir fent-ho perquè només queden elements iguals. És fàcil veure que, al final, els únics elements no aparellats seran còpies d' x . L'algorisme bàsicament simula aquest procés. En tot moment té uns quants elements iguals a y encara no aparellats, i el comptador num de quants són. Si z coincideix amb y , sumem un al comptador. Si són diferents, fem una parella amb z i una y , i restem un al comptador. El cas especial amb num igual a 0 es produeix quan no tenim cap element no aparellat (en particular, a l'inici de l'execució). En aquest cas guardem el nou element com a y i ens apuntem que en tenim 1.

Una altra manera de demostrar-ho és considerar quantes vegades num val 0. Si només val 0 al principi de l'execució, com que x apareix més de la meitat de les vegades, y s'ha hagut d'inicialitzar amb x i el resultat final és correcte. Altrament, si han calgut $2p$ passos fins arribar a 0 per primera vegada, llavors no es poden haver gastat més de p còpies d' x , per la qual cosa segueix sent un influencer de la resta del vector, i l'algorisme és correcte per inducció en el nombre de vegades que num val 0.

Parcial, 29 d'octubre de 2021

9.1 Ordenació boja

a) Sigui $m = n/3$. Després de la primera ordenació, els m elements més grans de V es troben dins de $V[e..j]$. Després de la segona ordenació, aquests m elements es troben a $V[d+1..j]$, i en la posició adequada. Per tant, després de la tercera ordenació, cada element es troba a la posició correcta.

Per exemple, si $V = [7, 4, 2, 9, 1, 8]$, l'evolució del vector és $[2, 4, 7, 9, 1, 8]$, $[2, 4, 1, 7, 8, 9]$, $[1, 2, 4, 7, 8, 9]$.

b) El cost ve donat per la recurrència $T(n) = \Theta(1) + 3T(2n/3)$. Aplicant el teorema mestre, obtenim $T(n) = \Theta(n^{\log_{3/2} 3}) \simeq \Theta(n^{2.71})$.

c) Com que l'algorisme de selecció té cost quadràtic, el nou cost ve donat per $T(n) = \Theta(n^2) + 2T(2n/3)$. Amb el teorema mestre, obtenim $T(n) = \Theta(n^2)$.

d) Si substituïm la primera crida recursiva per una partició en dues meitats que tinguin m elements cadascuna, l'algorisme funcionarà perquè els m elements més grans segueixen trobant-se a $V[e..j]$. Podem fer-ho amb l'algorisme "mediana de medianes de 5", el qual té cost linial en el cas pitjor. La recurrència ara és $T(n) = \Theta(n) + 2T(2n/3)$, la solució de la qual és $T(n) = \Theta(n^{\log_{3/2} 2}) \simeq \Theta(n^{1.71})$.

9.2 Multiplicant matrius

a) L'algorisme de Strassen divideix la matriu en quatre submatrius de mida $n/2 \times n/2$, i calcula el producte en funció de productes recursius i sumes de matrius. En lloc de fer els 8 productes recursius obvis, l'algorisme aconsegueix fer-ne només 7. La recurrència per al cost és $T(n) = \Theta(n^2) + 7T(n/2)$, la qual té solució $T(n) = \Theta(n^{\log_2 7}) \simeq \Theta(n^{2.81})$.

b) Amb 6 crides recursives, el cost seria $\Theta(n^{\log_2 6}) \simeq \Theta(n^{2.58})$. Amb 5 crides recursives, el cost seria $\Theta(n^{\log_2 5}) \simeq \Theta(n^{2.32})$, el qual ja és millor que $\Theta(n^{2.37})$.

9.3 Codi misteriós

a) El codi escriu 12 23 10 42 9.

b) Cada vegada s'escriu l'element més gran d'entre $m + 1$. Per tant, no s'escriu mai cap dels m elements més petits de V . Així doncs, s'escriuen els m elements més grans de V .

c) El cost en el cas pitjor és

$$\sum_{1 \leq i \leq m} \Theta(\log i) + \sum_{1 \leq i \leq m} \Theta(\log m) = \Theta(m \log m) = \Theta(n \log n).$$

d) Per escriure els mateixos nombres, n'hi hauria prou amb particionar el vector en dues meitats amb cost $\Theta(n)$ en el cas pitjor.

9.4 Xips defectuosos

Si connectem un xip X amb un xip Y i la resposta és BD (és a dir, X diu que Y és bo, i Y diu que X és dolent), llavors sabem que X és dolent. Com que en tots els apartats considerem només casos pitjors, podem suposar que no tindrem mai respostes BD o DB, per garantir que els tests ens donin la mínima informació possible. En particular, si connectem un xip bo amb un de dolent, la resposta serà sempre DD.

a) Per identificar el xip dolent d'entre $n \geq 3$ xips, els ajuntem per parelles. Si la resposta és BB, sabem que els dos xips són bons i els descartem. Altrament, si la resposta és DD, amb un sol test més podem identificar el xip dolent. Si n és parell, el cas pitjor es dona quan les $n/2 - 1$ primeres respostes són BB. En aquest cas cal fer un test final entre un xip bo i qualsevol dels dos últims candidats. Si n és senar, fem $(n - 3)/2$ tests amb resultat BB, i dos tests finals per identificar el xip dolent d'entre els tres candidats finals. El nombre total de test en el cas pitjor és doncs $\lceil n/2 \rceil$.

b) Agrupem els xips dolents en dos subconjunts amb 10 xips cadascun. Dos xips dolents respondran BB si són del mateix subconjunt, i DD si són de subconjunts diferents. Sabem que els xips bons diran BB quan es connectin entre si. En tots els altres casos les respostes seran DD. Així, tindrem tres grups de 10 xips que es comporten externament de forma idèntica, i que per tant són indistingibles.

[Fer que els xips dolents diguin BB quan es connecten entre si i DD quan es connecten amb xips bons és una resposta incorrecta. Tindríem un grup de 10 xips del mateix tipus (bo/dolent), i un grup de 20 xips del mateix tipus. Com que sabem que $b = 10$, podríem deduir trivialment que els xips del grup de 10 són els bons.]

c) Si $n \leq 2$, els xips són bons i no cal fer res. Altrament, suposem primer que n és parell. Posem els xips en $n/2$ parelles i fem els $n/2$ tests. Descartem un dels xips dels parells que responen BB, tots els xips dels parells que responen DD, i resollem el mateix problema recursivament amb els xips no descartats. Sigui $\alpha > 0$ el nombre de parelles amb dos xips bons, i $\beta < \alpha$ el nombre de parelles amb dos xips dolents. Encara que totes aquestes parelles responguin BB, en el subproblema recursiu tindrem α xips bons i β xips dolents, amb $\alpha > \beta$, mantenint l'invariant $b' = \alpha > (\alpha + \beta)/2 = n'/2$.

Si $n \geq 3$ és senar, una manera de resoldre el problema consisteix a fer els $n - 1$ tests d'un xip X amb tota la resta. Si X és bo, almenys $(n - 1)/2$ dels altres xips diran que ho és. Si X és dolent, com a màxim $(n - 1)/2 - 1$ dels altres xips diran que és bo. Així doncs, amb cost linial podem saber si X és bo o no. En el primer cas ja estem. En el segon, hem descartat un xip dolent i tenim $b' = b > (n - 1)/2 = n'/2$, així que tenim un problema més petit, amb n parell, i mantenint l'invariant.

En el cas pitjor, passem d' n xips a $n/2$ amb un nombre linial de tests. Per tant, el cost d'aquest algorisme en el cas pitjor ve definit per la recurrència $T(n) = \Theta(n) + T(n/2)$, amb solució $T(n) = \Theta(n)$.

9.5 I encara més codis misteriosos

a) El codi és una variant del garbell d'Eratóstenes, en la qual es calcula el factor primer més gran que divideix cada nombre entre 2 i $n - 1$.

El bucle intern té cost $\Theta(n/p)$. El codi sencer té cost

$$\Theta(n) + \sum_{2 \leq p < n, p \text{ primer}} \Theta(n/p) \leq \Theta(n) \cdot \sum_{1 \leq p < n} 1/p = O(nH_n),$$

on $H_n = \Theta(\log n)$ és l' n -èsim nombre harmònic.

b) El codi acumula a q la quantitat de factors primers diferents de cada nombre entre 2 i $n - 1$. Per tant, la quantitat que s'escriu és la mitjana de factors primers diferents dels naturals més petits que n .

Hi ha almenys dues maneres senzilles de demostrar que $q = O(n \log n)$. La primera és observar que cada primer p apareix en, com a molt, la factorització de n/p nombres, i que per tant contribueix amb una quantitat afitada per n/p a q . Això dona lloc a un càlcul molt similar al del primer apartat.

Una manera alternativa consisteix a veure que, per a cada nombre i , hi ha $O(\log i)$ primers diferents que el divideixen: Efectivament, siqui $\omega(i)$ aquest nombre. Per maximitzar-lo, suposem que els factors primers no estan repetits, i que són tan petits com sigui possible, és a dir, $i = 2 \cdot 3 \cdot 5 \cdot 7 \dots \geq 2^{\omega(i)}$. Llavors tenim $\omega(i) \leq \log_2 i$, i podem concloure que $q = \sum_{2 \leq i < n} O(\log i) = O(n \log n)$.

Extraordinari, 1 de juliol de 2021

10.1 Quicksort, de nou

Com que els triplets s'escullen de forma equiprobable, n'hi ha prou de considerar els casos favorables i els casos possibles. Per a cada posició i del pivot després de la partició, hi ha $i(m-i-1)$ triplets favorables (i elements possibles a l'esquerra del pivot, $m-i-1$ a la dreta del pivot). Així doncs, la probabilitat d'escollir aquest pivot és $i(m-i-1)/\binom{m}{3} \simeq 6i(m-i)/m^3$.

La recurrència que cal resoldre és

$$T_n = n + \sum_{0 \leq i < n} \frac{6i(n-i)}{n^3} (Q_i + Q_{n-i}) = n + \frac{12}{n^3} \sum_{0 \leq i < n} i(n-i)Q_i .$$

Suposem $T_n = cn \ln n + o(n \log n)$. Per comprovar aquesta conjectura i obtenir-ne la constant, substituïm $Q_n = cn \ln n$ a la recurrència, i obtenim

$$cn \ln n = n + \frac{12c}{n^2} \sum_{0 \leq i < n} i^2 \ln i - \frac{12c}{n^3} \sum_{0 \leq i < n} i^3 \ln i . \quad (*)$$

Per facilitar els càlculs, aproximem els sumatoris per integrals:

$$\sum_{0 \leq i < n} i^2 \ln i \simeq \int_0^n x^2 \ln x \, dx = \left. \frac{x^3 \ln x}{3} - \frac{x^3}{9} \right|_0^n = \frac{n^3 \ln n}{3} - \frac{n^3}{9} .$$

$$\sum_{0 \leq i < n} i^3 \ln i \simeq \int_0^n x^3 \ln x \, dx = \left. \frac{x^4 \ln x}{4} - \frac{x^4}{16} \right|_0^n = \frac{n^4 \ln n}{4} - \frac{n^4}{16} .$$

Substituint a (*), el terme $cn \ln n$ es cancel·la, i obtenim $c = 12/7$. Així doncs, concloem $T_n = 12/7 n \ln n + o(n \log n)$.

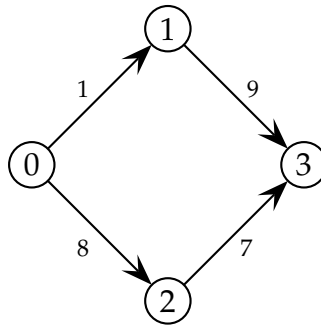
El quicksort en la variant bàsica té cost mitjà $Q_n = 2n \ln n + o(n \log n)$. Com que $12/7 \simeq 1.714 < 2$, aquesta variant mediana-de-3 fa menys comparacions en mitjana (un 15%, més o menys).

En el cas pitjor, el pivot és el segon element (o el penúltim). En aquest cas tenim la recurrència $Q(n) = \Theta(n) + Q(n-2)$, la qual té solució $Q(n) = \Theta(n^2)$ com el quicksort bàsic.

10.2 Dijkstra modificat

Aquesta variant de Dijkstra també calcula el cost mínim, però mesurat com el màxim dels pesos del camí, no com la suma.

Per exemple, suposem $u = 0$ i $v = 3$, i que el graf és



El camí òptim passa pel vèrtex 2, i té cost 8. El camí alternatiu que passa per 1 té cost 9.

El codi té el mateix cost en el cas pitjor que el Dijkstra bàsic, és a dir, $\Theta(n + m \log m)$.

10.3 Arbres aleatoris dolents

Cal calcular quantes de les $n!$ permutacions generen arbres amb alçada n . Anomenem aquesta quantitat $H(n)$. Tenim $H(1) = 1$. Per construir tots els arbres d'alçada n amb $n \geq 2$ claus, podem agafar tots els arbres d'alçada $n - 1$ amb $n - 1$ claus, i posar-ne un com a fill esquerre o com a fill dret d'una nova clau (que per tant ha de ser la màxima o la mínima de les n claus). Així doncs, tenim la recurrència $H(n) = 2H(n - 1)$, la solució de la qual és $H(n) = 2^{n-1}$. Com a conclusió, $P(n) = 2^{n-1}/n!$

10.4 Teoria de jocs

La frase és falsa. Considerem una variant molt simple, en la qual només es pot treure exactament una pedra de la pila que sigui. És molt fàcil veure (per inducció, per exemple) que el número d'una pila amb n pedres és $n \bmod 2$. Per tant, una posició amb una pila amb 2 pedres i una pila amb 3 pedres té número 1 (l'or exclusiu de 0 i 1), de manera que és guanyadora. Si treiem una pedra de la pila amb 2 pedres, la pila passa de número 0 a número 1, però la posició resultant té número 0 i, per tant, és perdedora. Així doncs, existeix una jugada guanyadora que incrementa el número de la pila on s'ha fet la jugada.

Com a curiositat, amb aquesta variant no caldrien números. Seria suficient veure que una posició és guanyadora si i només si la suma total de pedres és senar.

10.5 Un altre codi misteriós

El programa codifica tots els substrings de mida m com a nombres en base 26. El primer substring es codifica amb un bucle tal qual. La gràcia consisteix a calcular eficientment la resta de codificacions, cadascuna amb temps constant. Guardant tots els codis al conjunt, el qual elimina repetits, al final obtenim el nombre de substrings diferents de mida m .

En el cas pitjor tots els substrings són diferents, de manera que el conjunt va creixent de mida. El cost de la primera part del codi és $\Theta(m)$. La segona part té cost en el cas pitjor $\Theta(n - m) + \sum_{i=1..n-m} \Theta(\log i) = \Theta((n - m) \log(n - m))$.

Amb m gran tenim un sobreiximent de les variables h i pw . Per evitar-ho, es poden fer els càlculs mòdul un nombre M proper a $2^{31}/26$. D'aquesta manera, si dues h són diferents, és segur que els substrings representats també seran diferents. Si dues h són iguals, hi ha una petita probabilitat (propera a $1/M$) que els dos substrings siguin diferents però casualment tinguin el mateix "hash".

Com a curiositat, una manera de reduir la probabilitat d'error fins a zero a efectes pràctics consisteix a calcular dues (o més) funcions de "hash", de manera que aproximadament la probabilitat d'error s'eleva al quadrat (o al cub, ...).

10.6 Pintant un graf

Quan un graf connex és 2-colorable, només te dues possibles coloracions: 1) la que pinta un vèrtex qualsevol d'un color, pinta els veïns amb l'altre color, i itera això fins a esgotar tots els vèrtexs del graf, i 2) la coloració amb els dos colors intercanviats. Per tant, és suficient amb fer un recorregut del graf (un dfs o bfs, tant és) i pintar cada nou vèrtex que es visita amb el color contrari al color amb el que hem pintat el vèrtex d'on venim. El cost és el d'un recorregut del graf, és a dir $\Theta(n + m)$.

Donada una coloració parcial del graf, el veïnatge d'un vèrtex que encara està per colorar només pot haver usat d dels $d + 1$ colors disponibles. Per tant, fem un algorisme *greedy*: amb un recorregut del graf (un dfs o un bfs, tant és), pintem cada nou vèrtex que es visita amb el primer dels $d + 1$ colors que tingui disponible, i marquem aquell color com a no disponible per als seus veïns abans de continuar amb el recorregut. Podem mantenir el conjunt de colors disponibles amb un vector de $d + 1$ bits associat a cada vèrtex.

El cost d'aquest algorisme és $\Theta(nd + m)$ si la cerca del color disponible es fa linealment des del principi del vector de bits cada vegada, o $\Theta(n + m)$ independentment de d si la cerca del color disponible es fa a partir del punt on hem acabat l'anterior cerca, mantenint un apuntador al primer color disponible per a cada vèrtex.

L'observació clau és que, donat que el graf és 3-colorable, el veïnatge de qualsevol vèrtex és 2-colorable. Si el graf te tots els vèrtexs de grau més petit que $a = \lceil \sqrt{n} \rceil$, llavors podem fer servir l'algorisme de l'apartat segon per colorar tot el graf amb com a molt a colors. Si el graf té algun vèrtex, diguem-ne u , de grau més gran o igual que a , llavors podem 2-colorar el veïnatge d' u amb dos colors nous fent servir l'algorisme de l'apartat primer (gràcies a l'observació clau), pintar u amb un tercer color nou, i iterar sobre el graf en què el vèrtex u i el seu veïnatge s'han esborrat. Com que a cada iteració esborrem com a mínim a vèrtexs, l'algorisme acaba en com a molt $\lceil n/a \rceil$ iteracions. Com que a cada iteració fem servir tres colors nous, el nombre total de colors és com a molt $3\lceil n/a \rceil + a$, i això són $\Theta(\sqrt{n})$ colors. El cost de l'algorisme és polinòmic en la mida del graf.

SÍ, SÍ, NO SE SAP. Sabem que 3-COL és NP-complet, i que 4-COL i 2-COL pertanyen a NP (de fet, 2-COL fins i tot pertany a P). Com que qualsevol problema d'NP es redueix a qualsevol problema NP-complet, ja tenim els dos primers apartats. Justificació per al tercer: 2-COL pertany a P i 3-COL és NP-complet. Per tant, si es sabés que 3-COL es redueix a 2-COL, llavors sabríem que 3-COL està a P, i per tant que qualsevol problema d'NP està a P, i conclouríem que $P = NP$, la qual cosa no se sap.

11

Final, 18 de gener de 2021

11.1 Parells de primers bessons

Suposem que existeix una funció

```
bool para(string F, int e);
```

tal que, donats un string F que conté el codi d'una funció $f(\mathbf{int})$ en C++, i un $\mathbf{int}$ e , retorna **true** si $f(e)$ s'atura, i retorna **false** si $f(e)$ es penja.

Comencem programant una funció tal que, donat un enter n , retorna el parell de primers bessons $(x, x + 2)$ amb la mínima $x \geq n$ (si existeix, altrament la funció es penja):

```
pair<int, int> seguent(int n) {  
    while (not es_primer(n) or not es_primer(n + 2)) ++n;  
    return {n, n + 2};  
}
```

Sigui S un string amb la codificació de la funció $\text{seguent}(\mathbf{int})$. Per a tot enter n , aquesta funció retorna **false** si la conjectura és falsa, i es penja si la conjectura és certa:

```
bool a_mitges(int n) {  
    while (para(S, n)) ++n;  
    return false;  
}
```

Sigui A un string amb la codificació de la funció $a_mitges(\mathbf{int})$. Aquesta funció retorna **true** si la conjectura és certa, i **false** si és falsa:

```
bool conjectura () {  
    return not para(A, 42);  
}
```

11.2 Dividint un vector

El problema es pot pensar recursivament. Si $k = 1$, no cal fer res. Altrament, si k és parell, particionem el vector en dues meitats exactes, i resollem el mateix problema dues vegades amb $n' = n/2$ i $k' = k/2$. Si k és senar, sigui $m = \lfloor k/2 \rfloor$. En aquest cas, particionem el vector en dos subvectors amb $n' = (m+1)n/k$ i $n' = mn/k$ elements, i resollem els dos subproblemes amb $k' = m+1$ i $k' = m$, respectivament.

L'arbre de recursió d'aquest algorisme té $\lceil \log_2 k \rceil + 1$ nivells. Si per particionar usem mediana-de-medianes-de-5, el cost sencer de cada nivell és $\Theta(n)$. El cost total és doncs $\Theta(n \log k)$, el qual és $\Theta(n \log \log n)$ quan $k = \Theta(\log n)$.

Cal adonar-se que particionar per l'element n/k , després pel $2n/k$, pel $3n/k$, ... té cost total $\Theta(n \cdot k)$, el qual arriba a ser $\Theta(n^2)$ amb $k = n$. Això és pitjor que la solució trivial d'ordenar el vector amb cost $\Theta(n \log n)$.

11.3 Arbres de cerca aleatoris

Tenint en compte que totes les claus tenen la mateixa probabilitat d'estar a l'arrel, i que el mínim d'un arbre de cerca és el mínim del seu fill esquerra (si aquest no és buit), la recurrència que cal resoldre és

$$M_n = 1 + \frac{1}{n} \sum_{0 \leq i < n} M_i ,$$

amb $M_0 = 0$. Suposem $M_n = c \ln n + o(\log n)$. Per comprovar aquesta conjectura i obtenir la constant, substituïm $M_n = c \ln n$ a la recurrència, i obtenim

$$c \ln n = 1 + \frac{c}{n} \sum_{0 \leq i < n} \ln i . \quad (*)$$

Per facilitar els càlculs, aproximem el sumatori per una integral:

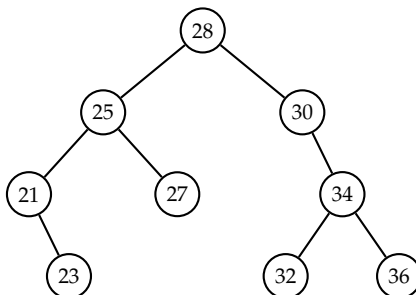
$$\sum_{0 \leq i < n} \ln i \simeq \int_0^n \ln x \, dx = x \ln x - x \Big|_0^n = n \ln n - n .$$

Substituint a (*), el terme $c \ln n$ es cancel·la, i obtenim $c = 1$. Així doncs, concloem $M_n = \ln n + o(\log n)$.

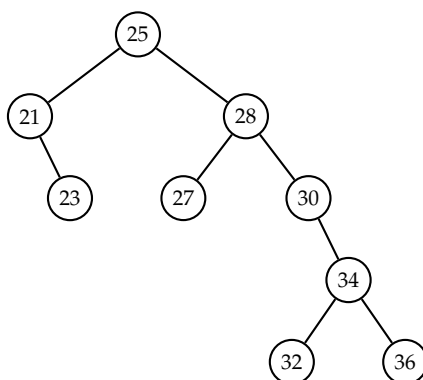
Una manera alternativa de resoldre la recurrència consisteix a calcular uns quants termes, observar que surten els nombres harmònics H_n , conjecturar $M_n = H_n$, i demostrar-ho per inducció, per exemple.

11.4 Arbres de cerca aleatoritzats

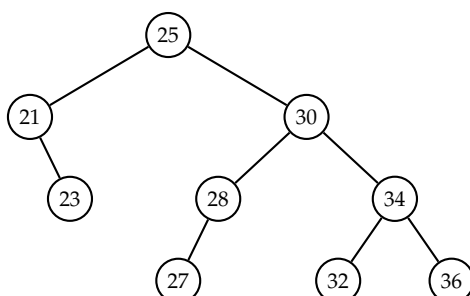
Amb probabilitat $1/(8+1) = 1/9$ posem el 28 a l'arrel, amb resultat



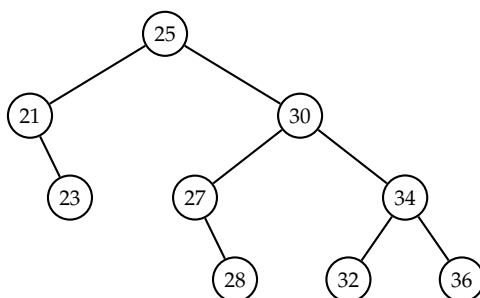
Altrament (amb probabilitat $8/9$) inserim el 28 recursivament al fill dret. Aquí, amb probabilitat $1/(5+1) = 1/6$ el posem a l'arrel, amb resultat



Altrament (amb probabilitat $8/9 \times 5/6$) inserim el 28 recursivament al fill esquerre. Aquí, amb probabilitat $1/(1+1) = 1/2$ el posem a l'arrel, amb resultat



Finalment (amb probabilitat $8/9 \times 5/6 \times 1/2$) inserim el 28 recursivament al fill dret, que és buit. El posem aquí amb probabilitat $1/(0+1) = 1$, amb resultat



Les probabilitats absolutes de cadascun d'aquest arbres són doncs, en ordre, $1/9 = 3/27$, $8/9 \times 1/6 = 4/27$, $8/9 \times 5/6 \times 1/2 = 10/27$, i $8/9 \times 5/6 \times 1/2 = 10/27$.

11.5 Kruskal

Sigui T qualsevol arbre generador de G de cost mínim, i sigui K l'arbre generador trobat per l'algorisme de Kruskal. Demostrarem que han de ser el mateix. En particular, això implica que hi ha un únic arbre generador de cost mínim quan els costos són tots diferents.

La demostració és per contradicció. Si T i K són diferents, sigui e_1 la primera aresta descartada per Kruskal que pertany a T , i sigui w_1 el cost d'aquesta aresta. Kruskal va descartar e_1 perquè generaria un cicle c_1 . Com que e_1 sí que pertany a T , almenys una de les arestes de c_1 (diguem-li e_2) no pot pertànyer a T . (Altrament, T tindria un cicle.) A més, com que Kruskal considera les arestes en ordre creixent de cost, e_2 té cost $w_2 < w_1$.

Si afegíssim e_2 a T , això generaria un cicle c_2 . Com que e_2 sí que pertany a K , almenys una de les arestes de c_2 (diguem-li e_3) no pot pertànyer a K . A més, com que T és òptim, e_3 té cost $w_3 < w_2$. (Si $w_3 > w_2$, intercanviant e_3 per e_2 a T obtindríem un arbre generador alternatiu de cost inferior.)

Com a conclusió, hem trobat una aresta e_3 que pertany a T , que no pertany a K , i que té cost $w_3 < w_1$. Això és una contradicció amb la hipòtesi que e_1 sigui la primera aresta descartada per Kruskal que pertany a T . Per tant, $T = K$.

11.6 Teoria de jocs

a) Com a casos base, tenim $\text{nim}(0) = \text{nim}(1) = 0$. Amb $n = 2$, només podem anar a 0, que té númer 0. Per tant, $\text{nim}(2) = 1$. Amb $n = 3$, només podem anar a 0 o 1, que tenen númer 0. Per tant, $\text{nim}(3) = 1$. Amb $n = 4$, podem anar a 0, 1 o 2, que tenen nímbers 0 i 1. Per tant, $\text{nim}(4) = 2$. Amb $n = 5$, podem anar a 1, 2 o 3, que tenen nímbers 0 i 1. Per tant, $\text{nim}(5) = 2$. Amb $n = 6$, podem anar a 2, 3 o 4, que tenen nímbers 1 i 2. Per tant, $\text{nim}(6) = 0$... El patró 0, 0, 1, 1, 2, 2 es repeteix per sempre. (Rigorosament, aquest fet s'hauria de demostrar per inducció, considerant els sis residus mòdul 6 similarment als arguments anteriors.) Una possible fórmula és $\text{nim}(n) = \lfloor (n \bmod 6) / 2 \rfloor$.

b) Tenim $\text{nim}(10) = 2$ i $\text{nim}(20) = 1$. L'or exclusiu de 2 i 1 és 3, que no es pot convertir en 0 combinant-lo ni amb un 0, ni amb un 1, ni amb un 2. Per tant, sigui quina sigui x , la posició és guanyadora. Malgrat això, com que qualsevol jugada a la tercera pila deixaria una posició guanyadora per al rival, no hi ha cap jugada guanyadora a la tercera pila.

11.7 Taules de dispersió

(a) El cost és $\Theta(s + s\alpha)$. El terme $\Theta(s)$ surt de calcular $h(c)$. El terme $\Theta(s\alpha)$ de comparar les claus dels parells $\langle c', v' \rangle$ dels vector $T[h(c)]$ amb c . El cost en mitjana d'aquesta segona part és l'indicat perquè l'esperança de la longitud del vector $T[h(c)]$ és n/m , que és α .

[Nota: La resposta $\Theta(s\alpha)$ només seria correcta si puguéssim assegurar que α és $\Omega(1)$. Però res no ens diu que α pugui ser $1/s$, la qual cosa faria que $\Theta(s\alpha)$ fos $\Theta(1)$, que no és el cost real.]

(b) Aquest apartat té diverses solucions. La solució esperada era fer servir una segona funció de hash per a cada taula $T[i]$, diguem-ne h_i , amb rang $\{1, \dots, n_i^2\}$, on n_i és el nombre de claus c amb $h(c) = i$. Al vector $T[i]$ s'hi guardarien els triplets $\langle c, v, h_i(c) \rangle$ tals que $h(c) = i$. Per buscar el valor associat a la clau c , primer calculem $i = h(c)$ i $j = h_i(c)$ amb cost $\Theta(s) + \Theta(s)$, i després recorrem els triplets $\langle c', v', j' \rangle$ del vector $T[h(c)]$ comparant j amb j' i, només en cas d'igualtat, c amb c' . Com en l'anàlisi del hashing perfecte vist a teoria, es pot aconseguir que el nombre esperat de vegades que haguem de fer la comparació de c amb c' sigui $\Theta(1)$. Per tant, el cost de buscar a $T[i]$ és $\Theta(\alpha) + \Theta(s)$ i el cost total és doncs $\Theta(s + \alpha)$.

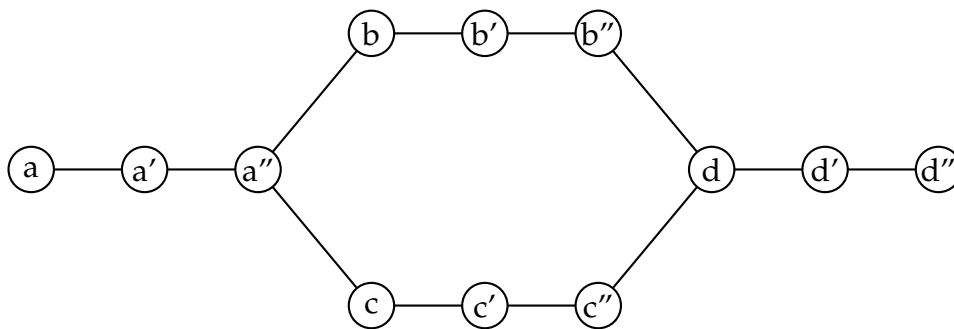
[Nota: La solució anterior té l'avantatge que manté la mateixa funció de hash h , la mateixa mida m de la taula, i la mateixa distribució de claus dins la taula. Només canvia la implementació de les subtaules. Però l'enunciat no demana tot això. Per tant, una solució perfectament vàlida seria redimensionar amb $m = \Theta(n)$ posicions, la qual cosa aconseguiria que el factor de càrrega α fos $\Theta(1)$, i per tant el cost de l'algorisme directe (el del primer apartat) seria $\Theta(s)$. Una altra solució vàlida seria fer servir un hashing perfecte per aconseguir també cost $\Theta(s)$. Una solució que *no* es pot considerar vàlida és construir una taula de mida 256^s i fer accés directe. Tot i que el cost de l'operació de

consulta seria $\Theta(s)$, no es pot dir que això sigui una implementació d'una taula de hash com demana l'enunciat, perquè l'espai que es fa servir no és lineal (ni polinòmic) en el nombre de claus n .]

11.8 Cicles hamiltonians

Siguin V i E el conjunt de vèrtexs i el conjunt d'arcs del graf dirigit G , i siguin $n = |V|$ i $m = |E|$ les seves mides respectives. Per a cada $x \in V$, el graf no dirigit G' tindrà tres vèrtexs x , x' i x'' , i les arestes $\{x, x'\}$ i $\{x', x''\}$. A més, per a cada arc $(x, y) \in E$, G' tindrà l'aresta $\{x'', y\}$. En total, G' tindrà $3n$ vèrtexs i $2n + m$ arestes, i el cost de construir G' serà com cal polinòmic en la mida de G .

En l'exemple demanat, aquest seria G' :

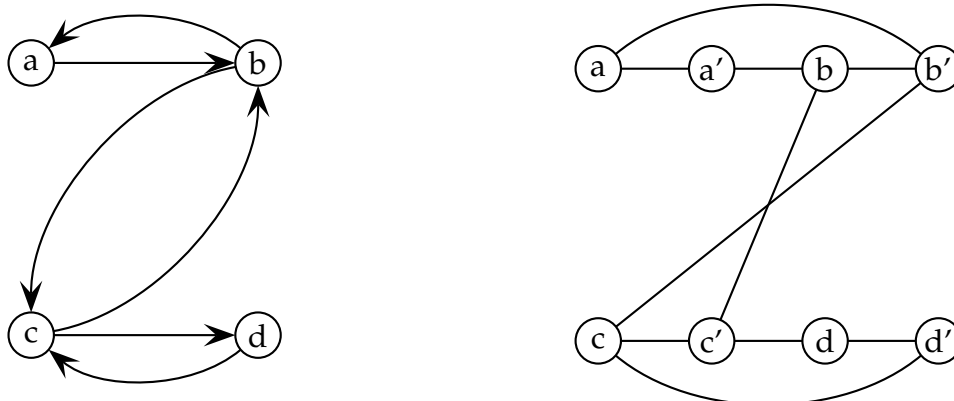


Per veure que aquesta transformació és efectivament una reducció, cal demostrar que G té un cicle hamiltonià si i només si G' té un cicle hamiltonià:

a) Suposem que G té un cicle hamiltonià $x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_n \rightarrow x_1$. Llavors G' té el cicle hamiltonià $x_1 - x'_1 - x''_1 - x_2 - x'_2 - x''_2 - \dots - x_n - x'_n - x''_n - x_1$.

b) Suposem que G' té un cicle hamiltonià. Per a cada vèrtex de tipus x' , l'única manera de passar-hi és venint d' x i anant a x'' , o en sentit invers. És a dir, el cicle ha de contenir o bé el tros $x - x' - x''$ o bé el tros $x'' - x' - x$. Com que els vèrtexs de tipus x no estan connectats a cap altre vèrtex del mateix tipus, l'única manera de connectar aquests n triplets és orientar-los tots de la mateixa manera, suposem arbitràriament que $x - x' - x''$. Per tant, el cicle hamiltonià ha de ser del tipus $x_1 - x'_1 - x''_1 - x_2 - x'_2 - x''_2 - \dots - x_n - x'_n - x''_n - x_1$, i podem concloure que G té el cicle hamiltonià $x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_n \rightarrow x_1$.

Val a dir que una reducció similar, però que només dobli el nombre de vèrtexs, no seria correcta, com es pot veure a l'exemple següent. El graf dirigit de l'esquerra no té cap cicle hamiltonià, mentre que el graf no dirigit corresponent sí que en té.



Parcial, 6 de novembre de 2020

12.1 Potències de 2

Primer, per comoditat assegurem que n sigui una potència de 2, afegint per la dreta del vector tants $+\infty$ com calgui. Això té cost $O(n)$.

Ara podem resoldre el problema recursivament. Si $n = 1$, guardem l'únic element a R . Altrament, busquem la mediana (per la dreta) d' n elements amb l'algorisme mediana-de-medianes-de-5, amb cost $\Theta(n)$. Guardem aquesta mediana a R , i fem una crida recursiva amb els $n/2$ elements més petits que la mediana, els quals coneixem com a resultat de la partició. El cost d'aquest algorisme segueix doncs la recurrència

$$C(n) = \Theta(n) + C(n/2) ,$$

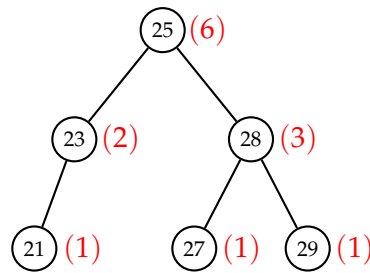
la qual segons el teorema mestre per a les recurrències divisores té solució $C(n) = \Theta(n)$.

És possible que al final el vector R quedi amb els element en ordre invers. Si fos així, només caldria girar-lo amb cost $\Theta(\log n)$. El cost total és $O(n) + \Theta(n) + O(\log n) = \Theta(n)$.

12.2 Rang d'un element

És suficient amb emmagatzemar a cada node el nombre d'elements del subarbre arrelat en aquell node. És important veure que aquesta informació es pot mantenir eficientment quan l'arbre es modifica, ja sigui per inserir o per esborrar algun element, perquè la informació dels nodes afectats es pot calcular trivialment en funció de la dels seus fills.

Aquest és un possible arbre binari de cerca amb sis claus. En vermell es mostren els comptadors de les mides:



Si baixem recursivament per una sola branca de l'arbre fins a trobar x (o arribar a una fulla), aquesta informació addicional ens permet calcular fàcilment la posició d' x . El cost està doncs afegit per l'alçada de l'arbre. En el codi, cal vigilar de no intentar accedir a la mida d'un arbre buit, i tenir en compte que les crides recursives (en particular, la de l'arbre dret) poden retornar -1:

```

using ABC = struct Node *;

struct Node {
    Clau x;
    ABC esq, dre;
    int n;
};

int mida(ABC a) {
    return (a ? a->n : 0);
}

int posicio (Clau x, ABC a) {
    if (a == 0) return -1;
    if (x < a->x) return posicio(x, a->esq);
    int m = mida(a->esq);
    if (x == a->x) return m;
    int res = posicio(x, a->dre);
    return (res == -1 ? -1 : m + 1 + res);
}
  
```

12.3 És un min-heap?

N'hi ha prou de comparar cada element $V[i]$ que no sigui l'arrel amb el seu pare $V[i/2]$, per assegurar que $V[i/2]$ no sigui més gran que $V[i]$. És a dir,

```

bool es_min_heap(const vector<int>& V) {
    for (int i = 2; i < n; ++i)
        if (V[i/2] > V[i]) return false;
    return true;
}
  
```

En el cas millor, quan la primera comparació falla, el cost és $\Theta(1)$. En el cas pitjor V conté un min-heap i cal fer $n - 2$ comparacions, amb cost $\Theta(n)$.

12.4 Quicksort millorat?

a) En el cas pitjor, la partició d'un vector amb n elements diferents deixa uns $n/4$ elements a una banda i la resta a l'altra. Així doncs, tenim la recurrència

$$P_n = 3n + P(n/4) + P(3n/4) .$$

No és difícil veure que $P_n = \Theta(n \log n)$: Primer, definim $C_n = P(n)/n$, i substituïm per obtenir la recurrència

$$C_n = 3 + \frac{1}{4}C(n/4) + \frac{3}{4}C(3n/4) .$$

Si ara definim $A_n = 3 + A(n/4)$ i $B_n = 3 + B(3n/4)$, resulta que tant A_n com B_n són $\Theta(\log n)$. Tenint en compte que $A_n \leq C_n \leq B_n$, podem concloure $C_n = \Theta(\log n)$ i $P_n = \Theta(n \log n)$.

Per obtenir la constant, conjecturem $P_n = cn \ln n + o(n \log n)$. Substituint a la recurrència per a P_n i quedant-nos amb els termes principals, obtenim

$$cn \ln n = 3n + \frac{cn}{4} \ln(n/4) + \frac{3cn}{4} \ln(3n/4) = cn \ln n + 3n - \frac{cn}{4} (4 \ln 4 - 3 \ln 3) .$$

D'aquí obtenim $c = 12/(4 \ln 4 - 3 \ln 3) \simeq 5'33$. En el cas pitjor, aquesta variant és molt millor que la variant bàsica de quicksort, la qual té cost $\Theta(n^2)$.

b) En la versió habitual, el nombre esperat de comparacions per ordenar amb quicksort un vector amb n claus diferents bàsicament segueix la recurrència

$$Q_n = n + \frac{2}{n} \sum_{0 \leq i < n} Q_i .$$

Sigui M_n el nombre esperat de comparacions amb la nova variant. Ara la i només té la meitat de valors possibles, bàsicament entre $n/4$ i $3n/4$, els quals per tant tenen el doble de probabilitat. A més, la partició necessita $3n$ comparacions en lloc de només n . És a dir,

$$M_n = 3n + \frac{4}{n} \sum_{n/4 \leq i < 3n/4} M_i .$$

Tenint en compte que la nova variant particiona de forma més equilibrada que la variant clàssica, la qual té cost $2n \ln n$, també hem de tenir $M_n = \Theta(n \log n)$. Per obtenir la constant, substituïm $M_n = cn \ln n$ a la recurrència, i obtenim

$$cn \ln n = 3n + \frac{4c}{n} \sum_{n/4 \leq i < 3n/4} i \ln i . \quad (*)$$

Per facilitar els càlculs, aproximem el sumatori per una integral:

$$\begin{aligned} \sum_{n/4 \leq i < 3n/4} i \ln i &\simeq \int_{n/4}^{3n/4} x \ln x \, dx = \frac{x^2 \ln x}{2} - \frac{x^2}{4} \Big|_{n/4}^{3n/4} \\ &= \frac{9n^2 \ln(3n/4)}{32} - \frac{9n^2}{64} - \frac{n^2 \ln(n/4)}{32} + \frac{n^2}{64} = \frac{n^2 \ln n}{4} + \frac{9n^2 \ln 3}{32} - \frac{n^2 \ln 4}{4} - \frac{n^2}{8} . \end{aligned}$$

Substituint a (*), el terme $cn \ln n$ es cancel·la, i simplificant l'equació obtinguda arribem a $c = 24/(8 \ln 4 + 4 - 9 \ln 3) \simeq 4'61$, la qual lògicament és més petita que la constant per al cas pitjor, però pitjor que la constant 2 (en base natural del logaritme) del quicksort clàssic.

12.5 Maps i sets

El codi aprofita que els maps i els sets estan ordenats per clau per bàsicament calcular la diferència $M - S$ en temps linial. Si interpretem el codi recursivament, el que fa depèn de la comparació de la primera paraula d' M i la primera paraula d' S (anomenem-les t i s , respectivament). Si $t > s$, sabem que s no servei per anular cap paraula d' M , així que avancem a S (és com si resolguéssim el mateix problema després de llençar s). Si $t = s$, hem d'evitar escriure la informació de t , així que avancem a M (és com si llencéssim t). [Podríem també avançar en S però no cal fer-ho.] Si $t < s$, cal escriure la informació de t , i avançar a M .

El codi donat pot intentar accedir a $*it2$ quan $it2$ apunta a $S.end()$. Aquesta és una manera d'evitar-ho:

```

auto it1 = M.begin();
auto it2 = S.begin();
while (it1 != M.end()) {
    if (it2 != S.end() and it1->first > *it2) ++it2;
    else {
        if (it2 == S.end() or it1->first < *it2) cout << it1->second << endl;
        ++it1;
    }
}

```

En el cas millor, només cal recórrer M . En el cas pitjor, també cal recórrer S . Tenint en compte que recórrer un map o un set completament té cost proporcional a la mida del map o del set (encara que incrementar un iterador pugui tenir cost pitjor que constant), el cost és sempre $\Theta(n)$.

12.6 Ordre d'una permutació

(a) Fem servir exponenciació ràpida: Si $m = 0$, llavors σ^m és la identitat. Si $m \geq 1$ és parell, llavors $\sigma^m = (\sigma^{m/2})^2$. Si $m \geq 1$ és senar, llavors $\sigma^m = (\sigma^{(m-1)/2})^2 \sigma$. Com que cada producte es calcula amb cost $\Theta(n)$, el cost és $T(m) = T(m/2) + \Theta(n)$, que és $\Theta(n \log m)$.

(b) Comencem a $i = 1$ i visitem les posicions $p[i], p[p[i]], p[p[p[i]]], \dots$ fins que tanquem el cicle i tornem a la posició i . A mida que visitem aquestes posicions, incrementem un comptador de longitud, i marquem cada posició que ens trobem en aquest recorregut com que ja ha estat visitada (per exemple, posant-la a 0). Un cop determinada la longitud $\ell[1]$ del primer cicle, incrementem i fins que trobem la primera posició $j > i$ que no estigui marcada, i tornem a començar amb $i := j$. Quan totes les posicions estiguin marcades, ja haurem acabat. Aquest algorisme visita cada posició dues vegades i per tant té cost $\Theta(n)$.

(c) Sigui $x = \sqrt{n \ln(n)}/2$, i siguin $p_1, \dots, p_k$ tots els nombres primers continguts a l'interval $(x/2, x]$. El Teorema dels Nombres Primers ens diu que $k = \pi(x) - \pi(x/2) = x/(2 \ln(x)) + o(x/\ln(x))$. Per tant, per a n suficientment gran tenim que $p_1 + \dots + p_k \leq kx \leq x^2/(2 \ln(x)) + o(x^2/\ln(x)) < n$.¹ Ara muntem una permutació $\sigma \in S_n$ amb k cicles de longituds $p_1, \dots, p_k$ i un cicle de longitud $n - (p_1 + \dots + p_k)$. L'ordre d'aquesta permutació serà com a mínim $\text{mcm}(p_1, \dots, p_k) = p_1 \cdots p_k$, i per tant com a mínim $(x/2)^k \geq (x/2)^{x/(3 \ln(x))} = e^{\ln(x/2)x/(3 \ln(x))}$, que és $2^{\Omega(\sqrt{n \log n})}$.

¹Detalls: $x^2/\ln(x) = n \ln(n)/(4 \ln(1/2) + 2 \ln(n) + 2 \ln(\ln(n))) < n \ln(n)/(2 \ln(n)) = n/2$, i per tant $x^2/(2 \ln(x)) + o(x^2/\ln(x)) < n/4 + n/2 < n$.

(d) Algorisme 1: Obtenim $\ell_1, \dots, \ell_k$ amb cost $\Theta(n)$ amb l'algorisme de l'apartat (b). Per calcular $N := \text{mcm}(\ell_1, \dots, \ell_k)$, factoritzarem cada ℓ_i en nombres primers. Com que cada ℓ_i és com a molt n i l'aritmètica de nombres més petits que n té cost $O(1)$, calcular la factorització de ℓ_i té cost $O(\ell_i)$. La factorització d'un nombre com N , que té tots els factors primers més petits o iguals que n , es pot representar amb un vector $F[1, \dots, n]$, on $F[p]$ és l'exponent de p en la factorització de N si p és primer, i $F[p]$ és 0 altrament. Ara, donada la factorització $F[1, \dots, n]$ de $N_i = \text{mcm}(\ell_1, \dots, \ell_{i-1})$ i la factorització $F'[1, \dots, \ell_i]$ de ℓ_i , podem obtenir la factorització de N_{i+1} fent $F[j] := \max\{F[j], F'[j]\}$ per a $j = 1, \dots, \ell_i$. El cost total d'aquestes operacions serà $O(\ell_1 + \dots + \ell_k)$, i per tant $O(n)$, perquè $\ell_1 + \dots + \ell_k = n$.

Ara, donada la factorització F de $N = \text{mcm}(\ell_1, \dots, \ell_k)$, obtenim els dígit del seu valor numèric multiplicant les potències dels primers de la seva factorització. Com que cada factor primer és com a mínim 2 i com a molt n , i $N = 2^{O(\sqrt{n \log n})}$, això és un producte de $m := O(\sqrt{n \log n})$ nombres primers de $O(\log n)$ bits cadascun. Primer els posem en un vector de mida m amb cost $O(n)$, i després els multipliquem tots en un esquema de dividir i vèncer usant l'algorisme quadràtic de multiplicació per a les recombinacions.

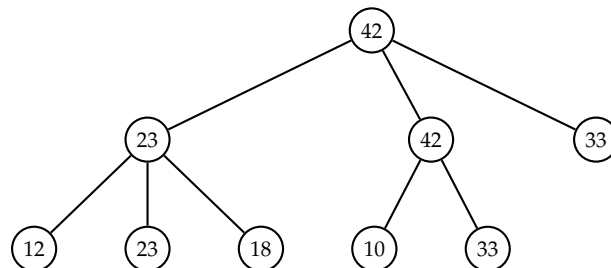
El producte d' m nombres de $O(\log n)$ dígit té $O(m \log n)$ dígit, i per tant la recurrència a analitzar és $T(m) = 2T(m/2) + O((m \log n)^2)$. Definint $T'(m) = 2T'(m/2) + m^2$ es veu per inducció que $T(m) \leq T'(m)O((\log n)^2)$, i com que $T'(m) = O(m^2)$, el resultat és $T(m) = O(m^2(\log n)^2)$. Substituint-hi $m = O(\sqrt{n \log n})$, finalment obtenim cost total $T(m) = O((\sqrt{n \log n})^2(\log n)^2) = O(n^{1.0001})$.

Algorisme 2: És una variant de l'algorisme anterior on, en comptes de fer servir l'algorisme escolar de multiplicació per a nombres de $O(m \log n)$ dígit amb cost $O((m \log n)^2)$, usem Karatsuba per multiplicar-los amb cost $O((m \log n)^{\log_2(3)})$, o la FFT per multiplicar-los amb cost $O((m \log n)^{1.0001})$, respectivament. En els dos casos la recurrència surt $T(m) = 2T(m/2) + O(m^{1.9}(\log n)^2)$. Definint $T'(m) = 2T'(m/2) + m^{1.9}$ es veu per inducció que $T(m) \leq T'(m)O((\log n)^2)$, i com que $T'(m) = O(m^{1.9})$, tenim $T(m) = O(m^{1.9}(\log n)^2)$. Substituint-hi $m = O(\sqrt{n \log n})$, obtenim $T(m) = O((\sqrt{n \log n})^{1.9}(\log n)^2) = o(n)$. El cost $\Theta(n)$ del càlcul inicial de les longituds dels cicles $\ell_1, \dots, \ell_k$ domina i, per tant, el cost total és $\Theta(n)$.

Extraordinari, 7 de juliol de 2020

13.1 Heaps

(a) Aquest és un possible arbre ternari amb 9 elements. Els fills de cada node contenen nombres més petits o iguals que l'element del node. L'arbre és ple per nivells, excepte l'últim, on tots els elements es troben a l'esquerra.



En un vector, els elements es guarden segons es van trobant en un recorregut per nivells de l'arbre, d'esquerra a dreta:

42	23	42	33	12	23	18	10	33
----	----	----	----	----	----	----	----	----

Si, com en els heaps binaris, la primera posició usada del vector és la 1, llavors els fills de cada posició i (si existeixen) es troben a les posicions $3i - 1$, $3i$ i $3i + 1$, i el pare (si existeix) es troba a la posició $\lfloor (i + 1) / 3 \rfloor$.

(b) Sigui $n \geq 1$ el nombre d'elements del heap. És fàcil veure que el nombre de nivells de l'arbre és aproximadament $\log_3 n$. Així doncs, tant flotar un element com enfonsar-lo visiten $O(\log n)$ posicions.

Com amb els heaps binaris, el màxim està a la posició 1. El cost de trobar-lo és $\Theta(1)$.

Per afegir un element ($push(x)$), cal fer un $push_back(x)$, i després flotar x fins trobar un pare més gran o igual, o fins arribar a l'arrel. El cost és doncs $O(\log n)$.

Per esborrar un màxim ($\text{pop}()$), cal posar l'últim element x a l'arrel, fer un $\text{pop.back}()$, i després enfonsar x fins que tots els fills siguin més petits o iguals, vigilant de no sortir del vector. El cost és doncs $O(\log n)$.

Un *heapify* seria molt similar al cas binari: des de $i = n$ fins a $i = 1$, s'enfonsa l'element de la posició i . La manera més senzilla de calcular el cost és adonar-nos que és $\Omega(n)$, i que la sèrie que l'afita és del tipus $\sum_{i=1}^{\infty} 2i/3^i \cdot n$ en comptes de $\sum_{i=1}^{\infty} i/2^i \cdot n$, tot i que ambdues sèries són $O(n)$. El cost és doncs $\Theta(n)$.

En resum, podem veure que totes les operacions tenen el mateix cost asimptòtic que en el cas binari.

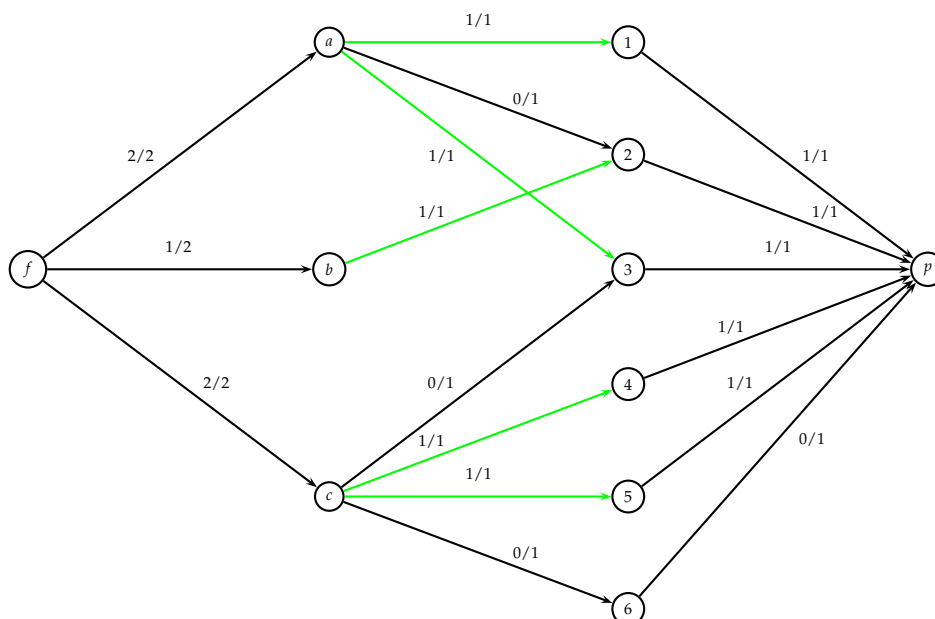
(c) Si només comptem els nodes dels quals s'ha canviat el contingut, tant un $\text{push}(x)$ com un $\text{pop}()$ visiten aproximadament $\log_2 n$ nodes en el cas binari, i $\log_3 n$ en el cas ternari, el qual és un valor significativament més petit.

Si també comptem els nodes dels quals cal consultar el contingut, llavors el cost del $\text{pop}()$ canvia. Es consulten fins a aproximadament $2 \log_2 n$ nodes en el cas binari, i fins a $3 \log_3 n = 3(\log_3 2) \log_2 n \simeq 1.89 \log_2 n$ nodes en el cas ternari, el qual també és un valor més petit.

13.2 L'agència matrimonial

(a) Construïm un graf bipartit amb dos vèrtexs addicionals: una font f i un pou p . Connectem amb arcs de capacitat k la font amb tots els homes, amb arcs de capacitat 1 totes les dones amb el pou, i amb arcs de capacitat 1 cada parell (home, dona) factible. El graf té doncs $n = h + d + 2$ vèrtexs i $m = h + d + p$ arcs. El flux màxim que es pot enviar des de la font fins al pou coincideix amb el nombre màxim de dones que es poden casar. Sabem que es pot calcular el flux màxim d'un graf amb cost polinòmic en la mida del graf, que en aquest cas és com a màxim proporcional a la mida de l'entrada del problema original.

A continuació es mostra el graf i una solució òptima corresponents a la taula de l'exemple. El primer nombre de cada arc és el flux que hi passa, i el segon nombre és la capacitat de l'arc. En verd es mostren els 5 casaments d'aquesta solució òptima.



(b) Anomenem $M = h \cdot d$ a la mida de l'entrada. Sabem que el cost de trobar cada camí augmentatiu és $O(n + m)$. Amb aquest graf, cada camí augmentatiu incrementa en 1 el flux. Com que es fan $x + 1$ iteracions, el cost està afitat per $O(x(n + m))$. Això és polinòmic en M , perquè $x \leq d$ i $n + m = O(h + d + p) = O(M)$.

(c) Si amb una certa k hi ha solució, també n'hi ha per a tota $k' > k$. Similarment, si amb una certa k no hi ha solució, tampoc n'hi ha per a cap $k' < k$. A més, sabem que amb $k = d$ hi ha solució, i que amb $k = 0$ no n'hi ha. Això ens permet fer una cerca binària començant entre 0 i d , aturant-nos quan els dos extrems es toquin. Com que cada pas només costa $\Theta(1)$, el cost total és $\Theta(\log d)$.

13.3 Cub de Rubik

Com que RUBIK és un problema NP-complet, sabem que pertany a NP, i també que és NP-difícil, és a dir, que tot problema dins de NP es pot reduir polinòmicament a RUBIK.

(a) Com que no sabem si X pertany a NP, no podem assegurar que existeixi una reducció polinòmica de X a RUBIK. Però sí que existeix segur una reducció polinòmica en sentit contrari, perquè RUBIK pertany a NP i X és NP-difícil.

(b) Com que Y pertany a P, i $P \subseteq NP$, sabem que Y pertany a NP, i per tant existeix segur una reducció polinòmica de Y a RUBIK. Existeix una reducció polinòmica de RUBIK a Y si i només si Y és NP-difícil, cosa que no sabem. Però si fos el cas, llavors $P = NP$, perquè Y pertany a P.

(c) Un motiu plausible (de fet, cert) que explica que RUBIK pertanyi a la classe NP malgrat que la k pot ser exponencialment gran en n és que estigui demostrat que el nombre de moviments necessari per resoldre un cub de mida n està afitat per un polinomi $p(n)$. Així, si el problema té solució, sempre hi ha un testimoni de mida $\min(k, p(n))$, el qual es pot verificar amb cost polinòmic en la mida de l'entrada.

13.4 Algunes demostracions

a) Hi ha p^2 parells $(a, b) \in \mathbb{Z}_p \times \mathbb{Z}_p$. D'aquests, exactament p satisfan $ax_1 + b \equiv y_1 \pmod{p}$: per a cada $a \in \mathbb{Z}_p$ hi ha un únic $b \in \mathbb{Z}_p$ que ho satisfà, concretament $b = y_1 - ax_1 \pmod{p}$. Per tant, la probabilitat que $f_{a,b}(x_1) = y_1$ quan (a, b) s'escull a l'atzar és $p/p^2 = 1/p$. L'existència i la unicitat de b no depenen del fet que p sigui primer, per tant no cal que ho sigui.

b) Hi ha p^2 parells $(a, b) \in \mathbb{Z}_p \times \mathbb{Z}_p$. De tots aquests, exactament 1 satisfà les equacions $ax_1 + b \equiv y_1 \pmod{p}$ i $ax_2 + b \equiv y_2 \pmod{p}$, concretament $a = (x_1 - x_2)_p^{-1}(y_1 - y_2) \pmod{p}$ i $b = y_1 - ax_1 \pmod{p}$, on $(x_1 - x_2)_p^{-1}$ denota l'invers de $x_1 - x_2$ mòdul p , el qual existeix perquè $x_1 - x_2 \neq 0$ i p és primer. Per tant, la probabilitat que $f_{a,b}(x_1) = y_1$ i $f_{a,b}(x_2) = y_2$ quan (a, b) s'escull a l'atzar és $1/p^2$. Per veure que $(x_1 - x_2)_p^{-1}$ existeix ens ha calgut assumir que p és primer. De fet, per a $p = 4$, la probabilitat que $f_{a,b}(0) = 0$ i $f_{a,b}(2) = 1$ és $0 \neq 1/4^2$.

c) Escollir g a l'atzar dins de F_p vol dir escollir cadascuna de les imatges $g(x)$ de g independentment i uniformement a l'atzar dins de $\mathbb{Z}_p$. Per tant, la probabilitat que $g(x_1) = y_1$ i $g(x_2) = y_2$ quan g s'escull a l'atzar a F_p és $(1/p)(1/p) = 1/p^2$, la mateixa que quan g s'escull a l'atzar a H_p . Per tant, en ambdós casos la probabilitat que $g(x_1) = g(x_2)$ és $p/p^2 = 1/p$, on el factor p té en compte els p possibles valors de la imatge comú $g(x_1) = g(x_2) = y_1 = y_2$.

13.5 Taules dinàmiques

(a) Suposem que (n, N) satisfà $N \geq a$, i $n \geq N/a$ en cas que $N \geq ab$.

- `push_back(x)` amb $n < N - 1$: L'estat següent és $(n + 1, N)$ i satisfà $N \geq a$ perquè ja el satisfia abans, i $n + 1 \geq n \geq N/a$ en cas que $N \geq ab$.
- `push_back(x)` amb $n \geq N - 1$: L'estat següent és $(n + 1, bN)$ i satisfà $bN \geq N \geq a$ perquè $b \geq 2$ i $N \geq a$, i $n + 1 \geq N \geq bN/a$ perquè $n \geq N - 1$ i $a \geq b$.
- `pop_back()` amb $N < ab$: L'estat següent és $(n - 1, N)$ i satisfà $N \geq a$ perquè ja el satisfia abans, i no cal comprovar res més perquè $N < ab$.
- `pop_back()` amb $N \geq ab$ i $n > N/a + 1$: L'estat següent és $(n - 1, N)$ i satisfà $N \geq a$ perquè ja el satisfia abans, i $n \geq N/a$ perquè de fet $n > N/a + 1$.
- `pop_back()` amb $N \geq ab$ i $n \leq N/a + 1$: L'estat següent és $(n - 1, N/b)$ i satisfà $N/b \geq ab/b = a$ perquè $N \geq ab$, i $n - 1 \geq N/a - 1 = (N/ab)(b - ab/N) \geq N/ab$ perquè $n \geq N/a$ i $N \geq ab$ i $b \geq 2$.

(b) Ens trobarem un estat de la forma $(N - 1, N)$ per a cada $N = 2^k$ entre 64 i m . El `push_back(x)` en cadascun d'aquests casos tindrà cost $\Theta(2^k)$, i la resta tindran cost $\Theta(1)$. Per tant, el cost total serà $\sum_{k=6}^{\lfloor \log_2(m) \rfloor} \Theta(2^k) + \Theta(m) = \Theta(2^{\log_2(m)}) + \Theta(m) = \Theta(m)$.

(c) Sigui $M = 2^k$, amb una k arbitràriament gran. Considerem una seqüència de $M - 32$ `push_back(x)` inicials, seguits de M parells $(\text{pop_back}(), \text{push_back}(x))$. En total són $m = 3M - 32$ operacions. Els $M - 32$ primers `push_back(x)` duen a l'estat $(M, 2M)$ i tenen cost $\Theta(M)$ per l'apartat (a). Els M parells $(\text{pop_back}(), \text{push_back}(x))$ següents fan transitar la taula pels estats $(M - 1, M)$, $(M, 2M)$, $(M - 1, M)$, $(M, 2M)$, $(M - 1, M)$, ... en bucle. El cost total d'aquesta part és $M \cdot \Theta(M) = \Theta(M^2)$. Donat que $m = \Theta(M)$, el cost total és $\Theta(m^2)$, tal com calia comprovar.

(d) No existeix una tal seqüència: el cost total sempre és $\Theta(m)$. La raó intuïtiva és que els estats on les operacions són "cares" (cost $\Theta(N)$) sempre duen a un estat de la forma $(N, 2N)$, i per a que es doni un estat $(n, 2N)$ on torni a fer-se una operació "cara" cal que $n \geq 2N - 1$ o $n \leq 2N/4 + 1 \leq N/2 + 1$, la qual cosa vol dir que entremig s'han hagut de fer com a mínim $\Omega(N)$ operacions "barates" (cost $\Theta(1)$).

Si us interessa veure un argument més complet, aquí va: Abans que res, observem que just després d'una operació que fa doblar/dividir la taula, l'estat tindrà la forma $(N, 2N)$, on N és una potència de 2. En efecte, doblar fa passar de $(N - 1, N)$ a $(N, bN) = (N, 2N)$, i dividir fa passar de $(N + 1, aN)$ a $(N, aN/b) = (N, 2N)$. Primer, comptem a part el cost de les operacions que no són de doblar/dividir. Clarament, tenen cost $O(m)$. A partir d'ara, ignorem aquestes operacions.

Per acabar, afitem el cost total de les operacions de doblar/dividir. Entre dues operacions consecutives d'aquest tipus haurem transitat de l'estat $(N_i, 2N_i)$ a l'estat $(N_{i+1}, 2N_{i+1})$, on N_{i+1} serà el doble o la meitat que N_i . Sigui a_i el cost d'aquest trànsit. Sigui b_i el nombre d'operacions d'usuari en aquest trànsit. Per una banda, $a_i = \Theta(N_i)$. Per l'altre, $b_i \geq N_i/2$, perquè per transitar de $(N_i, 2N_i)$ a $(N_{i+1}, 2N_{i+1})$ cal haver inserit almenys N_i elements o esborrat almenys $N_i/2$ elements. Per tant, $a_i = O(b_i)$. La conclusió és que si s'acaben fent r operacions que fan doblar/dividir, llavors el cost total és $\sum_{i=1}^r a_i = \sum_{i=1}^r O(b_i) = O(m)$. Tenint en compte també les operacions que no fan doblar/dividir, el cost total és $\Theta(m)$.

Final, 9 de gener de 2020

14.1 Element comú

Sigui $m = 4n$ el nombre d'elements del vector. Hi ha almenys dues maneres de resoldre aquest problema:

(a) Podem usar quickselect per buscar els elements n -èsim, $2n$ -èsim, $3n$ -èsim, i $4n$ -èsim. És fàcil veure que algun d'aquests quatre nombres ha de ser un element comú. Per a cada candidat, n'hi ha prou amb un bucle per comptar quantes vegades apareix. Això té cost esperat $4\Theta(m) = \Theta(n)$. [Essent més rigorosos, caldria implementar quickselect iterativament, per evitar l'espai logarítmic esperat de la pila de recursió.]

(b) Una manera més fàcil de programar consisteix a agafar un nombre de V a l'atzar, comprovar si és un element comú comptant quantes vegades apareix, i tornar a començar si no ho era.

Sigui $E(m, p)$ el cost esperat, mesurat com el nombre de comparacions entre elements, quan el vector té m nombres dels quals $p \cdot m$ són elements comuns. Tenim la igualtat $E(m, p) = m + (1 - p)E(m, p)$, per la qual cosa $E(m, p) = m/p$. [No és imprescindible calcular la quantitat que divideix m . N'hi ha prou amb adonar-nos que és una constant que creix amb p .] Com que $p \geq 1/4$, tenim que $E(m, p)$ és com a molt $4m$, i el cost esperat és $\Theta(m) = \Theta(n)$.

[Si es volgués evitar el cost infinit d'aquest algorisme en el cas pitjor, mantenint el cost lineal esperat, es pot començar desordenant el vector a l'atzar amb cost lineal, i després considerar com a candidat cada nombre com a molt una vegada, recorrent V d'esquerra a dreta. El cas pitjor amb aquesta variant trivialment seria $\Theta(n^2)$. I fixem-nos que el cost esperat després de desordenar el vector no és pitjor que el de la variant més senzilla, perquè els candidats inútils són descartats per sempre, així que la probabilitat d'encertar un candidat bo creix a cada pas.]

14.2 Heaps

(a) Primer, afegim el 33 al final del vector. Després, mentre el 33 sigui més gran que el seu pare actual (i no arribem a l'arrel del heap), els intercanviem. En aquest cas cal fer dos intercanvis:

1	2	3	4	5	6	7	8	9	10	11
42	28	23	27	25	15	23	17	22	22	33
42	28	23	27	33	15	23	17	22	22	25
42	33	23	27	28	15	23	17	22	22	25

(b) Primer, esborrem l'últim element (un 22) i el posem en lloc de l'arrel. Després, mentre el 22 sigui més petit que algun dels seus dos fills (i no arribem a les fulles del heap), intercanviem el fill més gran amb el 22. [Si hi hagués empat entre els fills caldria decidir quin triar, però això no passa en aquest problema.] En aquest cas també cal fer dues iteracions:

1	2	3	4	5	6	7	8	9
22	28	23	27	25	15	23	17	22
28	22	23	27	25	15	23	17	22
28	27	23	22	25	15	23	17	22

14.3 RSA

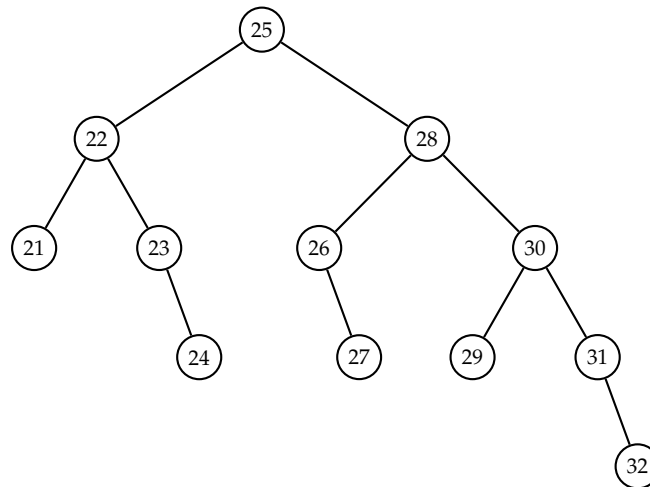
(a) Multipliquem per 2^{E_i} i reduïm mod N_i . Si s era l'incentiu, llavors la taula original conté $s^{E_i} \bmod N_i$, i per tant la taula modificada conté $(2s)^{E_i} \bmod N_i$, que és $2s$ xifrat.

(b) Per a cada treballador, només cal provar els 4000 candidats de l'interval $[1000, 4999]$. Per a cada candidat s , xifrem s amb la clau pública del treballador, i comparem el resultat amb el de la taula. L'únic s per al qual el resultat coincideixi serà l'incentiu. Executar 4000 xifrats per a cada treballador no sembla cap feina excessiva.

(c) Aquesta és una possible manera: Per cada treballador, l'empresa publica una clau simètrica R de 31 bits escollida a l'atzar, xifrada amb la clau pública del treballador, junt amb l'incentiu s xifrat amb la clau simètrica R que s'ha escollit. La taula està formada per triples (i, A, B) , on $A = R^{E_i} \bmod N_i$ per a un nombre aleatori $R \in \{0, \dots, 2^{31} - 1\}$, i $B = s + R \bmod 2^{31}$, on s és l'incentiu que es vol xifrar.

14.4 Arbres de cerca AVL

(a) Aquest arbre AVL amb 12 nodes té la clau més petita a l'alçada mínima possible:



(b) La profunditat de la clau mínima en un arbre de cerca AVL amb n claus és trivialment $O(\log n)$, perquè totes les claus d'un AVL tenen aquesta profunditat.

Una manera intuïtiva de veure que la profunditat és $\Omega(\log n)$ és la següent: Si l'arbre té alçada h , el mínim nombre de passos que s'han de fer per trobar el mínim es produeix quan el subarbre esquerre té alçada $h - 2$ (i el dret $h - 1$), i aquest patró es reproduïx recursivament en el subarbre esquerre, com es pot veure a l'exemple de l'apartat (a). Així doncs, cal fer almenys aproximadament $h/2$ passos fins arribar al mínim, cosa que ens dona una fita inferior proporcional a $\log n$.

Com a conclusió, el cost en el cas millor de trobar el mínim és $\Theta(\log n)$.

[Un error habitual responent aquest problema consisteix a afirmar que les profunditats de les fulles d'un arbre AVL són idèntiques o difereixen només en 1, cosa que és falsa segons es pot veure a l'exemple de l'apartat (a).]

14.5 Quick-sort

Com es va veure a classe, la solució de la recurrència

$$Q(n) = \Theta(n) + \frac{2}{n} \sum_{0 \leq i < n} Q(i)$$

és $Q(n) = \Theta(n \log n)$. Usarem aquest resultat més avall.

Per analitzar aquesta variant de quick-sort, cal veure com es particiona el vector: Es fa servir sempre $V[d]$ com a pivot. A més,

- Si els elements són tots diferents, es deixen a l'esquerra del pivot els elements més petits, i a la dreta els elements més grans. El particionat respecta l'ordre relatiu dels elements petits, i fa el mateix amb els grans, amb una excepció: posa el primer element gran al final de la banda dreta.
- Si els elements són tots iguals, deixa tots els elements a la dreta del pivot.

(a) Amb una permutació aleatòria d' n nombres diferents, tenim una probabilitat $1/n$ d'escollir qualsevol nombre com a pivot. A més, és fàcil veure que el particionat deixa a l'esquerra i a la dreta dues permutacions aleatòries (i independents) dels elements petits i grans. Per tant, l'anàlisi del cost ens du a la recurrència mencionada anteriorment, amb cost $\Theta(n \log n)$.

(b) Si els nombres són diferents i apareixen en ordre creixent, a cada pas obtenim un subvector amb $n - 1$ elements en ordre creixent, i un altre buit. La recurrència en aquest cas és

$$C(n) = \Theta(n) + C(n - 1) ,$$

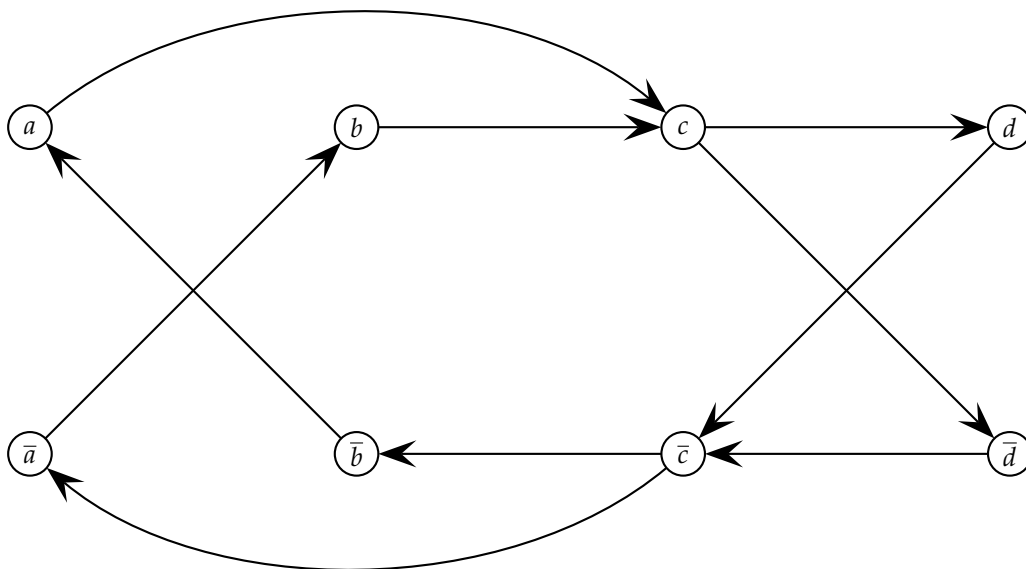
la qual té solució $C(n) = \Theta(n^2)$.

(c) Amb n nombres iguals tenim una situació simètrica a la de l'apartat (b), és a dir, 0 elements a l'esquerra i $n - 1$ a la dreta. La recurrència és la mateixa, i la solució també és quadràtica.

(d) El cost dels casos (a) i (c) no canvien: El primer ja estava aleatoritzat, i amb tots els nombres iguals l'intercanvi de dos elements no fa res. El cost del cas (b) passa a ser com el del (a), és a dir, $\Theta(n \log n)$, perquè la probabilitat de que a cada pas escollim qualsevol nombre com a pivot passa a ser $1/n$.

14.6 2-SAT

Sigui n el nombre de variables. Comencem creant un graf dirigit amb $2n$ vèrtexs, un per a cada literal. Cada clàusula genera dos arcs, corresponents a les dues implicacions equivalents a la clàusula. Per exemple, $(\bar{c} \vee d)$ genera $c \rightarrow d$ i $\bar{d} \rightarrow \bar{c}$. Amb l'exemple donat, obtenim

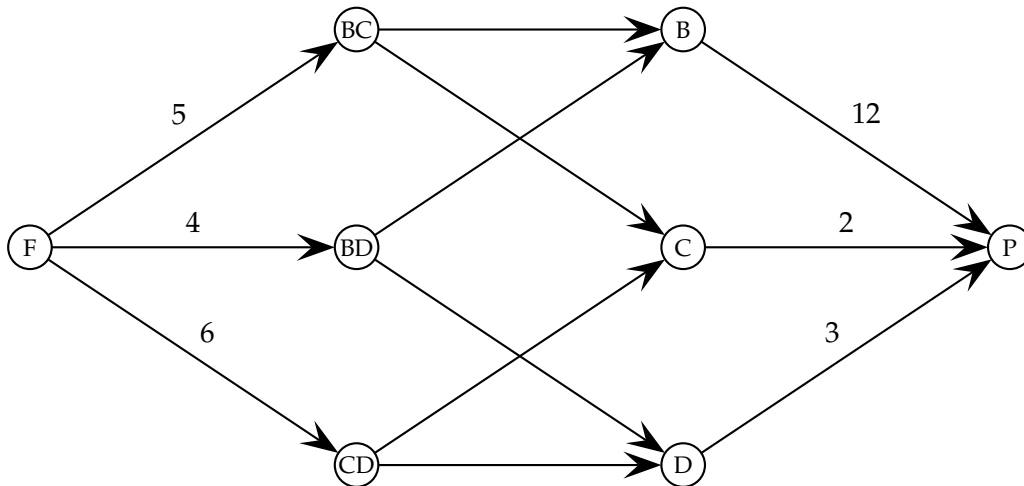


Ara calculem els components fortament connexos del graf. Sabem que la fórmula té solució si i només si cap component fortament connex conté una variable i la seva variable negada (perquè tindríem una situació tipus $x \rightarrow \bar{x} \rightarrow x$, que és insatisfactible). Com que, en aquest cas, només tenim un sol component fortament connex, no hi ha cap assignació de valors a les variables que satisfaci la fórmula.

14.7 Torneig de go

Suposem el cas més favorable al jugador A en què guanya totes les partides que li falten per jugar: En l'exemple, A obtindria $p = 8 + 6 + 3 + 6 = 23$ punts. Per a cada altre jugador j , calculem G_j , el nombre de partides que pot guanyar sense superar p punts. En l'exemple, $G_B = 23 - 11 = 12$, $G_C = 23 - 21 = 2$, $G_D = 23 - 20 = 3$. Si algun d'aquests nombres és negatiu no cal continuar, perquè A ja ha perdut.

Altrament, construïm un graf amb una font, un vèrtex per a cada enfrontament pendent on no juga A (a l'exemple, BC, BD, CD), un vèrtex per a cada jugador que no és A, i el pou. En l'exemple, tenim:



Les capacitats dels arcs de la banda esquerra del graf es corresponen als valors de la matriu d'enfrontament pendents. Tots els arcs intermedis tenen capacitat ∞ (que el graf no mostra explícitament). Els arcs de la banda dreta tenen capacitat G_j .

Ara apliquem l'algorisme de max-flow en aquest graf. Si aconseguïssim passar totes les $5 + 4 + 6 = 15$ unitats que surten d'F, es que hi ha alguna manera de repartir les victòries a les partides pendents de manera que cap jugador obtingui més de 23 punts, i per tant A encara pot guanyar el torneig. En l'exemple, el flux màxim és 14, així que A no pot guanyar.

14.8 NP-completesa

(a) D'una banda, sabem que tot problema dins d'NP es pot reduir en temps polinòmic a 3-SAT perquè, d'acord amb el teorema de Cook, 3-SAT és NP-difícil. D'altra banda, GRAPH ISOMORPHISM pertany a NP, perquè existeix un testimoni de mida polinòmica (la permutació) que ens permet comprovar en temps polinòmic que dos grafs donats tenen les mateixes arestes. Per tant, si es descobrís un algorisme de cost polinòmic per a 3-SAT, necessàriament es podria resoldre GRAPH ISOMORPHISM en temps polinòmic, via una reducció.

(b) Per demostrar que PARTITION és NP-complet, ens cal demostrar:

Que PARTITION pertany a NP: Si ens donen n booleans b_i que indiquen si $a_i \in S$, podem comprovar en temps polinòmic que la suma dels a_i amb $b_i = \text{true}$ és x .

Que PARTITION és NP-difícil: Podem reduir SUBSET SUM, que sabem que és NP-difícil, a PARTITION. Sigui $C = \{c_1, \dots, c_m\}$ el conjunt de nombres de SUBSET SUM, sigui $y = \sum_{c \in C} c$, i sigui t la suma objectiu. Definim $\alpha = y + t$, $\beta = 2y - t$, i $A = C \cup \{\alpha, \beta\}$. Si cridem a PARTITION amb A , i existeix una partició d'aquests $n = m + 2$ nombres, llavors tenim una solució per a SUBSET SUM:

Fixem-nos que $\sum_{a \in A} a = 4y$, per la qual cosa l'objectiu és $x = 2y$. Com $\alpha + \beta = 3y$, la solució ha de tenir un d'aquests dos nombres a un subconjunt de la partició, i l'altre nombre a l'altre. Quedem-nos amb el subconjunt que conté β . La resta de nombres d'aquest subconjunt sumen $2y - \beta = t$, així que efectivament hem obtingut una solució per a SUBSET SUM.

Per completar la demostració, calen dos detalls més: Primer, que els costs d'afegir α i β i de triar el subconjunt que conté β no incrementen el cost més que de forma polinòmica. I segon, que la crida a PARTITION troba una solució si i només si el problema original sobre SUBSET SUM també en tenia.

Parcial, 5 de novembre de 2019

15.1 Exponenciació de matrius

a) Començant en el cas base $x = 0$, cal fer x productes de matrius, cadascun de cost $\Theta(n^3)$. El cost total és doncs $\Theta(n^3x)$.

b) Ara només cal fer $\Theta(\log x)$ productes, amb un cost total $\Theta(n^3 \log x)$.

c) Si considerem la representació de x en base dos, n'hi ha prou de fer tants productes com bits estiguin a 1, aprofitant les matrius precalculades.

En el cas millor, tots els bits són 0 i no cal fer cap producte. El cost en aquest cas és $\Theta(n^2)$, corresponent a escriure el resultat.

En el cas pitjor, tots els bits són 1 i cal fer B productes. El cost en aquest cas és $\Theta(n^3B)$.

En el cas mitjà, cada bit pot ser 0 o 1 amb la mateixa probabilitat $1/2$. Per linialitat de l'esperança, el cost en aquest cas és $\Theta(n^3B)/2 = \Theta(n^3B)$.

15.2 Misteri

a) La funció *misteri* () indica si és possible escollir un subconjunt d'elements de T que sumi almenys *sum*. Si és el cas, la funció escriu les posicions dels elements del subconjunt. A més, es minimitza el nombre d'elements escollits, perquè es trien de més gran a més petit fins a tenir-ne prou.

Per exemple, suposem $T = [3, 9, 2, 5]$ i *sum* = 15. Després d'omplir V i ordenar-la amb el *sort* (), tenim $V = [\langle 2, 2 \rangle, \langle 3, 0 \rangle, \langle 5, 3 \rangle, \langle 9, 1 \rangle]$. L'algorisme trobarà que és suficient sumar els dos nombres més grans, i escriurà les seves posicions: 3 i 1.

b) El cost està dominat pel *sort* (), perquè la resta té cost constant excepte el bucle **while**() de cost $O(n)$. El cost total és doncs $\Theta(n \log n)$.

c) La funció *misteri* () també funcionaria bé amb zeros o nombres negatius: si hi ha alguna solució, la trobarà abans d'intentar usar cap d'aquests nombres. Altrament, perdrà com a molt temps linial sumant zeros i nombres negatius sense èxit.

15.3 Els k més propers a la mediana

Primer trobem la mediana m en temps $\Theta(n)$ usant l'algorisme de mediana de medianes de 5.

Després, creem una taula on cada posició tingui un parell $\langle |x_i - m|, x_i \rangle$. Si hi busquem el k -èsim element més petit amb el mateix algorisme de mediana de medianes de 5, la taula ens queda particionada amb els k elements més petits (que són els que ens interessin) a una banda, i els altres a l'altra. Ara n'hi ha prou d'escriure el segon camp dels k parells més petits. El cost total en el cas pitjor és $\Theta(n)$.

Per exemple, sigui $k = 3$ i $V = [8, 4, 1, 9, 2, 6, 7]$. Buscant-hi la mediana, obtenim $m = 6$. Ara creem la taula T , amb contingut inicial

$$T = [\langle 2, 8 \rangle, \langle 2, 4 \rangle, \langle 5, 1 \rangle, \langle 3, 9 \rangle, \langle 4, 2 \rangle, \langle 0, 6 \rangle, \langle 1, 7 \rangle].$$

Després de particionar amb $k = 3$, un possible resultat és

$$T = [\langle 2, 8 \rangle, \langle 0, 6 \rangle, \langle 1, 7 \rangle, \langle 2, 4 \rangle, \langle 5, 1 \rangle, \langle 3, 9 \rangle, \langle 4, 2 \rangle].$$

El resultat en aquest cas seria 8, 6, 7.

15.4 Euclides i Fibonacci

Segui $R(k)$ el nombre de crides recursives durant l'execució de $\text{mcd}(F_{k+2}, F_k)$. Veurem que

$$R(k) = \begin{cases} 0 & \text{si } k = 0, \\ 1 & \text{si } k = 1, \\ k - 1 & \text{si } k \geq 2, \end{cases}$$

i que $\text{mcd}(F_{k+2}, F_k) = 1$ per a tot $k \geq 0$.

Per a $k = 0, 1, 2$, les dues igualtats es poden comprovar a ma.

Per a $k \geq 3$, comencem notant que $F_{k+1} \% F_k = (F_k + F_{k-1}) \% F_k = F_{k-1} \% F_k = F_{k-1}$, perquè $F_{k-1} < F_k$. També, $F_{k+2} \% F_k = (F_{k+1} + F_k) \% F_k = F_{k-1}$. Això vol dir que l'execució de $\text{mcd}(F_{k+2}, F_k)$ comença fent la crida recursiva $\text{mcd}(F_k, F_{k-1})$, segueix amb les crides $\text{mcd}(F_{k-1}, F_{k-2}), \text{mcd}(F_{k-2}, F_{k-3}), \dots, \text{mcd}(F_3, F_2)$, i acaba amb la crida $\text{mcd}(1, 0)$, que retorna 1. Per tant, $R(k) = 1 + (k - 3) + 1 = k - 1$, i $\text{mcd}(F_{k+2}, F_k) = 1$.

[Error més comú: No adonar-se que la crida recursiva que fan $\text{mcd}(F_4, F_2)$ i $\text{mcd}(F_3, F_2)$ és directament $\text{mcd}(1, 0)$ i no pas $\text{mcd}(F_2, F_1)$. Aquest error duia a pensar que el nombre de crides recursives seria k també en el cas $k \geq 2$, la qual cosa es refuta provant-ho amb $k = 2$.]

15.5 Programacions dinàmiques

a) Per tal de fer la recurrència més elegant, definim $a_{n,t} = 0$ per a $t < 0$. Notem els casos base $a_{0,t} = 1$ quan $t = 0$ i $a_{0,t} = 0$ quan $t \neq 0$. Per a $n \geq 1$ i $t \geq 0$, tenim

$$a_{n,t} = a_{n-1,t-d_1} + \dots + a_{n-1,t-d_k} \quad (15.1)$$

b) L'algorisme $A(n)$ produirà n vectors $v[1], \dots, v[n]$ on $v[i]$ tindrà mida $9i + 1$ i contindrà els $a_{i,j}$ amb $0 \leq j \leq 9i$. El cost és $\sum_{i=1}^n \Theta(i) = \Theta(n^2)$.

c) Aquesta vegada els casos base de la recurrència seran els $a_{0,t}$ i els $a_{1,t}$ definits així:

$$\begin{aligned} a_{0,0} &= 1 & \text{si } t &= 0 \\ a_{0,t} &= 0 & \text{si } t &\neq 0 \\ a_{1,t} &= 1 & \text{si } t &\in \{d_1, \dots, d_k\} \\ a_{1,t} &= 0 & \text{si } t &\notin \{d_1, \dots, d_k\}. \end{aligned}$$

Com en el cas anterior, definim $a_{n,t} = 0$ per a tot $n \geq 0$ i $t < 0$, i per a $n \geq 2$ i $t \geq 0$ tenim

$$a_{n,t} = \sum_{i=0}^t a_{\lfloor n/2 \rfloor, i} \cdot a_{\lceil n/2 \rceil, t-i}. \quad (15.2)$$

d) Volem un algorisme $B(n, v)$ que, donat $n \geq 1$, produeixi un vector $v[0, \dots, 9n]$ de mida $9n + 1$ on $v[t] = a_{n,t}$.

Per a $n = 1$ el vector v té mida 10 i s'inicialitza amb $v[t] = 1$ si $t \in \{d_1, \dots, d_k\}$ i $v[t] = 0$ si $t \notin \{d_1, \dots, d_k\}$.

Per a $n \geq 2$, l'algorisme fa una crida a $B(n/2, w)$ amb un vector w auxiliar. Com que n és una potència de dos, $n/2$ torna a ser una potència de dos, i a més $\lfloor n/2 \rfloor = \lceil n/2 \rceil$. Per tant, el vector w conté els $a_{\lfloor n/2 \rfloor, s}$ i $a_{\lceil n/2 \rceil, r}$ que es necessiten per computar els $a_{n,t}$ segons l'expressió (15.2) i l'observació òbvia que $a_{n/2, s} = 0$ si $s > 9n/2$. De fet, el vector v que busquem és la convolució $w \otimes w$ del vector w amb 0's afegits al final fins que tingui longitud $9n + 1$. Per tant, podem aplicar la Transformada Ràpida de Fourier (FFT) per computar v a partir de w en temps $\Theta(n \log n)$.

L'algorisme final $B(n)$ farà una crida a $B(n, v)$ amb un vector v creat per a l'ocasió, i el retornarà.

Si $T(n)$ denota el cost d'aquest algorisme, llavors $T(n) = T(n/2) + \Theta(n \log n)$, recurrència que té solució $T(n) = \Theta(n \log n)$.

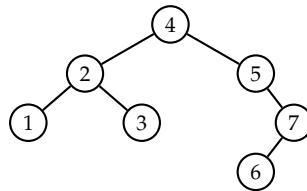
[Error més comú: No adonar-se que l'expressió (15.2) és una convolució que es pot computar amb la FFT. Sense la FFT, la recurrència del cost queda $T(n) = T(n/2) + \Theta(n^2)$, la qual és $\Theta(n^2)$, i per tant no millora la versió anterior.]

16

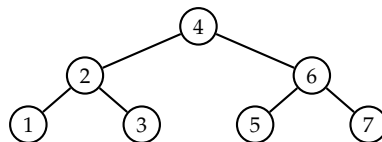
Final, 5 de gener de 2019

16.1 Arbres diversos

a) L'arbre binari de cerca resultant d'inserir les claus 4, 5, 7, 2, 1, 3 i 6 en aquest ordre en una arbre inicialment buit és



b) L'arbre AVL resultant d'inserir les claus 4, 5, 7, 2, 1, 3 i 6 en aquest ordre en una arbre inicialment buit és



Per obtenir-lo es fan dues rotacions simples en inserir les claus 7 i 1 i dues dobles en inserir les claus 3 i 6.

16.2 Heap, heap, ...

Què fa `misteri_1()` i amb quin cost? Es construeix un min-heap amb els n elements del vector original amb cost $\Theta(n)$. A continuació, s'extreuen m elements posant-los al final del vector, de manera semblant al heapsort. Aixó fa que els m elements més petits ocupin les m últimes components del vector, en ordre decreixent. Aquesta fase té cost $\Theta(m \log n)$

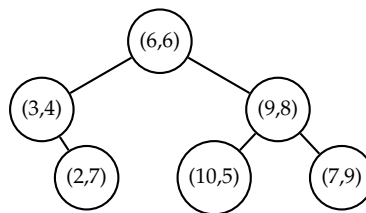
en cas pitjor. Per últim, invertim parcialment el vector amb cost $\Theta(m)$ per a ubicar els m elements més petits en les primeres m components del vector, en ordre creixent. El cost total és $\Theta(n + m \log n)$ en cas pitjor.

I *misteri_2* ()? Construïm un max-heap amb els primers m elements del vector, amb cost $\Theta(m)$. A continuació fem un recorregut sobre els $n - m$ elements restants fent el següent: comparem el màxim del heap amb l'element x en curs; si el màxim és més petit o igual que x , avancem al següent element del vector; si el màxim és més gran, llavors el màxim i x s'intercanvien i es fa un "enfosar" sobre l'element x . D'aquesta manera el heap contindrà els m elements més petits tractats fins al moment. Aquesta fase té cost $\Theta((n - m) \log m) = O(n \log m)$. Finalment, el max-heap situat a $v[0, m-1]$ s'ordena com es fa en el heapsort amb cost en cas pitjor $\Theta(m \log m)$. Tot plegat el cost d'aquest altre algorisme és $O(n \log m)$.

Comparant *misteri_1* () amb *misteri_2* (), *misteri_1* () és millor que *misteri_2* () excepte si $m = \Theta(n)$. Aleshores els dos algorismes tenen un cost asimptòticament equivalent.

16.3 Rangs en arbres 2d

a) L'arbre 2d resultant d'inserir les claus (6,6),(3,4),(9,8), (2,7), (7,9) i (10,5) en aquest ordre en una arbre inicialment buit és



La consulta $(1, 6.5) \times (2, 7.5)$ retorna les 3 claus (6,6), (3,4) i (2,7).

b) Per fer una consulta pel rang $r = (x_1, y_1) \times (x_2, y_2)$ podem procedir recursivament per a cada subarbre, començant per l'arrel. Si el subarbre és buit, retornem. Si no, i conté la clau (x, y) , verifiquem si $(x, y) \in r$ i la guardem si és el cas. Si l'arrel del subarbre discrimina per la primera coordenada i $x < x_1$, continuem recursivament pel subarbre dret, en cas que $x > x_2$ continuem recursivament pel subarbre esquerra i, finalment, si $x_1 < x < x_2$ s'ha de continuar recursivament per tots dos subarbres. S'ha de procedir anàlogament en cas que l'arrel del subarbre discrimini per la segona coordenada.

c) Cas millor: $\Theta(a)$. Cas pitjor: $\Theta(n)$.

16.4 Bonnie i Clide

Aquest problema pertany a la classe NP perquè donats dos conjunts de xecs, un per a Bonnie i l'altre per a Clide, es pot verificar en temps polinòmic que entre els dos conjunts hi ha tots els xecs de la bossa i que tots dos conjunts sumen la mateixa quantitat de diners. Aquest problema és NP-complet perquè és exactament el problema de PARTICIO, un problema NP-complet conegut.

De moment no se sap si PARTICIO pertany a la classe P o no, depèn de P i NP són iguals o diferents.

16.5 Triangle de Sierpinski

a) Tenim $f(1) = 1$. Quan $n = 2^k$, $f(n) = 3f(n/2)$, el qual per inducció és $f(n) = 3^k$. Altrament, si considerem la representació d' n en binari, amb 2^k com a valor del bit de pes més gran, aleshores tenim $f(2^k) = 3^k$ creus a la banda de dalt, més dues còpies d'un triangle amb $n - 2^k$ files a la banda de baix.

Per exemple, amb $n = 12$ tenim $k = 3$, i hi ha $3^3 = 27$ creus a la banda de dalt, i $2f(12 - 8) = 2f(2^2) = 2 \cdot 3^2 = 18$ creus a la banda de baix.

b) El cas millor és quan n només té un bit, és a dir, $n = 2^k$. En aquest cas només cal fer una crida a $f(n)$, de cost $\Theta(k) = \Theta(\log n)$.

El cas pitjor és quan $n = 2^k - 1$. En aquest cas es fa una crida recursiva per a cada bit. El cost total és $\Theta(\sum_{i=1}^k i) = \Theta(k^2) = \Theta(\log^2 n)$.

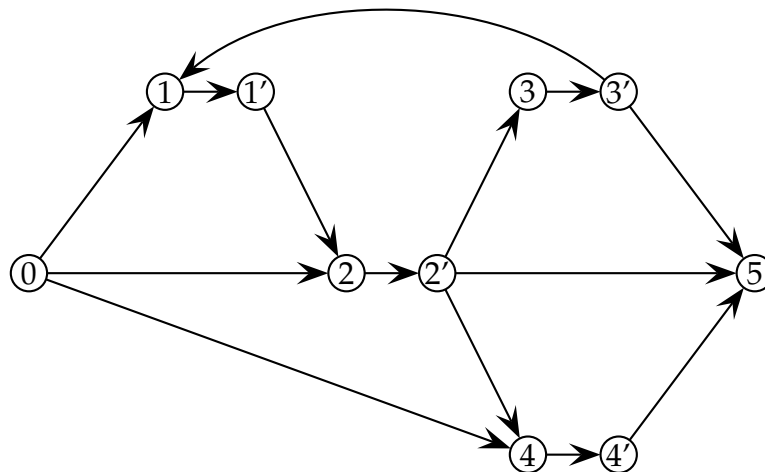
c) Si primer busquem el bit de més pes, i després ens movem cap als bits més petits, sense tornar a començar des del principi, el cost total sempre és $\Theta(\log n)$. Possible codi:

```
int f(int n, int i) {
    while (D[i] ≥ n) --i;
    if (n == D[i+1]) return T[i+1];
    return T[i] + 2*f(n - D[i], i);
}
```

```
int f(int n) {
    int i = 0;
    while (D[i] < n) ++i;
    return f(n, i);
}
```

16.6 Màxim nombre de camins independents

N'hi ha prou de calcular el flux màxim en un graf on tots els arcs tenen capacitat 1, però afegint la restricció que tots els vèrtexs (excepte 0 i $n - 1$) també tinguin capacitat 1. Això s'aconsegueix duplicant cada vèrtex, com es pot apreciar amb l'exemple:



16.7 Knuth-Morris-Pratt

a) "aabbbaabaa"

b) Siguin $c_1, c_2, \dots$ els caràcters de la paraula. El segon 0 implica $c_1 \neq c_2$. El penúltim 2 implica $c_7 = c_2$. L'últim 2 implica $c_7 = c_1$. Això és una contradicció.

16.8 Teoria de jocs

a) Sabem que una pila té número 0 si i només si és perdedora. Siguin b i n respectivament el nombre de pedres blanques i de pedres negres. Si $b = n = 0$, la pila és trivialment perdedora. Si $b \neq n$, es pot deixar la posició $b' = n' = 0$ a l'adversari, de manera que la posició actual és guanyadora. Si $b = n > 0$ només ens podem moure a posicions guanyadores, i per tant la posició actual és perdedora. En definitiva, una pila té número 0 si i només si $b = n$.

b) Necessitem calcular alguns nombres. Per simetria, $\text{nim}(b, n) = \text{nim}(n, b)$. I ja sabem que $\text{nim}(n, n) = 0$. A més, és trivial veure que $\text{nim}(n, 0) = n$. Fent uns pocs càlculs, obtenim $\text{nim}(2, 1) = 3$, $\text{nim}(3, 1) = 2$, $\text{nim}(4, 1) = 5$, i $\text{nim}(3, 2) = 4$. Com que el número de la composició és l'"or exclusiu" dels nombres, en aquest cas 5, 4 i 0, obtenim com a número total 1. Com que és diferent de 0, la posició és guanyadora. Un possible moviment guanyador consisteix a treure la pedra blanca de la primera pila.

16.9 Quatre nombres

Sigui V un vector que conté els nombres. En un "map", per a tot parell $0 \leq i < j < n$, hi guardem la suma $V[i] + V[j]$ com a clau i el parell (i, j) com a valor. Si trobem una suma repetida, tenim els índexs dels quatre valors demanats (els dos emmagatzemats i els dos actuals). Si no trobem cap suma repetida, l'equació no té solució.

El cost seria $\Theta(n^2)$ si no fos pel cost logarítmic del "map". Per evitar-lo, podem usar una "taula hash", amb cost esperat constant per operació.

Tal qual, l'algorisme fallaria si hi haguessin nombres repetits. Per exemple, amb el vector $V = [3, 4, 4']$ obtindríem la falsa solució $3 + 4 = 3 + 4'$, usant el mateix 3 dues vegades.

Parcial, 5 de novembre de 2018

17.1 Recurrències

a) $T(n) = \Theta(n^2)$.

b) Suposem que $f(n) = \Theta(n^x)$ però $0 \leq x \leq 2$. Si $x > \log_4 7 \approx 1.4$ llavors $T(n) = \Theta(n^x)$. Així, anem reduint $T(n)$ a mesura que reduim $f(n)$, mentre $x > \log_4 7$. Quan $x = \log_4 7$ llavors $T(n) = \Theta(n^{\log_4 7} \log n)$. Si $x < \log_4 7$ llavors $T(n) = \Theta(n^{\log_4 7})$ i no reduïrem més el cost, tot i reduir $f(n)$.

c) No es redueix el cost $T(n)$. Si reduïm el nombre de crides i fem $a < 7$ crides, continuem tenint $\log_4 a < 2$ i el cost $T(n) = \Theta(n^2)$.

17.2 Misteri

a) El cost en el cas pitjor és el mateix de quick-sort, $\Theta(n^2)$.

b) Aquest procediment recursiu troba un element de rang k , $i \leq k \leq j$, en el subvector $V[e..d]$, amb $0 \leq e \leq i \leq j \leq d < V.size()$; el subvector conte els elements e -èssim, $(e+1)$ -èssim, ..., d -èssim de V . Considerant el vector $V = [5, 6, 1, -2, 0, 3]$ la crida *misteri* ($V, 1, 3, 0, 5$) pot retornar 0, 1 o 3, que són els elements a les posicions 1, 2 i 3 amb V ordenat.

c) Es pot resoldre el mateix problema més eficientment fent servir l'algorisme de selecció en temps lineal i buscant pel k -èssim element amb $i \leq k \leq j$.

17.3 Ordenació i més

a) A més de la crida a *sort* (v) la crida a *misteri* (v, x) fa, en el cas pitjor, n cerques dicotòmiques amb un cost de $O(\log n)$ cadascuna. Cal observar però que aquest cost en cas pitjor de $O(n \log n)$ es asintòticament igual o menor que el cost de *sort* amb qualsevol dels tres algorismes d'ordenació en els casos mitjà i pitjor. Per tant el cost total en aquests dos casos és el cost de la crida a *sort* (v).

Així, si *sort* fa servir l'algorisme d'ordenació per selecció, amb cost $\Theta(n^2)$ en els casos mitjà i pitjor, el cost total seria $\Theta(n^2)$ en tots dos casos. En canvi, si *sort* fa servir merge-sort el cost total en els casos mitjà i pitjor seria $\Theta(n \log n)$. Finalment, si *sort* fa servir quick-sort, el cost total en cas mitjà és $\Theta(n \log n)$ mentre que en el cas pitjor és $\Theta(n^2)$.

b) *misteri* (v, x) determina si el vector v conté dos elements diferents que sumin x o bé un que sigui la meitat de x .

17.4 Eficiència

L'algorisme B és més eficient que l'algorisme A . Si prenem logaritmes de les respectives complexitats tenim $\sqrt{\log_2 n} \ll \frac{1}{2} \log_2 n$, és clar que la complexitat de B és molt menor que la d' A .

17.5 Multiplicació súper-eficient

$\Theta(1)$, perquè hi ha un nombre finit d'entrades possibles. Potencialment, es poden tenir precalculades totes les respostes en una matriu multidimensional enorme.

17.6 El nombre perdut

Fent servir la cerca dicotòmica es pot trobar el nombre perdut en temps $O(\log n)$. Suposem que tenim les posicions del vector numerades de 1 a $n - 1$. Llavors, durant la cerca dicotòmica, al mirar l'element a la posició i -èssima, si aquest element és igual a i l'element perdut correspon a una posició posterior a i del vector i si és diferent, a una posició anterior. Aturem la cerca quan quedi un únic element, la seva posició serà l'element perdut.

17.7 El rètol lumínic

Sigui M' la matriu d'enters de mida $n \times m$ tal que $M'[i][j]$ ($0 \leq i < n, 0 \leq j < m$) és la quantitat de llums enceses de $M[i_1..i_2][j_1..j_2]$. La matriu M' es pot calcular en temps $\Theta(nm)$. Cada pregunta p es pot calcular en temps $\Theta(1)$ calculant $r = M'[i_1][j_2] + M'[i_1][j_1] - M'[i_2][j_1] - M'[i_1][j_2]$ que dona el nombre de llums enceses a la graella demanada. El cost total d'aquesta solució és: $\Theta(nm + p)$.

Extraordinari, 29 de juny de 2018

18.1 Misteri

- a) En el cas pitjor, el codi executa dos bucles amb n iteracions cadascun, totes de cost constant. Per tant, el cost és $\Theta(n)$.
- b) Comprova si b és igual que a , potser amb els caràcters moguts (cíclicament) cap a un costat.
- c) Amb caràcters repetits, el codi podria fallar, dient que b no és equivalent a a quan sí que ho és. Per exemple, si a val "xxy" i b val "xyx".

18.2 Suma de dos elements

Una possibilitat de resoldre aquest problema en temps lineal es fer servir una taula de hash H . Els elements de V s'insereixen un a un a H . Cada vegada que s'insereix (amb cost mitjà $\Theta(1)$) un nou element amb valor y es comprova si a H hi ha l'element amb valor $x - y$ (en cost mitjà $\Theta(1)$). Aquest procediment es fa com a molt una vegada per cada element de V , per tant el cost mitjà total és $\Theta(n)$.

18.3 Heaps

Aquesta és l'evolució del vector:

[4, 1, 3, 2, 16, 9, 10, 14, 8, 7]

[4, 1, 3, 14, 16, 9, 10, 2, 8, 7]

[4, 1, 10, 14, 16, 9, 3, 2, 8, 7]

[4, 16, 10, 14, 7, 9, 3, 2, 8, 1]

[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]

18.4 NP-completesa

- a) Fals. Com que no sabem si X és a NP, no podem garantir que es pugui reduir a Y en temps polinòmic.
- b) Fals. Com que no sabem si Y és a NP, no podem concloure que sigui NP-complet. De fet, tampoc s'especifica si la reducció de X a Y és en temps polinòmic.
- c) Fals. No podem dir res sobre Y , excepte que es redueix a X .

18.5 Problema de l'aturada

El problema és decidible. En efecte, sigui N tal que per a tota $n \geq N$ existeix almenys un programa que para sempre, per exemple (hem omès les inclosions):

```
int main() {
    cout << "";
```

Sigui N la mida d'aquest programa. A dintre de la cadena "" podem posar qualsevol cadena més llarga, així que sempre hi ha algun programa que parará per a tota $n \geq N$.

Per a cadascuna de les $n < N$, o bé hi ha algun programa de mida n que para sempre, o bé no n'hi ha cap. Considerem aquest programa:

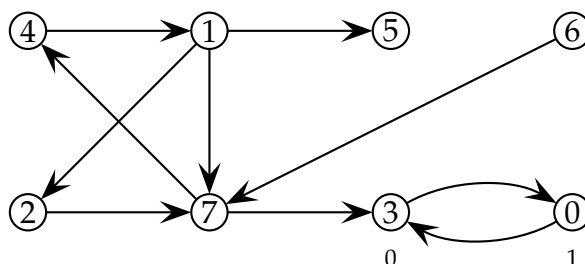
```
int n;
cin >> n;
if (n ≥ N) cout << "si" << endl;
else if (n == 0) cout << "no" << endl;
else if (n == 1) cout << "no" << endl;
...
else if (n == N - 2) cout << "no" << endl;
else cout << "no" << endl; // N - 1
```

A partir d'aquest programa en podem contruir 2^N , fent totes les combinacions de si/no (excepte per a la línia amb $n \geq N$). Encara que (els humans) potser no sapiguem quina és la bona, exactament una l'és. Per tant, existeix un programa que ho decideix, i el problema és decidible.

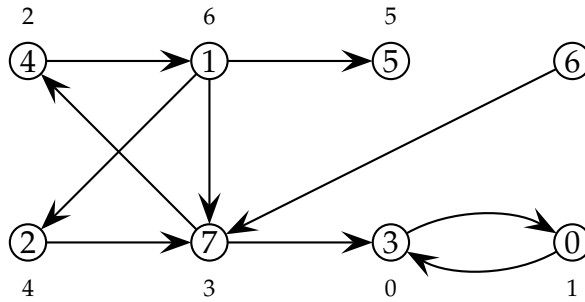
18.6 Components fortament connexos

En les explicacions que segueixen, per fixar un criteri, suposarem que els recorreguts es fan triant sempre el vèrtex més petit en tots els passos.

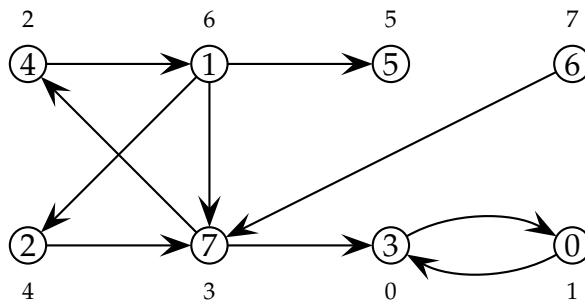
Primer, calculem el graf invers. Després, fem un recorregut en profunditat, marcant amb comptadors de temps els vèrtexs a mesura que sortim d'ells. Primer, marquem el 3 i el 0:



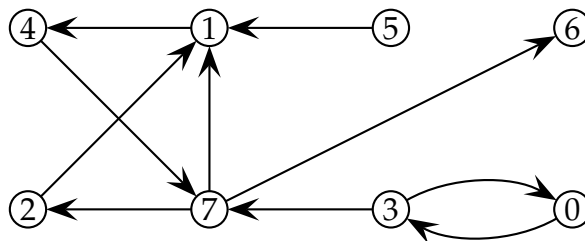
Després, a partir de l'1 marquem la resta de vèrtexs menys el 6:



Finalment, marquem el 6:



Ara tornem al graf original:

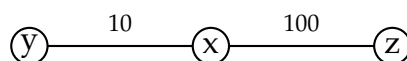


Hi fem recorreguts (en profunditat, per exemple), usant els comptadors de temps de gran a petit per triar per quin vèrtex comencem. Començant pel 6, que té el comptador de temps màxim, marcarem només el 6. Usant després l'1, marcarem l'1, el 4, el 7 i el 2. A partir del 5, el marcarem només a ell. Finalment, a partir del 0, trobarem el 0 i el 3.

Així doncs, i només amb cost linial en el nombre de vèrtexs i arestes, hem trobat els quatre components fortament connexos del graf: $\{6\}$, $\{1, 2, 4, 7\}$, $\{5\}$ i $\{0, 3\}$.

18.7 Tall mínim

Sí que és possible. Per exemple:



18.8 Teoria de jocs

Sabem que una posició és perdedora si i només si el seu número és 0. Anomenem X i Y les posicions, i nx i ny els seus números. La demostració és per inducció en la mida conjunta d' X i Y . Si X i Y són buides, la composició també ho és. Així doncs, per al cas base es compleix que la composició és perdedora i els números són iguals (en aquest cas, a 0).

Suposem que la propietat és certa fins a una mida determinada, i considerem la mida següent. Si $nx = ny$, per definició de número, qualsevol jugada que fem a X anirà a una posició amb número diferent a nx . (I similarment amb Y .) Per tant, per H.I. la composició resultant serà guanyadora. Com a conclusió, la posició actual és perdedora.

Altrament, i sense pèrdua de generalitat, suposem que $nx > ny$. Per definició de número, hi ha almenys una jugada a X que ens du a una posició amb número ny , la composició de la qual amb Y , per H.I., és perdedora. Per tant, la posició actual és guanyadora.

18.9 Què calcula?

El codi escriu 1, 2, 3, 5, 7, 11, ...

El seu cost és $\Theta(\sum_{x=1}^N \sum_{i=x}^N 1) = \Theta(N^2)$.

El codi resol un cas particular del problema de "el canvi de monedes". Aquí, estem suposant que hi ha un nombre infinit de monedes de cada tipus, i estem calculant de quantes maneres podem formar cada canvi.

Per exemple, amb $x = 4$ en tenim cinc:

$4 = 4$, $4 = 3 + 1$, $4 = 2 + 2$, $4 = 2 + 1 + 1$, $4 = 1 + 1 + 1 + 1$.

19

Final, 18 de gener de 2018

19.1 Recurrències

Una possibilitat seria aquest codi per a $n \geq 1$:

```
void escriu (int n) {  
    cout << ' ' << n;  
    if (n > 1) {  
        escriu (n - 1);  
        escriu (n - 1);  
    } } }
```

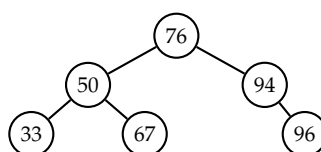
Sigui $T(n)$ el temps de la funció *escriu* () aplicada a una entrada $n \geq 1$. Llavors, tenim $T(n) = 2T(n - 1) + \Theta(1)$, que és $\Theta(2^n)$ utilitzant el teorema mestre.

19.2 Els k més petits, en ordre

Una possibilitat consisteix a transformar el vector en un min-heap amb cost $\Theta(n)$ (fent servir l'algorisme "heapify") i a continuació obtenir, en ordre, els $k = n / \log n$ elements més petits del heap, cadascun amb cost $\Theta(\log n)$, amb cost total $\Theta(n)$ en el cas pitjor.

19.3 Arbres AVL

L'arbre AVL resultant d'inserir les claus 94, 33, 50, 76, 96 i 67 en aquest ordre en una arbre inicialment buit és



Per obtenir-lo es fan dues rotacions dobles en inserir les claus 50 i 67.

19.4 NP-completesa

- El problema de factoritzar un nombre donat x pertany a NP. Efectivament, donat el nombre x i un conjunt de factors, es pot verificar en temps polinòmic en b que el nombre de factors està entre 1 i b , que cada factor es codifica en com a molt b bits, que cada factor és un nombre primer i que la multiplicació de tots els factors és x .
- Si es demostrés que no existeix cap algorisme polinòmic en b per factoritzar un nombre donat x podríem deduir $P \neq NP$, perquè sabem que P és a NP i hauríem trobat un problema a NP que no és a P.
- Si es trobés algun algorisme polinòmic en b per factoritzar un nombre donat x podríem deduir ni $P = NP$ ni $P \neq NP$, a menys que el problema de factoritzar un nombre donat x sigui NP-complet, amb la qual cosa sí que podríem deduir $P = NP$.

19.5 2-SAT

Primer, construïm un graf dirigit amb 8 vèrtexs (dos per a cada variable, una afirmada i l'altra negada), i dos arcs per a cada disjunció. Per exemple, $(\bar{a} \vee c)$ genera els arcs $a \rightarrow c$ i $\bar{c} \rightarrow \bar{a}$. Després, calculem en temps lineal els components fortament connexos del graf. En aquest cas, obtenim quatre "súper-nodes": Un només amb b , un altre amb a , c i $\bar{d}$, un altre amb $\bar{a}$, $\bar{c}$ i d , i un altre només amb $\bar{b}$. El primer apunta al segon i al tercer, i aquests apunten al quart.

Com que cap component conté una variable i la seva negada, la fórmula té almenys una solució. Per calcular quantes, n'hi ha prou d'observar que b implica transitivament $\bar{b}$, així que b ha de ser falsa. Respecte a les altres variables, o són certs tots els literals del segon component o ho són tots els del tercer. Per tant, hi ha exactament dues solucions: o bé $a = 1, b = 0, c = 1$ i $d = 0$, o bé $a = 0, b = 0, c = 0$ i $d = 1$.

19.6 Teoria de jocs

Primer, calculem el número de cada pila de pedres. Recordem que, per definició, el número d'una pila buida és zero, i el de les piles no buides es calcula inductivament com el mínim número al qual no es pot arribar amb cap jugada:

n	0	1	2	3	4	5	6	7	8
número(n)	0	1	2	0	1	2	3	4	5

Després de jugar, tenim quatre piles, amb 1, 3, 7 i 8 pedres, i números respectius 1, 0, 4 i 5. Com que el "or exclusiu" dels números és 0, la posició és perdedora, i per tant la jugada proposada és guanyadora.

19.7 Què fan?

Les tres funcions comproven si V conté algun element repetit, recorrent-lo d'esquerra a dreta. La primera usa un `set<int>` per guardar-hi els elements que s'han vist fins ara i poder comprovar amb cost logarítmic si l'element actual ja ha sortit. La segona ordena prèviament (una còpia de) el vector perquè els elements iguals quedin situats adjacentment. I la tercera implementa una taula de dispersió sencilla.

Per a les tres funcions, un cas millor consisteix per exemple en que tots els enters siguin iguals, i un cas pitjor en que tots siguin diferents. A més, per a $h(V)$ cal que la “funció de hash” de tots els enters apunti a la mateixa posició i .

Així doncs, per a $f(V)$, el cost en el cas millor és $\Theta(1)$ (només mirem dos elements) i el cost en el cas pitjor és $\sum_{1 \leq j \leq n} \Theta(\log j) = \Theta(n \log n)$. Tant el cost en el cas millor com en el cas pitjor de $g(V)$ és $\Theta(n \log n)$, que és el cost del `sort()` de la STL, perquè la resta del codi té com a molt cost lineal. Per a $h(V)$, el cost en el cas millor és $\Theta(n)$ (cal crear el vector H), i el cost en el cas pitjor ve dominat per $\sum_{1 \leq j \leq n} \Theta(j) = \Theta(n^2)$.

19.8 Codi estrany

El codi és una implementació senzilla de l'algorisme de Prim per trobar el cost d'un arbre generador mínim d'un graf no dirigit i connex amb pesos positius a les arestes.

El codi fa un `push()` (i un `top()`, i un `pop()`) per a cada aresta del graf (dues vegades). En el cas pitjor, el cost de la majoria d'aquestes operacions serà $\Theta(\log n)$, per a un cost total $\Theta(m \log n)$, on m és el nombre d'arestes.

< Falta una explicació breu de l'algorisme amb un graf petit. >

Parcial, 6 de novembre de 2017

20.1 Ordenació

Una solució senzilla consisteix a declarar un vector amb 26 enters, tots inicialment a zero, i recórrer la paraula una sola vegada, comptant el nombre d'aparicions de cada lletra. Al final, només cal escriure cada lletra tantes vegades com hagi aparegut. El cost és clarament $\Theta(26 + n + n) = \Theta(n)$.

Aquest cost no contradiu la fita inferior $\Omega(n \log n)$ per als algorismes d'ordenació de propòsit general, basats en comparacions, perquè aquest algorisme usa el fet que els elements del vector pertanyen a un interval fitat i petit de valors possibles, i no ha de comparar els elements entre si.

20.2 Cues de prioritats

- a) El codi escriu els x elements més petits del vector en ordre decreixent.
- b) Les operacions més costoses són $Q.push(V[i])$ i $Q.pop()$, amb cost logarítmic en el cas pitjor en el nombre d'elements a la cua de prioritats Q .

La contribució dels *push* és

$$\sum_{i=1}^x \log i + \sum_{i=x+1}^n \log x = \Theta(x \log x) + \Theta((n-x) \log x) = \Theta(n \log x) .$$

La contribució dels *pop* és similar. La contribució de la resta d'operacions, totes de cost constant, és $\Theta(n)$. El cost total és doncs $\Theta(n \log x)$.

- c) Una manera alternativa d'aconseguir els x elements més petits del vector consisteix a seleccionar l'element k -èsim amb un mètode lineal en n , ja sigui en el cas pitjor o en el mitjà. Després, el vector queda particionat directament amb els x elements més petits en una de les bandes. Quan $x = \sqrt{n}$, $\Theta(n \log x) = \Theta(n \log n)$ és pitjor que $\Theta(n)$.

20.3 Programació dinàmica

- a) La recurrència calcula el nombre de maneres diferents de sumar j , si es poden usar els i primers elements del vector V , però cadascun només dues vegades com a màxim.
- b) La programació dinàmica corresponent tindria cost en temps $O(n \times k)$, perquè cal calcular $f(i, j)$ per a cada possible combinació com a molt una vegada, cadascuna fàcilment en temps constant.

Com que $f(i, j)$ només depèn de valors de $i' = i - 1$, n'hi ha prou de guardar un vector de mida $k + 1$ per fer tots els càlculs, amb cost extra en espai $\Theta(k)$.

20.4 Multiplicació russo-francesa

- a) A cada crida recursiva es fan les operacions següents: una divisió per dos (amb cost lineal en el nombre de bits), una comprovació de paritat (mirant l'últim dígit amb cost constant), i una multiplicació per 2 (amb cost lineal en el nombre de bits), per a un total de $\Theta(n)$ operacions a cada crida recursiva. Com que cada crida disminueix en un bit la llargada de y , el cost total es $\Theta(n^2)$.
- b) Per exemple, fent servir l'algorisme de Karatsuba i Ofman, dos enters d' n bits es poden multiplicar en temps $O(n^{\log_2 3})$, el qual és un cost millor.

20.5 Notació asimptòtica

$f(n)$	$g(n)$	$f(n) = X(g(n))$
$n^{1/3}$	$n^{2/3}$	$f(n) = O(g(n))$
$100n + \log(n)$	$n + (\log(n))^2$	$f(n) = \Theta(g(n))$
$n2^n$	3^n	$f(n) = O(g(n))$
$10 \log(n)$	$\log(n^2)$	$f(n) = \Theta(g(n))$
2^n	$2^{n/2}$	$f(n) = \Omega(g(n))$

20.6 Zeros i uns

- a) El cost és clarament $\Theta(n^2)$, perquè l'algorisme fa n crides recursives reduint la talla del problema en 2 a cada crida: a la funció *parteix*, la i s'incrementa fins al valor d , i la d només es decrementa en 1.
- b) Analitzem la primera crida a la funció *parteix*. El pivot $v[0]$ serà 0 o 1 amb igual probabilitat. Si el pivot és 0, incrementem i fins a trobar el primer 1 del vector, o fins a arribar al valor de d . En mitjana, i només s'incrementa en 2. A continuació decrementem d fins al valor $i - 1$, perquè mentre es decrementa d no es troba cap valor més petit que el pivot. Si el pivot és 1, la situació és similar.

Per tant, tant si el pivot és 0 com si és 1, *parteix* deixa per ordenar recursivament dos vectors, un dels quals amb quasi tots els elements originals, per la qual cosa el cost asimptòtic mitjà també és $\Theta(n^2)$.

Extraordinari, 27 de juny de 2017

21.1 Cerques i insercions

Cerca	Cas pitjor	Cas mitjà
Taula	$\Theta(n)$	$\Theta(n)$
Taula ordenada	$\Theta(\log n)$	$\Theta(\log n)$
Arbre binari de cerca	$\Theta(n)$	$\Theta(\log n)$
Arbre AVL	$\Theta(\log n)$	$\Theta(\log n)$
Taula de hash	$\Theta(n)$	$\Theta(1)$
Perfect hashing	$\Theta(1)$	$\Theta(1)$

Inserció	Cas pitjor	Cas mitjà
Taula	$\Theta(n)$	$\Theta(n)$
Taula ordenada	$\Theta(n)$	$\Theta(n)$
Arbre binari de cerca	$\Theta(n)$	$\Theta(\log n)$
Arbre AVL	$\Theta(\log n)$	$\Theta(\log n)$
Taula de hash	$\Theta(n)$	$\Theta(1)$

Perfect hashing considera diccionaris o conjunts *estàtics*: no es poden inserir ni esborrar claus (totes les claus es donen quan es construeix).

21.2 Encara un altre codi misteriós

El codi escriu la longitud del cicle al qual s'arriba aplicant repetidament la funció *next()* començant en una *n* inicial donada, o bé es penja si no s'arriba a cap cicle. (A la pràctica, el codi podria petar per memòria si el cicle no existís, o si es trobés després de molts passos.)

Senzillament, primer es guarden tots els nombres a un *set* `<int>` fins a trobar-ne un de repetit. Després, es tornen a generar nombres fins a retornar a l'inici del bucle, comptant quants passos cal fer.

El cost és $O(m \log m)$, on *m* és el nombre d'elements abans i dins del cicle.

21.3 I un altre

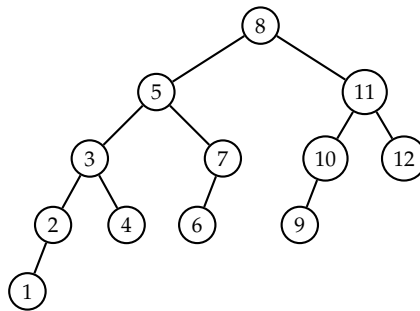
El codi és una senzilla variant de l'algorisme de Floyd-Warshall, on només es calcula l'anomenada clausura transitiva (qui està connectat amb qui), en lloc de les distàncies mínimes.

L'error és el mateix que es comet sovint amb l'algorisme de Floyd-Warshall, consistent a posar el bucle per a la k en tercer lloc, quan hauria de ser el primer de tots.

21.4 Arbres AVL

Un arbre binari és de cerca si, per a tota clau x , les claus contingudes en el seu subarbre esquerre són totes més petites que x , i les contingudes en el seu subarbre dret són totes més grans que x . Un arbre binari de cerca és AVL si la diferència d'alçades de cada parell de nodes germans és com a molt 1.

Sí que existeix algun arbre AVL amb 12 nodes i alçada 5. De fet, 12 és el mínim nombre de nodes necessaris per aconseguir aquesta alçada. Aquest n'és un:



21.5 La motxilla

1. Considerem que només hi ha dos objectes. El primer té pes 1 i benefici 2. El segon té pes C i benefici C . Llavors, l'algorisme proposat només col·locarà el primer objecte a la motxilla (perquè la seva relació benefici per pes és 2, mentre que per l'altre és 1) i tindrà benefici 2. El segon objecte ja no cap a la motxilla. En canvi, si s'hagués posat el segon objecte, el benefici seria C . Com que C pot ser arbitràriament gran, l'algorisme proposat no pot ser un algorisme d'aproximació de raó constant.

2. Considerem un nou algorisme: Com a solució es retorna la millor d'entre la de l'algorisme proposat i la que només inclou el primer objecte que j'ha no s'ha agafat.

Si la solució de l'algorisme proposat és subòptima, llavors ha de sobrar cert espai a la motxilla al final, diguem-ne $C - S$. Imaginem per un moment que el problema permetés agafar una fracció d'un objecte. Llavors, es podria agafar la solució de l'algorisme proposat (incloent els primers k objectes en ordre de raó benefici/pes) i afegir-hi una fracció del següent objecte (el $k + 1$), de forma que afegint $b_{k+1}(C - S)/p_{k+1}$ al benefici s'igualaria (o sobrepassaria) el valor del benefici òptim OPT (pel problema sense fraccions). Per tant, o bé $\sum_{i=1}^k b_i \leq \frac{1}{2} \text{OPT}$ o bé $b_{k+1} \geq b_{k+1}(C - S)/p_{k+1} \geq \frac{1}{2} \text{OPT}$.

21.6 Els del mig

L'algorisme de mediana de medianes permet trobar la mediana d'un vector en temps lineal en el cas pitjor. El podem utilitzar per trobar la mediana del vector i partir-lo entre els $n/2$ element més petits i els $n/2$ element més grans. Després, podem trobar la mediana dels petits i la dels grans, per obtenir els elements del mig. Tot plegat resol el problema en cost $\Theta(n)$ en el cas pitjor.

21.7 Collatz i Turing

Podem descriure el procés de Collatz amb aquesta funció:

```
void collatz (nat n) {
    if (n != 1) {
        if (n % 2 == 0) collatz (n/2);
        else collatz (3*n + 1);
    }
}
```

Per saber si el procés de Collatz s'atura per a un cert natural n , podem utilitzar la següent funció, que usa la hipotètica funció T :

```
bool collatz_atura (nat n) {
    return T(collatz, n);
}
```

Per saber si el procés de Collatz s'atura per a tot natural, es pot escriure una funció *tots* que s'atura només si per algun n , *collatz* (n) no s'atura:

```
void tots () {
    for (nat n = 1; ; ++n) {
        if (not collatz_atura (n)) return;
    }
}
```

Aleshores, la conjectura de Collatz és equivalent a **not** $T(\text{tots}, \text{void})$.

21.8 El valor d'un graf

Per veure que GRAPH-VALUE és **NP**, utilitzem una permutació dels vèrtexs com a testimoni i l'algorisme que en calcula la seva χ i la compara amb B . És clar que el testimoni té mida polinòmica i que l'algorisme es pot dur a terme en temps polinòmic.

Ara veiem que VERTEX-COVER es redueix a GRAPH-VALUE: Donada una entrada per al problema del recobriment de vèrtexs (és a dir, un graf G i una fita K), creem una llista a amb K zeros i $n - K$ uns, on n és el nombre de vèrtexs de G . Establim el valor de B a zero. Llavors, el graf té un recobriment de talla K si i només si hi ha una assignació de valors de a als vèrtexs que produeix un valor zero, perquè el recobriment de vèrtexs es correspon als vèrtexs que se'ls assigna zero al problema del valor del graf.

Dels dos paràgrafs anteriors en segueix que GRAPH-VALUE és **NP**-complet.

Final, 17 de gener de 2017

22.1 Hashing universal

a) Un conjunt de funcions de hash H és universal si, per a tot parell de claus diferents x i y , $\Pr[h(x) = h(y)] \leq 1/N$, on N és el nombre de posicions de la taula de hash i h és triada a l'atzar en H .

b) Multiplicar M per x correspon a sumar (mòdul 2) les columnes de M per a les quals el bit corresponent de x és 1. Siguin x i y dues claus diferents. Sense pèrdua de generalitat, podem suposar $x_i = 0$ i $y_i = 1$ per a alguna posició i . Suposem que fixem tota M excepte la columna i -èsima. Per a tota elecció de la i -èsima columna, Mx ja és correcta. Però per a cadascuna d'aquestes 2^m possibles eleccions, hi ha un valor diferent per a My : cada cop que se'n canvia un bit, es canvia el bit corresponent a My . Per tant, la probabilitat de que Mx i My siguin iguals és $1/2^m$, perquè els m bits es trien a l'atzar.

22.2 Recurrències

a) $H(n) = H(n-1) + 1$ i $W(n) = 2W(n-1) + 2$. El teorema mestre dona $H(n) = \Theta(n)$ i $W(n) = \Theta(2^n)$.

b) $L(n) = 2L(n-1) + 2$. El teorema mestre continua donant $L(n) = \Theta(2^n)$.

c) Siguin $A_1(n)$ i $A_2(n)$ les àrees per a un arbre complet de 4^n fulles en la primera i en la segona construcció, respectivament. Tenint en compte $4^n = 2^{2n}$, tenim $A_1(n) = H(2n)W(2n) = \Theta(2^{2n})\Theta(2n) = \Theta(2^{2n}n)$. Per altra banda, $A_2(n) = L(n)^2 = \Theta(2^{2n})$. Atès que $\lim_{n \rightarrow \infty} 2^{2n}n/2^{2n} = +\infty$, en deduïm que $A_2(n)$ és $O(A_1(n))$ però no $\Theta(A_1(n))$. Per tant, la segona construcció és asimptòticament més eficient que la primera.

22.3 MAX3SAT

a) Per exemple, l'assignació $x_1 = x_2 = x_3 = x_4 = \text{cert}$ satisfà ϕ i, per tant, maximitza el nombre de clàusules satisfetes.

- b) És clar que MAX3SAT és **NP**, i que podem usar MAX3SAT per resoldre 3SAT, el qual és **NP-complet**. Per tant, no se sap.
- c) Posem totes les variables a **cert**. Si se satisfan almenys la meitat de les clàusules, ja estem. Sinó, posem totes les variables a **fals**. Llavors s'han de satisfer més de la meitat de les clàusules (les que no se satisfien abans).
- d) Assignem **cert** o **fals** a cada variable a l'atzar. La probabilitat de no satisfer una clàusula és $1/8$. Per tant, el nombre esperat de clàusules satisfetes és $\frac{7}{8}n$. Si l'esperança té aquest valor, és que existeix alguna assignació que el té igual o més gran.

22.4 Flux màxim

El flux màxim entre dos vèrtexs coincideix amb el seu tall de cost mínim. Sabem que hi ha un tall de x i y amb cost 100. En aquest tall, z ha d'estar a la banda de y . (Si estés a la banda de x , hi hauria un tall entre y i z amb cost 100, i el flux màxim entre y i z no podria ser 200.) Com que hi ha un tall entre x i z amb cost 100, el flux màxim entre x i z és com a molt 100.

Suposem que fos més petit que 100, per exemple 50. Llavors, en el tall entre x i z amb cost 50, y no podria estar ni a la banda de x ni a la banda de z , perquè en qualsevol dels dos casos s'entraria amb contradicció amb el flux màxim conegut de 100 o de 200. Com a conclusió, el flux màxim entre x i z és exactament 100.

Cal dir que qualsevol resposta tipus “com que es pot enviar 100 de flux entre x i y , i es pot enviar 200 de flux entre y i z , llavors es pot enviar almenys 100 de flux entre x i z passant per y ”, tal qual, és incorrecta, perquè no considera les possibles arestes comunes entre els dos fluxos màxims (de x a y i de y a z).

22.5 Teoria de jocs

La demostració és per inducció en el nombre de jugades que queden per fer. Com a base de la inducció, el joc buit té per definició número 0, i la posició és perdedora. Ara, suposem que l'enunciat és cert per a tots els jocs on queden n jugades per fer, i considerem una posició qualsevol X amb $n + 1$ jugades. Si X té número 0, per definició no és possible deixar a l'adversari cap posició amb número 0. Per tant, per inducció totes les posicions de l'adversari seran guanyadores, i X és perdedora. Si X té número estrictament positiu, per definició hi ha almenys una jugada que deixa a l'adversari en una posició amb número 0, que per inducció és perdedora, i per tant X és guanyadora.

22.6 Codi espatllat

a) La funció f calcula si una matriu M , corresponent a una posició del tres-en-ratlla, és guanyadora, perdedora o és empat, suposant que els dos adversaris 1 i -1 juguen de forma òptima, i que ha començat a jugar 1. El paràmetre *juga* indica a qui li toca jugar. Les caselles amb un 0 són les que encara estan lliures. Si el resultat ja és conegut, es retorna i prou. Altrament, es busca si hi ha alguna jugada guanyadora. En aquest cas no cal buscar més. Si no és així, es retorna empat (0) a no ser que totes les jugades duguin a posicions perdudes. És un exemple senzill de l'algorisme mini-max, que és una variant de programació dinàmica.

b) El codi falla quan no queden jugades a fer. En aquest cas, la funció no retorna empat com hauria de fer, sinó que retorna que el jugador perd. Per arreglar-ho, el més senzill és

afegir un booleà que marqui si s'ha trobat alguna casella lliure en el **for** del **for** sobre tots els parells i, j . Si no és el cas, cal retornar (i guardar) un 0.

22.7 Més codis misteriosos

La funció *misteri*(G) calcula si el component connex del vèrtex 0 és bicolorejable (és a dir, si és bipartit, o si no té cicles de longitud senar). És un recorregut en profunditat que intenta assignar els colors 1 i 2 alternativament a tots els vèrtexs accessibles des del 0, de manera que no hi hagi dos vèrtexs veïns del mateix color. Si en algun moment troba una contradicció, l'algorisme retorna fals. Si és incapaç de trobar-la, l'algorisme retorna cert. El cas pitjor és un graf connex i bicolorejable. En aquest cas, el cost és el d'un recorregut complet del graf, $\Theta(n + m) = \Theta(m)$, que pot arribar a ser $\Theta(n^2)$.

Parcial, 15 de novembre de 2016

23.1 Recurrència simpàtica

El teorema mestre mostra que la solució de $T(n) = 7T(n/7) + \Theta(n)$ és $T(n) = \Theta(n \log n)$.

Demostrem primer per inducció que $T(n) = O(n \log n)$: És suficient veure que si $T(n) \leq 7T(n/7) + cn$, llavors $T(n) \leq \beta n \log n$ per a alguna β .

Prenem doncs com a hipòtesi d'inducció que existeix alguna β per la qual, per a tota $m \leq n$, $T(m) \leq \beta m \log m$. Llavors,

$$\begin{aligned} T(n) &\leq 7T\left(\frac{n}{7}\right) + cn \\ &\leq 7\beta \frac{n}{7} \log \frac{n}{7} + cn \\ &\leq \beta n \log n - \beta n \log 7 + cn. \end{aligned}$$

I, per tant, $T(n) \leq \beta n \log n$ és cert quan $\beta \geq c / \log 7$.

La demostració per inducció que $T(n) = \Omega(n \log n)$ és semblant.

23.2 Els més petits

- Aplicant l'algorisme de mediana-de-medianes per trobar el k -èsim element més petit, els elements del vector a l'esquerra de k seran els k més petits. El cost d'aquest algorisme és $\Theta(n)$ en el cas pitjor.
- Aplicant l'algorisme de quick-select per trobar el k -èsim element més petit, els elements del vector a l'esquerra de k seran els k més petits. El cost d'aquest algorisme és $\Theta(n)$ en mitjana.
- L'algorisme a) és més eficient en el cas pitjor, ja que b) podria té cost $\Theta(n^2)$ en el cas pitjor. En mitjana, els dos algorismes tenen el mateix cost asimptòtic, però les constants amagades a l'algorisme a) el fan més lent que b). A més, programar l'algorisme a) és més complicat que b).

23.3 Mínim local

- a) És clar que X té algun mínim absolut i que aquest mínim absolut és un mínim local.
- b) Es pot utilitzar una estratègia semblant a la cerca binària: Donat un vector, es mira l'element del mig. Si és un mínim local, ja està. Altrament, almenys un dels seus elements adjacents ha de ser menor que ell. Llavors, es continua recursivament en el subvector de l'element menor fins a l'extrem oposat al del mig.

Com que cada cop es divideix per dos la talla del problema, el cost en el cas pitjor és logarítmic.

23.4 Ordenació

Considerem l'arbre de decisió T de qualsevol algorisme d'ordenació de propòsit general. Llavors, T ha de tenir, almenys, $n!$ fulles. A més, com que T és un arbre de decisió, és un arbre binari. Com que $\log_2(\frac{1}{2} \cdot n!) = \Theta(n \log n)$, no és possible que cap meitat d'aquestes $n!$ fulles es trobin a alçada $O(n)$. Per tant, per almenys la meitat de les entrades, el cost ja és més que lineal. Per tant, en mitjana també ha de ser més que lineal. Per tant, no pot existir cap algorisme d'ordenació de propòsit general que trigui temps lineal en mitjana.

23.5 Codi misteriós

El codi calcula les “sumes prefixades” del vector. Després, en troba la menor diferència en valor absolut entre dos elements eficientment, primer ordenant el vector, i després restant només els elements consecutius. Per tant,

- a) El cost total és $\Theta(n \log n)$, i ve del *sort* (). Les altres parts del codi tenen cost constant o lineal.
- b) Les sumes prefixades són 0 -6 4 -4 -14 10.
Una vegada ordenades queden -14 -6 -4 0 4 10.
El resultat és doncs $-4 - (-6) = 2$.
- c) El codi calcula la suma d'elements més petita en valor absolut de totes les subseqüències consecutives no buides del vector. Per exemple, el 2 es correspon als elements 10 -8.

23.6 El problema d'en Nikochan

Si s'ordenen els parells amb l'ordre per defecte, és a dir, pel *first*, amb cost $\Theta(n \log n)$, llavors cal trobar una LIS (*longest increasing subsequence*) respecte dels *second*. Com s'ha vist a classe, una LIS es pot calcular amb una programació dinàmica quadràtica, o amb un codi senzill de cost $\Theta(n \log n)$.

Extraordinari, 7 de juliol de 2016

24.1 Recurrències

Aplicant el teorema mestre, és immediat que $A(n) = O(n^2)$.

Sigui $C(n) = 3C(n/3) + \Theta(\sqrt{n})$. És clar que $B(n) \leq C(n)$ per a tota n prou gran. Aplicant el teorema mestre, és immediat que $C(n) = \Theta(n)$. Per tant, $B(n) = O(n)$.

24.2 Escacs

Sigui ESCACS el problema descrit. És evident que ESCACS és un problema decisonal.

Malgrat ser molt gran, el nombre de posicions vàlides del joc dels escacs és finit. A més, com que les regles dels escacs impliquen que no hi ha partides amb un nombre infinit de torns, el nombre de partides, malgrat ser enormement gran, és finit també. Per tant, per a cada posició inicial es pot calcular en temps constant si és o no guanyadora. Consequentment, ESCACS es pot resoldre en temps constant.

Com que ESCACS es pot resoldre en temps constant, és evident que ESCACS pertany a la classe $\mathbf{P}$ i que ESCACS és decidible. A més, com que $\mathbf{P} \subseteq \mathbf{NP}$, també es té que ESCACS pertany a $\mathbf{NP}$.

ESCACS és $\mathbf{NP}$ -complet o no segons l'estatus del problema $\mathbf{P} \stackrel{?}{=} \mathbf{NP}$:

- Si $\mathbf{P} \neq \mathbf{NP}$, ESCACS no pot ser $\mathbf{NP}$ -complet, perquè si ho fós $\mathbf{P} = \mathbf{NP}$, ja que $\text{ESCACS} \in \mathbf{P}$.
- Si $\mathbf{P} = \mathbf{NP}$, tots els problemes no trivials de $\mathbf{P}$ també són $\mathbf{NP}$ -complets, per tant, ESCACS també ho seria.

24.3 Consultes en un llibre electrònic

- (a) L'estructura que cal és allò que hi ha al final dels llibres: un índex. Un índex indica, per a cada paraula, a quines pàgines apareix. Es tracta doncs d'un diccionari de paraules cap a llistes ordenades de pàgines. Com que, al contrari dels llibres tradicionals, no ens cal que les paraules de l'índex estiguin ordenades alfabèticament, podem representar l'índex amb una taula de hash:

```
unordered_map<string, vector<int>> index;    // els vectors estan ordenats
```

[Una solució amb `map<string, vector<int>>` és també perfectament correcta, però els costos serien diferents.]

El preprocés crea l'índex. Per fer-ho, es processen totes les paraules de cada pàgina, en ordre creixent de pàgina. Per a cada paraula x a la pàgina p , si la clau x no és al diccionari, se li afegeix, juntament amb el vector $[p]$ com a valor; si x ja és al diccionari, i el darrer element del seu vector no és p , s'afegeix p al final del vector. Això garanteix que no hi ha repeticions i que els vectors queden ordenats.

Les consultes es resolen simplement tot retornant el vector corresponent a la paraula demanada (o retornant `[]` si no hi és).

- (b) Un `unordered_map` utilitza espai lineal respecte del nombre de claus i de valors que conté. En aquest cas, tenim n possibles paraules diferents on cadascuna d'elles pot aparèixer en, com a molt, P pàgines. Per tant, l'espai és $O(nP)$.
- (c) L'accés i la inserció a les taules de hash és constant en mitjana i el pre-procés en fa en ordre de N . Com que consultar el final del vector i afegir al final del vector té cost constant, es cost mitjà total del pre-procés és $\Theta(N)$.
- (d) El temps de fer la cerca és $\Theta(1)$ en mitjana perquè només cal fer un accés a la taula de hash. [Es podria afegir el cost $O(P)$ de la còpia del vector.]
- (e) Per a cada paraula de la consulta, s'obté el seu vector de pàgines. Cal retornar doncs la intersecció de tots aquests vectors. La intersecció de dos vectors es fa amb l'algorisme de fusió, que té cost lineal respecte al nombre d'elements als dos vectors. La intersecció de molts vectors es fa incrementalment.
- (f) Com que les interseccions no poden fer més que reduir el nombre d'elements dels vectors, el cost de les $k - 1$ interseccions necessàries és $O(kP)$ en el cas pitjor. Sobre això, cal afegir el cost $\Theta(k)$ en mitjana de fer les k cerques.

24.4 Codis misteriosos

- (a) El codi escriu 12 12 23 23 42 42.
- (b) En general, el codi escriu el màxim de $V[0..i]$ per a tota i entre $m - 1$ i $n - 1$.
- (c) El cost està dominat per les n insercions, ja que la resta d'operacions tenen cost constant. El cost en el cas pitjor és $\Theta(\sum_{i=1}^n \log i) = \Theta(n \log n)$.
- (d) El codi escriu 4 2 2 1 1 1.
- (e) En general, el codi escriu el mínim de $V[i..i + m - 1]$ per a tota i entre 0 i m .
- (f) El cost en el cas pitjor està dominat per les n insercions, ja que la resta d'operacions tenen cost constant o, en el cas dels esborrats, el mateix cost que les insercions. Així doncs, igual que a l'apartat (c), el cost en el cas pitjor és $\Theta(n \log n)$.

- (g) El primer codi no es veu afectat, perquè les cues de prioritats admeten repetits sense problemes. Els sets, però, no admeten repetits, cosa que fa variar el comportament del codi. Per exemple, amb un vector amb dos 42, el programa primer escriu 42 i després aborta.
- (h) Aquest codi fa el mateix que el primer, només amb cost linial:

```
int mx = V[0];
for (int i = 1; i < m; ++i) mx = max(mx, V[i]);
cout << mx << endl;
for (int i = m; i < n; ++i) {
    mx = max(mx, V[i]);
    cout << mx << endl;
}
```

24.5 Algorisme de Kruskal

- (a) L'algorisme troba un arbre generador mínim, i funciona afegint les arestes en ordre creixent de cost, sempre i quan no formin cap cicle, cosa que es pot comprovar eficientment amb un mf-set. L'algorisme pot parar quan ha afegit $n - 1$ arestes.

A l'exemple donat, l'ordre en què es tracta les arestes és: 1-5, 0-7, 3-4, 3-6, 1-6, 4-7, 5-7 (s'ignora), 0-4 (s'ignora), i 2-6. L'aresta 1-4 no cal considerar-la.

- (b) És fàcil veure que Kruskal genera un arbre. Per veure que és de mínim cost, sigui K l'arbre generat per Kruskal, i sigui M un arbre generador mínim. Sigui $a = u - v$ l'aresta més barata que està a un arbre però no a l'altre.

No pot ser que a estigui a M i no a K : Si fos així, voldria dir que u i v estan connectats dins de K amb arestes més barates que a (perquè Kruskal va rebutjar a en el seu moment), cosa que implicaria que M té un cicle. Per tant, a està a K però no a M .

Si afegim a a M , això forma un cicle. Si les altres arestes del cicle fossin totes més barates que a , estarien totes a K , i K tindria un cicle. Per tant, almenys una (anomenem-la b) ha de ser més cara. Però això implicaria que, afegint a i esborrant b de M , obtindríem un arbre més barat que M , contra la hipòtesi. Com a conclusió, a no pot existir, i per tant $K = M$.

24.6 Teoria de jocs

Usant la definició, es pot calcular incrementalment el nombre de piles de pedres des de 0 fins a 7. Obtenim $n(0) = 0$, $n(1) = 1$, $n(2) = 0$, $n(3) = 2$, $n(4) = 1$, $n(5) = 3$, $n(6) = 0$, i $n(7) = 4$. No cal calcular $n(999)$, perquè n'hi ha dos i es cancel·len entre si.

Ara, el nombre del joc complet és el or exclusiu de 1, 0, 2, 1, 3, 0 i 4, que és 5. Com que és diferent de 0, la posició és guanyadora. L'única jugada guanyadora que no involucra les piles amb 999 pedres consisteix a treure 3 pedres de la pila amb 7 pedres, canviant $n(7) = 4$ per $n(4) = 1$, cosa que fa anar a una posició amb or exclusiu igual a zero, i per tant perdadora.

Final, 19 de gener de 2016

25.1 Costos

Sigui $T_x(n)$ el temps de la funció f_x aplicada a l'entrada n per a $x \in \{1..5\}$. Llavors tenim:

- $T_1(n) = \sum_{i=1}^n \Theta(1) = \Theta(n)$.
- $T_2(n) = \sum_{i=1}^n \sum_{j=1}^i \Theta(1) = \Theta(n^2)$.
- $T_3(n) = 2T_3(n/2) + \Theta(n) = \Theta(n \log n)$ utilitzant el teorema mestre.
- $T_4(n) = T_4(n-1) + T_4(n-2) + \Theta(1)$. [És la mateixa recurrència que la dels nombres de Fibonacci calculats recursivament.] Com que $T_4(n-1) \geq T_4(n-2)$, es té que $T_4(n) \leq 2T_4(n-1) + \Theta(1)$ i $T_4(n) \geq 2T_4(n-2) + \Theta(1)$. Aplicant el teorema mestre a cadascuna d'aquestes noves recurrències es troba que $T_4(n) = O(2^n)$ i $T_4(n) = \Omega(2^{n/2})$.

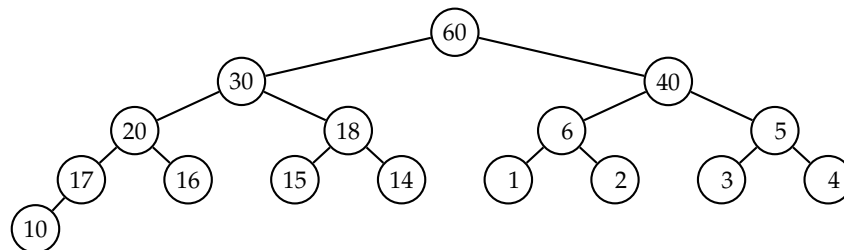
[De fet, la solució és $T_4(n) = \Theta(\phi^n)$ on ϕ és el nombre d'or.]

- Condicionat els sumatoris per la condició del **if**, tenim

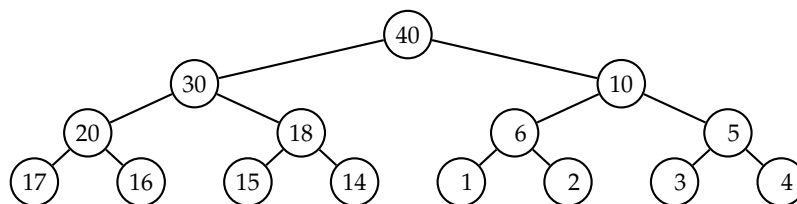
$$\begin{aligned}
 T_5(n) &= \sum_{i=1}^n \left(\sum_{\substack{j=0 \\ j \% i = 0}}^{i^2} \sum_{k=0}^n \Theta(1) + \sum_{\substack{j=0 \\ j \% i \neq 0}}^{i^2} \Theta(1) \right) \\
 &= \sum_{i=1}^n \left(\sum_{\substack{j=0 \\ j \% i = 0}}^{i^2} \Theta(n) + \sum_{\substack{j=0 \\ j \% i \neq 0}}^{i^2} \Theta(1) \right) \\
 &= \sum_{i=1}^n \left(\sum_{j=0}^i \Theta(n) + \sum_{j=0}^{i^2-i} \Theta(1) \right) \\
 &= \sum_{i=1}^n (\Theta(in) + \Theta(i^2)) \\
 &= \sum_{i=1}^n \Theta(in) + \sum_{i=1}^n \Theta(i^2) \\
 &= \Theta(n^3) + \Theta(n^3) \\
 &= \Theta(n^3).
 \end{aligned}$$

25.2 Heaps

Quan s'esborra l'element a l'arrel del max-heap



queda



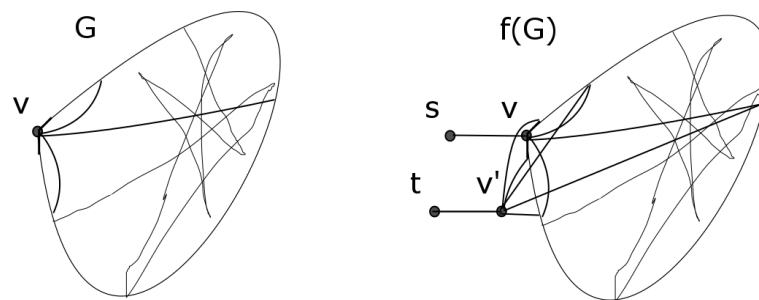
i només s'haurà fet un intercanvi.

25.3 Cicles i camins Hamiltonians

Per veure que $\text{CAM} \in \text{NP-C}$ és suficient de veure que $\text{CAM} \in \text{NP}$ (1.) i que HAM es redueix a CAM en temps polinòmic (2.):

1. El testimoni de CAM pot ser una seqüència de vèrtexs. L'algorisme verificador ha de comprovar que el testimoni és una permutació dels vèrtexs del graf i que, per a cada parell de vèrtexs consecutius a la seqüència, hi ha l'aresta corresponent al graf. La talla del testimoni és lineal amb el nombre de vèrtexs al graf i és clar que es pot implementar l'algorisme verificador en temps polinòmic. Per tant, $\text{CAM} \in \text{NP}$.
2. Donat un graf $G = (V, E)$, construïm un graf $f(G) = G' = (V', E')$ de la forma següent:

Sigui v un vèrtex de G , i siguin v', s, t tres vèrtexs nous. Llavors fem $V' := V \cup \{v', s, t\}$ i $E' = E \cup \{(v', w) \mid (v, w) \in E\} \cup \{(s, v), (v', t)\}$. Esquemàticament:



- ($\Rightarrow$) Si G té un cicle Hamiltonià, el podem escriure com a $(v, u), \ell, (u', v)$ on e és una llista d'arestes encadenades entre u i u' passant per tots els vèrtexs tret de v . Llavors, $(s, v), (v, u), \ell, (u', v'), (v', t)$ és un camí Hamiltonià entre s i t en G' .
- ($\Leftarrow$) D'altra banda, si G' té un camí Hamiltonià, per força ha d'anar entre s i t perquè tenen grau 1. En aquest cas, el camí ha de ser $(s, v), (v, y), \ell', (y', v'), (v', t)$. I, llavors, G té un cicle Hamiltonià $(v, y), \ell', (y', v)$.

Com que la reducció f es pot implementar en temps polinòmic respecte la talla de G , tenim que HAM es redueix a CAM .

25.4 Inaproximabilitat

Suposem que per algun $\rho > 1$, existeix un algorisme d'aproximació A de factor ρ per a TSP. En aquest cas, veurem que podem resoldre HAM en temps polinòmic utilitzant A . Com que sabem que HAM és NP-complet , això no és possible sota la hipòtesi $\text{P} \neq \text{NP}$. Per tant, $\text{TSP} \notin \text{APX}$ si $\text{P} \neq \text{NP}$.

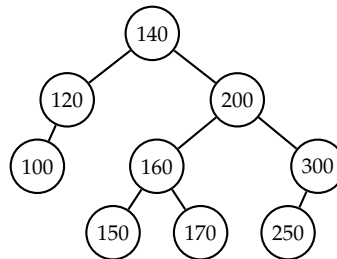
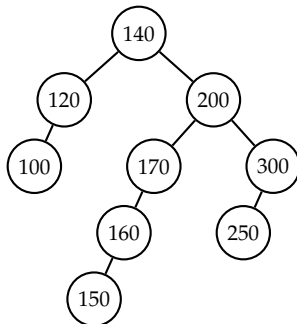
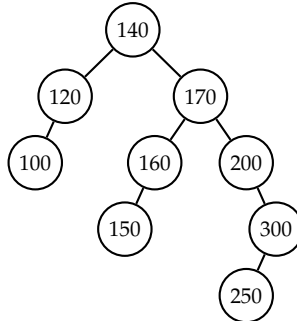
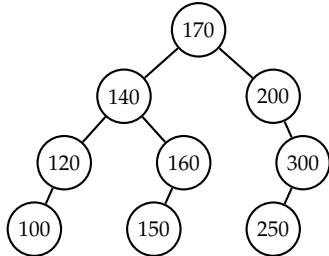
La reducció parteix d'una instància de HAM i calcula una instància de TSP. És a dir, donat un graf G amb $V = \{1..n\}$, retorna una matriu M de mida $n \times n$. Concretament, per a cada parell u, v , es fa $M[u, v] := 1$ si $\{u, v\} \in E(G)$ i $M[u, v] := \rho n + 1$ altrament.

Per tant, si G té un cicle Hamiltonià, el cost del TSP òptim en M és n , i si G no té un cicle Hamiltonià, el cost del TSP òptim en M és estrictament més gran que ρn perquè haurà d'utilitzar una de les no-arestes de G .

Així, al executar A sobre M , en el primer cas obtindrem una solució de cost $\leq \rho n$ (perquè l'òptim és n i A és algorisme d'aproximació de factor ρ). En el segon cas, trobarem una solució de cost estrictament més gran que ρn . Per tant, A pot ser usat per decidir si un graf és o no Hamiltonià en temps polinòmic.

25.5 Arbres aleatoritzats

Aplicant l'algorisme, es poden obtenir els arbres següents, amb probabilitats respectives $1/9$, $8/9 \cdot 1/6$, $8/9 \cdot 5/6 \cdot 1/3$, i $8/9 \cdot 5/6 \cdot 2/3 \cdot 1$.



25.6 Components fortament connexs

És fàcil veure que un graf dirigit amb n vèrtexs té n components fortament connexos si i només si no té cap cicle. Per tant, el graf accepta almenys una ordenació topològica. Com que cada ordenació topològica prohibeix un conjunt d'arcs (els que anirien de dreta a esquerra), i volem maximitzar aquest nombre, suposarem que només n'admet una. En aquest cas, podem tenir $(n-1) + (n-2) + \dots + 2 + 1 = n(n-1)/2$ arcs.

25.7 Algorisme de Prim

a) L'algorisme troba un arbre generador mínim, i funciona afegint a un subarbre format inicialment per un sol vèrtex (en el nostre cas, el 6) el vèrtex encara no afegit que estigui connectat al subarbre per l'aresta més barata. Així doncs, les arestes s'afegeixen en aquest ordre: 6-3, 3-4, 6-1, 1-5, 4-7, 7-0, i 6-2.

b) És fàcil veure que Prim genera un arbre P . Per demostrar que és òptim, suposem que n'hi ha un altre de millor M , i arribarem a una contradicció. Sigui $x-y$ una aresta que estigui a P i no a M . Aquesta aresta, quan es va escollir, era la més barata entre C , el conjunt de vèrtexs connectats al vèrtex origen, i la resta de vèrtexs R . Sense pèrdua de generalitat, suposem $x \in C$ i $y \in R$. Si afegim $x-y$ a M , es forma un cicle. El camí antic que va de x a y , com que comença a C i acaba a R , ha de contenir almenys una aresta $u-v$ tal que $u \in C$ i $v \in R$. Com que aquesta aresta és més cara que $x-y$, si l'esborrem de M i hi afegim $x-y$, obtenim un nou arbre generador millor que M , contra la hipòtesi.

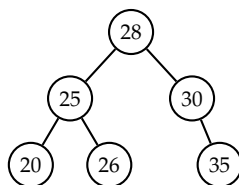
25.8 Flors i colors

El problema proposat és exactament el de trobar el màxim aparellament possible en un graf bipartit. La solució habitual (explicada a classe) consisteix a trobar el flux màxim en el graf incrementat amb una font i un pou addicionals. L'algorisme trobarà un camí augmentatiu exactament f vegades, cadascuna de les quals triga temps $O(n + c)$ per ser un recorregut. Així doncs, una fita superior al cost és $O((n + c) \cdot f)$.

Parcial, 3 de novembre de 2015

26.1 Pica-pica

1. És clar que $f(n) \leq \max\{f(n), g(n)\}$ i que $g(n) \leq \max\{f(n), g(n)\}$. Ara, sumant ambdues inequacions, $f(n) + g(n) \leq 2 \max\{f(n), g(n)\} = O(\max\{f(n), g(n)\})$.
2. Una taula de dispersió amb n elements implementat amb llistes requereix temps $\Theta(n)$ per cercar/inserir/esborrar claus en el cas pitjor, i temps $\Theta(1)$ en el cas mitjà (per a un bon factor de càrrega). Anàlogament, una taula de dispersió amb AVLs requereix temps $\Theta(\log n)$ en el cas pitjor, i temps $\Theta(1)$ en el cas mitjà. Per tant, una taula de dispersió millora el cas pitjor però no afecta asimptòticament el cas mitjà.
D'altra banda, la implementació amb AVLs és més complicada, utilitza una mica més d'espai (el doble de punters) i requereix que els elements siguin ordenables.
3. Les claus 26, 27 i 29 requereixen una rotació doble a l'arrel de l'arbre. La inserció de 26 donaria l'arbre



26.2 Maleïdes recurrències

- La recurrència és $T_1(n) = 3T_1(n/2) + \Theta(n^2\sqrt{n})$. Aplicant el teorema mestre, s'obté $T_1(n) = \Theta(n^2\sqrt{n})$.
- La recurrència és $T_2(n) = 4T_2(n/2) + \Theta(n^2)$. Aplicant el teorema mestre, s'obté $T_2(n) = \Theta(n^2 \log n)$.

- La recurrència és $T_3(n) = 5T_3(n/2) + \Theta(n^2 \log^2 n \log \log n)$, que no es pot resoldre directament. Siguin $T_4(n) = 5T_4(n/2) + \Omega(n^2)$ i $T_5(n) = 5T_5(n/2) + O(n^{2+\varepsilon})$ per a algun $\varepsilon \in (0, \frac{1}{2})$. Aplicant el teorema mestre, s'obté $T_4(n) = \Omega(n^{\log_2 5})$ i $T_5(n) = O(n^{\log_2 5})$. Però $T_3(n) = \Omega(T_4(n))$ i $T_3(n) = O(T_5(n))$, i per tant $T_3(n) = \Theta(n^{\log_2 5})$.
- Com que $\log_2 5 > 2$, la segona alternativa és la més ràpida.

26.3 Element solitari

1. Utilitzem una taula de dispersió amb n posicions. Per a cada element de l'entrada, si no és a la taula de dispersió, l'afegim; i si ja hi és, l'esborrem. Al final, l'únic element que queda a la taula de dispersió és l'element solitari.

Cal fer unes n cerques, $n/2$ insercions i $n/2$ esborrats, cadascun de cost constant en mitjana. I, després, cal cercar l'element solitari a les n posicions. El temps total en el cas mitjà és, doncs, $\Theta(n)$. L'espai és clarament $\Theta(n)$.

[Solució una mica millor: L'element solitari és la XOR de tots els enters de l'entrada. El temps en el cas pitjor és lineal, i l'espai és constant.]

2. Fem una variació de la cerca binària: Per trobar l'element solitari en el subvector $v[i..j]$ amb $j - i + 1$ senar (i prou gran), calculem la posició central $c = (i + j)/2$, i mirem si $v[c - 1] = v[c]$. En cas afirmatiu, cal continuar cercant entre i i c , on hi ha un nombre senar d'elements i, per tant, l'element solitari. En cas contrari, cal continuar cercant entre c i j .

El temps en el cas pitjor és logarítmic, i l'espai és constant.

26.4 En posició!

És fàcil veure que el nombre esperat de passos de la funció està afitat superiorment per $n/2^{n-1} + \sum_{i \geq 1} i/2^i = n/2^{n-1} + 2$. Per tant, el cost és $O(1)$. Com que el cost trivialment també és $\Omega(1)$, podem deduir que el cost en el cas mitjà és $\Theta(1)$.

26.5 Comptatge

Una manera de resoldre aquest problema consisteix a fer una còpia del vector original i ordenar-la. Sigui $[y_1, \dots, y_n]$ el resultat. Llavors, cal comptar un per cada posició i tal que $x_i = y_i$, i tal que o bé $i = n$ o bé $x_i < y_{i+1}$. El cost en temps és $\Theta(n \log n)$, i l'espai addicional necessari és $\Theta(n)$.

26.6 Selecció

La recurrència per al cas pitjor és $M(n) = \Theta(n) + M(n/3) + M(2n/3)$. El primer terme es correspon al cost no recursiu de calcular $n/3$ medianes, i de particionar el vector. El segon terme, a la primera crida recursiva per calcular la mediana de $n/3$ medianes. I el tercer, a la segona crida recursiva considerant la pitjor partició possible.

La solució és $M(n) = \Theta(n \log n)$. Una possible manera de veure-ho consisteix a iterar la recurrència, i veure que la contribució de cada nivell de l'arbre de recursió al cost és lineal. Finalment, n'hi ha prou d'adonar-se que aquest arbre té $\Theta(\log n)$ nivells.

Extraordinari, 26 de juny de 2015

27.1 Punts propers a l'origen

Podem adaptar l'algorisme de quick-select per trobar el k -èsim element d'un vector. És igual que el quick-sort, però en lloc d'ordenar les dues particions, només ordena la que cal (és a dir, la que conté l'element k -èsim). Aquest algorisme té cost $\Theta(n)$ en el cas mitjà i $\Theta(n^2)$ en el cas pitjor. La versió mediana de medianes té cost $\Theta(n)$ sempre.

27.2 Propietats dels algorismes d'ordenació

	selection-sort	insertion-sort	merge-sort	quick-sort	heap-sort
Temps en cas pitjor	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n \log n)$	$\Theta(n^2)$	$\Theta(n \log n)$
Temps en cas millor	$\Theta(n^2)$	$\Theta(n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n) \star$
Temps en mitjana	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$
Mida espai addicional	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$	$\Theta(n^2) \star$	$\Theta(1)$
És estable?	no	sí	sí	no	no

* L'espai de quick-sort pot ser $\Theta(n)$ en el cas pitjor, per les crides recursive que cal emprar a la pila de la recursivitat, però és fàcil reduir-lo a $\Theta(\log n)$.

$\star$ Quan tots els elements són iguals.

27.3 Grafs

En general, Dijkstra té cost en el cas pitjor $\Theta(n + m \log n)$. Com que aquí $m = O(n)$, tenim que el cost en el cas pitjor és $O(n \log n)$ (i no inferior a $\Theta(n)$).

Si els pesos de les arestes són naturals entre 1 i 10, per a cada arc de v a u amb cost c es poden afegir $c - 1$ vèrtexs intermedis entre v i u , i fer que tots els arcs tinguin cost unitari. Ara només cal fer un recorregut en amplada començant en x , la qual cosa té cost $O(n + 10n) = O(n)$.

27.4 Teoria de jocs

L'or exclusiu dels cinc nombres dona 2, que com que no és 0 implica que la posició és guanyadora. Fent una mica de casuística, és fàcil veure que l'única jugada guanyadora consisteix a treure dues pedres de la pila que en té tres.

27.5 Arbres AVL

a) El cas pitjor es dona quan l'element és en una fulla de l'arbre. Com que l'arbre és AVL, el cost en el cas pitjor és $\Theta(\log n)$.

Com que la meitat dels nodes són a les fulles de l'arbre, el cost en el cas mitjà també és $\Theta(\log n)$.

b) L'arbre del mig no és AVL. El node amb un 130 està desequilibrat, ja que té alçada 2 per l'esquerra i 0 per la dreta. La resta d'arbres són arbres binaris de cerca i tenen tots els nodes equilibrats, i per tant són AVLs.

27.6 Un altre codi espifiat

El programa llegeix una seqüència de paraules i al final escriu en ordre alfabètic les que han aparegut un nombre senar de vegades. L'error és $it \leq S.end()$, que hauria de ser $it \neq S.end()$. En el cas millor totes les paraules són iguals, de manera que cada operació en el conjunt (que té com a molt un element) té cost constant. El cost és doncs $\Theta(n)$, on n és el nombre de paraules llegides. En el cas pitjor totes les paraules són diferents, de manera que cada operació en el conjunt té cost $\Theta(\log(i+1))$, on i és el nombre d'elements actual del conjunt. El cost és doncs $\sum_{i=1}^n \Theta(\log i) = \Theta(n \log n)$.

27.7 NP-completesa

Per començar, és evident que $DIVISOR \in \mathbf{NP}$: Donat un testimoni x , es pot comprovar en temps polinòmic que compleix els requeriments demanats.

- a) Si es demostrés $\mathbf{P} = \mathbf{NP}$, com que $DIVISOR \in \mathbf{NP}$, tenim que $DIVISOR \in \mathbf{P}$.
- b) Si es demostrés $\mathbf{P} \neq \mathbf{NP}$, això implicaria $DIVISOR \notin \mathbf{P}$ només si se sabés que $DIVISOR$ és $\mathbf{NP}$ -complet.
- c) Si es demostrés que $DIVISOR \notin \mathbf{P}$, com que $DIVISOR \in \mathbf{NP}$, tenim que $\mathbf{P} \neq \mathbf{NP}$.
- d) Si es trobés un algorisme polinòmic per resoldre $DIVISOR$, això implicaria $\mathbf{P} = \mathbf{NP}$ només si se sabés que $DIVISOR$ és $\mathbf{NP}$ -complet.

Final, 16 de gener de 2015

28.1 Recurrència

(solució d'Albert Atserias)

Demostrem un resultat més fort: $T(n) \leq 2n \ln n$ per a tot $n > 0$. Ho fem per inducció sobre n .

El cas base és per a $n = 1$. Tenim $T(1) = 0 \leq 2 \cdot 1 \cdot \ln 1 = 0$.

Per al cas inductiu ($n > 1$), assumim $T(i) \leq 2i \ln i$ per a tot $0 < i < n$. Llavors,

$$\begin{aligned} T(n) &= (n-1) + \frac{2}{n} \sum_{i=1}^{n-1} T(i) \\ &\leq (n-1) + \frac{2}{n} \sum_{i=1}^{n-1} 2i \ln i \\ &= (n-1) + \frac{4}{n} \sum_{i=1}^{n-1} i \ln i. \end{aligned}$$

Demostrem ara el següent petit lemma:

Lemma: Per a tot enter $n > 0$, tenim $\sum_{i=1}^{n-1} i \ln i \leq \frac{1}{2}n^2 \ln n - \frac{1}{4}n^2 + \frac{1}{4}$.

Demostració: Afitem la suma $\sum_{i=1}^{n-1} i \ln i$ superiorment per la integral $\int_1^n x \ln x \, dx$.

Una primitiva de $x \ln x$ és $\frac{1}{2}x^2 \ln x - \frac{1}{4}x^2$. Per tant, podem afitar la suma en qüestió per

$$\left[\frac{1}{2}x^2 \ln x - \frac{1}{4}x^2 \right]_1^n = \frac{1}{2}n^2 \ln n - \frac{1}{4}n^2 + \frac{1}{4}$$

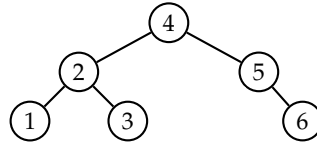
tal com volíem demostrar.

Continuem amb $T(n)$ i tenim

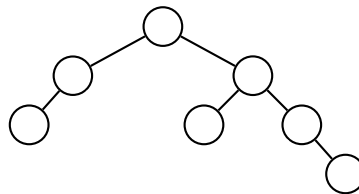
$$\begin{aligned} T(n) &\leq (n-1) + \frac{4}{n} \left(\frac{1}{2}n^2 \ln n - \frac{1}{4}n^2 + \frac{1}{4} \right) \\ &= (n-1) + 2n \ln n - n + \frac{1}{n} \\ &\leq 2n \ln n. \end{aligned}$$

28.2 Arbres AVL

- a) Recordem que el cost en el cas pitjor de cercar un element en un arbre AVL amb m nodes és $\Theta(\log m)$. Per tant, per a $m = n2^n$ nodes, el cost és $\Theta(\log(n2^n)) = \Theta(\log n + n \log 2) = \Theta(n)$.
- b) L'arbre resultant és aquest bonic dibuix:



- c) L'alçada és 4, amb un arbre com el següent:



28.3 Heaps ternaris

Podem representar els heaps ternaris en un vector col·locant consecutivament els seus elements de l'arrel cap a les fulles, i d'esquerra a dreta. Un node a la posició i té el seu fill k -èsim a la posició $3i + k$ per a $k \in \{1, 2, 3\}$ i el seu pare a la posició $(i - 1)/3$, en el ben entès que aquestes posicions existeixen. L'arrel és a la posició 0.

Declarant el tipus dels heaps ternaris com a vectors d'elements,

```
typedef vector<Elem> Heap3;
```

el codi de la inserció podria ser

```
void insert (Heap3& h, Elem x) {
    h.push_back(x);
    sinkup(h, h.size() - 1);
}

void sinkup (Heap3& h, int i) {
    int p = (i-1)/3;
    if (i != 0 and h[p] > h[i]) {
        swap(h[p], h[i]);
        sinkup(p);
    }
}
```

Com que l'algorisme comença des d'una fulla, i es mou cap a l'arrel pujant un nivell i prenent un temps constant cada cop, el cost de l'algorisme és proporcional a l'alçada de l'arbre, que és aproximadament $\log_3 n$, és a dir, el cost és $\Theta(\log n)$ on n és la talla del heap ternari.

28.4 NP-completesa

(solució d'Antoni Lozano)

La reducció de GRAN-CLICA a CLICA és senzilla: Donada una entrada $G = (V, E)$ de GRAN-CLICA, l'algorisme de reducció retorna (G, k) amb $k = |V|/2$.

La reducció de CLICA a GRAN-CLICA és més laboriosa:

Donada una entrada de CLICA $(G = (V, E), k)$, hem de retornar un graf G' tal que $G' \in \text{GRAN-CLICA}$ si i només si $(G, k) \in \text{CLICA}$. Per descriure la reducció, definim $n = |V|$ i distingim dos casos:

1. Quan $k \geq \lceil n/2 \rceil$: Aquí n'hi ha prou a retornar un nou graf G' que és G al qual se li afegeixen $2k - n$ vèrtexs aïllats.
2. Quan $k < \lceil n/2 \rceil$: Com que un subgraf complet de k vèrtexs en G seria massa petit, definirem G' com una còpia de G més un subgraf complet de i vèrtexs connectats a tots els de la còpia de G . D'aquesta manera, un hipotètic subgraf complet de k vèrtexs a G es convertirà en una subgraf complet de $k + i$ vèrtexs a G' . Perquè $k + i$ representi la meitat dels vèrtexs de G' , cal que $2(k + i) = n + i$, és a dir, $i = n - 2k$. Per tant, $G' = (V', E')$, on

- $V' = V \cup \{u_1, \dots, u_{n-2k}\}$, sent $u_1, \dots, u_{n-2k}$ vèrtexs nous.
- $E' = E \cup \{\{u_i, v\} \mid 1 \leq i \leq n - 2k \wedge v \in V\}$.

En el cas (1), és evident que la reducció és correcta. En el (2), si $(G, k) \in \text{CLICA}$, hem construït G' de manera que pertanyi a GRAN-CLICA; en l'altre sentit, si $G' \in \text{GRAN-CLICA}$, llavors G' té un subgraf complet amb la meitat dels seus vèrtexs, és a dir, $n - k$, dels quals almenys k han de pertànyer a la còpia de G . Per tant, $(G, k) \in \text{CLICA}$.

És clar que ambdues reduccions es fan en temps polinòmic.

28.5 Ordenació topològica

Suposem que l'única ordenació topològica és $0, \dots, n - 1$. Llavors els arcs mostrats en el graf de l'esquerra (per a $n = 5$) són imprescindibles ja que, altrament, hi hauria altres ordenacions possibles. Un cop fixat un ordre entre els vèrtexs, tots els arcs orientats cap a l'esquerra estan prohibits. I la resta d'arcs orientats cap a la dreta són opcionals. Per exemple, a la dreta es pot veure un dels diversos grafs amb la mateixa ordenació topològica $0, 1, 2, 3, 4$.



Així doncs, el nombre d'ordenacions topològiques és $n!$ i, per a cadascuna, el nombre d'arcs opcionals és $(n - 2) + \dots + 0 = (n - 1)(n - 2)/2$. Per tant, podem deduir que el nombre de grafs és $n!2^{(n-1)(n-2)/2}$.

28.6 Codi espifiat

L'algorisme comprova si un graf no dirigit és connex o no. Ho fa usant un MF-set, fins que només queda una component connexa, o fins que les arestes s'acaben. L'error està a la línia

$$\text{pare}[y] = x;$$

que hauria de ser

$$\text{pare}[ry] = rx;$$

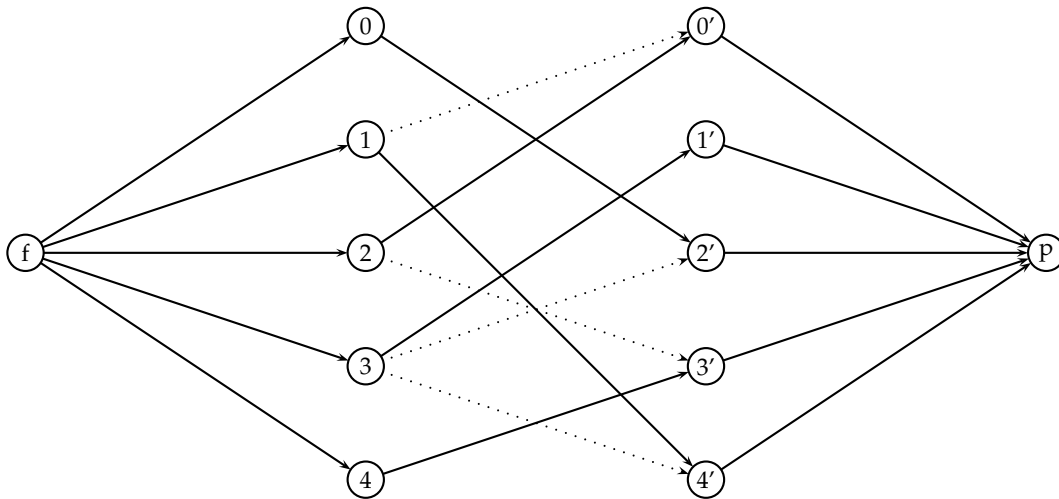
En el cas millor, les primeres $n - 1$ arestes (si existeixen) connecten tot el graf. En aquest cas es fan $\min(n - 1, m)$ iteracions, cadascuna de cost "constant". Addicionalment, la línia

$$\text{vector} \langle \text{int} \rangle \text{pare}(n, -1);$$

té cost $\Theta(n)$. Tot plegat, el cost en el cas millor és $\Theta(n)$. En el cas pitjor cal fer m iteracions, i el cost és $\Theta(n + m)$.

28.7 Max-flow

És fàcil veure que un graf dirigit és un conjunt de cicles si i només si tots els vèrtexs tenen exactament un arc d'entrada i un arc de sortida. Per tant, si dupliquem els vèrtexs del graf original, afegim una font i un pou, i fem que tots els arcs tinguin capacitat u , com es pot veure en l'exemple (on s'han marcat amb punts els arcs per on no acaba passant flux), llavors es poden escollir n arcs adequats si i només si el flux màxim és n .



28.8 Teoria de jocs

És impossible. Sigui n el número de la composició de la resta de piles, i suposem que a la pila en qüestió hi ha dues jugades guanyadores que deixen x i y pedres respectivament, amb $x \neq y$. Com que el resultat de les jugades ha de ser perdedor, tenim $x \wedge n = y \wedge n = 0$, cosa que implica $x = y = n$, contra la hipòtesi.

Parcial, 14 de novembre de 2014

29.1 Ordenació per selecció

1. Les assignacions entre elements només es fan dins dels *swap()*. Cada *swap()* realitza tres assignacions (i no dues!). Com que a cada iteració del bucle de les *i* s'executa un *swap()*, en total se n'executen *n* i, per tant, es fan $3n$ assignacions.
2. Les úniques comparacions entre elements tenen lloc dins del **if**. Per tant, se'n fan $\sum_{i=0}^{n-1} \sum_{j=i+1}^{n-1} 1 = n(n-1)/2$.
3. Sigui *X* la variable aleatòria que compta el nombre de cops que s'executa ($\star$). Per trobar l'esperança de *X*, definim $X_{i,j}$ com la variable aleatòria que val 1 si la instrucció ($\star$) s'executa a la iteració *j* del bucle interior dins de la iteració *i* del bucle exterior, i 0 altrament. Llavors,

$$\mathbf{E}[X] = \mathbf{E} \left[\sum_{i=0}^{n-1} \sum_{j=i+1}^{n-1} X_{i,j} \right].$$

Per linearitat de l'esperança i definició d'esperança,

$$\mathbf{E}[X] = \sum_{i=0}^{n-1} \sum_{j=i+1}^{n-1} \mathbf{E}[X_{i,j}] = \sum_{i=0}^{n-1} \sum_{j=i+1}^{n-1} \Pr[X_{i,j} = 1].$$

Per trobar $\Pr[X_{i,j} = 1]$, observem que $X_{i,j} = 1$ quan $v[j]$ és el valor més petit del subvector $v[i, \dots, j]$. Com que els elements de *i* a *n* - 1 sempre estan distribuïts a l'atzar i sense repeticions, la probabilitat de que l'element $v[j]$ sigui el mínim de $v[i, \dots, j]$ és $1/(j - i + 1)$. Per tant, tenint en compte que $H(k)$ és el *k*-èsim nombre

harmònic,

$$\begin{aligned}
 \mathbb{E}[X] &= \sum_{i=0}^{n-1} \sum_{j=i+1}^{n-1} \frac{1}{j-i+1} \\
 &= \sum_{i=0}^{n-1} \sum_{j=2}^{n-i} \frac{1}{j} \\
 &= \sum_{i=0}^{n-1} (H(n-i) - 1) \\
 &= \sum_{i=0}^{n-1} H(n-i) - n \\
 &= \sum_{i=1}^n H(i) - n.
 \end{aligned}$$

[A partir d'aquí, es pot veure que $\mathbb{E}[X] = \Theta(n \log n)$.]

29.2 Recurrència

[Com a referència, aquesta recurrència és la que caracteritza el cost de l'algorisme de mediana de medianes per trobar el k -èsim element ordenat d'un vector en temps lineal en el cas pitjor.]

Sigui $R(n) = R(\frac{1}{5}n) + R(\frac{3}{4}n) + \beta n$. Demostrarem per inducció que, per a tota n , $R(n) \leq \alpha n$ per a una certa constant α . D'aquí, es dedueix que $T(n) = T(\frac{1}{5}n) + T(\frac{3}{4}n) + O(n)$ és $O(n)$.

Com a hipòtesi d'inducció, suposem que, donada una n , per a tota $m < n$ tenim que $R(m) \leq \alpha m$. Llavors,

$$\begin{aligned}
 R(n) &= R(\frac{1}{5}n) + R(\frac{3}{4}n) + \beta n \\
 &\leq \alpha \frac{1}{5}n + \alpha \frac{3}{4}n + \beta n \\
 &\leq (\frac{19}{20}\alpha + \beta) n \\
 &\leq \alpha n
 \end{aligned}$$

sempre que $\alpha \geq 20\beta$.

29.3 Primer element repetit

1. Per a cada element (d'esquerra a dreta), cal mirar si aquest té algun element igual a la seva dreta. El cost temporal és $\Theta(n^2)$ i l'espacial és $\Theta(1)$.
2. Utilitzem un conjunt d'enters inicialment buit. Per a cada element, si el trobem al conjunt, és que és una repetició; si no hi és, l'afegim.

Si implementem els conjunts amb un arbre equilibrat (com ara un *set* `<int>`), en el cas pitjor cada iteració es fa en temps $O(\log n)$ i el cost total és $\Theta(n \log n)$.

D'altra banda, si implementem els conjunts amb taules de dispersió (com ara un *unordered.set* `<int>`), en mitjana cada iteració es fa en temps $\Theta(1)$ i el cost total mitjà és $\Theta(n)$. El cas pitjor, però, és $\Theta(n^2)$.

En ambdós casos, l'espai necessari és $\Theta(n)$.

29.4 Subvector de suma zero

Si calculem la suma prefixada del vector (és a dir, per a cada element, la suma dels elements a la seva esquerra), una subseqüència d'elements consecutius de suma zero es correspon a un element repetit a la suma prefixada.

Per tant, podem utilitzar la solució del problema anterior per resoldre el problema en $\Theta(n \log n)$ passos en el cas pitjor utilitzant un *set* o bé en $\Theta(n)$ passos en el cas mitjà utilitzant un *unordered_set*.

29.5 Nombres de Fibonacci

Sigui $M = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$. Llavors tenim $\begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix} = M \begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix} = \dots = M^n \begin{pmatrix} 1 \\ 0 \end{pmatrix}$.

Per tant, F_n és l'element de baix a l'esquerra de la matriu M^n , la qual es pot calcular amb exponenciació ràpida amb cost $\Theta(\log n)$.

29.6 Heaps

Els passos després d'inserir un 43 són:

47	41	31	37	29	13	19	17	11	23	43
47	41	31	37	43	13	19	17	11	23	29
47	43	31	37	41	13	19	17	11	23	29

Els passos després d'esborrar el màxim són:

23	41	31	37	29	13	19	17	11
41	23	31	37	29	13	19	17	11
41	37	31	23	29	13	19	17	11

29.7 Ordenació topològica

El nombre màxim d'ordenacions topològiques amb n vèrtexs és $n!$, i aquestes es poden assolir només amb un graf sense arcs.

Com que el graf no té cicles, sempre tindrà, almenys, una ordenació topològica. Un dels molts grafs que només en tenen una és $1 \rightarrow 2 \rightarrow \dots \rightarrow n$.

Extraordinari, 7 de juliol de 2014

30.1 Recurrència

Tenim $F(n) = 2F(n/2) + 2F(n/3) + \Theta(n^2)$. Definim $R(n) = 2R(n/2) + 2R(n/2) + \Theta(n^2)$. Pel teorema mestre, $R(n) = \Theta(n^2 \log n)$. Com que $F(n) \leq R(n)$ per a tot n prou gran, $F(n) = O(n^2 \log n)$.

30.2 Interseccions

- a) Primer s'ordenen els K vectors amb l'algorisme de heap-sort. Després, es van fusionant els K vectors per anar-se quedant amb els elements comuns.

Les K ordenacions prenen temps $\Theta(Kn \log n)$ en el cas pitjor. Amb heap-sort, el cost en espai és constant.

La fusió de dos vectors de, com a molt, n elements pren temps $\Theta(n)$ en el cas pitjor i requereix un vector auxiliar de mida $\Theta(n)$.

Així doncs, en total, el temps és $\Theta(Kn \log n + Kn) = \Theta(n \log n)$ i l'espai és $\Theta(n)$.

- b) Primer, per a cada vector, es guarden els seus elements en un conjunt implementat amb una taula de hash. Després, per a cada element del primer vector, es mira si apareix a tots els altres conjunts.

Com que, en mitjana, les insercions i les cerques en taules de hash prenen temps constant i se'n fan Kn de cada, el temps mitjà és $\Theta(n)$. L'espai necessari és $\Theta(Kn) = \Theta(n)$ per guardar les K taules de hash amb n elements cadascuna.

30.3 Anagrames

Dues paraules són anagrames si els seus vectors de caràcters ordenats són iguals.

Utilitzarem una estructura del tipus `unordered_map<string, vector<string>> M` que té com a claus paraules ordenades i com a informacions totes les paraules de L que són anagrames de la clau.

Per contruir l'estructura de dades, per a cada paraula s de L , s'insereix s al final de la llista de $M[o(s)]$, on $o(s)$ és s ordenada.

Per a cada consulta x , cal retornar $M[o(x)]$.

Suposant que les paraules tenen una mida constant, les seves ordenacions tenen també cost constant. Per crear M , calen n insercions, cadascuna de cost constant en mitjana. Per tant, en total, temps $\Theta(n)$ en mitjana. L'espai necessari serà el de les n paraules i la taula de hash, que és $\Theta(n)$.

Les cerques es fan en temps constant en mitjana.

[Si s'usa un *map* enlloc d'un *unordered_map* cal modificar els costos constants en mitjana per costos logarítmics en el cas pitjor.]

30.4 Creació de heaps

Vegeu problema 32.3.

30.5 Quadrat d'una matriu

Per calcular el producte de dues matrius X i Y de mida $n \times n$, es pot calcular el quadrat de

$$M = \begin{pmatrix} 0 & Y \\ X & 0 \end{pmatrix} \quad \text{que és} \quad M^2 = \begin{pmatrix} YX & 0 \\ 0 & XY \end{pmatrix}$$

i quedar-se amb el quadrant inferior dret.

Com que M és una matriu de mida $2n \times 2n$, el seu quadrat es podria calcular en temps $\Theta((2n)^2) = \Theta(n^2)$ i l'extracció del quadrant en temps $\Theta(n^2)$.

30.6 Arbres de cerca

```
int mida (punter p) {
    return p ? p->m : 0;
}

int posicio (double y, punter p) {
    if (not p) return -1;
    if (y == p->x) return 1 + mida(p->dre);
    if (y < p->x) return posicio(y, p->esq);
    int pos = posicio(y, p->dre);
    return pos == -1 ? -1 : mida(p->dre) + 1 + pos;
}
```

Com que cada cop es va per la dreta o per l'esquerra, el cost és proporcional a l'alçada de l'arbre.

30.7 Teoria de jocs

L'afirmació és falsa. Per exemple, si els números són 5, 4 i 2, l'únic moviment guanyador que existeix segur canvia el 2 per un 1. (Potser es pot canviar el 5 per un 6, però d'això no en podem estar segurs.)

Final, 16 de gener de 2014

31.1 Sumeu zero

1. Cal aplicar l'operació de cerca estàndard dels AVL. El cost és $\Theta(\log n)$.
2. Es pot fer un algorisme recursiu que, començant a l'arrel, miri el signe de cada node: si és zero, ja s'ha trobat, si és negatiu ja es pot acabar, i si és positiu es continua pels seus dos fills. El cost és $\Theta(n)$ perquè el zero pot trobar-se (o no) a qualsevol posició de l'arbre.
3. S'ordena el vector i s'utilitza la solució del punt següent (4). Amb un algorisme d'ordenació $\Theta(n \log n)$ es cost és $\Theta(n \log n) + \Theta(n) = \Theta(n \log n)$.
4. Es situen dos índexos a cada extrem del vector. Mentre no es creuin ni els valors a què apunten sumen zero, si la suma és positiva es mou el de la dreta cap a l'esquerra, i sinó es mou el de l'esquerra cap a la dreta. El cost és $\Theta(n)$.
5. Només cal mirar uns pocs casos quan el vector és prou petit. El cost és $\Theta(1)$.
6. Es recorre l'arbre en inordre per deixar tots els seus elements ordenats en un vector (en temps lineal) i s'utilitza el punt 4. El cost és $\Theta(n)$.
7. S'ordena el vector i, per a cadascun dels seus elements x , es cerca si dos elements sumen $-x$ utilitzant una generalització del punt 4. El cost és llavors $\Theta(n^2)$.

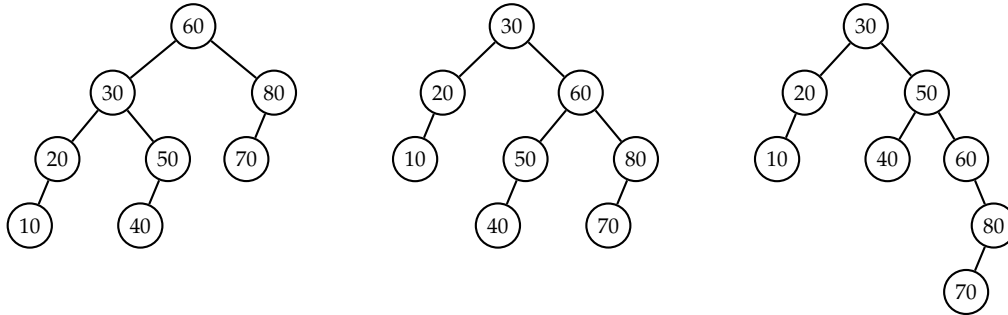
31.2 Ordenació de vectors ordenadets

Ens podem valer d'un min-heap de $2 \log n + 1$ elements. Al principi de l'algorisme, es fiquen els $2 \log n + 1$ primers elements del vector (si existeixen) en el heap. A partir d'aquell moment, a cada iteració, s'extreu el mínim del vector (que ha de ser el següent element del vector ordenat) i s'hi insereix el següent element de la taula (si en queda algun) fins que el heap quedi buit.

L'espai necessari és $\Theta(\log n)$ per emmagatzemar el heap. El temps és $O(n \log \log n)$, perquè es fan n insercions i extraccions del mínim en un heap amb $O(\log n)$ elements.

31.3 Arbres de cerca

Si l'algorisme escull l'arrel del segon arbre (60) com arrel del resultat, cosa que passa amb probabilitat $3/8$, obtenim el primer arbre. Altrament (amb probabilitat $5/8$), l'arrel del resultat serà la del primer arbre (30). En aquest cas cal fer recursivament el join del subarbre dret del primer arbre i del segon arbre. Amb probabilitat $(3/5) \times (5/8) = 3/8$ obtindrem el segon arbre, i amb probabilitat $(2/5) \times (5/8) = 1/4$ el tercer.



31.4 Teoria de jocs

La variant del joc presentada té les característiques necessàries per poder-lo analitzar amb nombres. És fàcil veure per inducció que el nombre d'una pila amb n pedres és $n \bmod 4$. Així doncs, els nombres de cada pila donada són 1, 3, 0, 1, 3, 1, 2, 0 i 0. A més, sabem que el nombre de la composició de jocs és el "xor" dels nombres. Podem concloure que la posició de la partida té nombre 3, que no és 0, i per tant la posició és guanyadora. Una possible jugada guanyadora consisteix a treure 3 pedres de la segona pila, deixant al rival una posició amb nombre 0.

31.5 Fluxos

La reducció és incorrecta, perquè els fluxos que arriben a cada x_i es poden repartir entre els dos arcs que en surten, de manera que sempre pot arribar tot el flux s fins al pou, independentment de si la instància del problema *subset sum* que estem intentant resoldre té solució o no.

Si existís una reducció correcta (i en temps polinòmic) de *subset sum* a *max-flow*, tenint en compte que el segon pertany a **P**, llavors el primer també hi pertanyeria, i com que sabem que és **NP**-complet, tots els problemes de la classe **NP** es podrien resoldre en temps polinòmic, així que **P** seria igual a **NP**.

31.6 Codi mort

El problema d'en Genís és indecidible: Suposem que existís un algorisme per resoldre'l. Aleshores, podríem resoldre el problema de la totalitat (donat un programa p , acaba per a totes les seves entrades?) tot escrivint un nou programa que sigui p al qual li hem afegit una instrucció qualsevol i al final. Si es pogués decidir que i és codi mort, es podria decidir el problema de la totalitat per a p . Però aquest problema és indecidible i, per tant, també ho és el d'en Genís.

Parcial, 8 de novembre de 2013

32.1 Heu preparat la lliçó?

- $O(f) = \{g : \mathbb{N} \rightarrow \mathbb{N} \mid \exists n_0 > 0, c_0 > 0 : \forall n \geq n_0 : g(n) \leq c_0 f(n)\}.$
- $\Omega(f) = \{g : \mathbb{N} \rightarrow \mathbb{N} \mid \exists n_0 > 0, c_0 > 0 : \forall n \geq n_0 : g(n) \geq c_0 f(n)\}.$
- $\Theta(f) = O(f) \cap \Omega(f).$

32.2 Polinomis

1. Utilitzant la Regla de Horner es pot avaluar $p(x)$ en temps $\Theta(n)$:

$$p(x) = p[0] + x(p[1] + x(p[2] + \dots))$$

2. El resultat de multiplicar $p(x) = p[0] + p[1]x + \dots + p[n-1]x^{n-1}$ per $q(x) = q[0] + q[1]x + \dots + q[n-1]x^{n-1}$ és $r(x) = r[0] + r[1]x + \dots + r[2n-1]x^{2n-1}$ amb

$$r[k] = \sum_{i=0}^k p[i]q[k-i]$$

sent $p[i]$ i $q[i]$ zero quan $i \geq n$.

El temps necessari és doncs $2 \sum_{k=0}^{n-1} \Theta(k) = \Theta(n^2)$.

3. Es té

$$(ax + b)(cx + d) = acx^2 + (a + b)(c + d)x + bd - bdx - acx$$

i, a la banda dreta, només hi ha tres productes de reals: ac , $(a + b)(c + d)$ i bd .

4. Podem escriure els polinomis $p(x) = \sum_{i=0}^{n-1} p[i]x^i$ i $q(x) = \sum_{i=0}^{n-1} q[i]x^i$ com a

$$p(x) = \sum_{i=0}^{n/2-1} p[i]x^i + x^{n/2} \sum_{i=0}^{n/2} p[i+n/2]x^i = b(x) + x^{n/2}a(x)$$

i

$$q(x) = \sum_{i=0}^{n/2-1} q[i]x^i + x^{n/2} \sum_{i=0}^{n/2} q[i + n/2]x^i = d(x) + x^{n/2}c(x).$$

i aplicar la idea de la pregunta anterior per obtenir

$$p(x)q(x) = acx^n + (a+b)(c+d)x^{n/2} + bd - bdx^{n/2} - acx^{n/2}$$

que estableix una forma recursiva de fer la multiplicació de polinomis de grau $n - 1$ utilitzant tres multiplicacions de polinomis de grau $(n - 1)/2$, a part d'operacions de suma, de restes i de multiplicació per potències de x (que tenen cost lineal).

L'algorisme resultant té cost $T(n) = 3T(n/2) + \Theta(n)$ que, aplicant el teorema mestre, resulta en $\Theta(n^{\log_2 3})$.

32.3 Heaps i heapsort

1. Cal verificar que cada node (menys l'arrel) sigui menor o igual que el seu pare:

```
bool is_max_heap (const vector<double>& v) {
    int n = v.size ();
    for (int i=1; i<n; ++i) {
        if (v[i] > v[(i-1)/2]) {
            return false;
        }
    }
    return true;
}
```

El cas pitjor és quan el vector és un max-heap. En aquest cas, l'algorisme fa $n - 1$ iteracions, cadascuna de cost constant. Per tant, el cost en el cas pitjor és $\Theta(n)$. El cas millor és quan a la primera iteració es veu que l'arbre no satisfà la propietat de forma. El cost en el cas millor és doncs $\Theta(1)$.

2. Es pot fer enfonsant cada node, des de les fulles de l'arbre cap a l'arrel:

```
void heapify (const vector<double>& v) {
    int n = v.size ();
    for (int i = n/2 - 1; i ≥ 0; i--) {
        sink(v, n, i);
    }
}

void sink (vector<double>& v, int n, int i) {
    double x = v[i];
    int c = 2*i + 1;
    while (c < n) {
        if (c+1 < n and v[c] < v[c + 1]) ++c;
        if (x ≥ v[c]) break;
        v[i] = v[c];
        i = c;
        c = 2*i + 1;
    }
    v[i] = x;
}
```


El cost en el cas pitjor ve donat per

$$T(n) = \sum_{i=1}^{\log n} i \frac{n}{2^i}$$

perquè els $n/2$ nodes del darrer nivell s'han d'enfonsar un nivell, els $n/4$ nodes del darrer nivell s'han d'enfonsar dos nivells, ..., i l'arrel s'ha d'enfonsar $\log n$ nivells.

Per trobar $T(n)$, es pot veure que

$$T(n) - 2T(n/2) = \sum_{i=1}^{\log n} i \frac{n}{2^i} - \sum_{i=1}^{\log n-1} i \frac{n}{2^i} = \log n$$

i, aplicar el teorema mestre per obtenir $T(n) = \Theta(n)$.

```
3. void heapsort (vector<double>& v) {
    int n = v.size ();
    make_heap(v);
    for (int i = n - 1; i ≥ 1; --i) {
        swap(v[0], v[i]);
        sink(v, i, 0);
    } }
```

El cost d'aquest algorisme bé donat per $T(n) = \sum_{i=1}^n \Theta(\log i) = \Theta(n \log n)$.

32.4 Comparacions del Quicksort

Per tal que quicksort realitzi el nombre màxim de comparacions possibles cal que cada pivot que tria sigui el màxim o el mínim del subvector a ordenar. Per tant, hi ha 2^{n-1} permutacions que provoquen el màxim nombre de comparacions.

32.5 Múltiples de 66 en un conjunt

```
void elimina_multiples_de_66 (set<int>& s) {
    auto it = s.begin ();
    while (it ≠ s.end()) {
        if (*s % 66 == 0) {
            s.erase (it ++); // C++ fa el post-increment abans de la crida
        } else {
            ++it;
        }
    } }
```

Com que el cost per esborrar un element d'un set és logarítmic respecte el nombre d'elements en ell, si s conté n elements, aquest algorisme té cost $O(n \log n)$.

Final, 15 de gener de 2013

33.1 El mateix que el del parcial

Vegeu una possible solució a la secció 34.2.

33.2 Algorisme curiós

Per començar, suposarem com és habitual a l'assignatura que dividir té cost constant. (Altrament, els càlculs es poden adaptar.) Si x és parell, n passa a valer $n - 1$. Si x és senar, n passa a valer aproximadament $\log_2 n$, que asimptòticament és molt més petit que $n - 1$.

Així doncs, en el cas pitjor tenim la recurrència $P(n) = \Theta(1) + P(n - 1)$, la solució de la qual òbviament és $P(n) = \Theta(n)$. El cas pitjor és $x = 2^{n-1}$.

En el cas millor, els nombres quasi sempre són senars (ja que cal passar pel 2 per arribar a 1), i tenim la recurrència aproximada $M(n) = \Theta(1) + M(\log_2 n)$, que no està coberta directament per cap "Teorema Mestre", però es pot iterar: $M(n) = \Theta(1) + M(\log_2 n) = 2\Theta(1) + M(\log_2(\log_2 n)) = 3\Theta(1) + M(\log_2(\log_2(\log_2 n))) \dots$ fins arribar a 1, és a dir, $\Theta(\log^* n)$ vegades. Per tant, tenim $M(n) = \Theta(\log^* n)$. La successió $a_1 = 1$, $a_2 = 2$, $a_3 = 3 = 2^2 - 1$, $a_4 = 7 = 2^3 - 1$, $a_5 = 127 = 2^7 - 1$, $a_6 = 2^{127} - 1, \dots$ és un exemple de cas millor en el sentit invers de les crides recursives.

33.3 Problemes petits i bonics sobre vectors

1.

Algorisme	Funciona	Temps CP	Temps CM	Espai CP
a	no			
b	sí	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$
c	sí	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$
d	sí	$\Theta(n^2)$	$\Theta(n \log n)$	$\Theta(n)$ o $\Theta(\log n)$ ★
e	sí	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$
f	no			
g	sí	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
h	sí	$\Theta(n^2)$	—	$\Theta(1)$
i	no			
j	sí	$\Theta(n!n)$	$\Theta(n!n)$	$\Theta(n)$

(★ Si s'implementa amb cura el QS, només cal espai logarítmic, però si es fa malament pot caler lineal.)

- Podem calcular la suma S de tots els elements del vector v i, llavors, l'element que falta és el $n(n+1)/2 - S$.
- Aquest cop calculem la suma S de tots els elements del vector v i la suma T de tots els quadrats dels elements del vector. Llavors, els elements x i y que falten han de complir

$$\begin{cases} x + y + S = \sum_{i=1}^n i = n(n+1)/2 = S', \\ x^2 + y^2 + T = \sum_{i=1}^n i^2 = n(n+1)(2n+1)/6 = T'. \end{cases}$$

Com que

$$x^2 + y^2 = (x + y)^2 - 2xy$$

obtenim

$$xy = ((S' - S)^2 - (T' - T))/2$$

i, per tant, x i y són les arrels de l'equació de segon grau

$$z^2 - (S' - S)z + ((S' - S)^2 - (T' - T))/2 = 0.$$

33.4 Agència matrimonial

Aquest problema es pot reduir a calcular un max-flow en un graf bipartit amb $h + d + 2$ vèrtexs. Afegim una font 0 connectada a cada home, amb capacitat de cada aresta igual al màxim nombre de dones que vol aquell home en qüestió, un pou $h + d + 1$ connectat a cada dona, amb capacitat 1 a cada aresta, i arestes amb capacitat 1 entre cada parella home-dona compatible entre si. És fàcil veure que el flux màxim d'aquest graf coincideix amb el màxim nombre de parelles que es poden fer tot respectant les restriccions.

33.5 Tasques i màquines

1. El problema decisional que hem de tractar (anomenem-lo TASQUES) és el següent:

Donades m màquines, n tasques de temps $t_1, \dots, t_n$ i un natural k , existeix alguna assignació que tingui temps de procés inferior o igual a k ?

El problema és en **NP**: Podem fer servir com a testimoni d'una entrada positiva n enters entre 1 i m que indiquen a quina màquina cal executar cada tasca. Aquest testimoni té clarament talla polinòmica i és ben fàcil escriure un algorisme polinòmic per comprovar si seu temps de procés és $\leq k$.

Per demostrar la completesa, podem reduir el problema PARTICIÓ (que sabem **NP**-complet) al problema TASQUES. Recordem que el problema PARTICIÓ és:

Donats q naturals $a_1, \dots, a_q$, es poden particionar en dos subconjunts que tinguin la mateixa suma?

A partir d'una entrada $a_1, \dots, a_q$ de PARTICIÓ, la reducció crea una entrada per TASQUES triant $m = 2$ màquines, $n = q$ tasques, $t_i = a_i$ per a tot $i \in [1, \dots, n]$ i $k = \lfloor S/2 \rfloor$ on $S = \sum_{i=1}^q a_i$. Aquest procés és clarament polinòmic. Per comprovar que la reducció funciona distingim dos casos segons la paritat de S :

En el cas que S és senar, l'entrada de PARTICIÓ és per força negativa, i és clar que l'entrada corresponent de TASQUES també.

En el cas que S és parell, quan l'entrada de PARTICIÓ és positiva, llavors és clar que l'entrada corresponent de TASQUES també és positiva. I, del revés, quan l'entrada generades de TASQUES és positiva, llavors és clar que l'entrada original de TASQUES també és positiva.

Nota: Una manera alternativa de resoldre el problema és adonant-se que el problema TASQUES és equivalent a $\text{BIN-PACKING}_{\leq}$ i recordar que a classe s'ha dit (però no demostrat) que aquest és **NP**-complet.

2. Sigui A el temps de procés de l'algorisme proposat i sigui OPT el temps de procés mínim.

És clar que $OPT \geq \sum_{i=1}^n t_i / m$ i que $OPT \geq t_i$ per a tota i .

D'altra banda, al moment que es posi en marxa la darrera tasca en ser processada, cal que totes les altres màquines estiguin processant altres tasques (altrament, alguna màquina hagués estat lliure anteriorment i l'algorisme li hagués assignat aquesta tasca). Sigui t_u el temps de tractament d'aquesta tasca i sigui t l'instant en què comença. Llavors, $A = t + t_u$ i $t \leq \sum_{i=1}^n t_i / m$.

Com que $OPT \geq t_u$ i $OPT \geq \sum_{i=1}^n t_i / m$, tenim que $2OPT \geq \sum_{i=1}^n t_i / m + t_u \geq t + t_u = A$ i, per tant, $A / OPT \leq 2$.

Com que l'algorisme obviament funciona en temps polinòmic, queda demostrat que és un algorisme d'aproximació de raó 2.

Parcial, 13 de novembre de 2012

34.1 Ompliu els blancs

- Definiu formalment $O(f)$:

$$O(f) = \{g : \mathbb{N} \rightarrow \mathbb{N} \mid \exists n_0 > 0, c_0 > 0 : \forall n \geq n_0 : g(n) < c_0 f(n)\}$$

- Si $T(n) = 2T(n-3) + n^2 - 2n + 1$, llavors $T(n) = \Theta(\boxed{2^{n/3}})$.
- Per ordenar un vector amb n elements, l'algorisme d'ordenació per inserció triga $\Theta(\boxed{n})$ passos en el cas millor i $\Theta(\boxed{n^2})$ passos en el cas pitjor.
- Si féssim una cerca dicotòmica partint el vector en tres trossos enlloc de dos i buscant en el terç adequat, el cost resultant seria $\Theta(\boxed{\log n})$.
- Suposeu que hi ha una manera de multiplicar dues matrius 3×3 amb 25 productes de reals i 50 sumes de reals. Assumint que n és una potència de 3, el cost que tindria un algorisme recursiu basat en aquest mètode per multiplicar matrius $n \times n$ seria $\Theta(\boxed{n^{\log_3 25}})$.

34.2 Comparacions del QuickSort

Per a $1 \leq i < j \leq n$, sigui $X_{i,j}$ la variable aleatòria que val 1 si l' i -èsim element del conjunt es compara amb el j -èsim element i 0 altrament. A més, sigui $p_{i,j}$ la probabilitat que $X_{i,j}$ sigui 1. Llavors, $\mathbb{E}[X_{i,j}] = p_{i,j}$.

El nombre esperat de comparacions efectuades per Quicksort és

$$C_n = \mathbb{E} \left[\sum_{i=1}^n \sum_{j=i+1}^n X_{i,j} \right]$$

i, per linearitat de l'esperança,

$$C_n = \mathbf{E} \left[\sum_{i=1}^n \sum_{j=i+1}^n X_{i,j} \right] = \sum_{i=1}^n \sum_{j=i+1}^n \mathbf{E} [X_{i,j}] = \sum_{i=1}^n \sum_{j=i+1}^n p_{i,j}.$$

Per calcular $p_{i,j}$, ens cal tenir en compte que i i j es comparen si i només si, d'entre tots els elements de i a j , es tria i o j com a pivot abans que qualsevol element entre $i+1$ i $j-1$. Com que la selecció del pivot es fa a l'atzar i hi ha dues possibles eleccions d'entre les $j-i+1$ possibles que condueixin a comparar i i j , tenim que $p_{i,j} = 2/(j-i+1)$. Aleshores,

$$C_n = \sum_{i=1}^n \sum_{j=i+1}^n \frac{2}{j-i+1} \leq 2 \sum_{i=1}^n \sum_{k=1}^n \frac{1}{k} = 2nH_n = O(n \log n).$$

D'altra banda, hem vist a classe que el cas millor de Quicksort és $\Theta(n \log n)$. Com que el valor esperat ha de ser, com a mínim, tant gran com el del més petit, obtenim $C_n = \Theta(n \log n)$.

34.3 L'Euclides no hi és

1. Si els dos paràmetres són parells, segur que el seu mcd conté un factor 2 que podem eliminar de tots dos.

Si a és parell i b és senar, el seu mcd no contindrà cap factor 2, així que podem calcular-lo en base de b i la meitat de a .

Si a i b són senars amb $a > b$, la identitat d'Euclides $\text{mcd}(a, b) = \text{mcd}(a-b, b)$ i l'aplicació de l'identitat anterior (perquè $a-b$ és parell) dona $\text{mcd}(a, b) = \text{mcd}((a-b)/2, b)$.

2. Cal escollir un cas base i evitar valors negatius als paràmetres:

```
int mcd (int a, int b) {
    if (a == b) return a;
    if (a%2==0 and b%2==0) return 2*mcd(a/2, b/2);
    if (a%2==1 and b%2==0) return mcd(a, b/2);
    if (a > b) return mcd((a-b)/2, b);
    return mcd((b-a)/2, a);
}
```

Cada crida recursiva divideix almenys un dels dos paràmetres per la meitat. Per tant, el nombre màxim de crides recursives que es poden fer és $O(\log a + \log b)$. Com que el cost imputable a la part no recursiva és constant, el cost total és $O(\log a + \log b)$.

3. Podem rescriure l'algorisme anterior per a nombres llargs:

```
bigint mcd (bigint a, bigint b) {
    if (iguals(a, b)) return a;
    if (parell(a) and parell(b)) return mul2(mcd(div2(a), div2(b)));
    if (senar(a) and parell(b)) return mcd(a, div2(b));
    if (major(a,b)) return mcd(div2(resta(a, b)), b);
    return mcd(div2(resta(b, a)), a);
}
```


Aquí, hem utilitat funcions per les operacions aritmètiques necessàries amb significat evident i implementació directa. El seu cost és, com a molt, lineal respecte la llargada dels seus paràmetres.

Suposem que el nombre de bits totals per codificar a i b és n . Com que cada crida recursiva escurça d'un bit la talla d'un dels dos paràmetres, el nombre de crides recursives és, com a molt, n . Com que el cost de cada crida no imputable a la part recursiva és lineal, el cost total és $O(n^2)$.

34.4 La funció misteri ataca de nou

1. Aquesta funció retorna el mínim d'un vector d'enters. En el cas que el vector sigui buit comet un error accedint a un element invàlid.
2. El cas millor es dona quan el mínim del vector es troba a la primera posició, llavors no s'executa mai.
3. El cas millor es dona quan el vector està ordenat decreixentment. Llavors s'executa $n - 1$ cops.
4. La probabilitat que l'element i -èsim sigui el més petit de i elements és $1/i$. Per tant, el nombre esperat de cops que s'executa (*) sobre una permutació aleatòria és $\sum_{i=1}^{n-1} 1/i = H_{n-1} = \Theta(\log n)$.

34.5 Nombres aborrits

Siguin x i y dos nombres de n bits descomposats en $x = x_1 2^{n/2} + x_0$ i $y = y_1 2^{n/2} + y_0$ on x_1, x_0, y_1 i y_0 són nombres de $n/2$ bits. Llavors $xy = (x_1 2^{n/2} + x_0)(y_1 2^{n/2} + y_0) = x_1 y_1 2^n + x_1 y_0 2^{n/2} + x_0 y_1 2^{n/2} + x_0 y_0$.

Quan x i y són aborrits, $x_1 = x_0$ i $y_1 = y_0$, de manera que $xy = x_0 y_0 2^n + x_0 y_0 2^{n/2} + x_0 y_0 2^{n/2} + x_0 y_0$. Per tant, podem calcular el producte recursivament únicament sobre $x_0 y_0$ i obtenir que el cost de l'algorisme resultant és $T(n) = T(n/2) + \Theta(n)$ que, aplicant el teorema mestre, dona $T(n) = \Theta(n)$.

34.6 Encara més misteris?

El programa escriu, en ordre i sense repeticions, els n primers naturals de la forma $2^\alpha 3^\beta 5^\gamma$, per a qualssevol naturals α, β i γ .

Durant la i -èsima iteració, el conjunt té entre i i $3i - 2$ nombres. Per tant, el cost de la i -èsima iteració és $\Theta(\log i)$, i el cost temporal total del programa és $\sum_{i=1}^n \Theta(\log i) = \Theta(n \log n)$. El moment en què s'usa més memòria és al final del programa, quan el conjunt té entre n i $3n - 2$ nombres. Per tant, el cost en espai és $\Theta(n)$.

Extraordinari, 29 de juny de 2012

35.1 Miscel·lània

- a) Com que el cost dels intercanvis és $\Theta(1)$, fent $n = d - e + 1$ el cost del codi segueix la recurrència

$$T(n) = 3T(n/2) + \Theta(1) ,$$

i del Teorema Mestre de Recurrències Divisores obtenim $T(n) = \Theta(n^{\log_2 3})$.

- b) El programa no fa res!
- c) El joc dels escacs o té una estratègia guanyadora o no la té. Així doncs, el problema té una solució trivial: l'algorisme que sempre diu *sí* o l'algorisme que sempre diu *no*, malgrat que ara mateix no sapiguem quin dels dos és. Per tant, el problema es pot resoldre en temps constant, per tant és a **Pi**, per tant, també a **NP**.
- d) No podem assegurar res, aquest problema té l'algorisme amb el cost proposat a l'enunciat, però en podria tenir un altre de cost més baix.
- e) Alan Turing (23 de juny de 1912 / 7 de juny de 1954) fou un matemàtic britànic. Va treballar en camps com la informàtica teòrica, la criptoanàlisi o la intel·ligència artificial. Se'l considera el pare de la informàtica moderna. [Text i foto: Wikipedia]



35.2 Algorismes d'aproximació

Per a cada aresta $e \in E$, hi ha tres possibilitats: Els seus dos extrems són en S , cap dels seus dos extrems són en S , o un dels seus dos extrems és en S i l'altre no. Com que cada extrem és o no en S amb probabilitat $\frac{1}{2}$, les probabilitats d'aquests tres esdeveniments són $\frac{1}{4}$, $\frac{1}{4}$, i $\frac{1}{2}$ respectivament. Així, la probabilitat que una aresta es compti al tall és $\frac{1}{2}$.

Per linearitat de l'esperança, el tall esperat és, doncs,

$$\mathbf{E}[t(S)] = \sum_{e \in E} \Pr[e \in \text{tall}] = \sum_{e \in E} \frac{1}{2} = \frac{1}{2}m.$$

D'altra banda, el tall màxim està afetat pel nombre total d'arestes: $t^*(S) \leq m$. Per tant,

$$\frac{\mathbf{E}[t(S)]}{t^*(S)} \leq \frac{1}{2}.$$

35.3 Complexitat i indecidibilitat

Per reduir SAT a ATURADA, hem de transformar les entrades de SAT a entrades d'ATURADA, és a dir, fórmules booleanes F a algorismes A i entrades x .

Per fer-ho, definim A com l'algorisme següent:

- entrada:** una fórmula F
- per a tota** assignació a de valors a les variable de F : (1)
- si** $F(a)$ és cert: (2)
- atura** (3)
- penja** (4)

La reducció és l'algorisme que, a partir de F , retorna el parell (A, x) on $x = F$.

Comprovem que realment és una reducció de temps polinòmic:

1. El cost de convertir F en (A, F) és lineal en la talla de F i, per tant, polinòmic.
2. Si F és una entrada positiva per al problema de SAT, vol dir que la fórmula F és satisfactible. Per tant, té alguna assignació que satisfà F . Per tant, $A(F)$ la trobarà i s'aturarà a la línia (3). Per tant, (A, F) és una entrada positiva per al problema de l'aturada.
3. Si F és una entrada negativa per al problema de SAT, vol dir que la fórmula F no és satisfactible. Per tant, no té cap assignació que la satisfà. Per tant, després de provar totes les assignacions possibles (són finites), $A(F)$ arribarà a la línia (4) on es penjarà. Per tant, (A, F) és una entrada negativa per al problema de l'aturada.

Aquesta reducció no implica que ATURADA sigui un problema **NP**-complet, només que és un problema **NP**-hard. Per tal que ATURADA fós **NP**-complet, a més caldria demostrar que ATURADA és a **NP**, cosa que és falsa, perquè sabem que ATURADA és indecidible.

35.4 Programació dinàmica

Definim la funció $sor(i, p, g)$ com el soroll màxim assolible utilitzant alguns dels i primers coets del catàleg, amb un preu total de p euros o menys i amb g grams o menys de pólvora. L'objectiu és doncs calcular $sor(N, P, G)$.

La funció sor es pot descriure així:

$$sor(i, p, g) = \begin{cases} 0 & \text{si } i = 0, \\ sor(i-1, p, g) & \text{si } i \neq 0 \wedge (p[i] > p \vee g[i] > g), \\ \max\{sor(i-1, p-p[i], g-g[i]) + s[i], sor(i-1, p, g)\} & \text{si } i \neq 0 \wedge p[i] \leq p \wedge g[i] \leq g. \end{cases}$$

El cost de l'algorisme de programació dinàmica resultant seria $\Theta(N \cdot P \cdot G)$, perquè caldria una taula de $N \cdot P \cdot G$ valors que es podria omplir amb cost $\Theta(1)$ per casella anant dels índexos petits als grans.

35.5 Teoria de jocs

Vegeu la solució a la secció 36.3.

35.6 Codi misteriós

La funció *modul*(*x*) retorna el resultat matemàticament correcte de $x \bmod P$. Cal fer mòduls per evitar sobreiximents, tot i que això faci que la funció *f*(*s*, *t*) es pugui equivocar.

La funció *num*(*c*) retorna el número corresponent a *c* entre 0 i 25. El programa fa càlculs tractant les cadenes com a nombres en base 26, aprofitant que les lletres són totes minúscules.

La funció *f*(*s*, *t*) diu si el string *s* està dins del string *t*. Per fer-ho, calcula el valor de "hash" corresponent a *s*, i mira si hi ha alguna subcadena de *t* de mida *n* que tingui aquest mateix valor. En aquest cas, suposa que són el mateix string i retorna **true**. Si no troba cap coincidència, retorna **false**.

En el cas pitjor (quan *s* no està dins de *t*, i no hi ha cap fals positiu) la funció fa tres bucles de mida *n*, (un d'ells per precalcular P^{n-1} , que cal per computar eficientment el valor de "hash" de cada subcadena de *t* de mida *n*), i un bucle de mida $m - n$. Com que cada pas dels bucles té cost $\Theta(1)$, i $n \leq m$, el cost en el cas pitjor és $\Theta(m)$. En el cas millor *t* comença amb *s*, i el cost només és $\Theta(n)$.

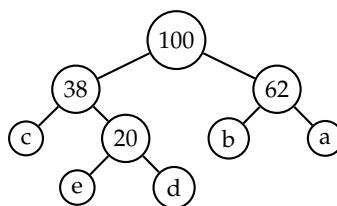
La funció no pot retornar **false** incorrectament. Si *s* està dins de *t*, la funció ho detectarà.

La funció sí que pot retornar **true** incorrectament, si *s* i algun dels substrings de *t* de mida *n* són diferents però per casualitat tenen el mateix valor de "hash". La probabilitat de que això *no* passi és aproximadament $(1 - \frac{1}{P})^{m-n+1}$.

Final, 11 de gener de 2012

36.1 Codis de Huffman

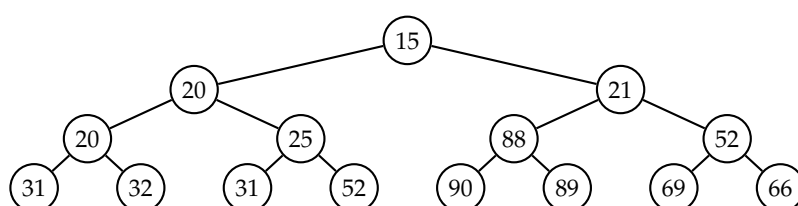
Si repetidament s'agafen els dos símbols més improbables i es posen com a fills d'un nou símbol que té probabilitat igual a la suma de les probabilitats, fins arribar a una arrel amb probabilitat 1, el resultat és aquest arbre (les fulles tenen els símbols inicials, i els altres nodes mostren les probabilitats sobre 100; suposeu que els arcs de l'esquerra tenen (arbitràriament) un 0 i els de la dreta un 1):

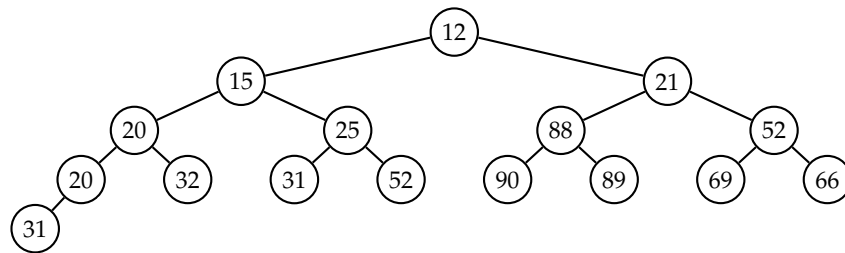


Lavors els codis serien $a = 11$, $b = 10$, $c = 00$, $d = 011$, i $e = 010$. El nombre mitjà de bits per símbol és, doncs, $0.34 \cdot 2 + 0.28 \cdot 2 + 0.18 \cdot 2 + 0.11 \cdot 3 + 0.09 \cdot 3 = 2.2$, així que calen $2.2n$ bits en mitjana per transmetre n símbols.

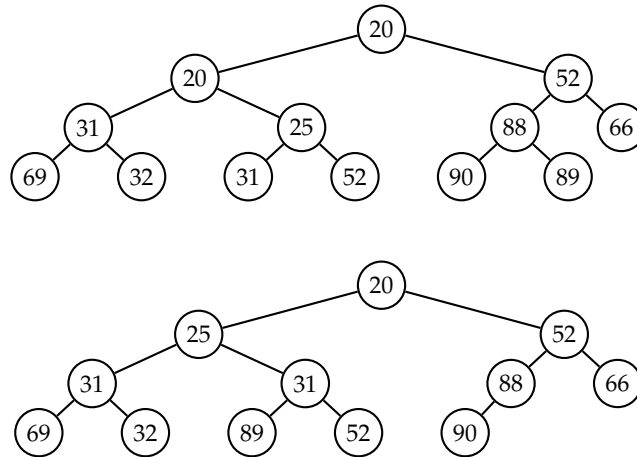
36.2 Heaps

a)





b)



36.3 Teoria de jocs

Sigui x el número de X , y el número de Y , i z el número de la composició de X i Y . Volem demostrar que $z = x \hat{y}$. La demostració es pot fer per inducció sobre el nombre de jugades que falten. Quan no falta cap jugada, tenim $X = Y = \emptyset$, i $x = y = z = 0 = 0 \hat{0}$. Ara, suposant que l'enunciat és cert fins a un cert nombre de jugades, anem a demostrar-ho per a jocs amb una jugada més. Cal veure:

- a) El valor $x \hat{y}$ no es pot aconseguir amb cap jugada.
- b) Per a tot valor $v < x \hat{y}$, v es pot aconseguir amb alguna jugada.

Considerem qualsevol moviment en (posem) Y que dugui a un Y' amb número y' . Per definició de número, $y' \neq y$. Per tant, el número de la composició de X i Y' , que per inducció és $x \hat{y}'$, és diferent de $x \hat{y}$. Això demostra (a).

Per a qualsevol $v < x \hat{y}$, la diferència més a l'esquerra en la representació binària d'ambdós nombres és un bit a 0 en v que és 1 en $x \hat{y}$. Aquest bit és 0 en x i és 1 en y (o la situació simètrica). Movent a un Y' tal que y' tingui aquest bit a 0, i escollint els altres bits a la dreta convenientment (en qualsevol cas tindrem $y' < y$), sempre es pot aconseguir $x \hat{y}' = v$. Això demostra (b).

36.4 Estructures de dades

- a) Utilitzem una taula de dispersió que enmagatzemi parells (u_i, v_i) . De primer, cal recórrer C tot inserint els m parells (u_i, v_i) a la taula. Després, cal recórrer de nou C cercant per a cada parell (u_i, v_i) si el seu invers (v_i, u_i) és a la taula. La relació és simètica si i només si tots els inversos són a la taula.

El cost de cada inserció i cerca en una taula de dispersió és constant en mitjana. Com que es fan m insercions i m cerques, el cost és $\Theta(m)$ en mitjana.

- b) Per saber si R és simètrica, aquest cop inserim els parells en un arbre AVL enlloc d'una taula de dispersió. Per saber si R és reflexiva, per a cada $u \in S$, cal mirar que (u, u) sigui a l'arbre AVL. Per saber si R és transitiva, construïm un graf on cada vèrtex es correspon amb cada element de S i cada aresta a cada parell de C i comprovem que cadascun dels seus components connexs és una clica.

El test per la simetria té cost $O(m \log m)$, ja que cal inserir primer i cercar després m elements en un arbre AVL que té com a molt m elements, i cada cerca i inserció tenen cost logarítmic en el cas pitjor.

El test de reflexivitat té cost $O(n + n \log m)$ que és $O(n + m \log m)$ si $n \leq m$. Si $m \leq n$, llavors segur que R no és reflexiva i només cal const constant.

El test de transitivitat es pot realitzar en temps $O(m + n)$ per construir el graf, $O(m + n)$ per etiquetar cada vèrtex amb l'identificador del seu component connex i $O(m + n)$ per comprovar que cada component és una clica.

Tot plegat és doncs $O(n + m \log m)$, tal com cal.

36.5 NP-completesa i programació dinàmica

- a) Per demostrar que MOTXILLA és **NP**-complet, demostrarem que és **NP** i que PARTICIÓ es redueix en temps polinòmic a MOTXILLA.

La pertinença a **NP** queda establerta a través de testimonis que consisteixen en vectors de n bits que indiquen si cal agafar o no cada objecte. L'algorisme verificador ha de comprovar que els objectes agafats no superen el pes requerit però sí superen el valor requerit. Així, és clar que els testimonis tenen talla polinòmica i el verificador pren temps polinòmic.

La reducció transforma els m valors $a_1, \dots, a_m$ de PARTICIÓ en $n = m$ objectes amb $p_i = a_i$ i $v_i = a_i$ i els valors P i V s'agafen com a $P = V = \frac{1}{2}S$ on $S = \sum_{i=1}^n a_i$. És clar que aquesta reducció pren temps polinòmic.

Si $a_1, \dots, a_m$ es pot particionar el dos subconjunts de mateixa suma, llavors hi ha un $A \subset [n]$ tal que $\sum_{i \in A} a_i = \sum_{i \notin A} a_i$ i, a més, $\sum_{i \in A} a_i = \sum_{i \notin A} a_i = \frac{1}{2}S$. En aquest cas, l'entrada corresponent per a MOTXILLA ha de ser positiva, perquè es poden agafar els objectes corresponents a A , amb $\sum_{i \in A} p_i = \sum_{i \in A} a_i = \frac{1}{2}S \leq P$ i $\sum_{i \in A} v_i = \sum_{i \in A} a_i = \frac{1}{2}S \geq V$.

A la inversa, si es poden agafar els objectes, cal que la suma dels seus pesos, la suma dels seus valors i $\frac{1}{2}S$ coincideixin. Per tant, agafant els a_i corresponents per una banda i els demés a_i en l'altra, la seva suma ha de ser la mateixa.

- b) Definim $M(i, p)$ com al valor màxim que es pot obtenir agafant objectes fins al i inclòs amb pes menor o igual a p . Llavors,

$$M(i, p) = \begin{cases} 0 & \text{si } i = 0, \\ \max\{M(i-1, p-p_i) + v_i, M(i-1, p)\} & \text{si } i > 0 \text{ i } p_i \leq p, \\ M(i-1, p) & \text{si } i > 0 \text{ i } p_i > p. \end{cases}$$

El cost resultant d'aplicar programació dinàmica utilitzant aquesta recurrència és $\Theta(nP)$ en espai i temps. El problema decisonal es respon consultant si $M(n, P) \geq V$.

- c) El problema MOTXILLA és **NP**-complet i la solució anterior de cost $\Theta(nP)$ sembla tenir cost polinòmic, però en realitat no ho és, perquè el cost creix amb el valor de P , que és exponencial respecte de la talla de P . Un cas clar seria quan $P = 2^n$. Així, el gran professor Petit no ha establert que $\mathbf{NP} = \mathbf{P}$ i haurà de continuar vivint del seu sou retallat.

36.6 Algorismes d'aproximació

- a) L'algorisme avança mentre pot incrementar el tall de S . Com que el tall comença a zero, com que creix de, com a poc, una unitat a cada iteració, i com que el tall està afitat pel nombre d'arestes del graf, l'algorisme no pot fer més de m iteracions.

- b) Sigui S el subconjunt de vèrtexs retornat per l'algorisme i sigui t^* el tall màxim.

Considerem v un vèrtex de S amb grau $g(v)$. Llavors, v té $g_d(v)$ veïns dins de S i $g_f(v)$ veïns a fora de S , amb $g_d(v) + g_f(v) = g(v)$. Com que l'algorisme s'ha acabat, sabem que $g_f(v) \geq g_d(v)$ (perquè, sinó, es compliria la segona condició dels ifs) i, per tant, $g_f(v) \geq \frac{1}{2}g(v)$.

El mateix passa per a vèrtexs u fora de S , definint aquest cop $g_f(u)$ com al nombre de veïns de u dins de S .

Tenim, doncs,

$$\begin{aligned} t(S) &= \frac{1}{2} \left(\sum_{u \in S} g_f(u) + \sum_{u \notin S} g_f(u) \right) \\ &\geq \frac{1}{2} \left(\sum_{u \in S} \frac{1}{2}g(u) + \sum_{u \notin S} \frac{1}{2}g(u) \right) \\ &= \frac{1}{4} \sum_{u \in V} g(u) = \frac{1}{4}2m = \frac{1}{2}m. \end{aligned}$$

Com que $t^* \leq m$ i hem establert que $t(S) \geq \frac{1}{2}m$, obtenim $t^* \leq 2t(S)$.

Per tant, com que és clar que l'algorisme es pot implementar en temps polinòmic, *cerca* és un algorisme d'aproximació de raó 2.

Parcial, 27 d'octubre de 2011

37.1 Ompliu els blancs

Ompliu els blancs de la forma més curta i precisa possible.

- Si $T(n) = 4T(n/2) + 3n^3 - 2n^2 + 1$, llavors $T(n) = \Theta(\boxed{n^3})$.
- Si $T(n) = 3T(n-2) + n^2 - 2n + 1$, llavors $T(n) = \Theta(\boxed{3^{n/2}})$.
- Qualsevol algorisme per trobar el k -èsim element més petit d'una taula de talla n necessita $\boxed{\Omega(n)}$ passos.
- En un min-heap amb n elements, en el cas pitjor, consultar l'element mínim té cost $\boxed{\Theta(1)}$, consultar l'element màxim té cost $\boxed{\Theta(n)}$, inserir un element té cost $\boxed{\Theta(\log n)}$ i esborrar el mínim té cost $\boxed{\Theta(\log n)}$.
- Per ordenar un vector amb n elements tots iguals, l'algorisme de merge sort triga $\Theta(\boxed{n \log n})$ passos i l'algorisme de quick sort (amb la partició de Hoare) en triga $\Theta(\boxed{n \log n})$.
- Quick sort triga $\Theta(\boxed{n \log n})$ passos per ordenar un vector amb n elements si sempre tria com a pivot la mediana dels elements a particionar.
- | | | |
|--------------------------|-----------------------------------|---------------------|
| $\Omega(1/n)$ | $1/(\log n)$ | $O(1)$ |
| $\Omega(n^{3/2})$ | $7n^5 - 3n + 2$ | $O(2^n)$ |
| $\Omega(n^2 / \log^2 n)$ | $(n^2 + n) / (\log^2 n + \log n)$ | $O(n^2 / \log^2 n)$ |
| $\Omega(5^n)$ | $2^{\log^2 n}$ | $O(n^n)$ |
| $\Omega(2^n)$ | 3^n | $O(5^n)$ |

37.2 Costs

- a) El cost de *cosa*(n) és

$$T(n) = \sum_{i=0}^n \sum_{j=0}^i \sum_{k=j}^i \Theta(1) = \Theta\left(\frac{11}{6}n + \frac{1}{3} + \frac{1}{2}(n+1)^2 + \frac{1}{6}(n+1)^3\right) = \Theta(n^3).$$

- b) El cost de *farandula*(n) ve donat per

$$R(n) = \begin{cases} \Theta(1) & \text{si } n = 0, \\ \Theta(n^3) + R(n/3) & \text{altrament.} \end{cases}$$

Utilitzant el teorema mestre, s'obté $R(n) = \Theta(n^3)$.

37.3 Sumar un

- a) El cas millor es dóna quan el bit de menys pes és 0. Per sumar una unitat, només cal canviar-lo per un 1. Així, el temps no depèn de la talla del nombre i és, per tant, constant, és a dir, $\Theta(1)$.
- b) El cas pitjor es dóna quan tots els bits són 1. Per sumar una unitat, cal canviar-los tots per 0. Així, el temps és lineal, és a dir, $\Theta(n)$.
- c) La probabilitat de que l'algorisme se n'emporti una fins al bit i és $1/2^i$ per a $i = 1, \dots, n$. Per tant, en mitjana triga

$$T_{\text{mig}}(n) = \Theta(1) \sum_{i=1}^n i/2^i = \Theta(1) (2 - n2^{-n} - 2^{1-n}) = \Theta(1).$$

Alternativament, podem establir la recurrència següent:

$$T_{\text{mig}}(n) = \frac{1}{2}\Theta(1) + \frac{1}{2}(\Theta(1) + T_{\text{mig}}(n-1)).$$

Amb el teorema mestre, s'obté $T_{\text{mig}}(n) = \Theta(1)$.

37.4 Heaps

- a) L'algorisme llegeix una seqüència de n enters i l'escriu de gran a petit.
- b) La cua de prioritats mai té més de n elements, per tant, les n operacions de *push*() i *front*() que s'hi apliquen sempre tenen cost $O(\log n)$ cadascuna (no Θ perquè moltes vegades no hi ha n elements!). Per tant, el cost és $O(n \log n)$.

Per veure que aquest cos és $\Theta(n \log n)$, hom pot observar que es tracta d'un algorisme d'ordenació basat en comparacions (les que fa el max-heap). Per tant, és $\Omega(n \log n)$.

Alternativament, es pot veure que, a la segona meitat de les iteracions del primer bucle i a la primera meitat de les iteracions del segon bucle, sempre hi ha, almenys, $n/2$ elements a la cua de prioritats. Per tant, les $n/2$ operacions de *push*() i *front*() que s'hi apliquen sempre tenen cost $\Omega(\log \frac{n}{2})$ cadascuna, és a dir, un total de $\Omega(\frac{n}{2} \log \frac{n}{2}) = \Omega(n \log n)$.

- c) El cas millor es dóna quan tots els elements de la seqüència són iguals: en aquest cas, cada *push*() i *front*() tenen cost constant, perquè al max-heap l'element ni es sura ni s'enfonsa perquè el seu pare i els seus fills són sempre iguals que ell. Per tant, en el cas millor, el cost és $\Theta(n)$.

37.5 Triangles en grafs

Com que el graf donat no té cicles de longitud 5 o més, el problema és equivalent a detectar si el graf té algun cicle de longitud senar, cosa que és equivalent a comprovar que no sigui 2-colorejable. Per tant, un simple recorregut (que costa temps lineal en el nombre de vèrtexos i arestes) resol el problema.

Extraordinari, 5 de juliol de 2011

38.1 Cues de prioritats

Una cua de prioritats enmagatzema un multiconjunt d'elements. Les seves operacions són:

- Crear un multiconjunt buit.
- Determinar si el multiconjunt és buit o no.
- Afegir un element al multiconjunt.
- Consultar quin és l'element més petit (gran) del multiconjunt.
- Esborrar quin és l'element més petit (gran) del multiconjunt.

Les cues de prioritat se solen implementar utilitzant heaps. Utilitzant heaps, el cost de les operacions anteriors sobre una cua de prioritats amb n elements és $\Theta(\log n)$, excepte per crear i consultar si és buida, que són $\Theta(1)$.

38.2 Anàlisi

a) El primer bucle té cost $\Theta(n)$ ja que k val 1 o 2. El segon bucle té cost $\Theta(n\sqrt{n})$ perquè el bucle de les i fa $\lceil \sqrt{n} \rceil$ interaccions. El tercer bucle té cost $\Theta(n \log n)$ ja que en el bucle de les i , aquesta es dobla a cada iteració. Així, la funció *intrigant* (n) té cost $\Theta(n\sqrt{n})$.

b) Podem escriure la recurrència següent per a la funció *confus* (n):

$$T(n) = \Theta(n\sqrt{n}) + T(n/2)$$

que, gràcies al teorema mestre, sabem que val $\Theta(n\sqrt{n})$.

38.3 NP-completesa

El primer problema és el problema de la PARTICIÓ, que sabem és NP-complet. Per tant, si $P \neq NP$, no pertany a P .

El segon problema es pot resoldre en temps $O(n^3)$ amb l'algorisme evident. Per tant, pertany a P amb independència de la hipòtesi.

38.4 Indecidibilitat

El problema de l'aturada total també es indecidible.

38.5 Aproximació

a) És clar que $\max_{i \in 1..n} p_i \leq OPT$ perquè no es pot trigar menys que el temps de la tasca més llarga. Igualment, $\frac{1}{m} \sum_{i \in 1..n} p_i \leq OPT$, perquè al dividir totes les tasques entre totes les màquines, pot ser que ens quedi un rest.

b)

És clar perquè l'algorisme assigna continuament tasques a les màquines mentre no s'exhaureixen.

c)

Sigui A el temps de finalització de tots els treballs. Sabem que el temps mentre totes les màquines estan treballant el temps és $\frac{1}{m} \sum_{i=1..n} p_i$ com a molt. Després, no pot haver-hi cap màquina que trigui més $\max_{i=1..n} p_i$. Per tant,

$$A \leq \frac{1}{m} \sum_{i=1..n} p_i + \max_{i=1..n} p_i \leq 2OPT.$$

38.6 Grafs

falta

38.7 Arbres

falta

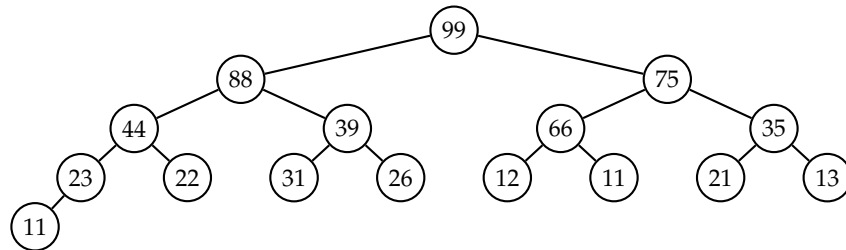
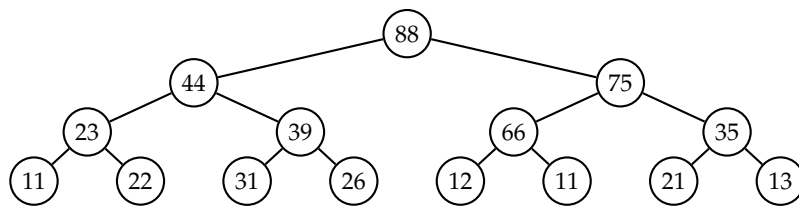
38.8 Més arbres

falta

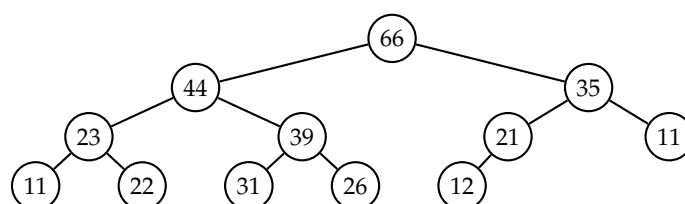
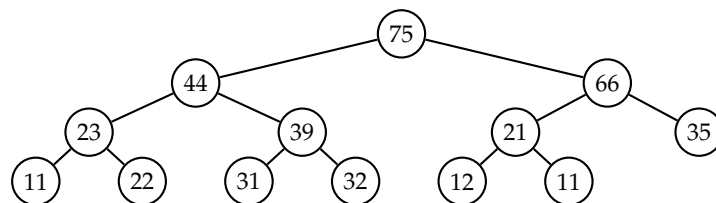
Final, 11 de gener de 2011

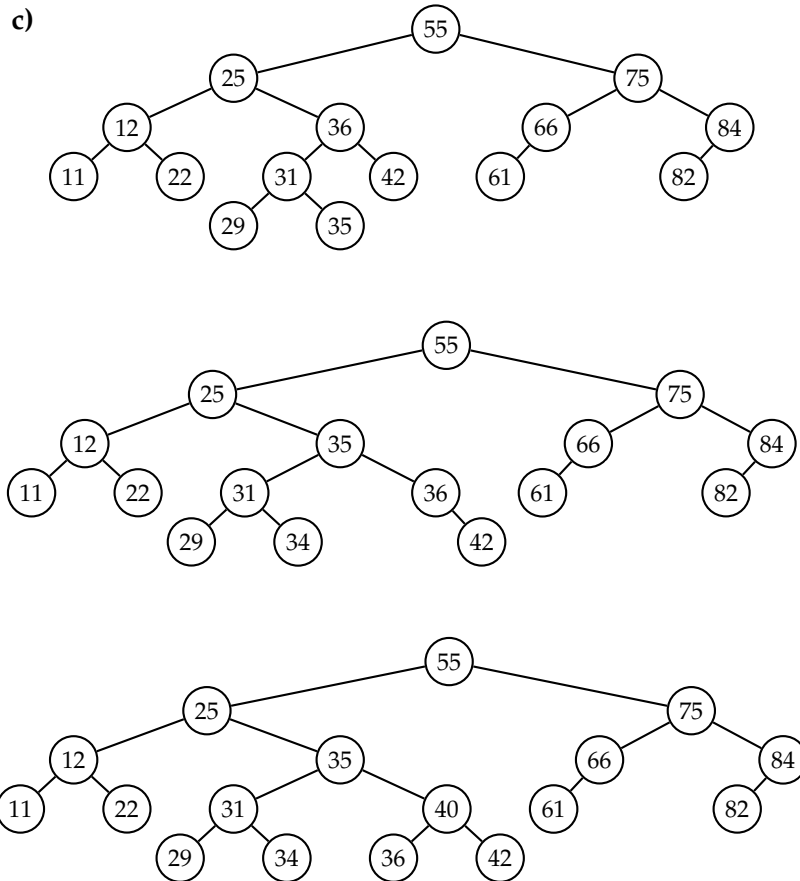
39.1 Heaps i AVLs

a)



b)





39.2 Anàlisi

- a) El cost del primer bucle és clarament $\Theta(n)$.

El cost del segon bucle és $\sum_{i=1}^n \sum_{j=1}^i \sum_{k=1}^j \Theta(1)$ que és $\Theta(n^3)$.

El cost del tercer bucle és clarament $\Theta(n \log n)$.

Així, el cost total és $\Theta(n) + \Theta(n^3) + \Theta(n \log n) = \Theta(n^3)$.

- b) La recurrència pel cost d'aquesta funció és $T(n) = \Theta(n^3) + T(n-1)$. Utilitzant el teorema mestre es troba que $T(n) = \Theta(n^4)$.

39.3 NP-completesa

- a) 2-COLORABILITAT es pot reduir polinòmicament a 3-COLORABILITAT.

Cert: 2-COLORABILITAT pertany a la classe **P**, modificant lleugerament l'algorisme BFS (recorregut en amplada) s'obté un algorisme de temps polinòmic que decideix si donat un graf G es pot pintar amb només dos colors. Com que tot problema a **P** és també a **NP** (**P** és una subclasse de **NP**) i 3-COLORABILITAT és **NP-complet**, aleshores 2-COLORABILITAT es pot reduir polinòmicament a 3-COLORABILITAT.

- b) 3-COLORABILITAT es pot reduir polinòmicament a 2-COLORABILITAT.

No es pot assegurar ni una cosa ni l'altra: Si 3-COLORABILITAT es reduís polinòmicament a 2-COLORABILITAT, aleshores 3-COLORABILITAT seria a **P** i com que és **NP-complet** tindríem **P** = **NP**, igualtat de la que no es coneix la seva veritat o falsetat. Sota

la hipòtesi **P** diferent de **NP** tindríem que aquesta reducció no és possible. Sota la hipòtesi **P** = **NP** aleshores 3-COLORABILITAT es reduiria polinòmicament a 2-COLORABILITAT.

39.4 Indecidibilitat

Suposem que existís un algorisme **bool** *acaba*(*string* *p*, *string* *x*) que, donat un programa *p* i una entrada *x*, indica si el programa *p* s'atura o no quan s'executa sobre l'entrada *x*.

Ara, considerem aquest algorisme que potencialment explora totes les *n, x, y, z*:

```
void fermat () {
    for (int i = 3; true; ++n)
        for (int n = 3; n < i; ++n)
            for (int x = -i; x ≤ i; ++x) if (x ≠ 0)
                for (int y = -i; y ≤ i; ++y) if (y ≠ 0)
                    for (int z = -i; z ≤ i; ++z) if (z ≠ 0)
                        if (pot(x, n) + pot(y, n) == pot(z, n))
                            return;
}
```

És clar que si la conjectura de Fermat és falsa, aquest algorisme trobarà tard o d'hora un contra-exemple i acabarà en el **return**. Igualment, si la conjectura és certa, aquest algorisme es penjarà.

Així, per saber si la conjectura de Fermat és certa o no només ens caldria calcular **not** *acaba*(*fermat*, ""). De fet, l'entrada és irrellevant.

39.5 Aproximació

- Totes les arestes amb un o dos node no-fula estàn cobertes. Ens cal veure que no hi ha cap aresta entre dues fulles de l'arbre de DFS: Suposem que hi hagués una aresta $\{u, v\}$ entre dues fulles *u* i *v* i que *u* hagués estat visitat abans que *v*. Com que hi ha aresta entre *u* i *v* i *v* encara no s'ha visitat, s'ha d'anar a visitar *v*. Però llavors *u* no és fulla!
- Trobem un aparellament maximal (*maximal matching*) *M* sobre l'arbre de DFS: Agafem l'arrel i un dels seus veïns i els aparallem. Els treiem de l'arbre donant lloc a més subarbres i continuem. Els subarbres amb dos o més nodes es poden continuar aparellant, els que només tenen un node, aquest es correspon amb un node fulla. Així, $|M| \geq |S|/2$ on *S* és el conjunt de nodes no fulla. Com que sabem que $|M| \leq OPT$, obtenim la 2-aproximació.

39.6 Grafs

- Sigui *G* el graf amb vèrtexs $V = \{x, a, b, c\}$, amb els arcs
 - $x \rightarrow a$, amb cost 10;
 - $a \rightarrow b$, amb cost 10;
 - $x \rightarrow c$, amb cost 30;
 - $c \rightarrow a$, amb cost -100.

L'algorisme de Dijkstra trobaria els costos següents en aquest ordre: cost 0 per a x , cost 10 per a a , cost 20 per a b , i cost 30 per a c . Així doncs, el cost mínim real d'anar des de x fins a b , que és -60 , no seria trobat per l'algorisme.

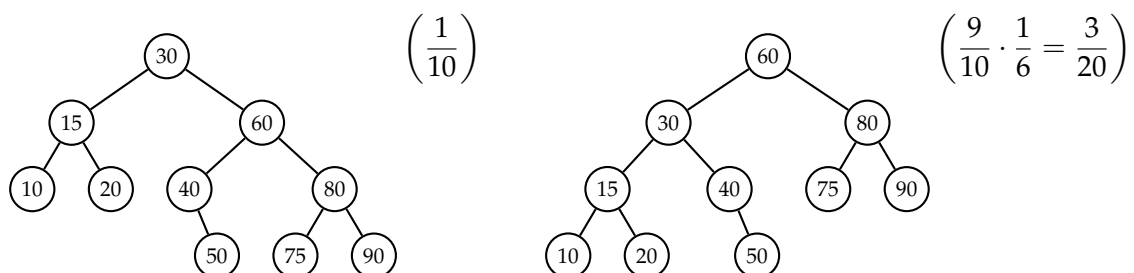
- b) La modificació proposada no permet calcular els costos mínims, perquè l'increment del cost de cada camí depèn del nombre d'arcs que es travessi. Els costos obtinguts per a G' només permeten calcular fites superiors per als costos de G , perquè poden existir camins amb més passos que els millors camins trobats per a G' que amb els costos originals de G siguin menys costosos.

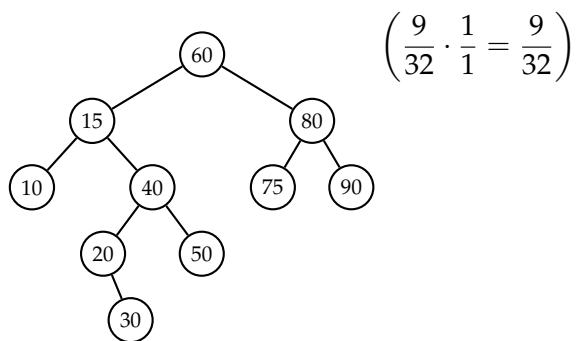
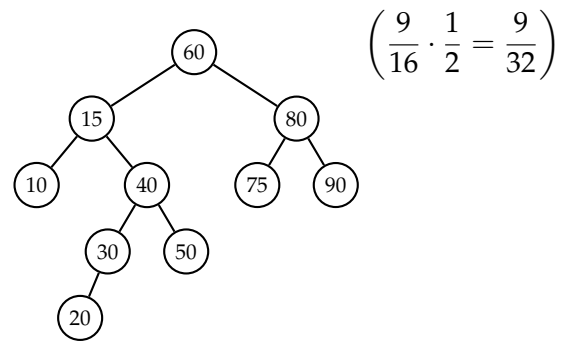
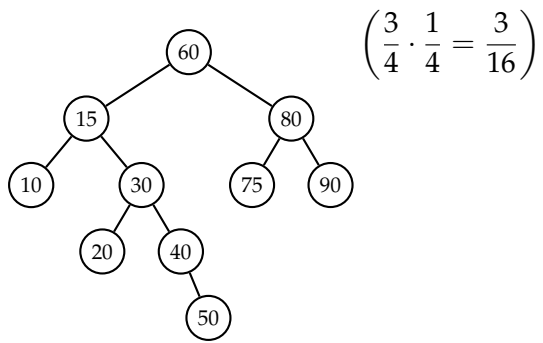
39.7 Llistes

```
Punter esborra(int x, Punter p) {
    if (p == 0) return 0;
    if (p->elem > x) return p;
    if (p->elem < x) {
        p->seguent = esborra(x, p->seguent);
        return p;
    }
    punter q = p->seguent;
    delete p;
    return q;
}
```

39.8 Arbres

L'algorisme insereix el 30 començant a l'arrel, i a cada pas considera si es guarda en aquella posició o més avall, en funció del nombre d'elements del subarbre en curs. El possibles arbres, amb les seves probabilitats (calculades com el producte de la probabilitat de no haver inserit encara el 30, per la probabilitat de fer-ho en aquell nivell), són





Parcial, 28 d'octubre de 2010

40.1 Ompliu els blancs

- Definiu $O(f)$:

$$O(f) = \{g \mid \exists c_0 > 0, \exists n_0 > 0, \forall n > n_0 : g(n) \leq c_0 f(n)\}$$

- El teoreme mestre de resolució de recurrències divisores afirma que si tenim una recurrència de la forma $T(n) = aT(n/b) + \Theta(n^k)$ amb $b > 1$ i $k \geq 0$, llavors, fent $\alpha = \log_b a$,

$$T(n) = \begin{cases} \Theta(n^k) & \text{si } \alpha < k, \\ \Theta(n^\alpha \log n) & \text{si } \alpha = k, \\ \Theta(n^\alpha) & \text{si } \alpha > k. \end{cases}$$

- Si $T(n) = 3T(n/2) + n^2 - 2n + 1$, llavors $T(n) = \Theta(\boxed{n^2})$.
- Si $T(n) = 2T(n-1) + n^2 - 2n + 1$, llavors $T(n) = \Theta(\boxed{2^n})$.
- Mergesort ordena n elements en temps $\boxed{\Theta(n \log n)}$ en el cas pitjor, en temps $\boxed{\Theta(n \log n)}$ en el cas mitjà, i en temps $\boxed{\Theta(n \log n)}$ en el cas millor. L'espai auxiliar requerit és $\boxed{\Theta(n)}$.
- Quicksort ordena n elements en temps $\boxed{\Theta(n^2)}$ en el cas pitjor, en temps $\boxed{\Theta(n \log n)}$ en el cas mitjà, i en temps $\boxed{\Theta(n \log n)}$ en el cas millor.
- Insertionsort ordena n elements en temps $\boxed{\Theta(n^2)}$ en el cas pitjor, en temps $\boxed{\Theta(n^2)}$ en el cas mitjà, i en temps $\boxed{\Theta(n)}$ en el cas millor.

- Per multiplicar dos nombres naturals grans eficientment podem fer servir l'algorisme de Kartabuba-Ofman .
- Tot algorisme per trobar la mediana de n elements necessita $\Omega(n)$ passos.
- Un algorisme d'ordenació és estable si preserva l'ordre relatiu de les claus repetides .

40.2 Ordenació

L'algorisme següent es coneix com a Counting Sort:

```
void ordena ( vector<int>& v) {
    int n = v.size ();
    int m = int(n*K + 1);
    vector<int> c(m, 0);
    for (int i = 0; i < n; ++i) ++c[v[i]];
    int j = 0;
    for (int i = 0; i < m; ++i) {
        for (int k = 0; k < c[i]; ++k) v[j++] = i;
    }
}
```

És clar que el cost en temps i es espai són $\Theta(nK)$. Com que K és una constant, aquests costos són lineals.

Aquest algorisme no és una algorisme de propòsit general: només serveix per ordenar enters i, a més, amb el seu valor afitat. Per això no hi ha cap contradicció amb la fita inferior d' $\Omega(n \log n)$ per ordenació. De fet, Counting Sort ni tan sols realitza compacions dels elements!

40.3 Dividir i vèncer

Si només tenim un cargol i una femella, ja hem acabat. Altrament:

Agafem un cargol qualsevol. Amb ell, classifiquem totes les femelles en femelles massa grans, massa petites i la femella que encaixa perfectament.

Amb aquesta darrera femella, classifiquem igualment tots els cargols: per un costat els massa grans, per l'altra els massa petits i deixem aparellat el que encaixa.

Un cop fet això, apliquem dos cops recursivament el mateix algorisme als dos grups de femelles i cargols.

Aquest algorisme té el mateix cost que Quicksort i és, doncs, $\Theta(n \log n)$.

40.4 Primalitat

Per saber si x és un nombre primer n'hi prou amb comprovar que no hi ha cap número entre 2 i $\sqrt{x}$ que divideixi x . Suposant que totes les operacions bàsiques entre enters tenen cost constant, el cost d'aquest algorisme és doncs $\Theta(\sqrt{x})$. Però si mesurem el seu cost en funció de la talla de x , és a dir, el seu nombre de bits, el cost és $\Omega(2^{n/2})$.

Es tracta doncs d'un algorisme exponencial, però això no vol dir que el problema de la primabilitat sigui intractable. De fet, existeixen algorismes polinòmics (en funció de la talla) per saber si un nombre és primer.

40.5 Producte de matrius

a) Quan n no és una potència de 2, podem “inflar” les matrius amb zeros a baix i a la dreta amb zeros fins arribar a fer que la seva mida sigui la potència següent de 2. Llavors podem usar l'algorisme de Strassen presentat a classe i “desinflar” el resultat (treure-li les files i columnes afegides al final), que serà correcte.

Al inflar les matrius, el seu nombre d'elements no creixerà més d'un factor de 4. Per tant, el cost asimptòtic es manté.

b) Sigui $s = n^2$ el nombre d'elements a les matrius. Si descomposem les matrius en 9 submatrius i cridem recursivament m vegades, tenim la recurrència $T(s) = mT(s/9) + \Theta(s)$. Sigui $\alpha = \log_9 m$ i $k = 1$. Llavors, per a $m > 9$, es té $\alpha > k = 1$. En aquest cas, el teorema mestre ens diu que $T(s) = \Theta(s^\alpha)$. Per comparar-ho amb Strassen, ens cal veure quan $\alpha = 2'81/2$. Això es dona per $m = 9^{1'405} = 21'9$. Per tant, ens cal que $m \leq 21$.

c) El plantejament és el mateix, ara ens cal $\alpha = 2'376/2$, i llavors ens cal que $m \leq 13$.